MAKING MINDS

A Constructive Introduction to the Computational Theory of Mind

Gabriel Greenberg

10.18.2026

“I never satisfy myself until I can make a mechanical model of a thing.
If I can make a mechanical model, I can understand it.”

— Lord Kelvin, 1884

Table of Contents

Unit 1: COMPUTATION

1. The Cognition-Body Problem 6
2. What Is a Computer? 10
3. Logic Gates & Combinatorial Circuits 14
4. Functions 19
5. Systems of Representation 24
6. Defining Computation 30
7. Functional Abstraction 36
8. Function Composition 38
9. The Story of Binary Successor 41
10. Lessons from Binary Successor 44
11. Real and Ideal Computers 47
12. Introducing Turbots 48
13. Thought as Symbol Processing 51
14. Artificial and Natural Computers 55
15. The Computational Theory of Mind 59

Unit 2: LEVELS

16. Memory 64
17. Physical Memory 65
18. Machines in Time: Sequential Circuits 69
19. Computing in Time 74
20. Computational Power 79
21. The Method of State Analysis 84
22. State Abstraction 89
23. The Concept of Abstraction 94
24. Finite State Machines 102
25. What Is an Algorithm? 107
26. Levels of Abstraction 110

Unit 3: COGNITION

27. Turing Machines 117
28. Computing with Turing Machines 125
29. Recursion 129
30. Turing Machine Abstraction and Composition 135
31. Binary Turing Machines 138
32. The Mathematical Tree 140
33. Computational Navigation 143
34. The Desert Ant 146
35. Physical Cognition 151
36. Machines and Programs 161
37. The Universal Machine 165
38. Cognitive Architecture 171
39. Computation as Consciousness 174
40. Computational Theory of Mind: The Long View 177

Unit 1

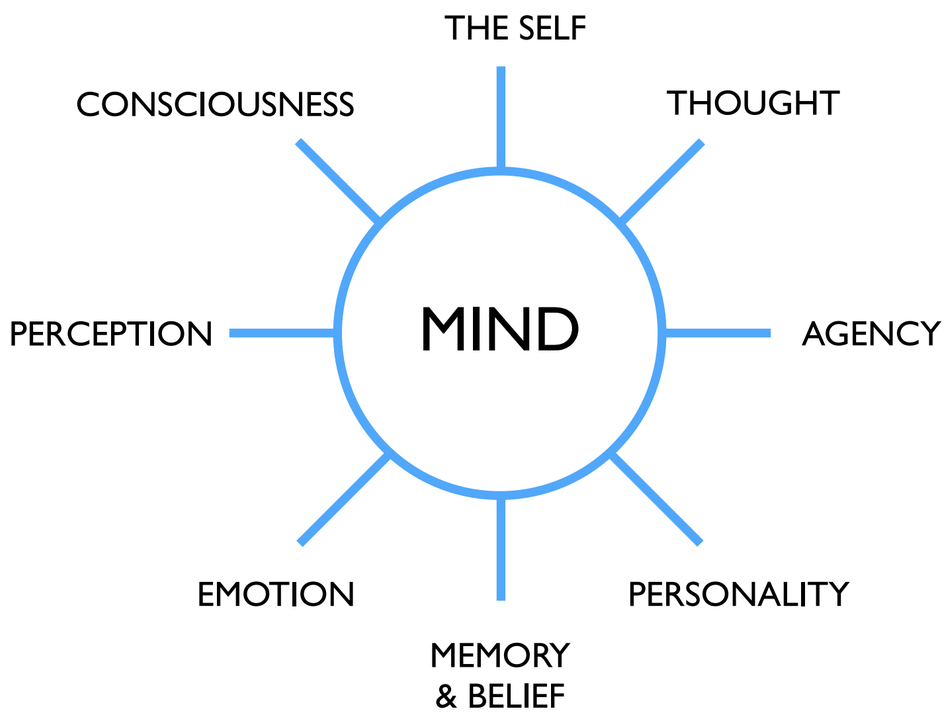
COMPUTATION

1. The Cognition-Body Problem

The Mind

What is a mind? What is *your* mind? There is no simple definition. The mind is the unity of **psychological** capacities, processes, and states.

Mental states are states of awareness, thinking, perceiving, feeling, hoping, imagining lying on the beach, and so on.



Dualism & Physicalism

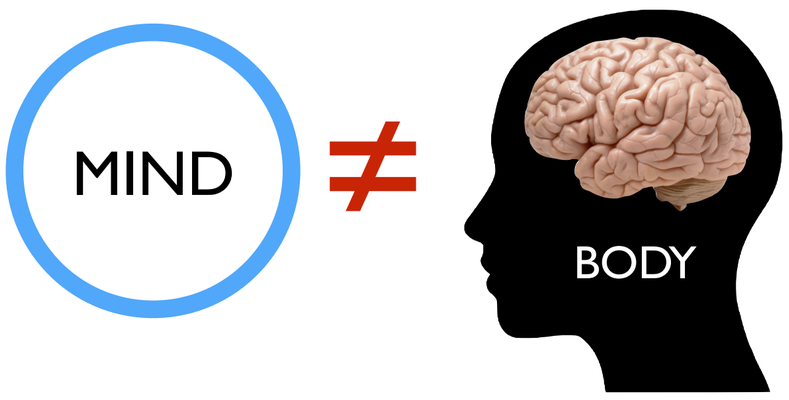
How do your own inner mental states fit into the objective physical universe? How do we reconcile mind and nature? The debate between two answers to this question is one of the deepest metaphysical debates in the history of philosophical thought.

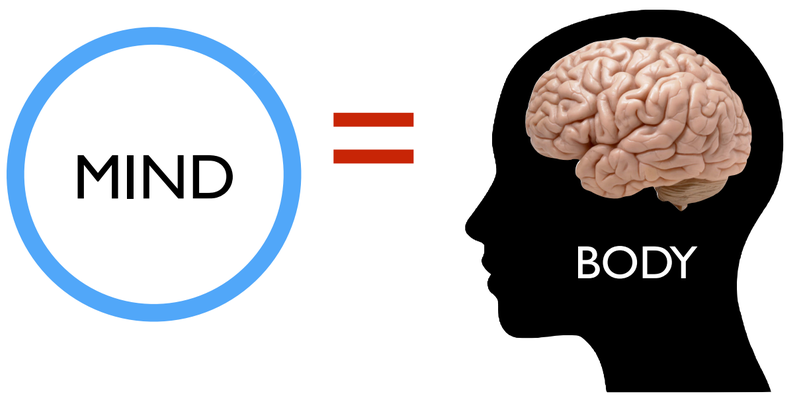
Dualism

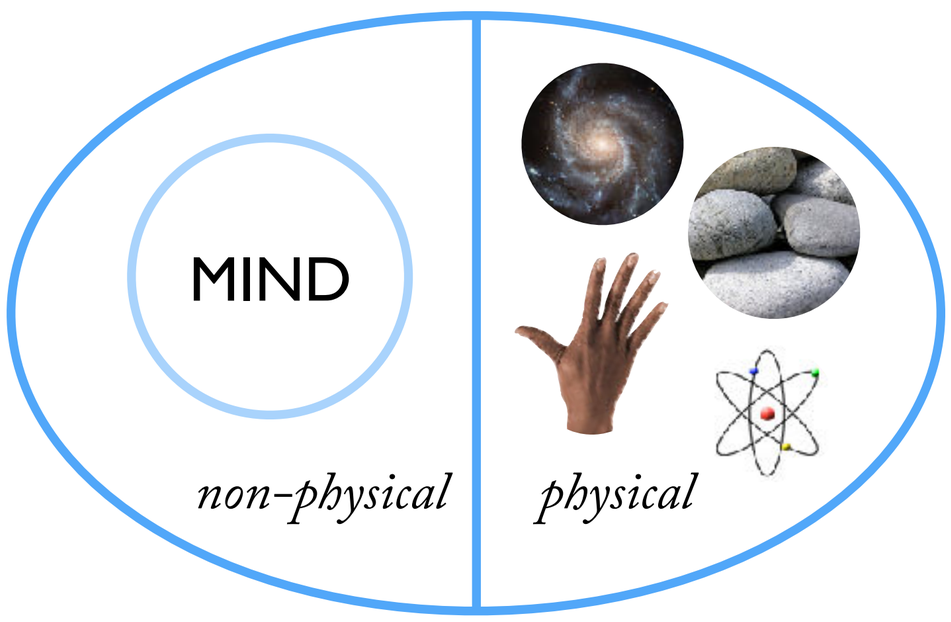
The mind is distinct from the body. No mental state is realized by a physical state.

Physicalism

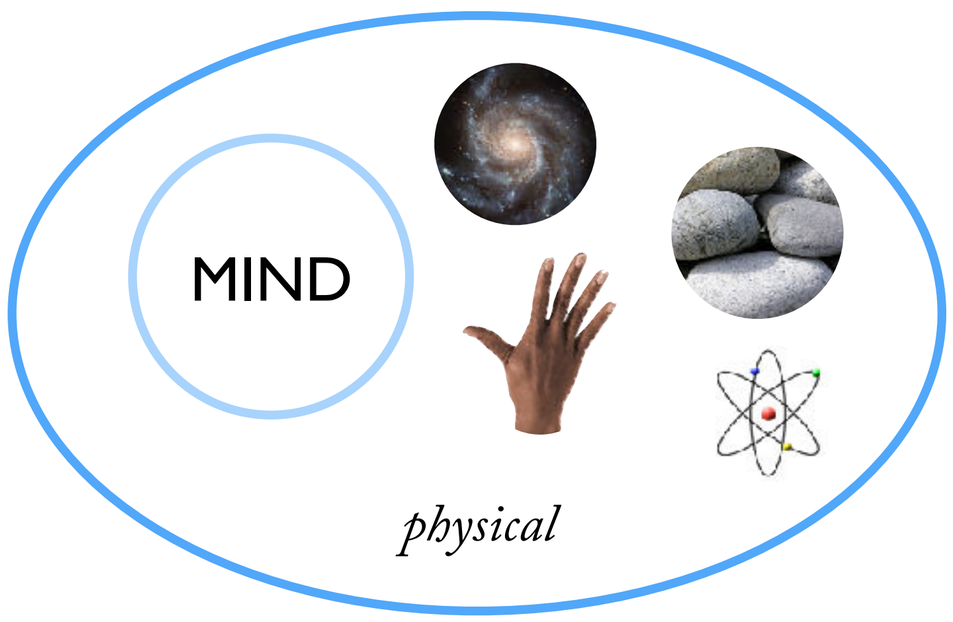
The mind is (part of) the body. Every mental state is realized by a physical state.







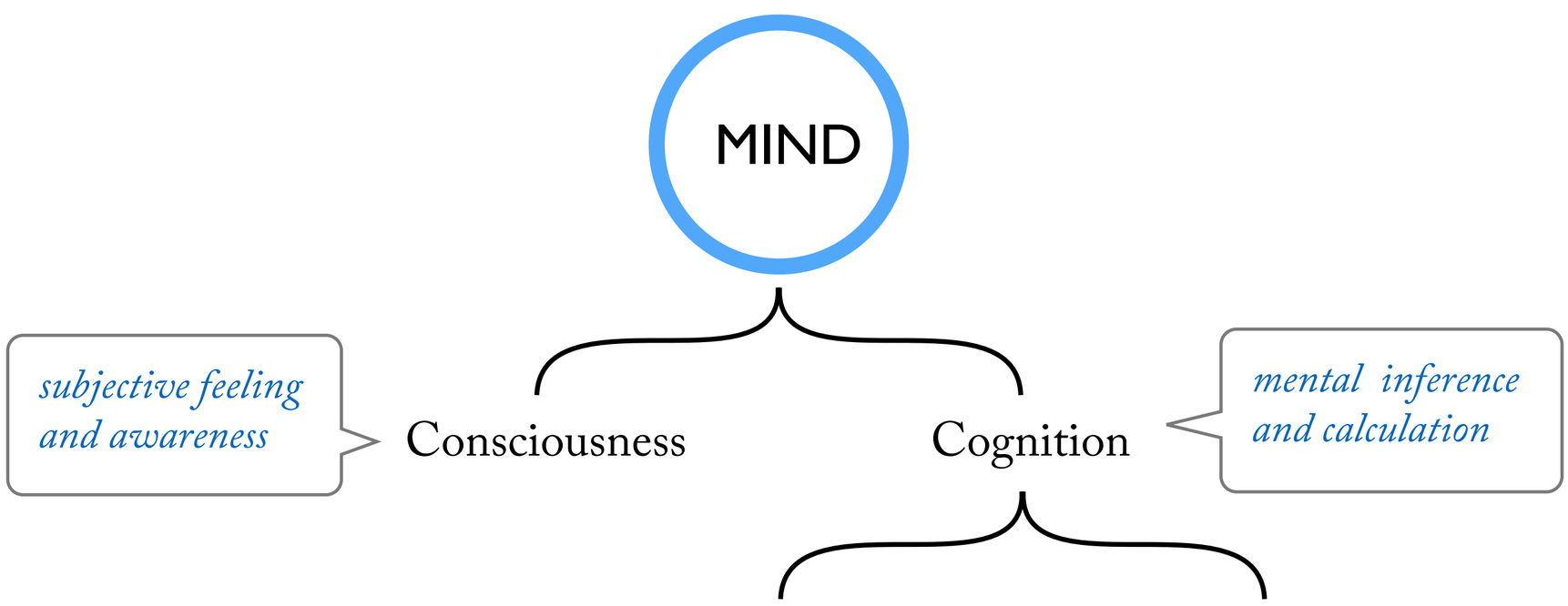
Reality according to **Dualism**



Reality according to **Physicalism**

Cognition & Consciousness

Two fundamental properties seem to distinguish minds from the rest of the non-mental universe. One is **consciousness**. The other is **cognition**.



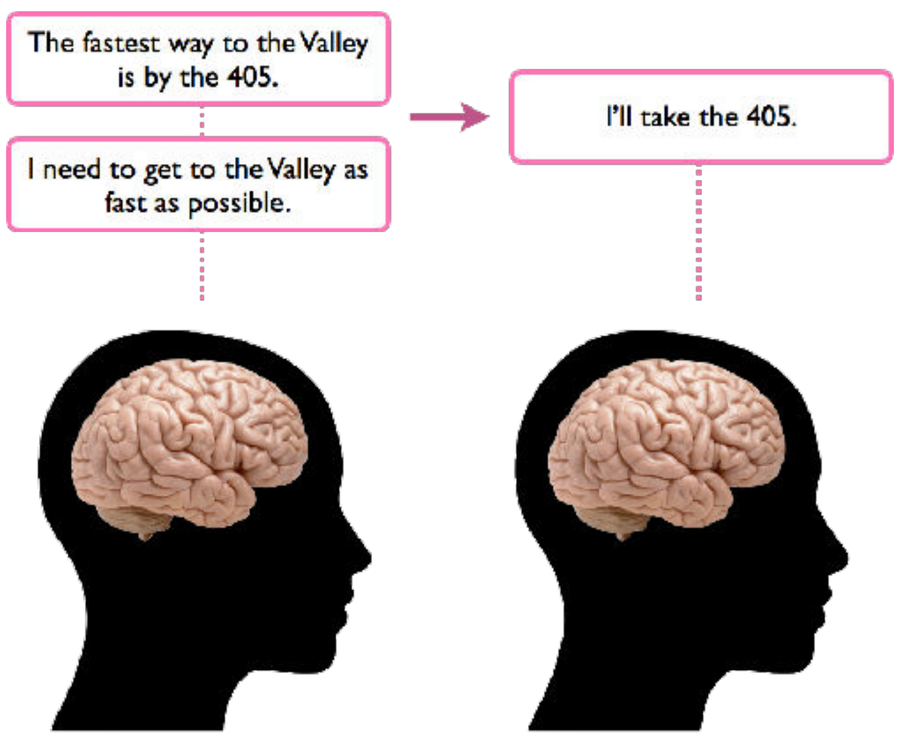
Contentful states: States possessing content.

content

Rational transitions: Rational transitions between contentful states.

content → content

A system has **cognition** if it undergoes rational transitions between contentful states.



A system undergoes **rational transitions** when it finds rational solutions to real problems.

The **content** of a state is whatever it is "about" or "of".

Individual words have particular individuals or properties as contents. Entire sentences have whole situations as their contents.

Examples of Mental States with Content

Believe: A believes *he is on Mars.*
Wish: B wishes *she could visit Mars.*
Know: C knows *she owns a G-Class.*
Think: D thinks *it's Tuesday.*
Learn: E learns *a dance move.*
See: F sees *a bicycle.*

Examples of Rationality

Logic: All X are Y. Q is X. → Q is Y.
Math: 3 + 4 → 3 + 4 = 7
Navigation: Maze → Path
Decision-Making: I want to see a movie. I like action movies. → I'll see an action movie.
Perception: [Light image] → It's a duck!
Prediction: Initial conditions → final conditions.

The Cognition-Body Problem

Leibniz (1646–1716) famously challenged the idea that cognition could emerge from purely physical elements. His objection is known as **The Windmill Argument:**

> One is obliged to admit that *perception* and what depends upon it is *inexplicable on mechanical principles*, that is, by figures and motions. In imagining that there is a machine whose construction would enable it to think, to sense, and to have perception, one could conceive it enlarged while retaining the same proportions, so that one could enter into it, just like into a windmill. Supposing this, one should, when visiting within it, find only parts pushing one another, and never anything by which to explain a perception. Thus it is in the simple substance, and not in the composite or in the machine, that one must look for perception.
>
> Leibniz 1714, *Monadology*

The argument has more or less the following form:

The Windmill Argument

Premise 1: Basic physical elements do not have cognition.

Premise 2: If a set of objects does not have cognition, then nothing composed of those objects alone has cognition.

Therefore: Nothing composed purely from basic physical elements has cognition.

Dualism is one conclusion to draw from this argument. If the premises are true, the argument implies that cognition is not physical. If it isn't physical, cognition must be the activity of a non-physical mind or soul.

Panpsychism is another possible reaction: deny Premise 1, and insist that even basic physical elements have cognition. In fact this was Leibniz' own view. (He called these basic minds "monads".) We won't discuss this interesting idea in this course.

Physicalism (or materialism) is the view that the mind (including cognition) *is* physical. Physicalists accept Premise 1, but deny Premise 2. They believe that non-cognitive parts *can* give rise to a complex whole which has genuine cognition.

The constructive challenge: physicalism is a promising *idea*, but how could you actually *make* a thinking thing from material parts? Where would you even begin?

The Computational Theory of Mind (CTM) is one version of physicalism. It seeks to understand the mind as the computational expression of the brain. CTM is more than just a philosophical idea; it is also the foundational assumption of cognitive science. The purpose of this course is to investigate CTM's answer to the cognition-body problem primarily as a philosophical and logical idea, and secondarily as an empirical hypothesis.

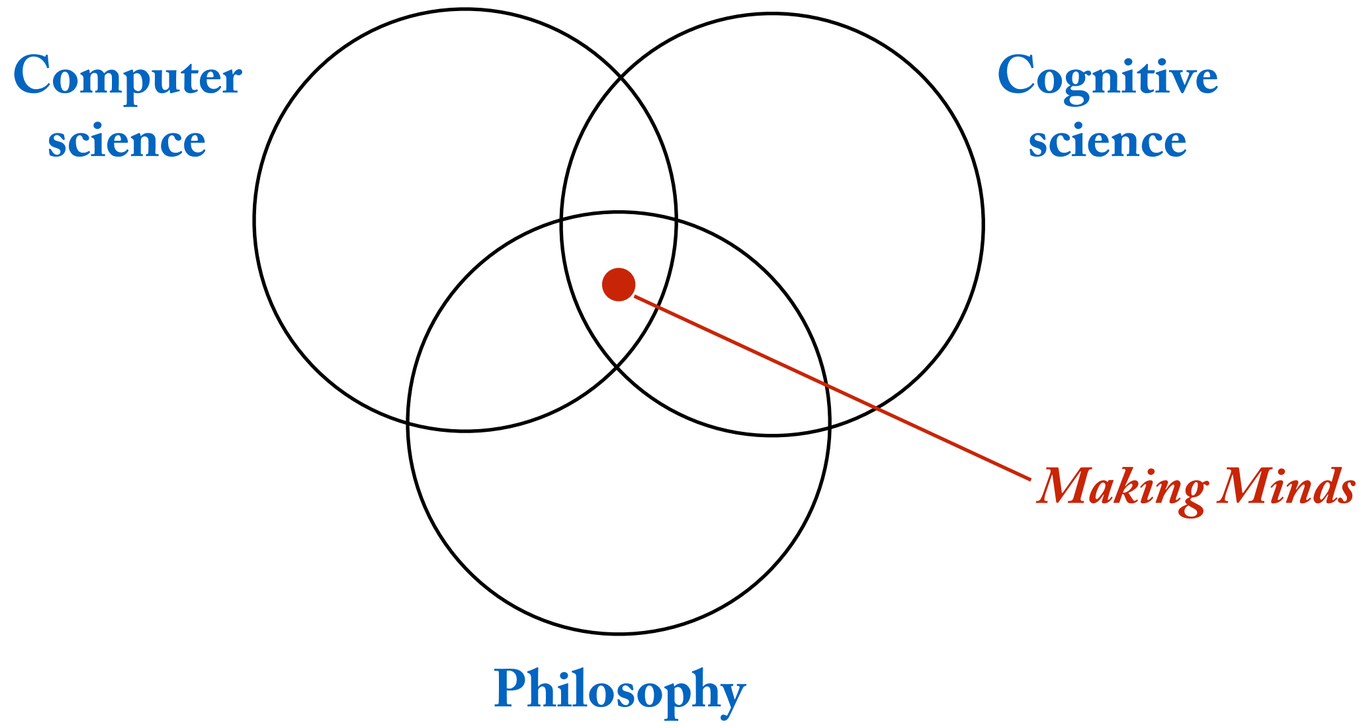
The Trajectory of This Course

This course is a sustained physicalist response to Leibniz' challenge.

Over 10 weeks we will try to see how far we can get, starting with the simplest non-thinking parts, and building towards more and more complex systems. The end goal is to design an organism whose physical construction is completely clear to us, but which seems to exhibit limited cognition. Our path to that destination is the computational approach to the mind.

The approach.

We will draw from computer science, cognitive science, and philosophy. We'll try to understand the big picture of the mind, by building detailed models of its key components. We'll use these simplified models to think about the complexity of the human brain.



Outline of the course:

Unit 1: Computation

We'll ask: what is computation? And we will construct simple logical computers from basic physical building blocks.

Unit 2: Levels

We'll expand our view of computers by considering higher levels of abstraction, and higher levels of computational power.

Unit 3: Cognition

We'll make the argument that computation in an embodied agent can give rise to genuine cognition.

Outline of Unit 1:

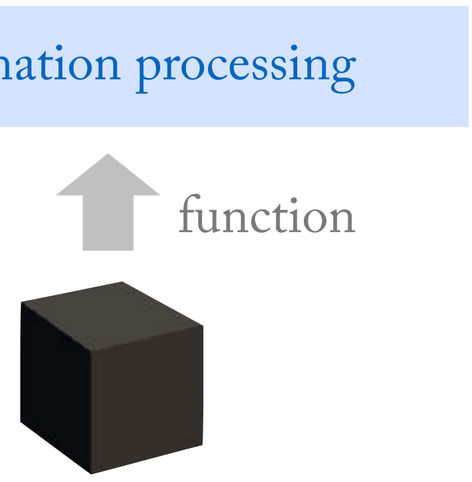
In this unit, we define the essential elements of computation.

- **Logic circuits:** our first machine.
- **Functions**: what computers do, defined precisely.
- **Representation:** how physical states relate to functions.
- **Computation:** a definition.
- **Computational theory of mind:** the conjecture at the heart of cognitive science.

2. What Is a Computer?

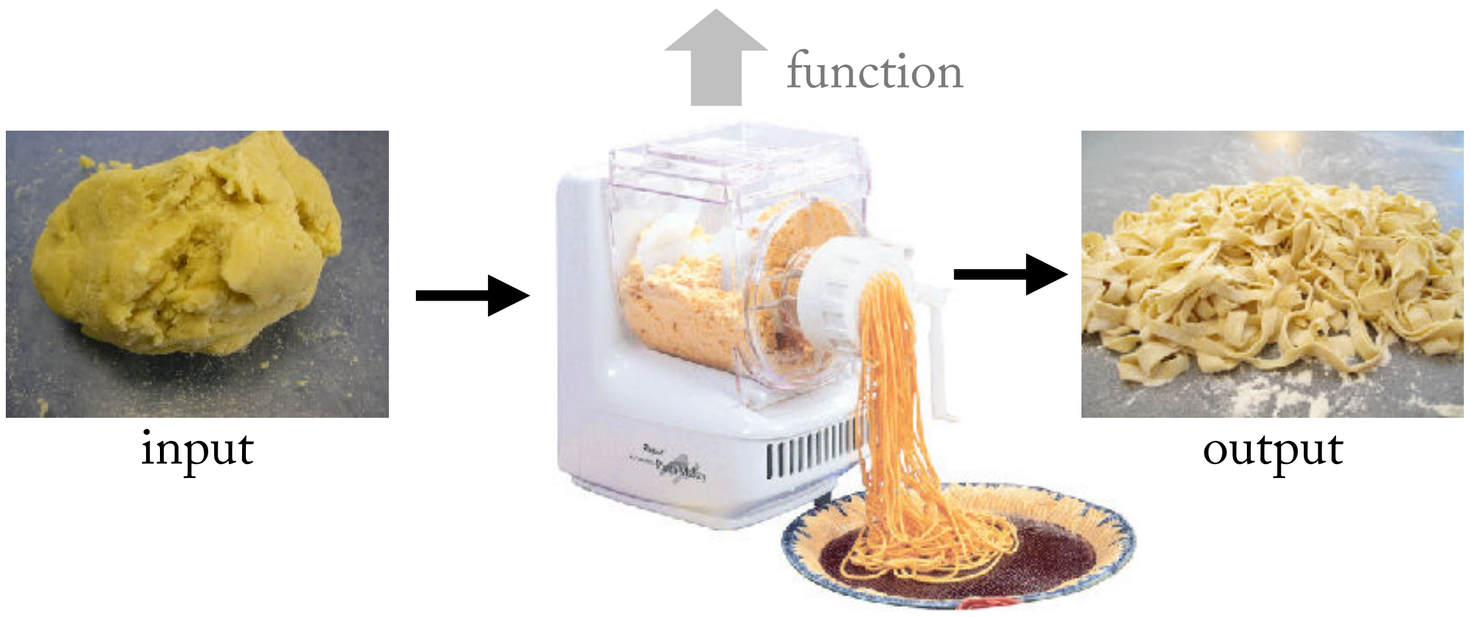
Information Processing

A computer is a machine that has the function of **processing information**.

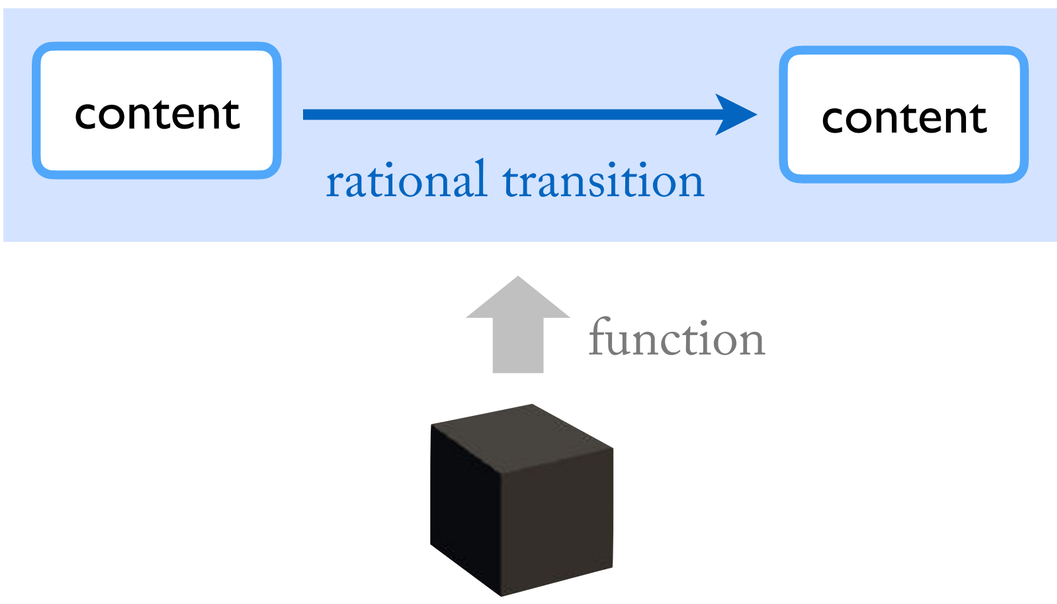


Like any other machine, it has a definite function…

… but the function of a computer isn't something that can be described in purely physical terms.

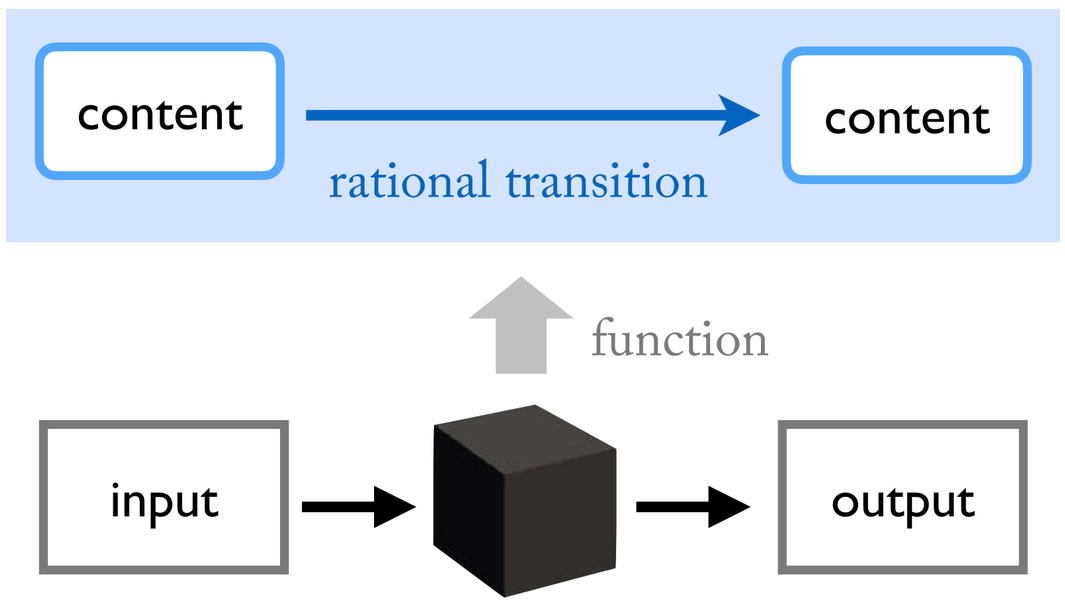


To process information is to perform **rational transitions** between **contentful states.**



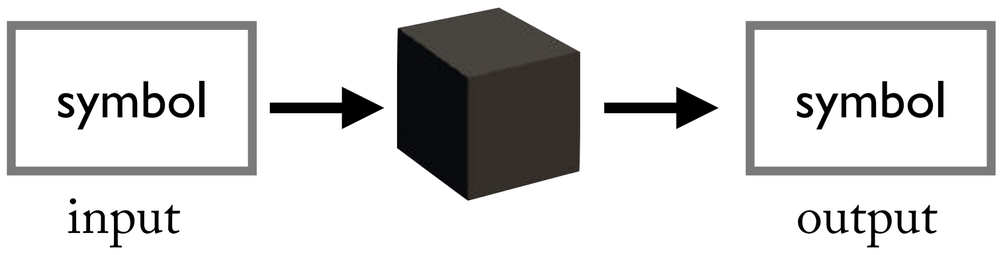
Ultimately, a computer is still a **physical object**, and it must perform its informational function *by* performing some physical process.

In fact, like the pasta machine, all computers are **input/output machines**. They take physical inputs and process them into physical outputs.



Symbol Processing

A computer takes physical signals as **input** and produces physical signals as **output**.



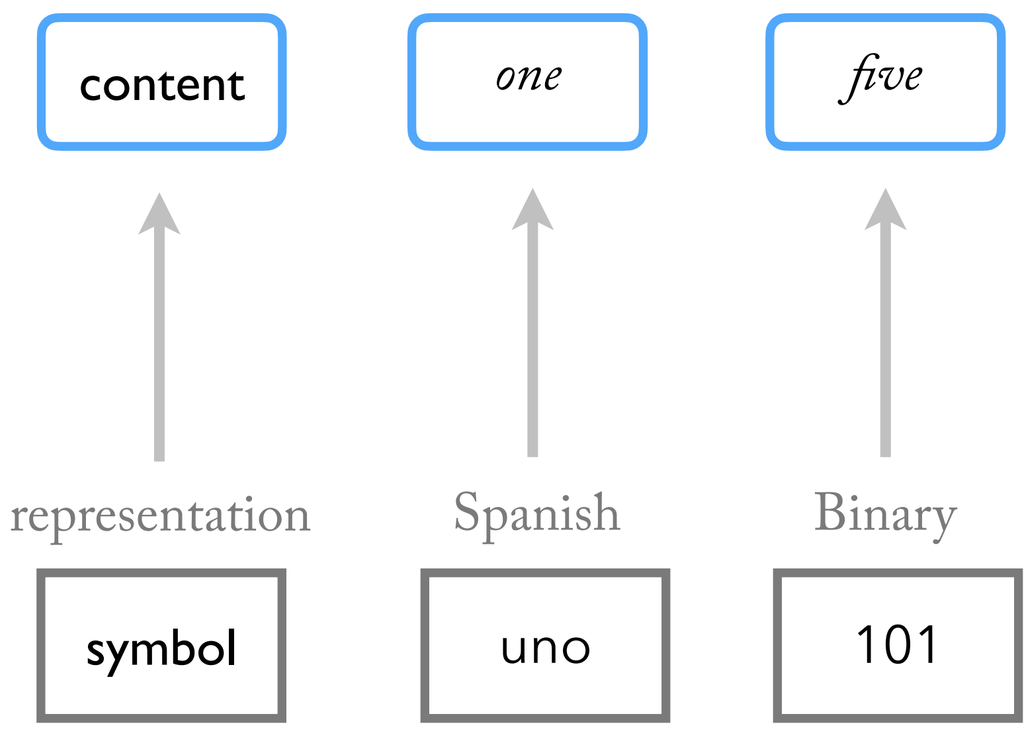
Symbols are physical objects or patterns. A computer takes symbols as inputs, processes symbols, and produces symbols as outputs.



A **representation** is a physical symbol that is *about* something or expresses a meaning.

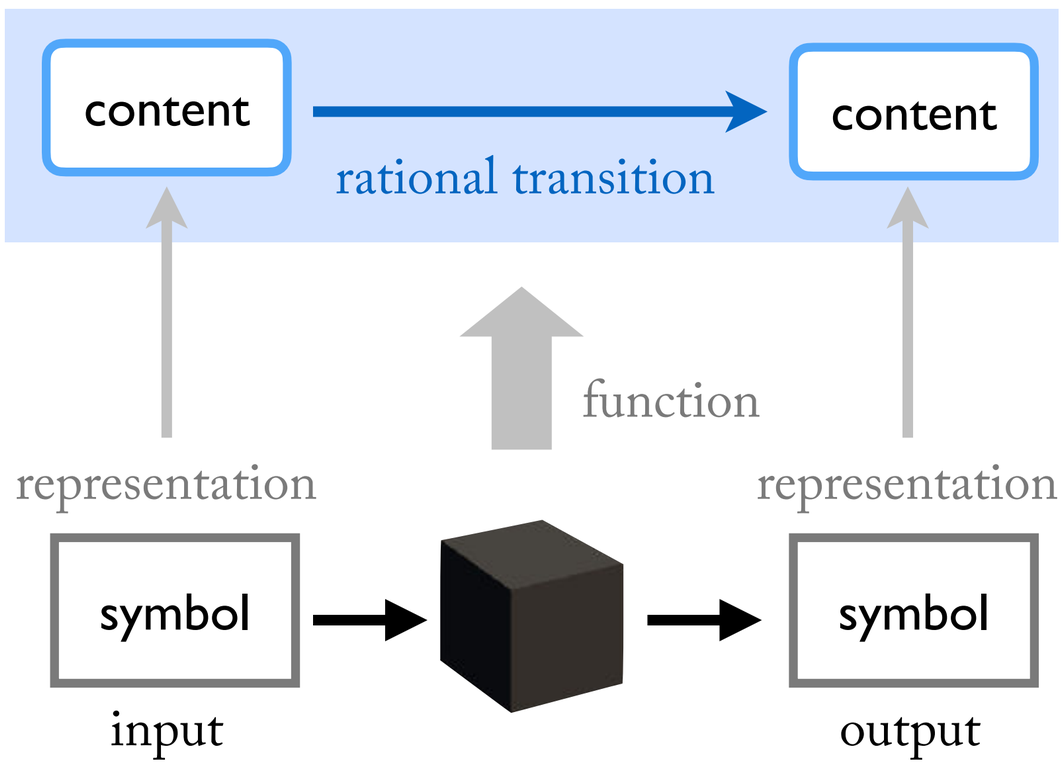
What a representation is about or what it means is its **content**. Symbols **represent** their contents.

Both **external** symbols, like words or pictures, and **internal** symbols, like computer code, can express content.

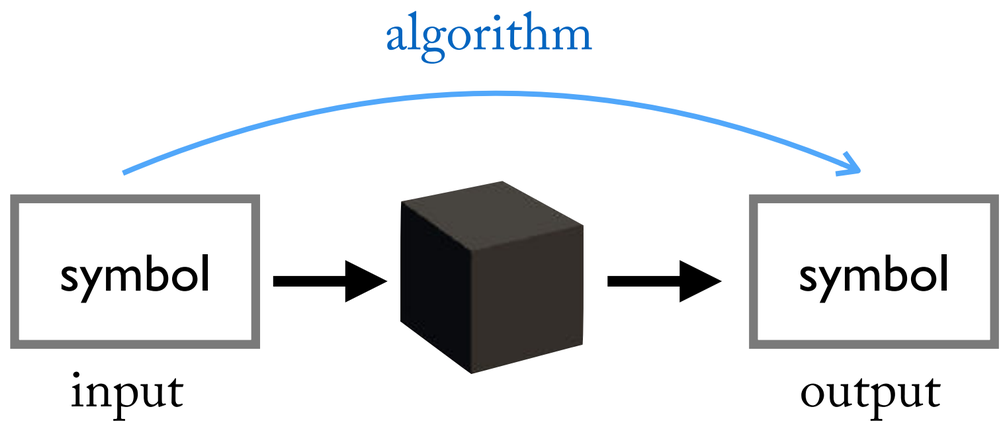


For a computer to perform a rational transition, its inputs and outputs must represent the initial and final contents of the rational transition.

The representations processed by the computer must **mirror** the rational transition.



In order to produce representations which mirror rational transitions, computers follow **algorithms:** mechanical procedures for manipulating symbols.

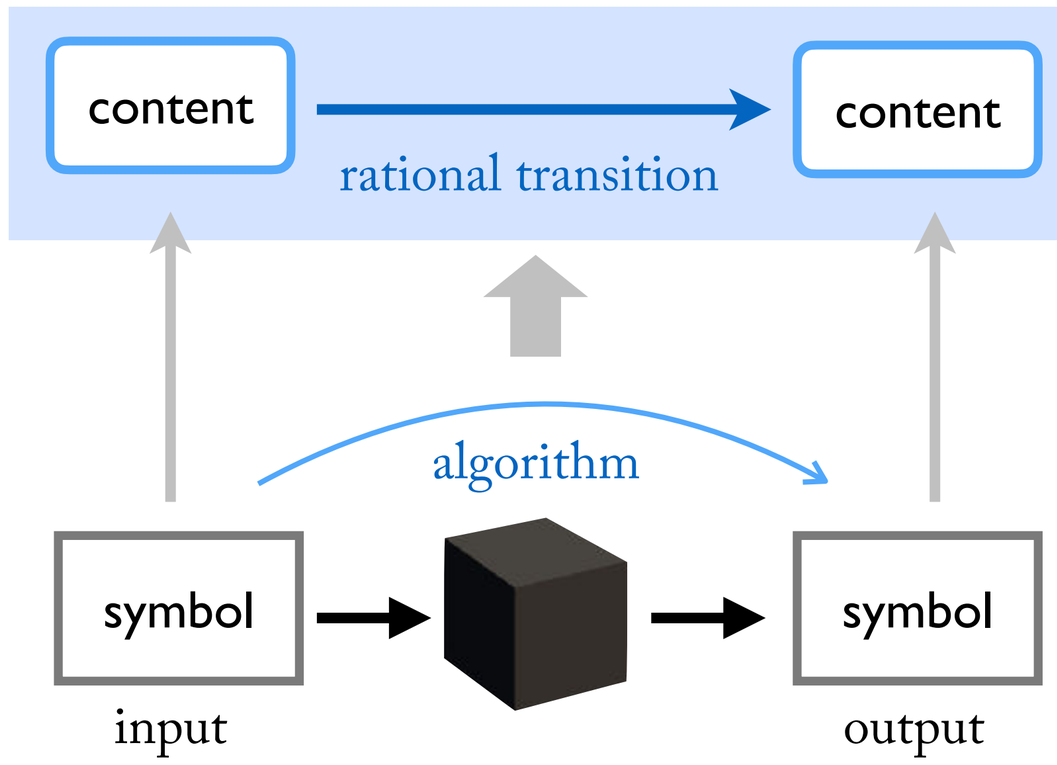


Computation

A computer processes information *by* processing symbols.

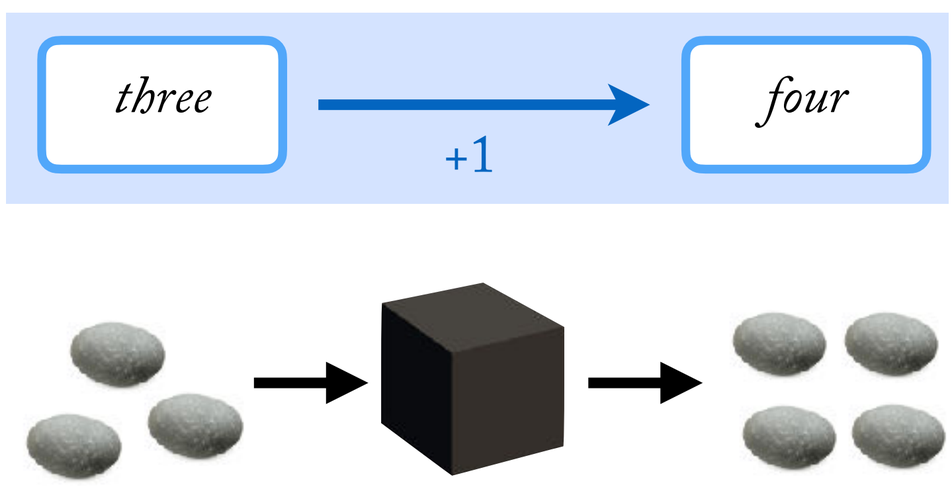
A computer functions to perform rational transitions by manipulating representational symbols according to an algorithm.

Computation as embodied logic.



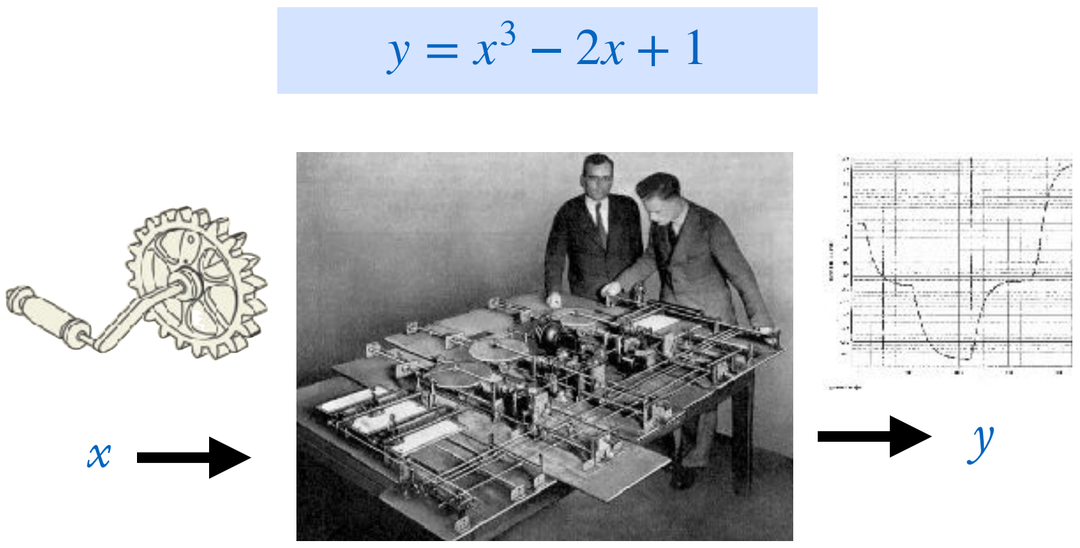
Example: Caveman computer

This computer takes buckets of rocks as input, and produces buckets of rocks as output. Each collection of rocks represents a number. The computer performs the operation +1.



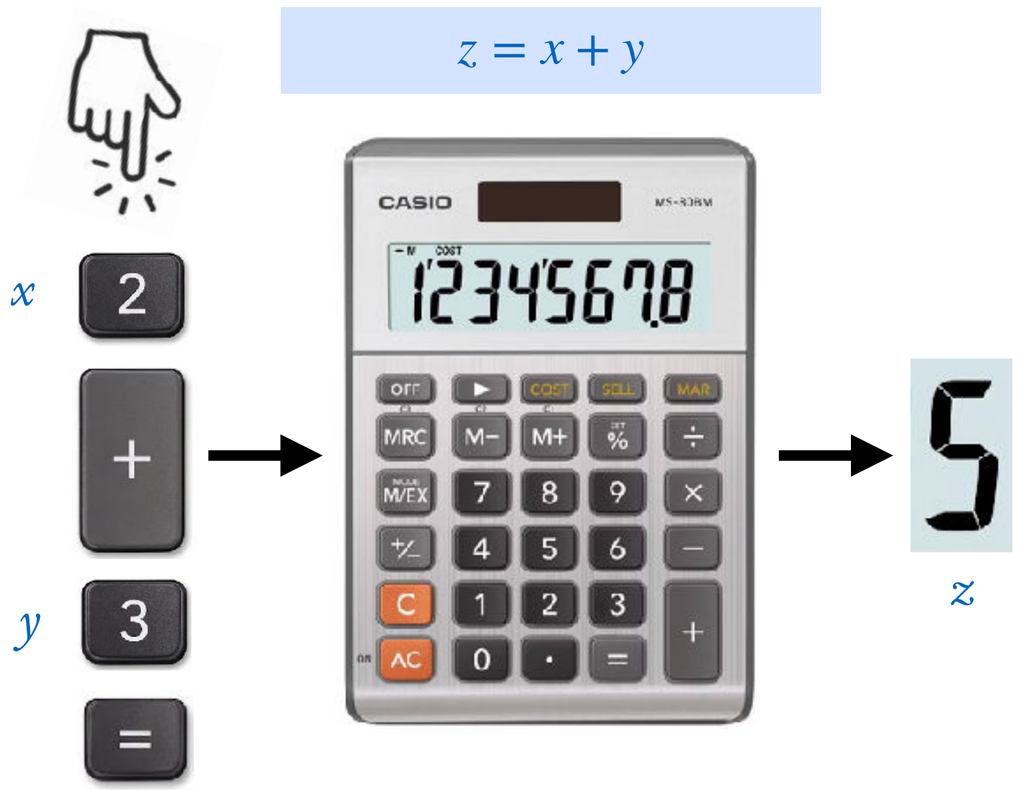
Example: Analog computer

Historical analog computers involved complex networks of gears. The input is entered as the rotation of a gear wheel. The output is the rotation of yet another gear wheel. They could solve polynomial equations.



Example: Digital calculator

The inputs to a digital calculator are button presses. The outputs are displayed numerals on a screen. The inner circuitry of a calculator is very much like the machines we will design in Unit 1.



Computational Theory of Mind (CTM)

Our definition of computation is broad enough that it may potentially include both artificial and naturally evolved information processors. In fact, according to the **computational theory of mind**, the brain itself is a kind of computer.

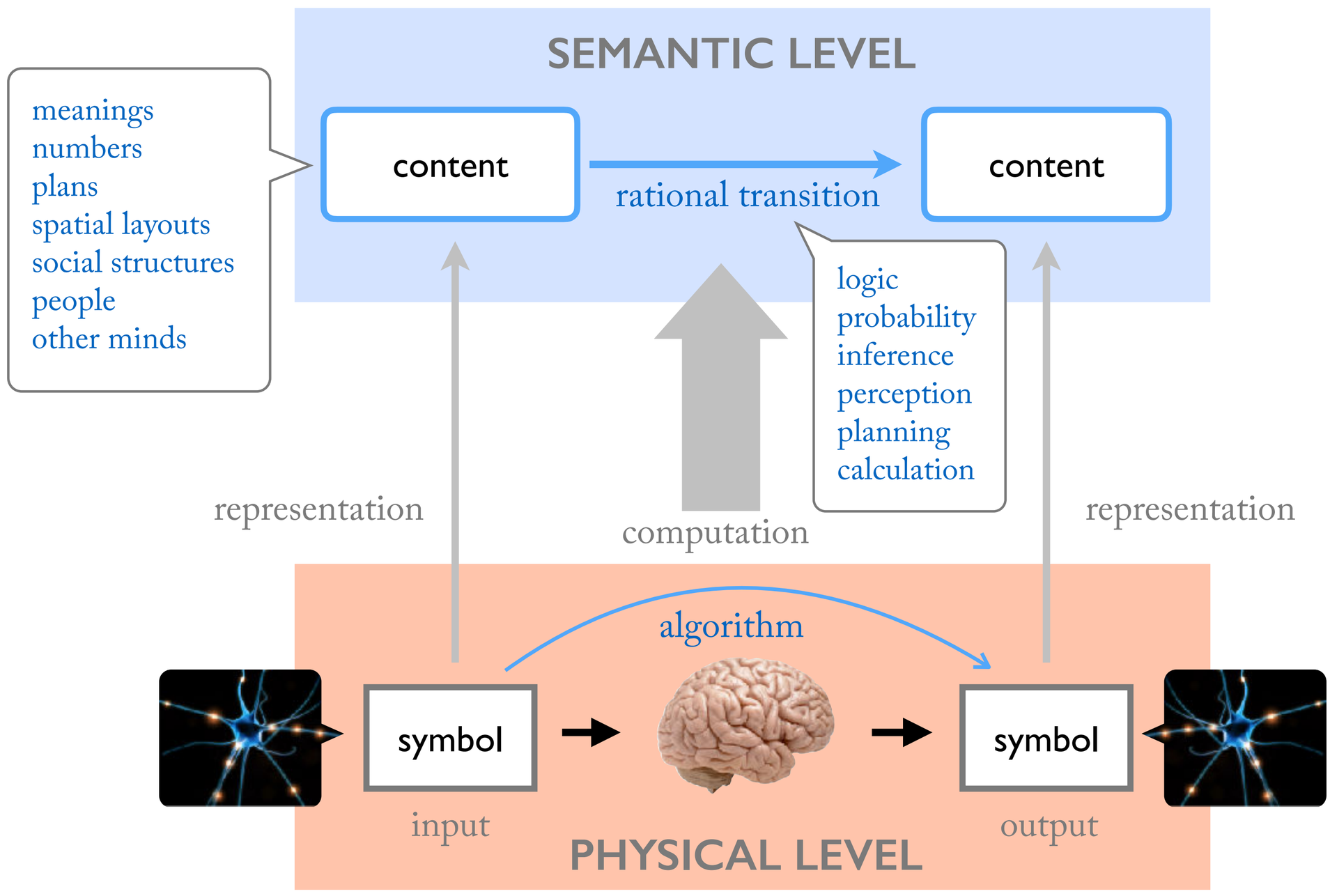
Information processing
The brain has the function of processing information through rational transitions. This is a biological function, not a design function.

Symbol processing
The brain takes neural symbols as inputs, processes neural symbols, and produces neural symbols as outputs. Neural symbols in the brain carry mental contents.

Computational Theory of Mind (v. 1)

1. The brain is a naturally occurring computer or information processor.
2. Cognition arises from the computational activity of the brain.

CTM uses the same computational framework that was traditionally applied only to artificial computers and extends it to the brain. CTM is the foundational hypothesis of contemporary cognitive science.



Computational Theory of Mind: the brain computes cognitive functions by manipulating symbolic representations in the brain according to biologically grounded algorithms.

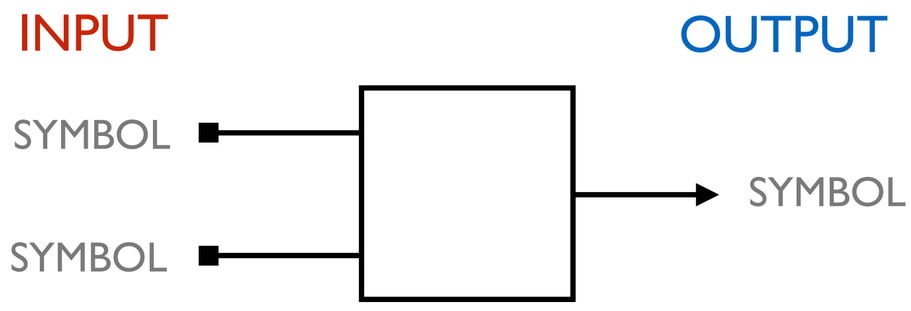
3. Logic Gates & Combinatorial Circuits

Logic Gates

Logic gates are the basic building blocks of most human-made computers. By combining logic gates into complex networks, we can ultimately achieve complete computational power.

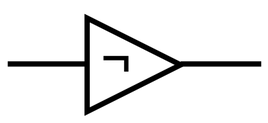
A single logic gate is actually a very, very simple computer. It takes one or two symbols as input (depending on the gate), and produces a single symbol as output.

There are billions of logic gates in a normal desktop processor.



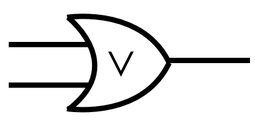
We'll use three basic types of logic gates. Each is defined by its input-output profile:

NOT-GATE



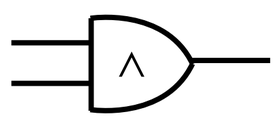
IN	OUT
0	1
1	0

OR-GATE



IN 1	IN 2	OUT
0	0	0
0	1	1
1	0	1
1	1	1

AND-GATE



IN 1	IN 2	OUT
0	0	0
0	1	0
1	0	0
1	1	1

To understand logic gates, don't think of the input and output symbols as actual "1"s and "0"s. They are abstract values that are defined simply as opposites.

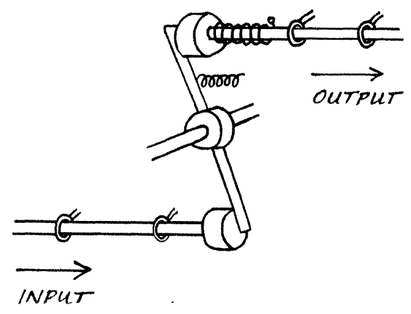
The logic gate itself is a kind of abstract machine which matches inputs with outputs.

IN	OUT
0	1
1	0

IN	OUT
off	on
on	off

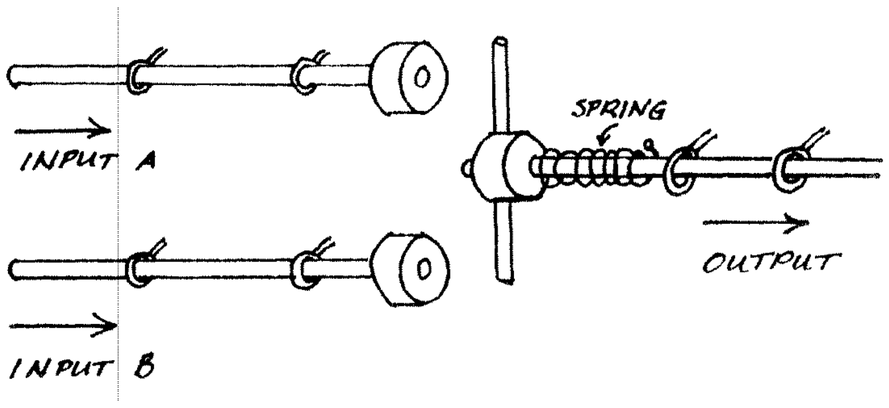
Thought of as abstract machines, logic gates work "instantaneously" and never make mistakes or wear out.

Physical implementations of logic gates take time to process, make mistakes, and are subject to entropy.

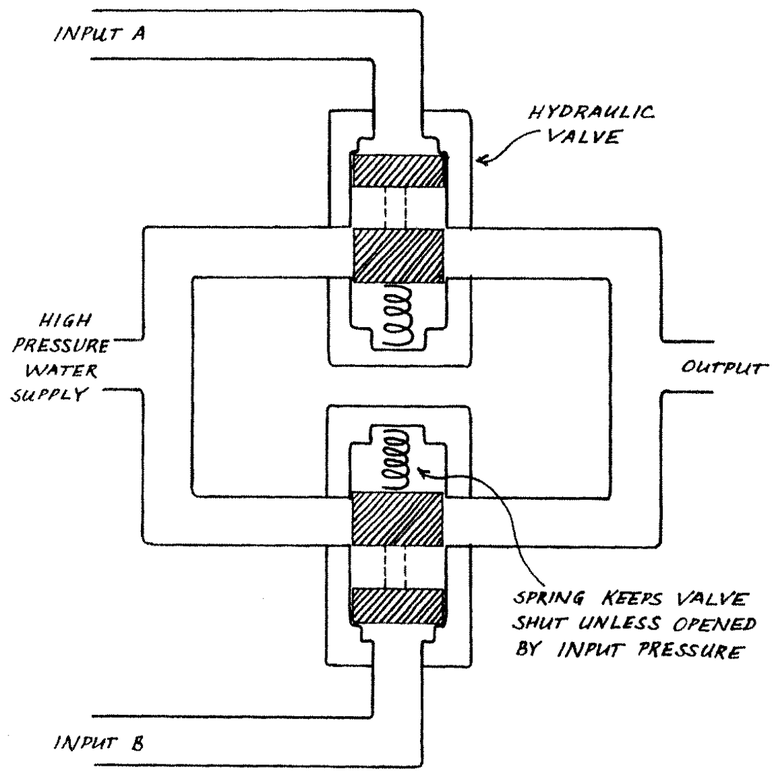


From Physics to Logic

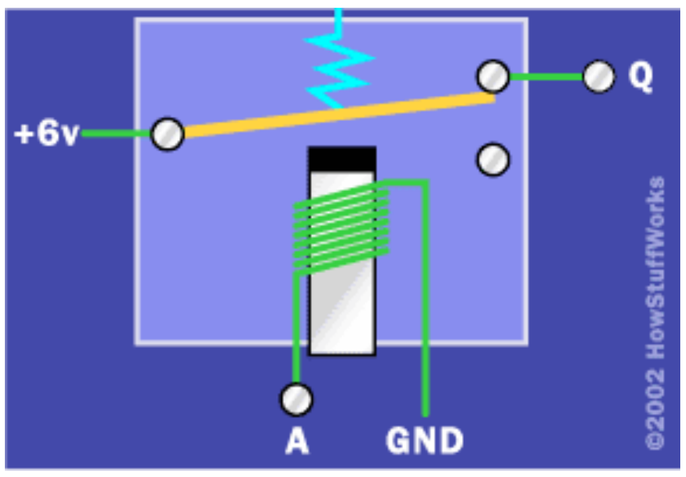
Logic gates can be implemented using any number of different physical designs and physical materials.



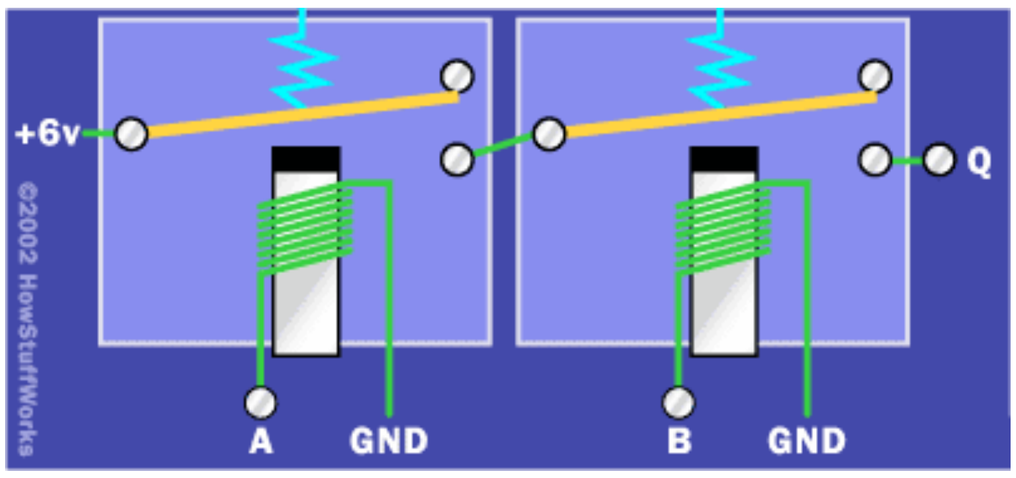
Physical implementation of an OR-gate with a stick and spring mechanism.



Physical implementation of an OR-gate with hydraulic valves.



Physical implementation of a NOT-gate with electromagnets.



Physical implementation of an AND-gate with electromagnets.

Logic gates themselves can be thought of as abstractions from concrete machines.

They abstract away from the internal description of a machine by specifying only inputs and outputs.

And they abstract away from the external description by specifying inputs and outputs only as “0” and “1”.

IN	OUT
0	1
1	0

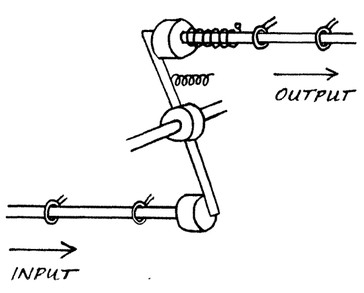
A NOT-gate.

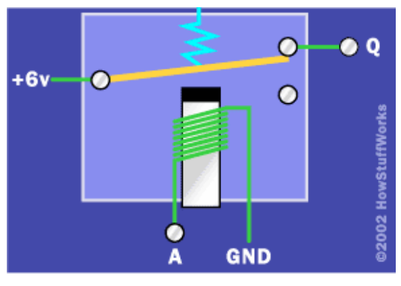
Abstraction over external description.

IN	OUT
→	←
←	→

IN	OUT
⚡	⚡
⚡	⚡

Abstraction over internal description.

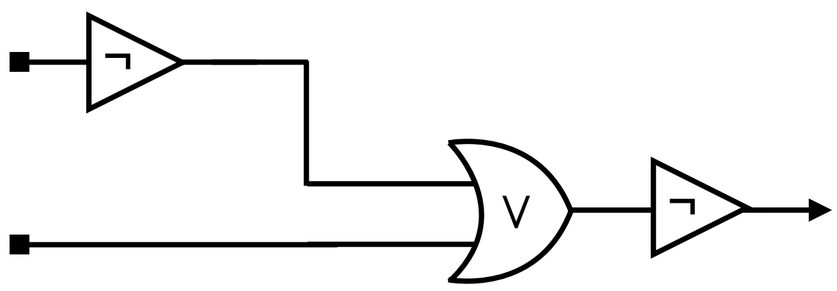




Physical implementations of a NOT-gate.

Combinatorial Circuits: Concept

Logic gates can be combined together into a circuit, called a **combinatorial circuit** or **CC.** Here's an example:



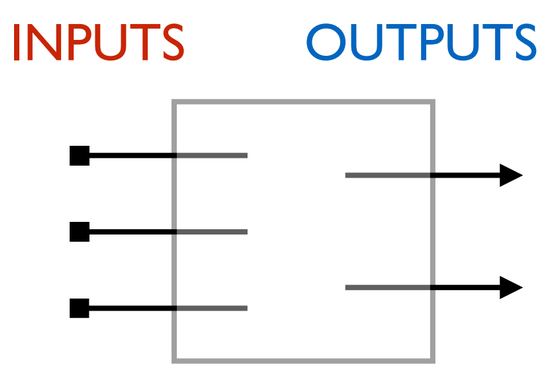
Combinatorial circuits are made up of **logic gates** connected by **links.** The links represent pathways that symbols may flow through.

In a combinatorial circuit, we abstract from time. Symbols pass "instantaneously" from inputs, through the network of links and logic gates, to the final output.

Input/output control. In a factory-made computer, the inputs are determined by the user, while the outputs are determined by the machine (given the input). In a "natural" computer, like the brain of an animal, the inputs are determined by the natural environment, but the outputs are still determined by the machine.

Combinatorial Circuits: Definition

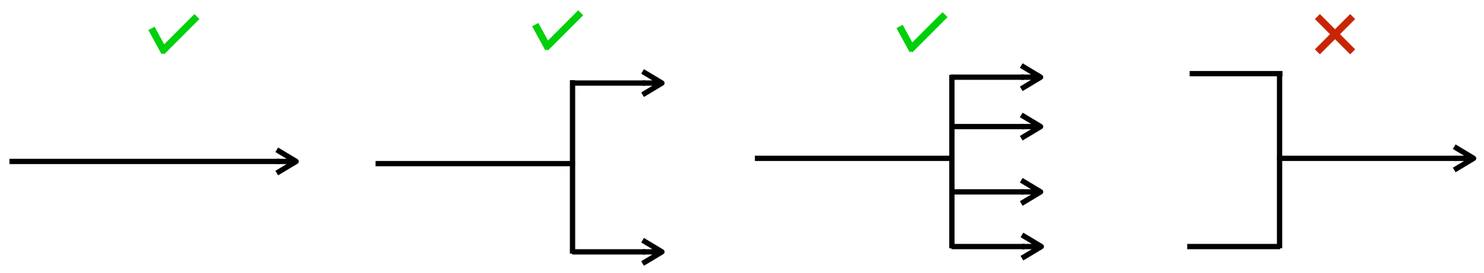
Input and output. A combinatorial circuit can have any number of inputs and any number of outputs.



For clarity, mark links which are connected to the input with small squares. Mark links connected to the output with small arrows.

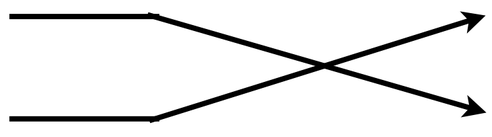
Each input leads either into a logic gate or directly to an output.

Links. Logic gates are connected to one another by **links.** Links may split any number of times. When a link splits, it carries the same symbol to both of its branches. Links may split, but they can't merge, because how would you combine a 0 and a 1?

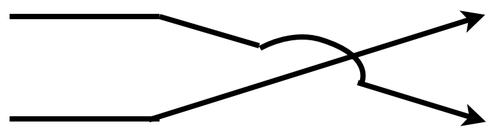


When drawing your circuit diagrams, the link-*lines* may sometimes cross. This is OK, it doesn't say anything about the circuit itself, which is an abstract network of logic gates. Line crossing is just a side effect of drawing in two dimensions.

So the following diagram is fine:



But for visual clarity, introduce a "hop":

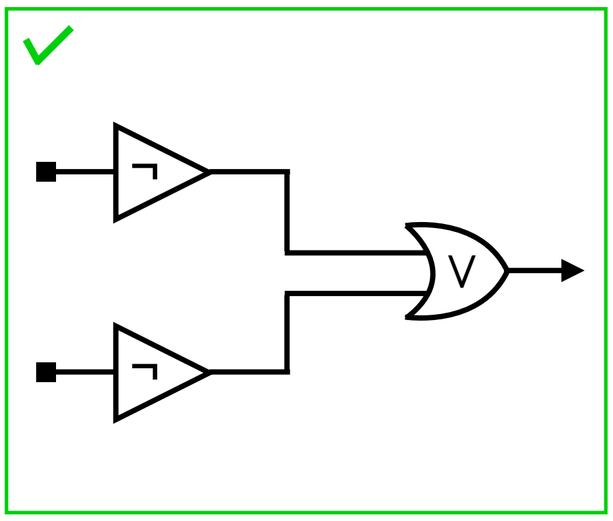


CC Definition. A combinatorial circuit can now be officially defined as follows:

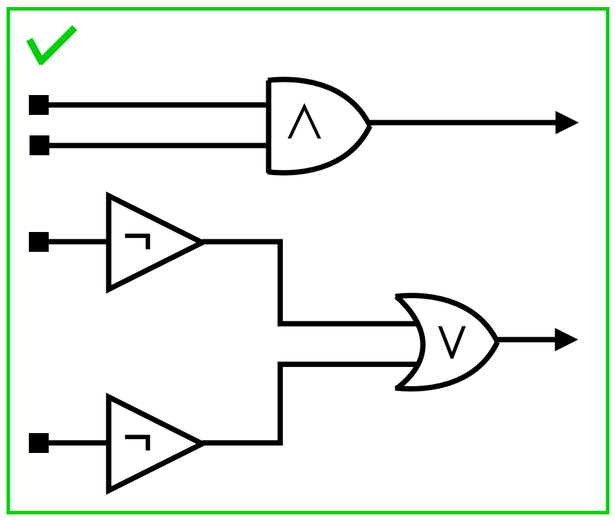
A **combinatorial circuit** is any network of logic gates and links where:

- there are no loops;
- there are no merged links;
- every free end is either an input or output.

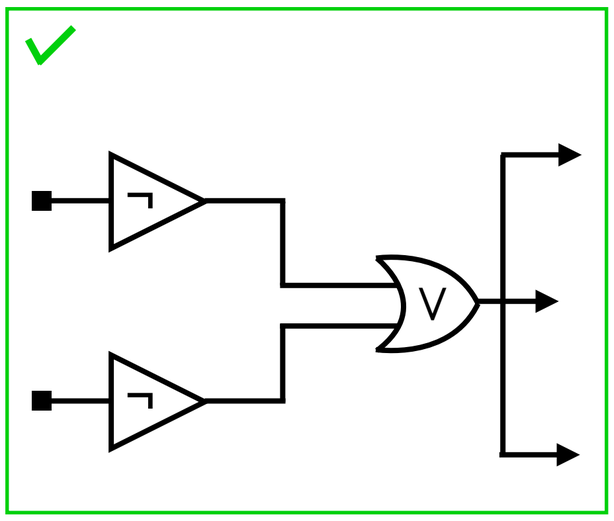
A standard case.



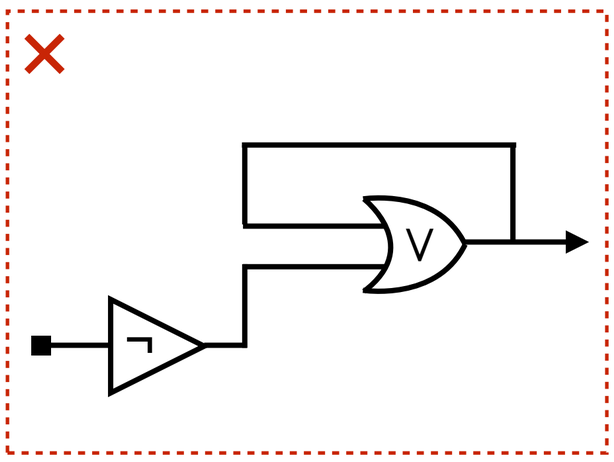
There may be disconnected parts.



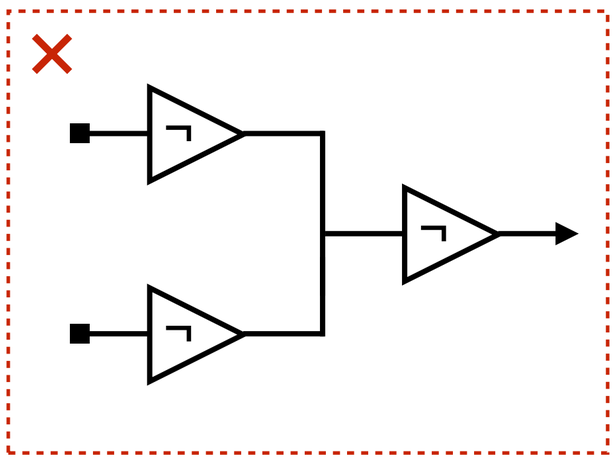
There may be more outputs than inputs.



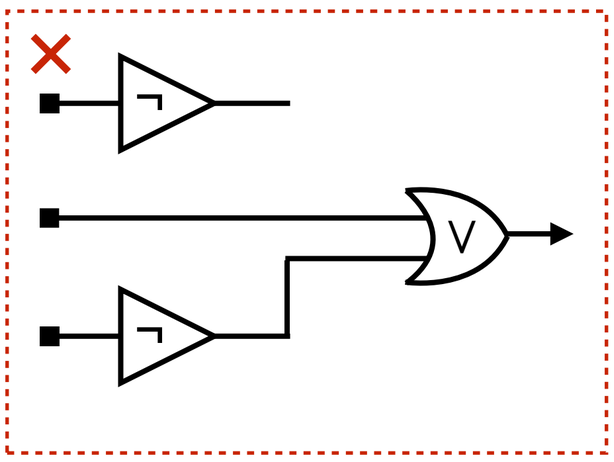
No loops allowed.



No merged links allowed.

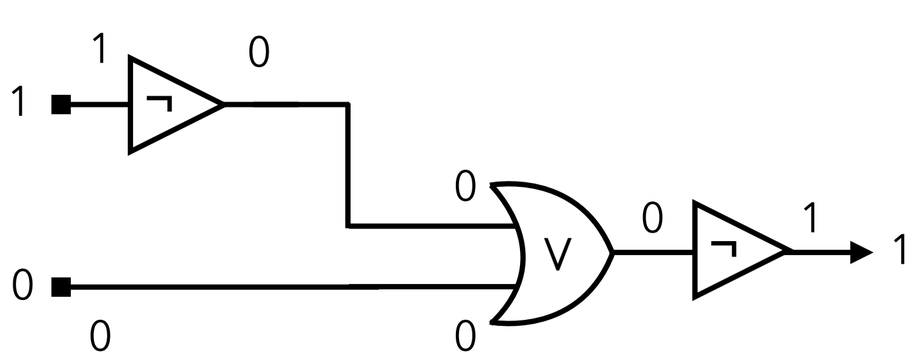


No free ends (except in/out).

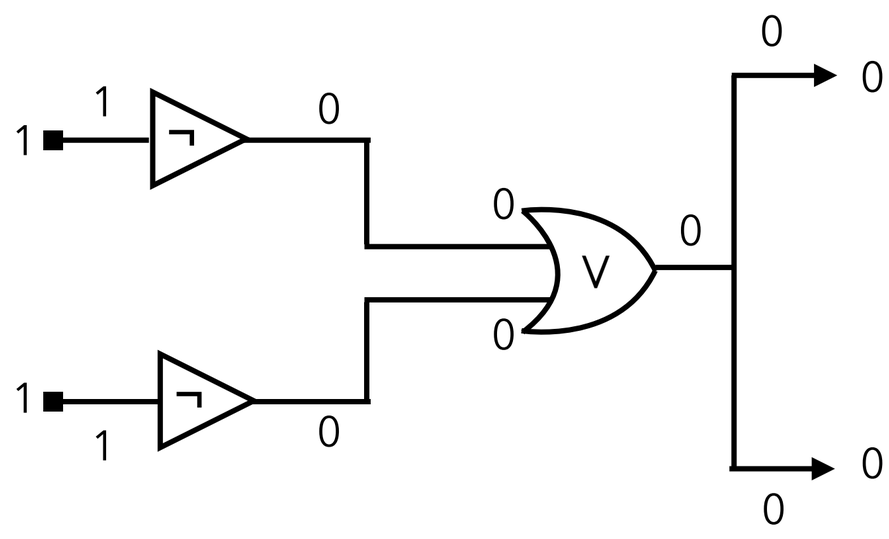


Combinatorial Circuits: Flow of Information

Here are two sample circuits and their input-output tables. Each circuit is labeled with one possible set of inputs, as well as their values as they propagate through the circuit.



IN 1	IN 2	OUT
1	0	1
1	1	0



IN 1	IN 2	OUT 1	OUT 2
1	0	1	1
1	1	0	0

In-Class Problems

Construct a combinatorial circuit for each input-output table:

1. Not

IN 1	OUT
0	1
1	0

2. Identity

IN 1	OUT
0	0
1	1

3. And

IN 1	IN 2	OUT
0	0	0
0	1	0
1	0	0
1	1	1

4. Nand

IN 1	IN 2	OUT
0	0	1
0	1	1
1	0	1
1	1	0

5. Or

IN 1	IN 2	OUT
0	0	0
0	1	1
1	0	1
1	1	1

6. Nor

IN 1	IN 2	OUT
0	0	1
0	1	0
1	0	0
1	1	0

7. All-1

IN 1	OUT
0	1
1	1

8. All-0

IN 1	OUT
0	0
1	0

9. Split

IN 1	IN 2	OUT 1	OUT 2
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

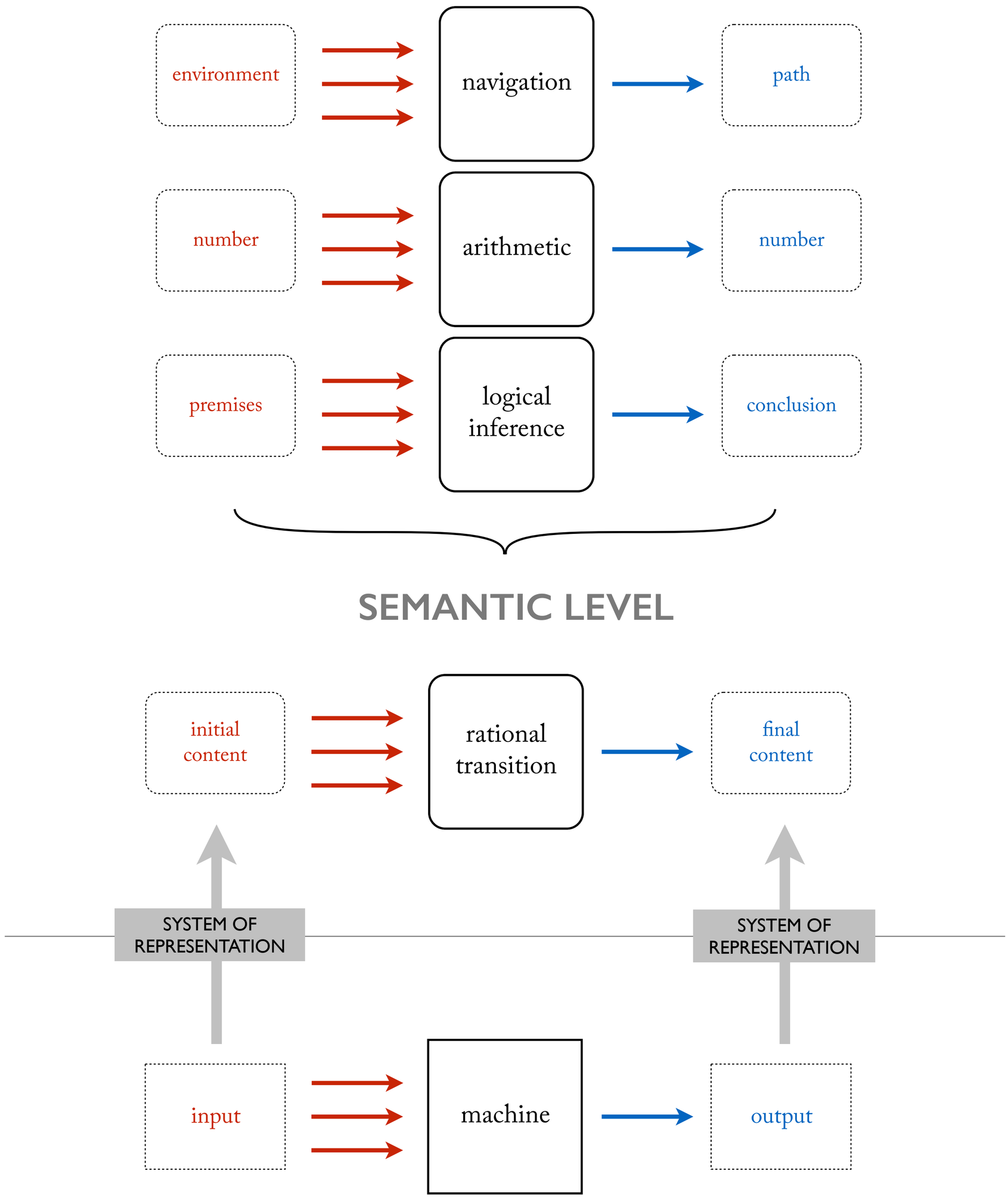
4. Functions

The Structure of Computation

So far we've described computational information processing in terms of rational transitions between informational content. Now we'll use the mathematical concept of a function to formalize this idea.

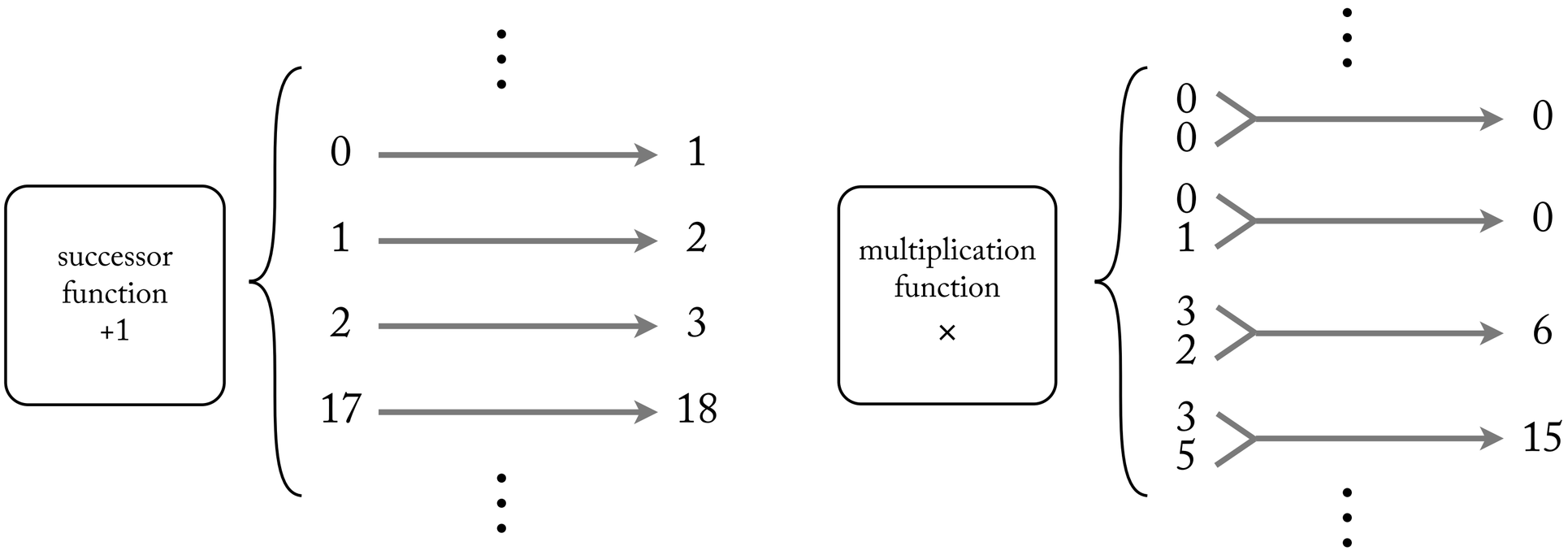
Officially, we'll say that a computer **computes a function** when it performs a given rational transition.

Rational transitions as computable functions…



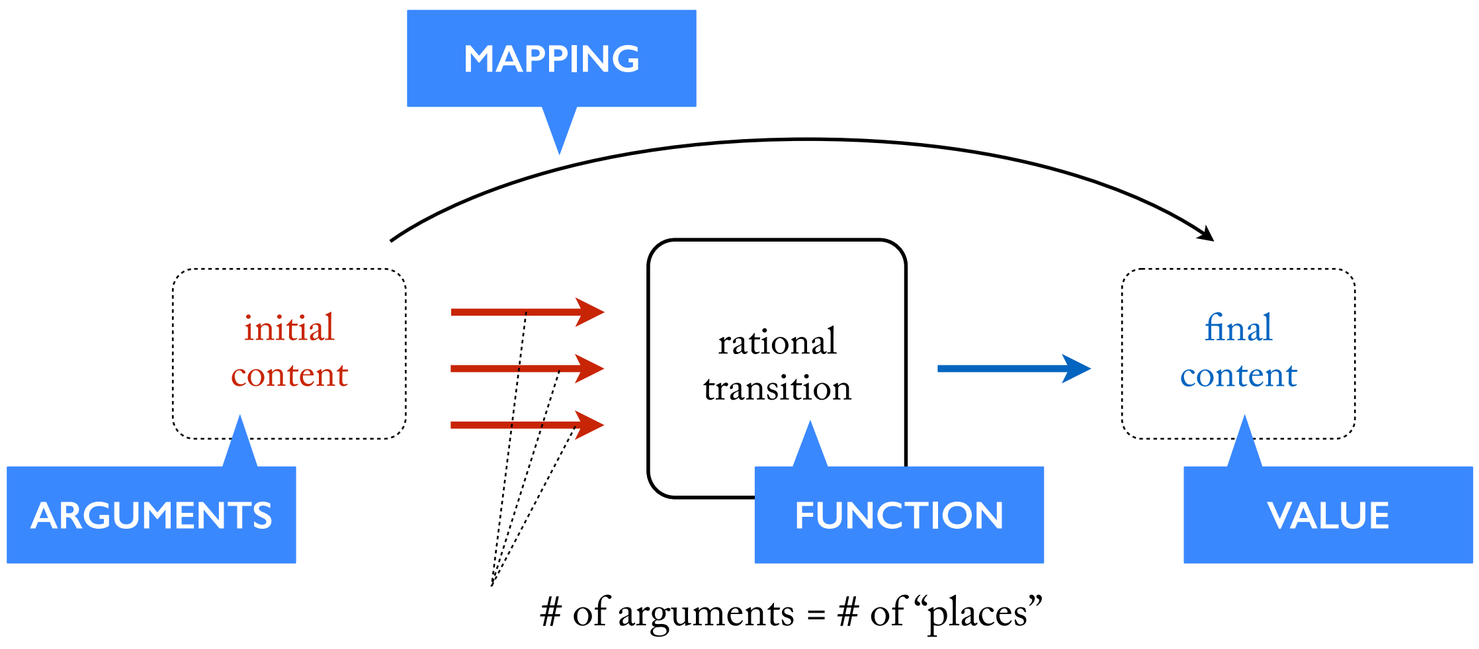
Numerical Functions

Familiar functions correspond to relations between numbers. Examples include *successor*, *addition*, *multiplication*, and so on.



If you think of numbers as abstract objects in Plato's heaven, then functions are abstract and infinite relations *between* those numbers.

The Structure of Functions



Functions are **deterministic**. Given the same arguments, they always return the same value.

Functions can have **many** arguments, but only **one** value.

Multiple arguments go in different **argument places**. What is a function for which argument place matters?

the addition function:

$$3 + 4 = 7$$

argument 1 argument 2 value

Functions: Terminology

How to **talk** about functions…

The function **takes X** (as an argument) and **returns Y** (as value).

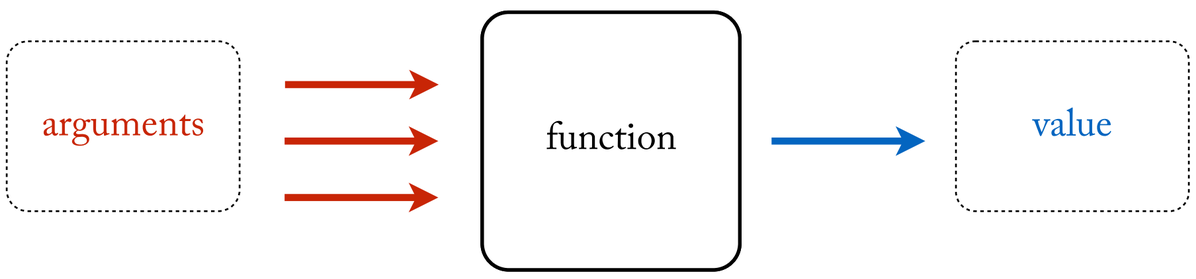
The function **applied to X, returns Y**.

The function **maps X to Y.**

Functions: Notation

How to **write** about functions…

Prefix notation: $f(x, y) = z$



Infix notation: $x \, f \, y = z$

Prefix notation puts the function symbol on the left side.

"add( )" means the addition function
"add(x, y)" means the addition function applied to arguments x and y

For example:
"add(3,4)" means the addition function applied to the numbers three and four
which is... the number seven

Which is why it is true to say: add(3,4) = 7

Infix notation (Grade School Notation) puts the function symbol in the middle.

"+" means the addition function
"x+y" means the addition function applied to arguments x and y

For example:
"3+4" means the addition function applied to the numbers three and four
which is... the number seven

Which is why it is true to say: 3+4 = 7

Functions: Composition

Functions can be combined together. This is called the **composition** of functions. Feel free to mix the two styles of notation together as you find convenient.

$(3 + 4) \times 2 = 14$

multiply(add(3,4), 2) = 14

add(3,4) × 2 = 14

These statements are all equivalent, and appropriate use of notation.

Defining a Function

The value of a function may be **undefined** for some arguments.

divide(3,0) = undefined *square.root*(-4) = undefined

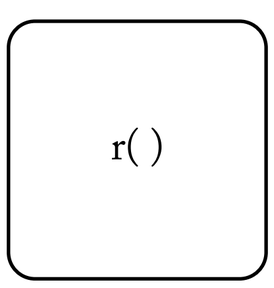
Most functions are defined for **infinitely many** arguments. Some functions are defined only for **finitely many** arguments. Call these **finite functions**.

To define an **infinite function** you typically have to **use an equation.**

Example 1
Let s( ) be the 1-place function which satisfies the following condition:
For any number x: $s(x) = x + 1$

Example 2
Let q( ) be the 2-place function which satisfies the following condition:
For any numbers x, y: $q(x,y) = x + (y \times 2)$

To define a **finite function** you typically have to **use a table.** (See below for an exception.) By convention, the function is undefined for the unlisted argument pairs.



=

argument 1	argument 2	value
0	0	15
0	1	3
1	0	0
1	1	1
1	2	2

The Identity of Functions

A function is defined by its arguments and its values *only*.
It doesn't matter *how* these are specified.

Example 1

Let f( ) be the function:
for any x: $f(x) = x^2$

Let g( ) be the function:
for any x: $g(x) = x \times x$

It follows that f( ) = g( ).

Example 2

Let h( ) be the function:

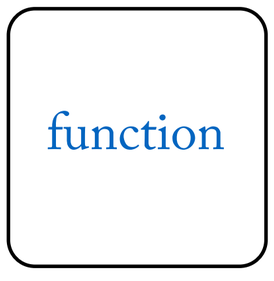
argument	value
1	2
2	3
3	4

Let j( ) be the function:
for any x:
$j(x) = x + 1$ if $1 \leq x \leq 3$:
$j(x)$ = undefined otherwise

It follows that h( ) = j( )

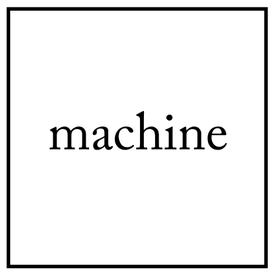
Functions vs Machines

In this class it's critical to keep your concept of **function** and your concept of **machine** clearly separated. (Even if, in some of the readings, authors sometimes run them together!)



A function is…

- An abstract entity, which often relates numbers.
- Has no temporal or spatial location.
- Up to infinite in size.
- "Instantaneously" relates one number to another.
- Requires no "processing" to do this.



A machine is…

- A concrete device, which processes symbols.
- Has a temporal and spatial location.
- Only finite in size.
- Takes time to take an input and produce an output.
- Requires processing to do this.

The Metaphysics of Functions

Mathematical functions are abstract infinite relations between numbers. There is nothing more to a function than the numbers it relates.

You can think of a mathematical function as an infinite "list" of numbers.

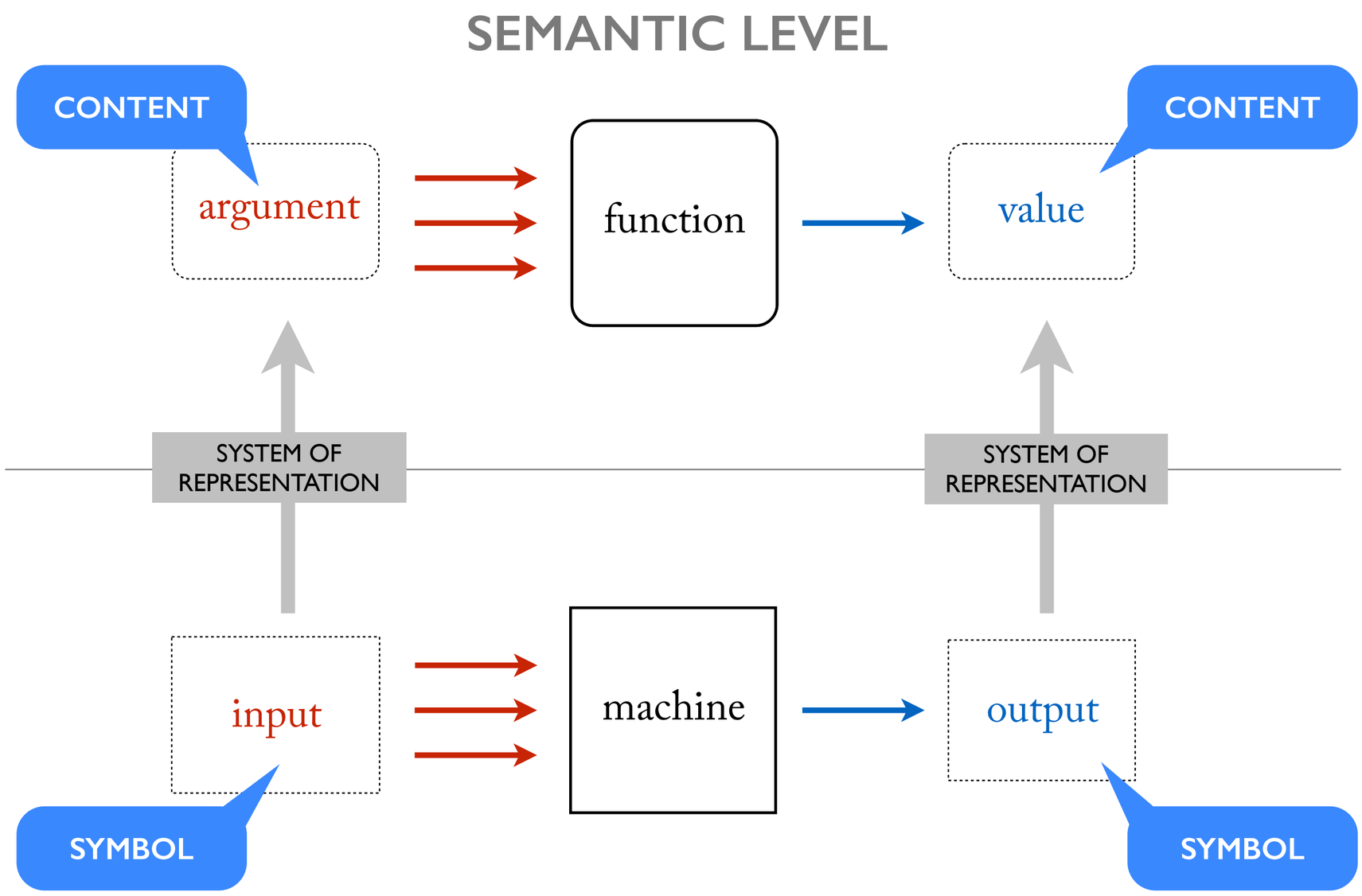
Other kinds of functions can relate other kinds of objects (sets, people, places, etc.). But the functions themselves are always abstract relations.

argument 1	argument 2	value
0	0	0
0	1	0
1	0	0
1	1	1
1	2	2
2	0	0
2	1	2
2	2	4
3	0	0

5. Systems of Representation

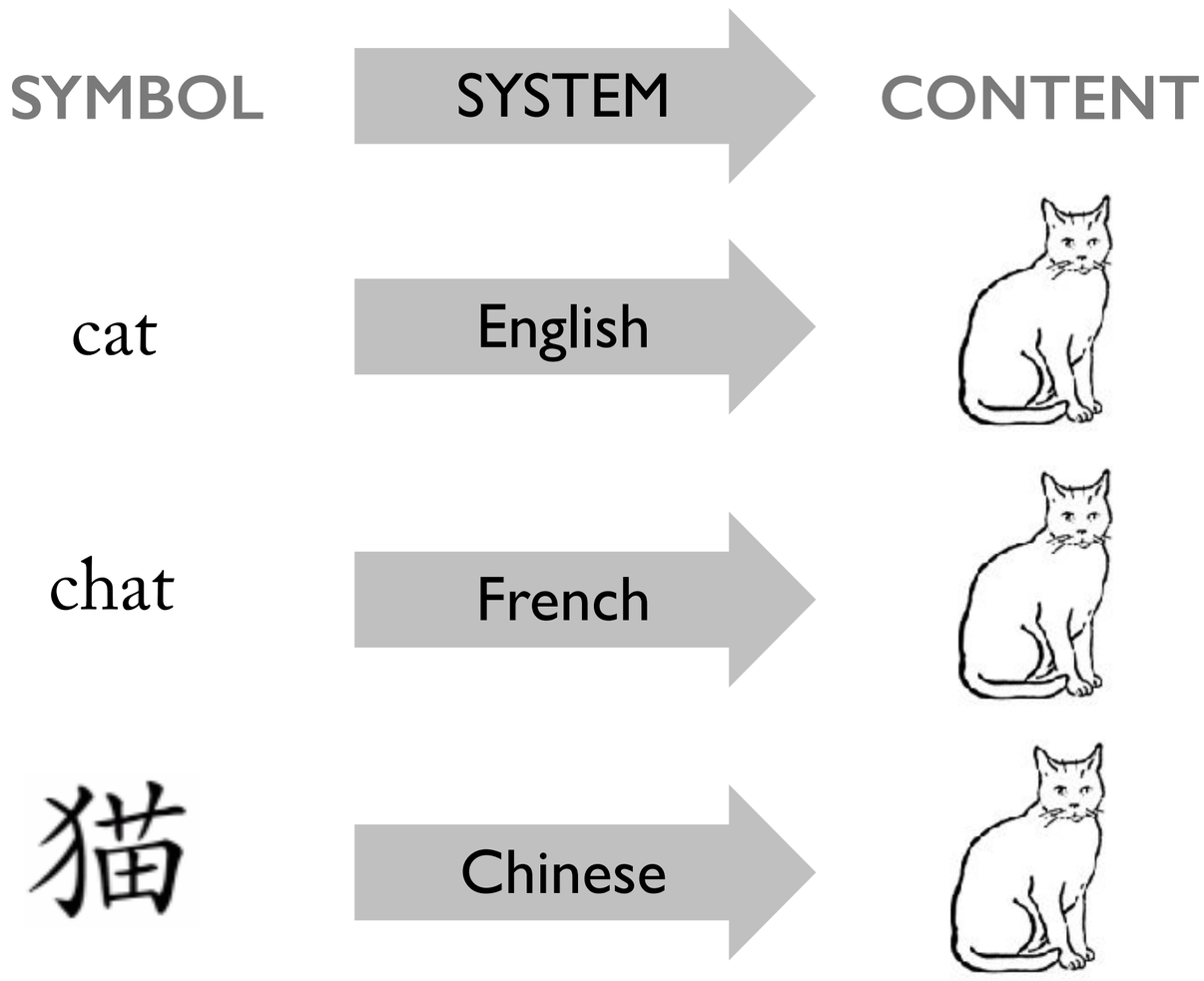
The Role of Representational Systems in Computation

Systems of representation associate symbols with contents. In computation, they map inputs to arguments and outputs to values. They are the glue that connects the physical and semantic levels.



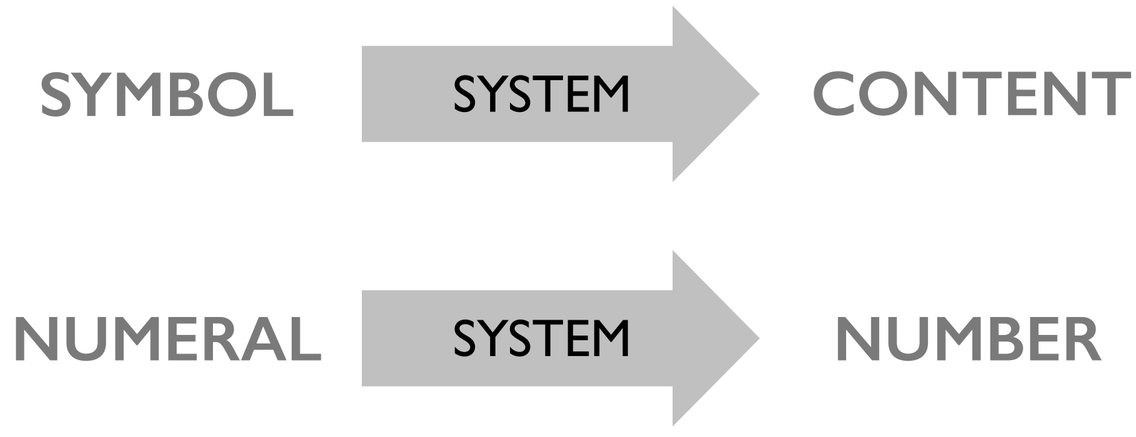
Symbol, Content, System

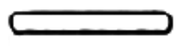
Symbols express contents only relative to a system. The same content can be expressed by different symbols in different systems. Here is an illustration for the case of words in a language.

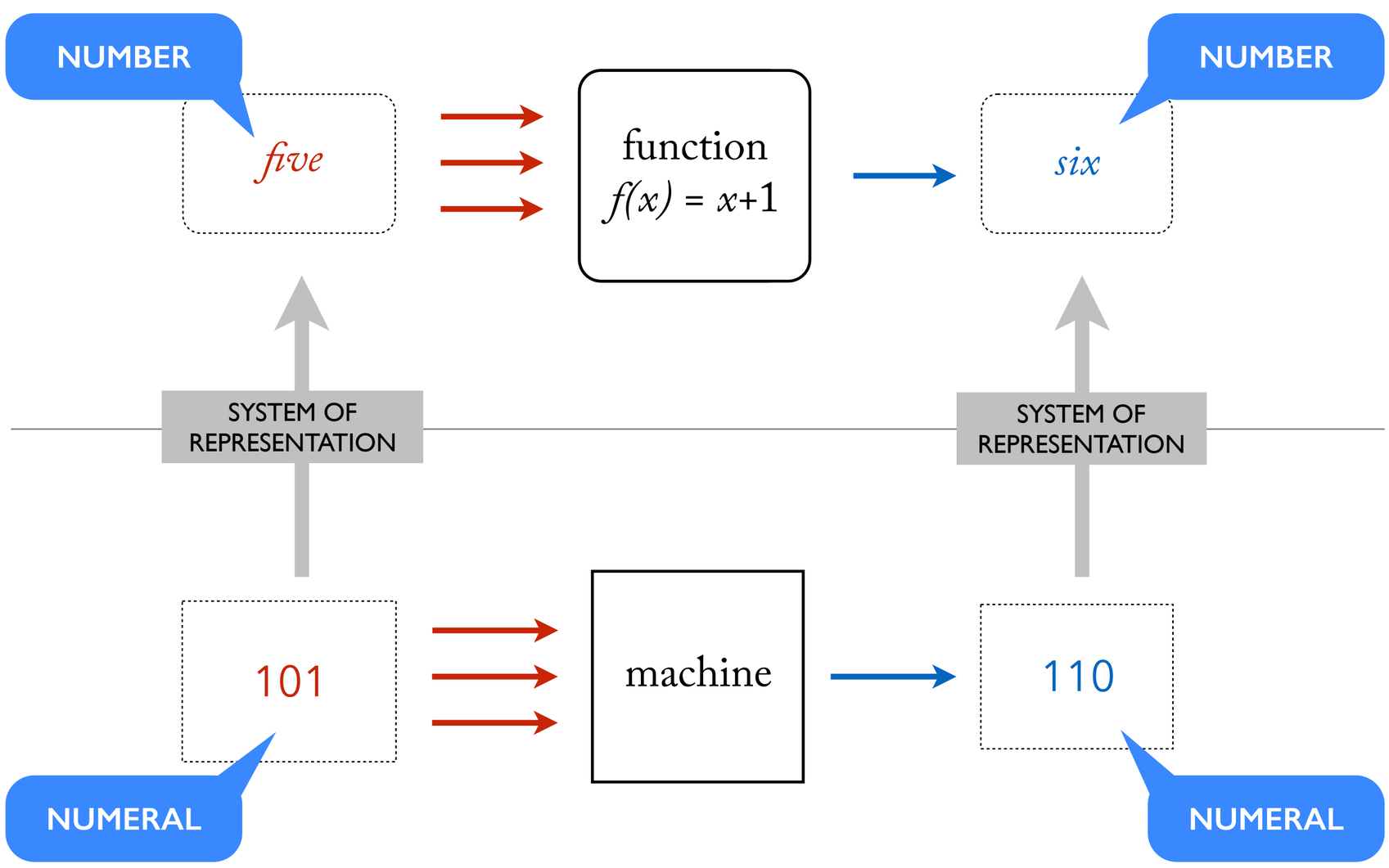


Numeral Systems

Numeral systems are systems of representation specifically for numbers. In these systems, the symbols are called **numerals**, and the contents are **numbers**. Numerals are types of physical symbols, while numbers are abstract quantities.



Different numeral systems use…	Roman numerals	Indo-Arabic numerals	Mayan numerals	The numbers represented
		0		*zero*
	I	1		*one*
	II	2		*two*
	III	3		*three*
	IV	4		*four*
	V	5		*five*



Anatomy of a Numeral

A **complex** numeral is made up of a string of **atomic** numerals. Each atomic numeral has a **position** within the complex numeral.

In the **Indo-Arabic numeral** system we all use, the atomic numerals are the numerals 0–9, and all other numerals are constructed from these.

3rd position	2nd position	1st position	
3	8	4	complex numeral
	atomic numeral	atomic numeral	

Atomic symbols in the **Indo-Arabic** system

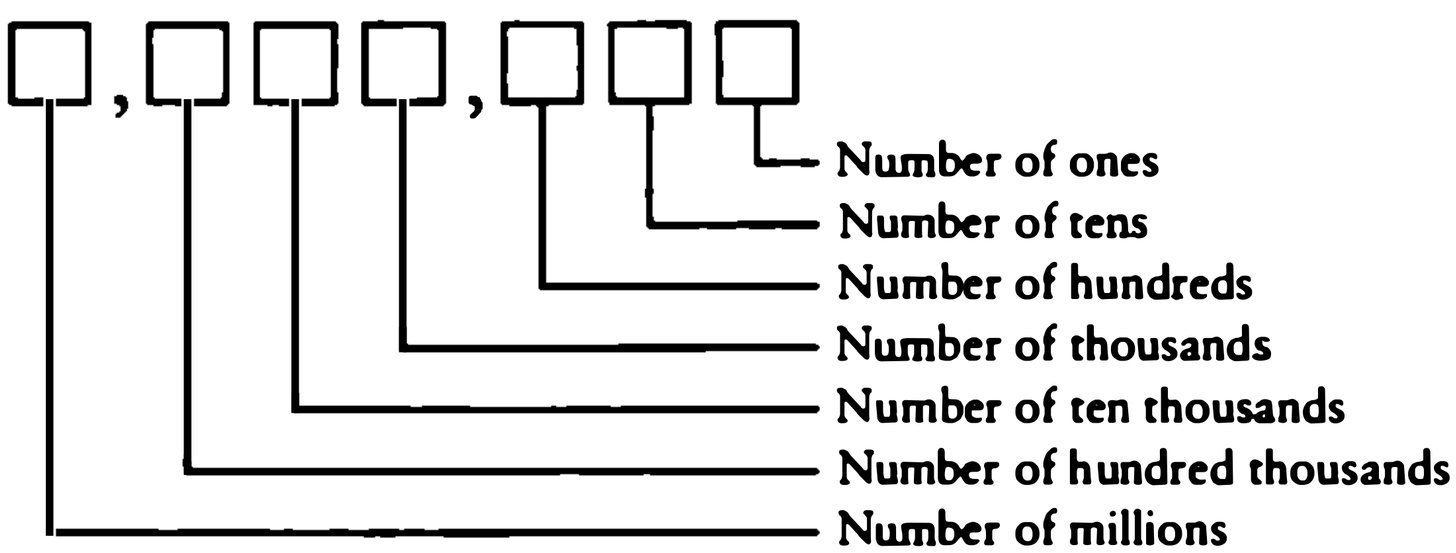
0 1 2 3 4 5 6 7 8 9

Mechanics of a Numeral System

A **positional** numeral system is defined by

(i) the **numeric values** it assigns to each **atomic numeral** (starting from zero);
(ii) the **numerical weight** of the **position rank** (starting from zero),
where the number of symbols in (i) = the weight in (ii).

In the Indo-Arabic system, the **weight** of the position rank is 10. That means that the weight of each position goes up by a power of 10. (The first position is considered position 0, so its weight is $10^0=1$.)



The Indo-Arabic numeral system is positional, but the Roman system is not. Both employ complex symbols to express large numbers, but they rely on different forms of representational system.

one hundred and one	101	CI
	Indo-Arabic numerals	Roman numerals

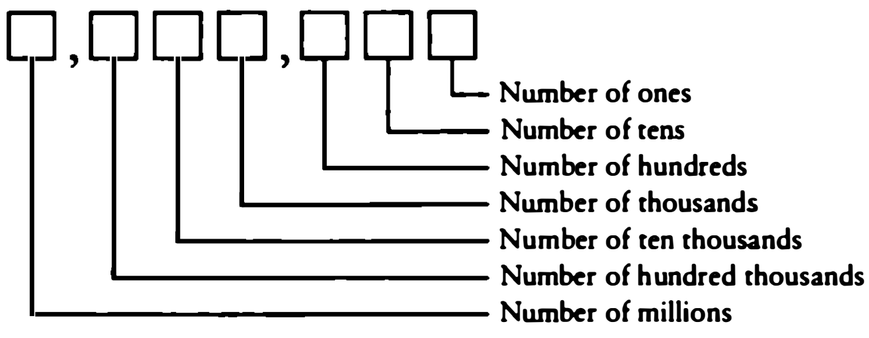
Note that the numeral 1 appears twice in the Indo-Arabic numeral, because it plays different roles in the different positions. In the Roman numeral, which is **additive** rather than positional, the numeral I only appears once, added to the value of C.

Base

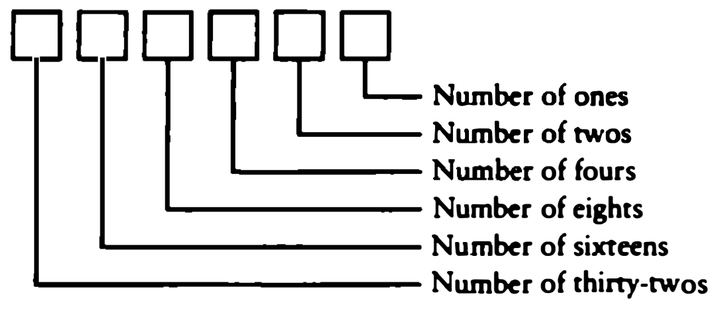
The **base** of a positional system corresponds to the number of atomic symbols. In the Indo-Arabic system, there are ten symbols, 0–9, so this system is called **base 10**. In the Binary system, there are only two symbols (0,1), so this system is **base 2**.

The base determines the weight of the positional rank. So in base 10, positions increase in weight by powers of 10, while in base 2, they increase by powers of 2.

Base 10



Base 2 (Binary)



Base 10	Base 7	Base 2 (Binary)	Base 1 (Tally)
1	1	1	1
2	2	10	11
3	3	11	111
4	4	100	1111
5	5	101	11111
6	6	110	111111
7	10	111	1111111
8	...	1000	11111111
9	15	1001	111111111
10	16	1010	1111111111
...	20	1011	11111111111
98	...	1100	111111111111
99	66	1101	1111111111111
100	100	1110	11111111111111

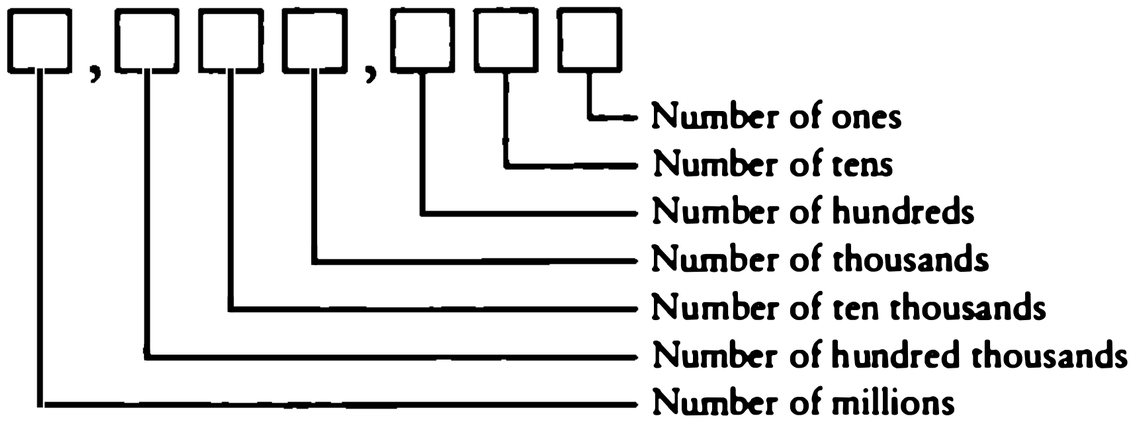
As you can see, Tally has the virtue of being very simple, but the vice of producing very large numerals— inefficient for creatures made of finite physical components.

Binary is of special significance in computer design because, like Tally, it only involves two possible digits, 1 and 0. But unlike Tally, the size of numerals increases non-linearly (~ logarithmically) as the number represented gets bigger. Today, binary is the representation system most used at the mechanical foundations of computing.

Calculating a Numeral's Value

Decimal System

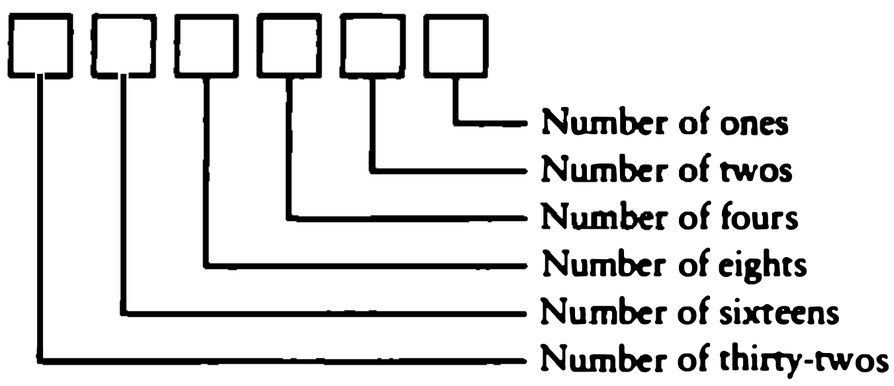
ten atomic symbols, *base* = 10



$$
\begin{aligned}
111_{D} &= 1_{D} \times 100 + \\
&\quad\ 1_{D} \times 10 + \\
&\quad\ 1_{D} \times 1 \\
&= 1 \times 100 + \\
&\quad\ 1 \times 10 + \\
&\quad\ 1 \times 1 \\
&= 100 + 10 + 1 \\
&= 111
\end{aligned}
$$

Binary System

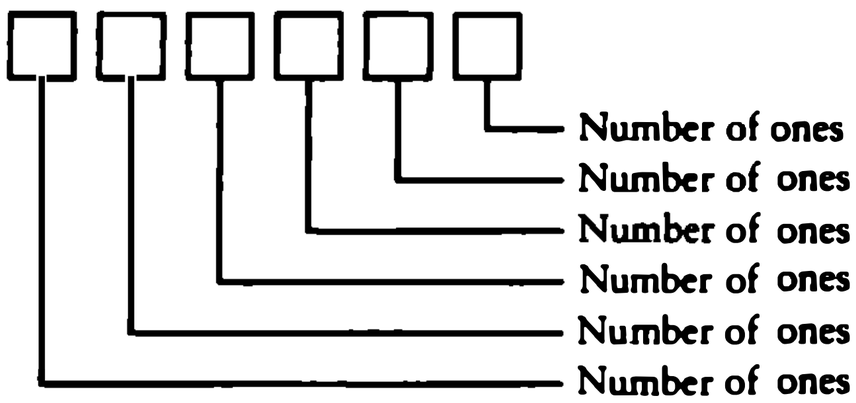
two atomic symbols, *base = 2*



$$
\begin{aligned}
111_{B} &= 1_{B} \times 4 + \\
&\quad\ 1_{B} \times 2 + \\
&\quad\ 1_{B} \times 1 \\
&= 1 \times 4 + \\
&\quad\ 1 \times 2 + \\
&\quad\ 1 \times 1 \\
&= 4 + 2 + 1 \\
&= 7
\end{aligned}
$$

Tally System

one atomic symbol, *base = 1*

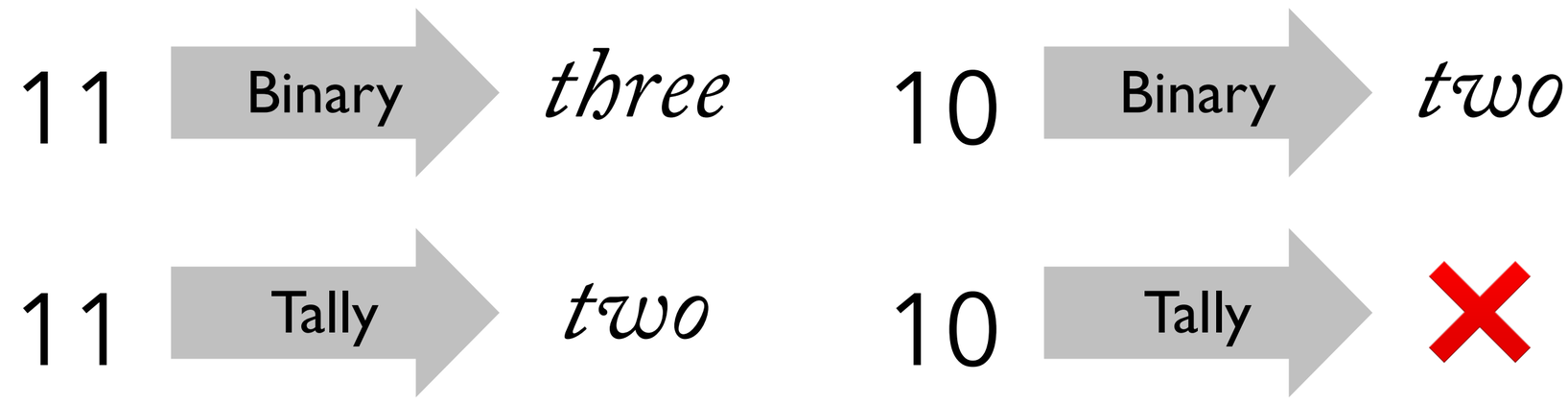


$$
\begin{aligned}
111_{T} &= 1_{T} \times 1 + \\
&\quad\ 1_{T} \times 1 + \\
&\quad\ 1_{T} \times 1 \\
&= 1 \times 1 + \\
&\quad\ 1 \times 1 + \\
&\quad\ 1 \times 1 \\
&= 1 + 1 + 1 \\
&= 3
\end{aligned}
$$

Binary and Tally as Representational Systems

In this course, the most important numeral systems will be **tally (T)** and **binary (B).** This is because you can express all numbers in these systems using only two symbols, 0 and 1.

Because these are two different ways of interpreting numerals, the same numerals can mean different things in the two systems.



The **syntax** of a representational system is a set of rules for constructing well-formed complex numerals in that system. When designing a machine to compute a function in tally or binary, we'll always assume that inputs are well formed.

Tally Syntax

Tally systems in common usage take a variety of forms.



> Here, a **well-formed tally numeral** is any string of bits, where the first n (≥ 0) bits from the right are 1's, and they are followed to the left by any number of 0's.

any number of 0's — any number of 1's

0 . . . 0 1 . . . 1

So the following are well formed:

000
111
0011

But these ones are not.

1000
1101
1110

Binary Syntax

> A **well-formed binary numeral** is any string of bits, where the first n (≥ 0) bits from the right are either 0's or 1's, and they are followed to the left by any number of 0's.

any number of 0's — any number of 1's and 0's

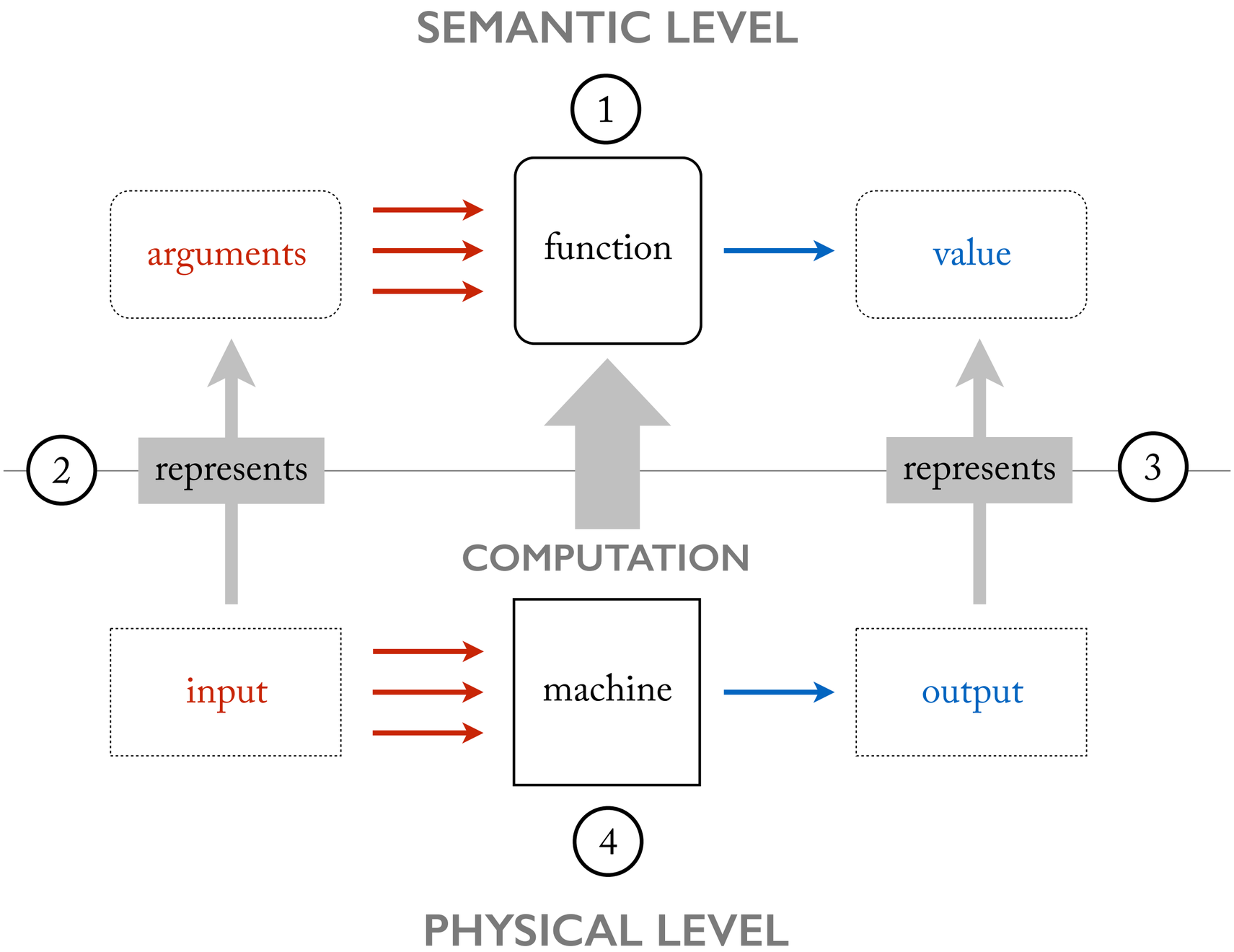
0 . . . 0 1|0 . . . 1|0

In binary, any sequence of 1's and 0's will be well formed.

6. Defining Computation

The Structure of Computation

Computation is a relation between a **machine** and a **function** that is brought about when the machine is designed to physically **mirror** or **simulate** the abstract argue-value profile of the function. In this section, we will define this relationship formally.



Definition of Computation

Machine M **computes** function *f* under system R if and only if:

For every possible argument *x* for function *f*:

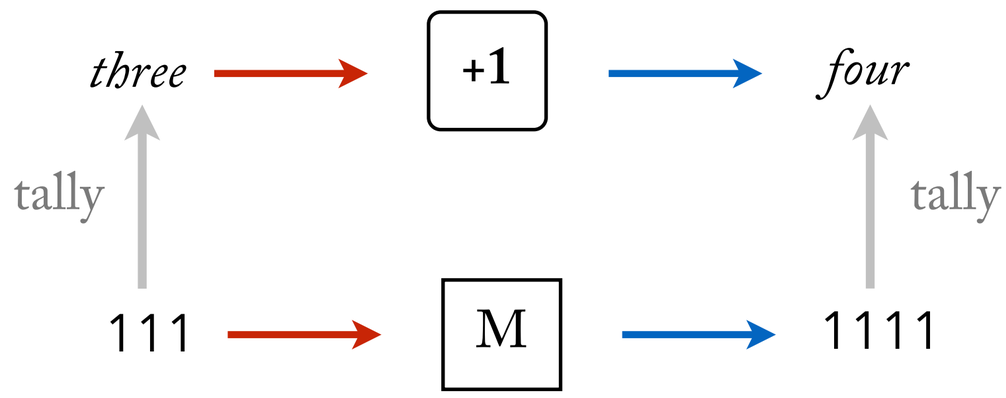
(1) there is a value *y* such that *f(x)=y* ;

(2) there is a symbol string S_1 such that S_1 represents *x* under R;

(3) there is a symbol string S_2 such that S_2 represents *y* under R;

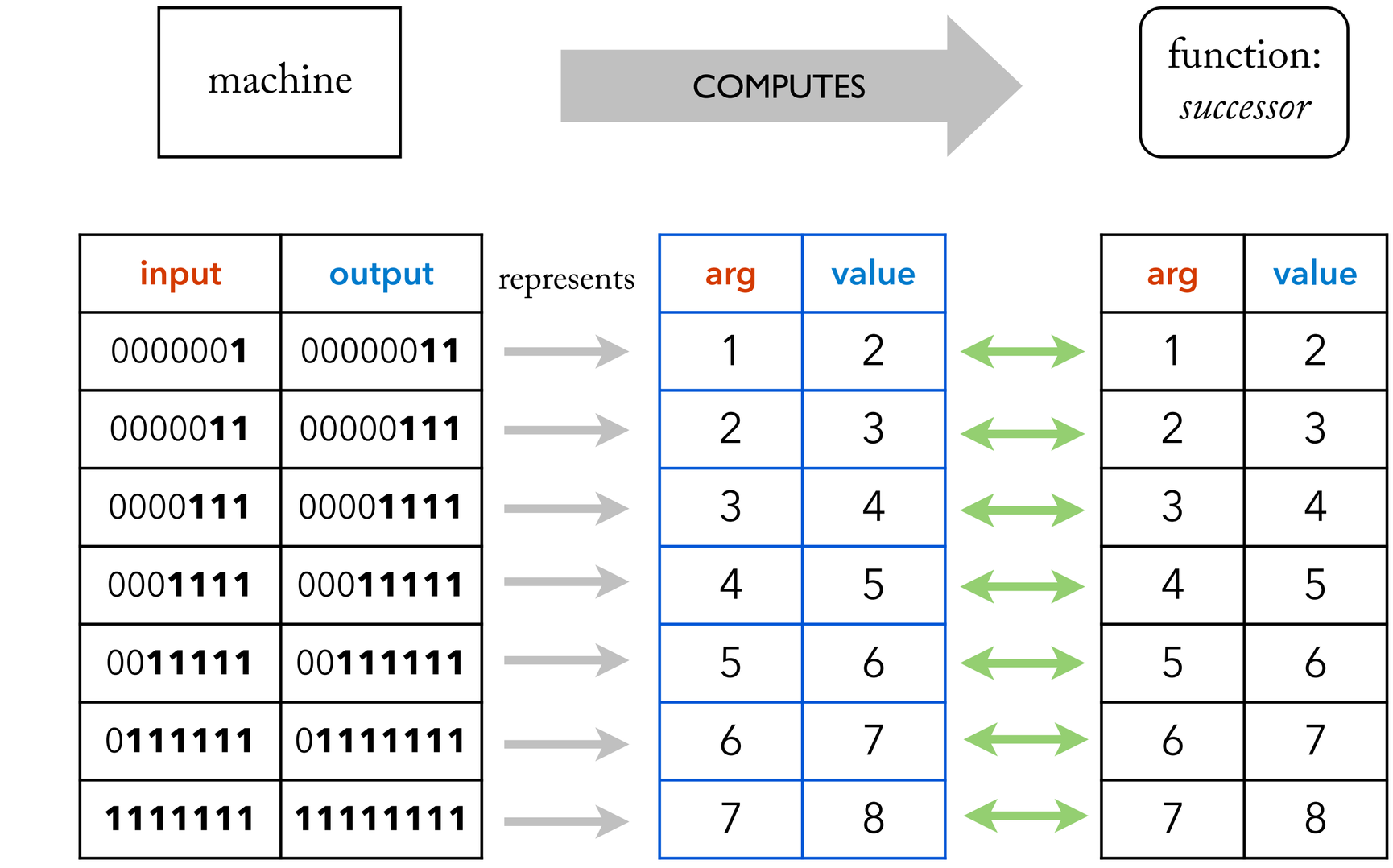
(4) if M takes S_1 as input, then M produces S_2 as output.

For at least one argument, the machine at right computes successor (+1) under the tally representation system.

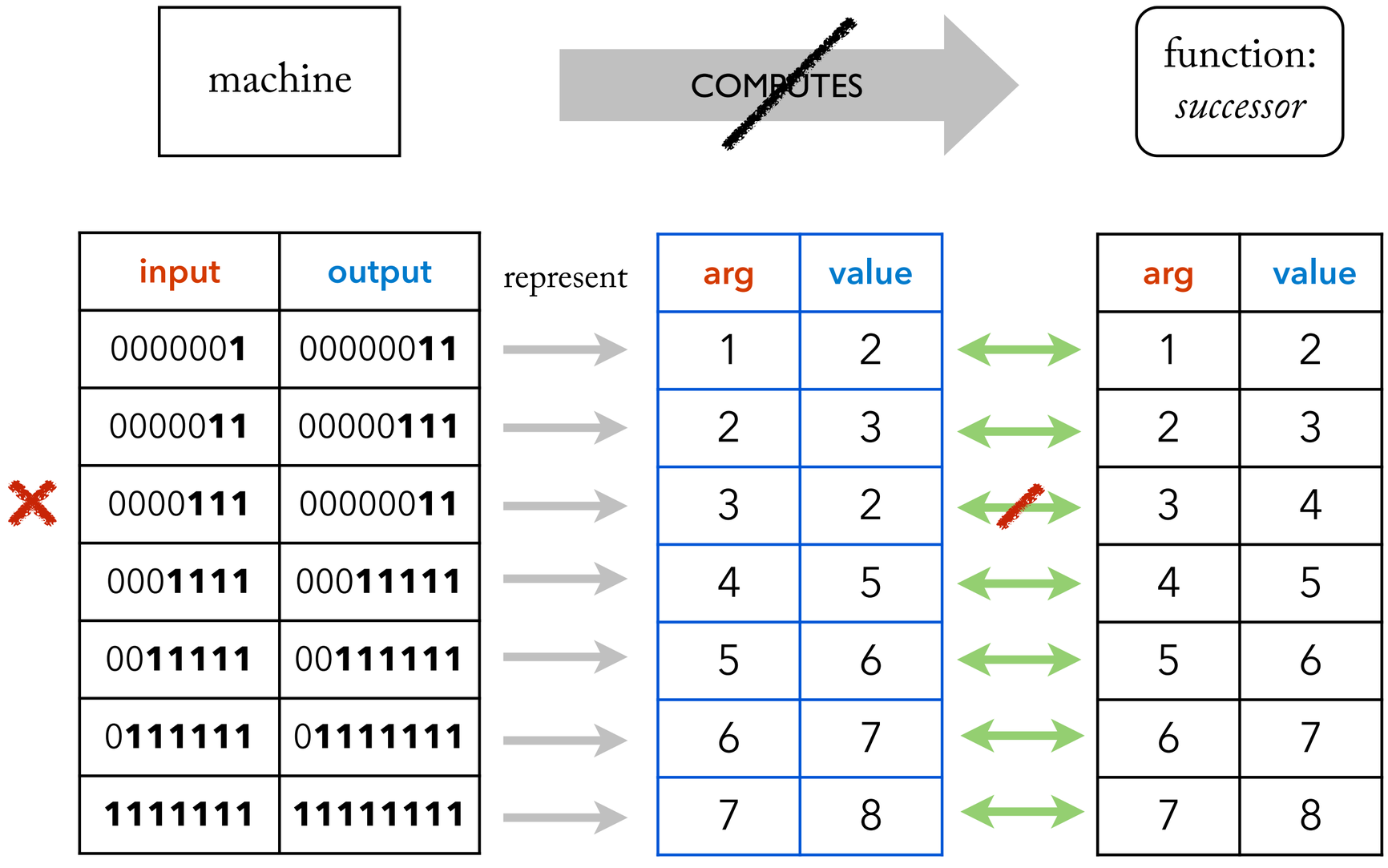


Examples of Computation

The following machine successfully computes the successor function, under the tally representation system. The machine's input-output profile, when interpreted in tally, perfectly matches the argument-value profile of the successor function (so far).



The following machine does *not* computes the successor function under the tally representation system. The machine's input-output profile, when interpreted in tally, does *not* perfectly match the argument-value profile of the successor function.



Bounded Functions

A **bounded** function is a function which is defined for only a finite range of arguments. To name a bounded function, we write:

f[0-*n*] e.g. "*successor*[0-3]"

Note: this names a function that is only defined for the *arguments* 0 through *n* (*not* the values). The same notation applies whether the function is 1-place, 2-place, or more. For example the following two tables are equivalent to the named function in each case.

successor[0-2]

arg	val
0	1
1	2
2	3

addition[0-1]

arg1	arg2	val
0	0	0
0	1	1
1	0	1
1	1	2

A machine **computes a bounded function** when it computes that function's values for all *defined* arguments. The values it computes for undefined arguments are irrelevant for now.

Notation for Functionally Describing Computers

We will often want to refer compactly to a computer in terms of what function it computes, and what system of representation it computes it under. This is actually a deep and important form of **functional abstraction**, as we'll see in a few weeks.

I will say that a machine computes

f[0-*n*] S

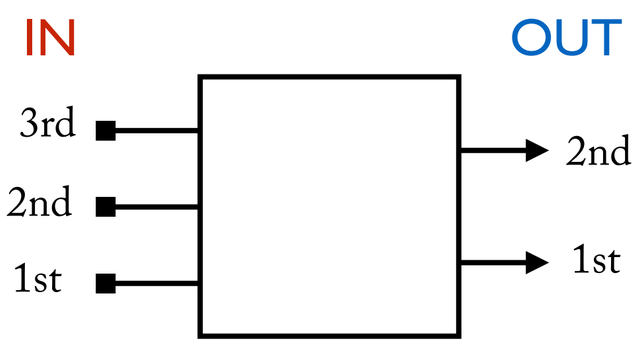
where "*f*[0-*n*]" is a (bounded) function, and S is a representational system. Here I will normally use **T** to stand for tally, and **B** to stand for binary. So I will often ask you to solve problems such as:

Design a machine which computes +1[0-9] B.
Design a machine which computes 2(*x*+3) T.

I'll sometimes omit variables in the definition of the function, if context makes their role clear, as in the first machine described above.

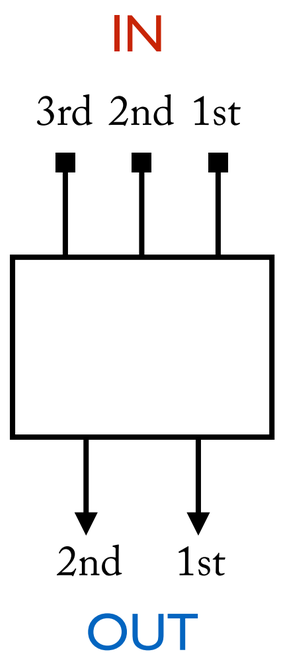
Inputs and Outputs

Input and output. A CC can have any number of inputs and any number of outputs. All the inputs are read at the same time (**synchronically**) and all the outputs are produced at the same time. By convention, the inputs and outputs are ordered from top-to-bottom, or from right-to left (following Arabic).

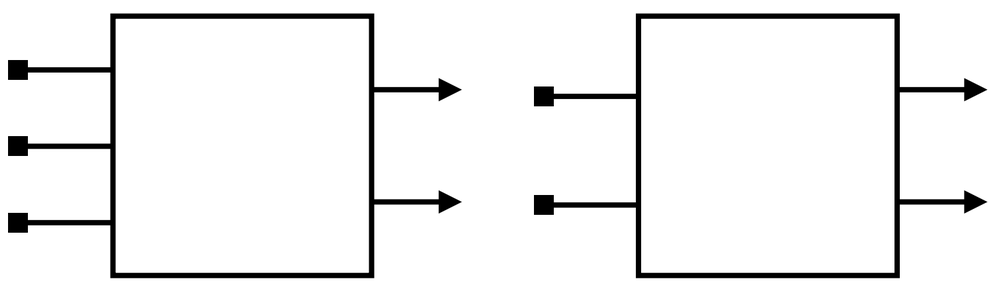


The positions here correspond to positions in a complex numeral:

1 1 0
3rd 2nd 1st



Local input. The **local** inputs to a circuit correspond to the number of input-wires the machine has. Each local input receives one symbol at a given time. A single symbol is called an **atomic** symbol.

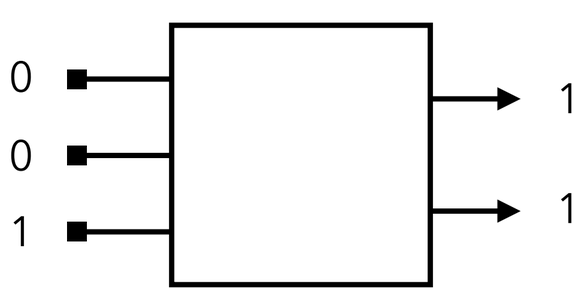


3 local inputs

2 local inputs

Global input. The **global** input to the machine is a collection of its local inputs. Where each local input carries an atomic symbol, a global input is the aggregate or **string** of atomic symbols. This string itself is called a **complex** symbol. The **order of local inputs** within a global input corresponds to the **order of symbols** in the input string.

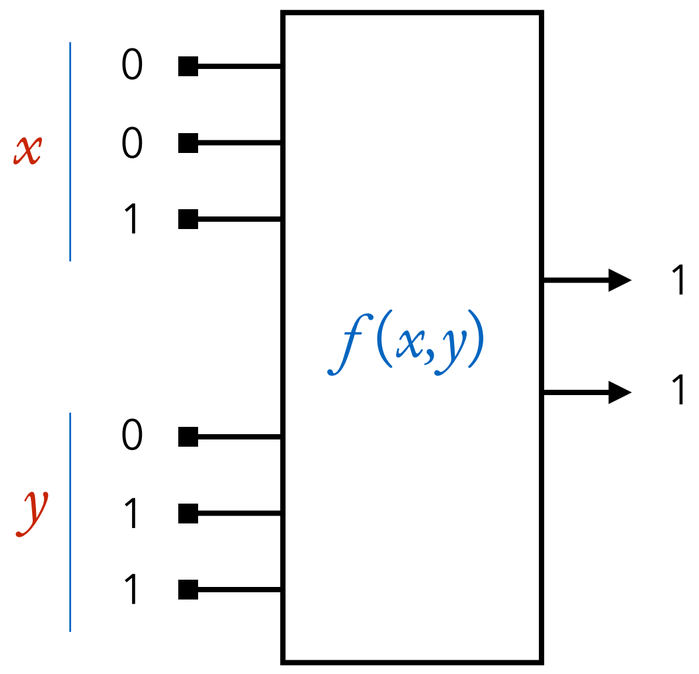
In a CC, the global input is read at one time, by reading each local input synchronically. Later we'll look at circuits where the global input is read over time, **diachronically**.



In the example on the left:
The local inputs are "1", "0", and "0".
The global input is the string "001".

Multiple global inputs. The number of global inputs corresponds to the number of arguments in the function being computed. Each global input represents one argument.

When designing a machine that computes a function with multiple argument places, indicate clearly which global input goes with which argument. The ordering convention again is top-to-bottom, or right-to-left.



Computation as a Three-Part Relation

Computation is a relation between a **machine** M, a **system of representation** R, and a **function** *f*.

Given any two, you can solve for the third.
M + R = *f*?
M + *f* = R?
f + R = M?

The machine and representational system together determined the function computed. But they are themselves logically independent from one another. This means that…

The same M can compute different functions *f* and *f** relative to different systems of representation R and R*.

The same function *f* can be computed by different machines M and M* relative to different systems of representation R and R*.

This demonstrates the importance of representational system for designing a computer. One must get the circuitry of the machine right, but it must also be tuned to the right representational system.

In-Class Problems

1. Suppose M1 computes x+1[0-4] T
    a. How many local inputs and outputs must M1 have?
    How many global inputs and outputs does M1 have?
    b. Make a global input-output table for M1. (Restrict your table to possible inputs and outputs.)
    c. Design a circuit for M1.
2. M2 computes x+1[0-5] T. Design a circuit.
3. M3 computes x+1[0-1] B. M3 has 1 local input and 2 local outputs.
    Make a global input-output table. Design a circuit.
4. M4 computes x+1[0-2] B. M3 has 2 local inputs and 2 local outputs.
    Make a global input-output table. Design a circuit.
5. M5 computes x+1[0-3] B. Make a global input-output table. Design a circuit.

Tip: for all these problems, turn your circuit 90° so the inputs and outputs are vertically aligned, the same way you would set up a long addition problem.

7. Functional Abstraction

Describing a Machine by Its Function

A **functional abstraction** is a description of a machine in terms of its function, rather than its internal mechanism. Functional abstraction is common. We say that a given machine keeps food cold (without saying how it accomplishes this); or we say that a given machine is for driving (without saying if it is gas or electric).

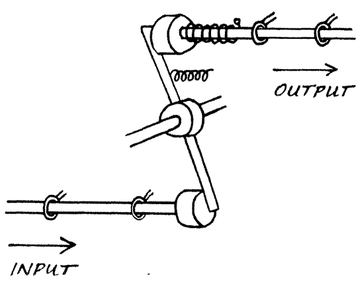
Functional abstraction is also one of the basic analytical tools of cognitive science. We have already encountered on form of functional abstraction.

Logic gates themselves can be thought of as functional abstractions from concrete machines.

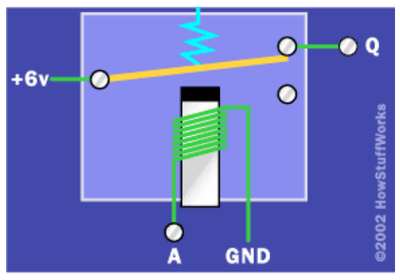
IN	OUT
0	1
1	0

A NOT-gate.

We describe a gate in terms of input-output profile, which is its function, rather than its internal machinery.



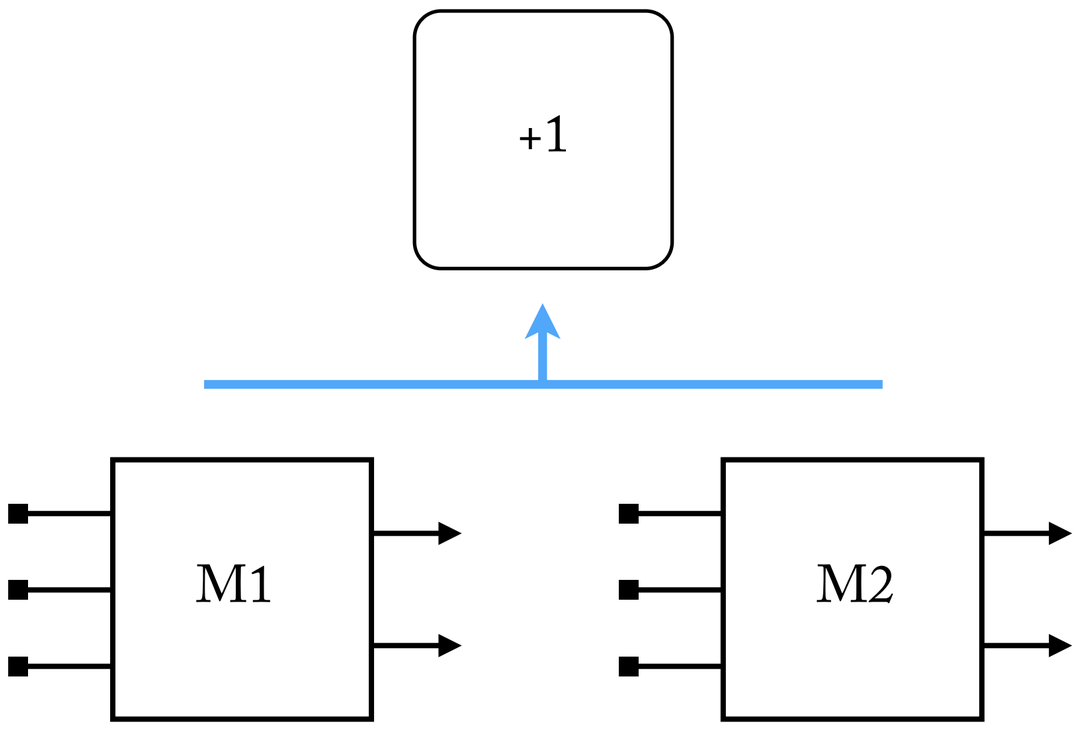
Abstraction over internal description.



Physical implementations of a NOT-gate.

Functional abstraction reoccurs at many levels. So logic gates are functional abstractions. But we can also create functional abstractions for machines *made out of* logic gates.

Now that we have introduced the concept of computation, we can also describe an entire circuit in terms of the function it computes.



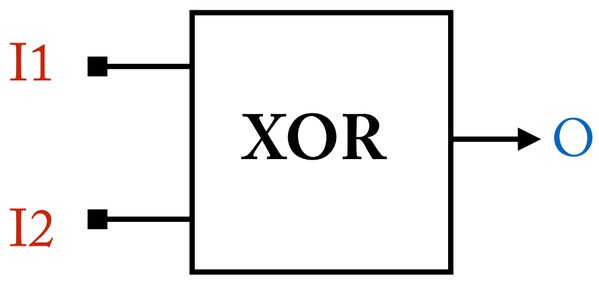
This is the kind of abstraction we engage in when we say that a calculator is **performs multiplication**, or that an app on your phone **finds the fastest route home.**

These are all forms of functional abstraction in terms of the function computed.

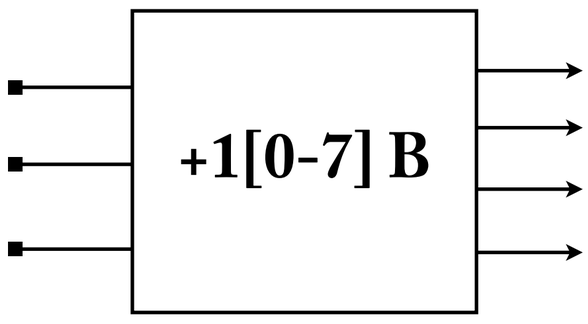
Diagramming Abstraction

Concretely, we'll abstract over lower-level descriptions by drawing a "box" around a circuit, and then labeling the box with the function of that circuit. I'll call these **boxed** circuits.

For example, once you have shown how to build an XOR from NOT, AND, and OR-gates, then we can "box" that circuit. Then you can use XOR as a basic element in your other circuits:



In general, once you have defined a circuit for a given function (say, binary +1[0-7]), you can use that circuit later as part of other circuits. For example:



8. Function Composition

It's a familiar fact that functions can be "stacked together" or **composed**. For example, I could define *f* as:

$$f(x) = (x + 3) \bullet 2$$

Here I've defined *f* in terms a +3 function and •2 function. Put another way, I've defined *f* as the composition of +3 and •2.

Because functions are identified only by their arguments and values, the same function can be composed in different ways. In fact, "a composition" of a function is really just a set of instructions to the reader for how to identify that function. (It is very much like a computer program for your mind to run.) The same function can be specified using different sets of instructions. For example, consider the function *g* where:

$$g(x) = x+7 = (x+5)+2$$

Here we describe *g* in two different ways, by composing different functions. Can you think of others we might have used?

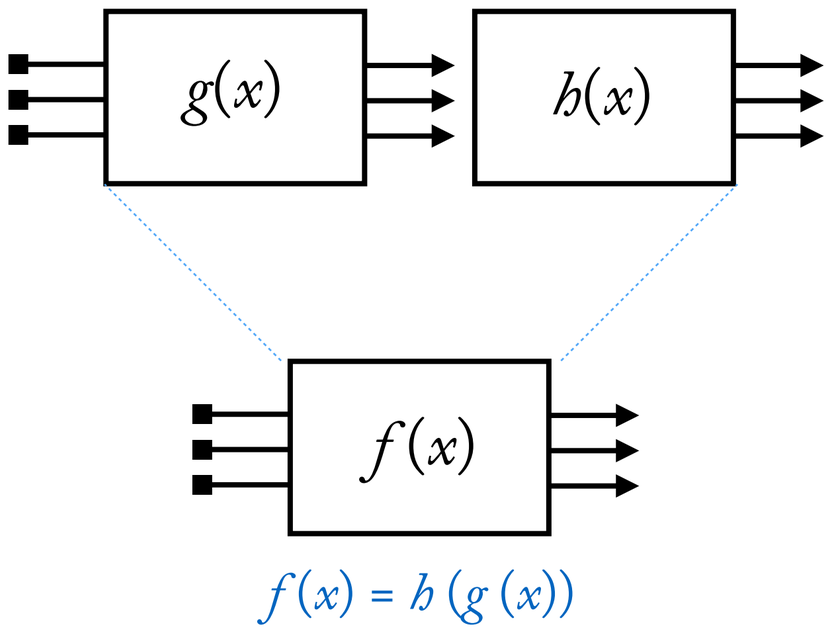
Machine Composition

Any time a function f can be defined in terms of the composition of functions g and h—then a *machine* which computes f can be designed by *putting together* a machines which computes g and a machine which computes h. (Actually, this is only true for certain ways of describing machines. But it holds for the circuit-based descriptions we're considering right now.)

For example, suppose: $f(x) = h(g(x))$. And suppose you have a CC that computes h and a CC that computes g:



Then you can construct a machine that computes f by *combining the machines themselves*. Like this:

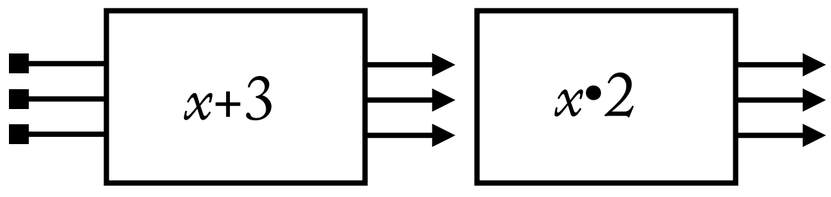


You'll have to be careful about the number of inputs and outputs lining up properly in each case. In the illustration here, I've abstracted away from this (important!) detail.

Concretely, applied to the example we started with, suppose:

$$f(x) = (x + 3) \bullet 2$$

And suppose you have machines that computes $x + 3$ and $x \bullet 2$. Then you can design a machine which computes f by putting together your $x + 3$ machine and your $x \bullet 2$ machine as follows:



You'll want to think about and experiment with these principles to make sure they work for both tally and binary, how to deal with multiple argument places, and other engineering questions that may arise.

Just as there are different ways of composing the same function, there are different ways of combining machines to compute the same function. If you have machines that compute +1, +2, and +5, can you describe different machines for computing $x + 7$?

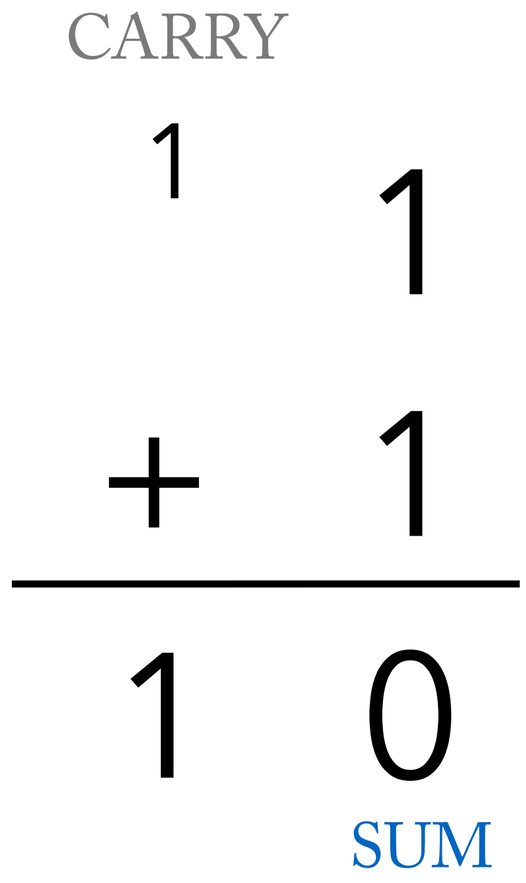
Note! When composing functions, and putting machines together, *order matters*. (Recall the order of operations PEMDAS from middle school arithmetic!) For example, $2 \bullet (3+x) \neq 3+(2 \bullet x)$. This means that order also matters in the way you put together your machines to derive a more complex machine. I'll leave it to you to determine exactly *how* order matters here.

9. The Story of Binary Successor

The Problem: Binary Successor

For computer scientists, the computation of binary successor is the basis for nearly all more sophisticated computations.

Intro: Long Addition in the Simplest Case

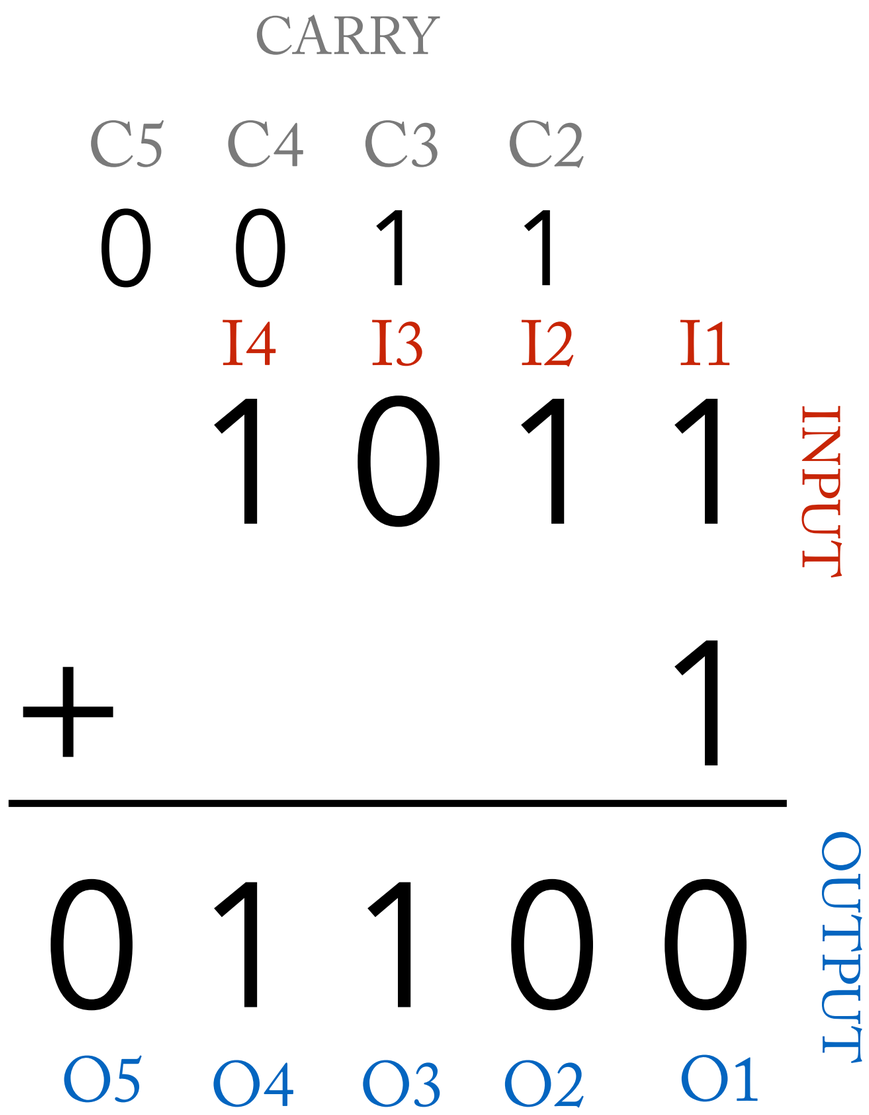


A *bit* = a single binary digit (or numeral).

The result of the initial sum is two bits long, which we break into parts.

The *first* bit is called the **SUM** bit (or S bit). The *second* bit is called the **CARRY** bit (or C bit).

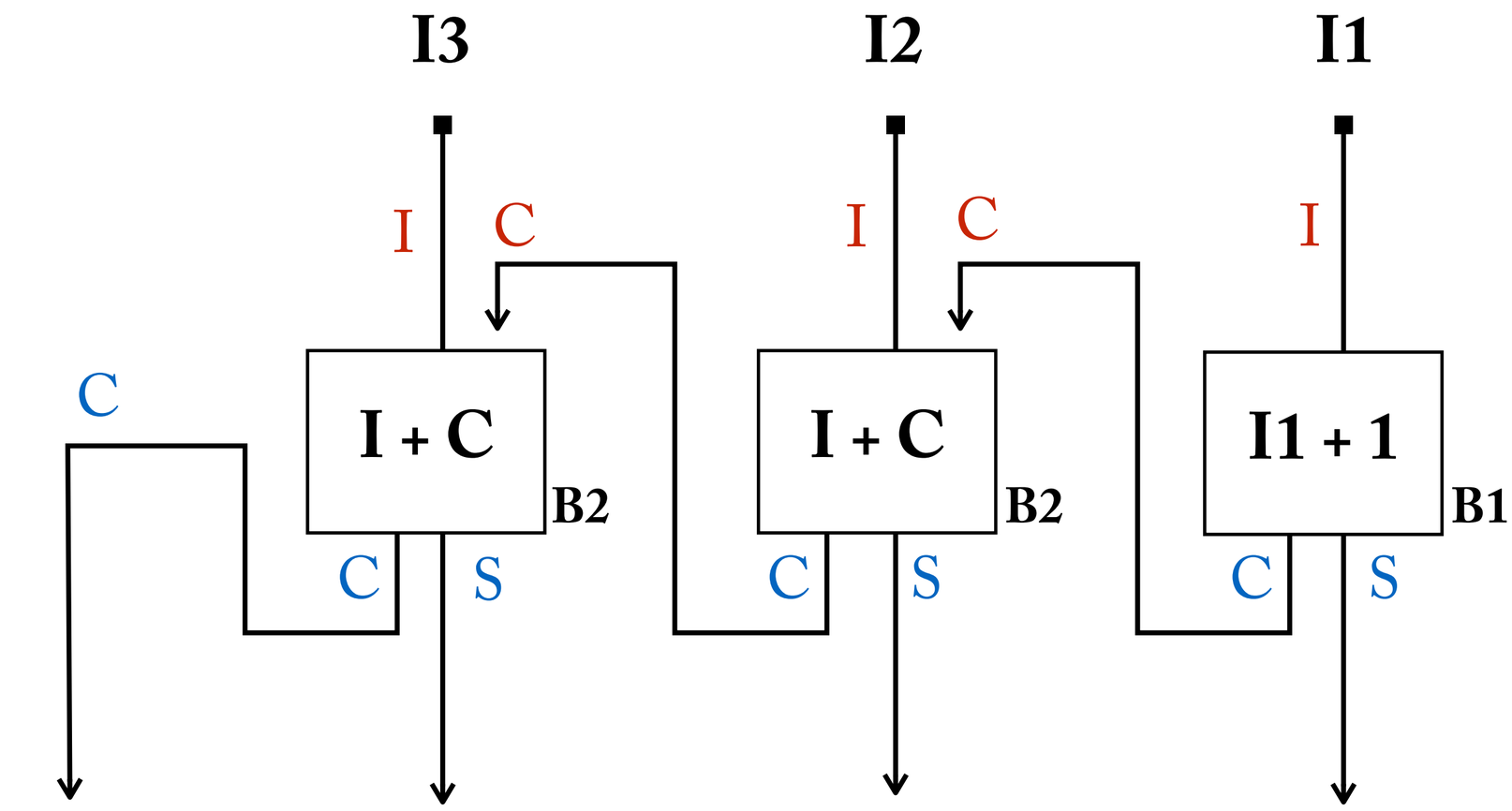
Step 1. Identify a Method (Carrying)



Binary successor algorithm

1. Compute I1 + 1.
   Results in a SUM bit and a CARRY bit.
2. O1 = SUM.
3. Compute I_{NEXT} + CARRY.
   Results in a new SUM bit and a new CARRY bit.
4. O_{NEXT} = SUM.
5. Repeat from 3, as long as there are inputs.
6. When there are no inputs left, O_{FINAL} = CARRY.

Step 2. Break It into Parts



We could keep doing this forever...!

Step 3. Analyze the Parts

Box 1: I1 +1

Input	Sum	Carry
1	0	1
0	1	0

Box 2: I +C

Input	Carry (IN)	Sum	Carry
1	1	0	1
1	0	1	0
0	1	1	0
0	0	0	0

Next you must design a circuit corresponding to each box….

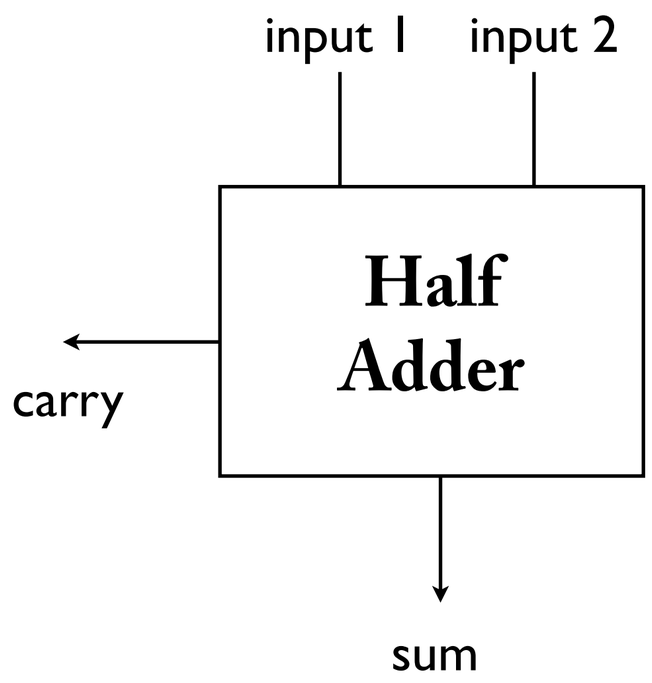
Interlude: Half-Adder

The Box 2 circuit is especially useful and important. It performs a key action: it takes any two bits as inputs, and outputs a SUM bit and a CARRY bit.

This mini-circuit is known as a "half adder". Half adders form the heart of any addition-based computation.

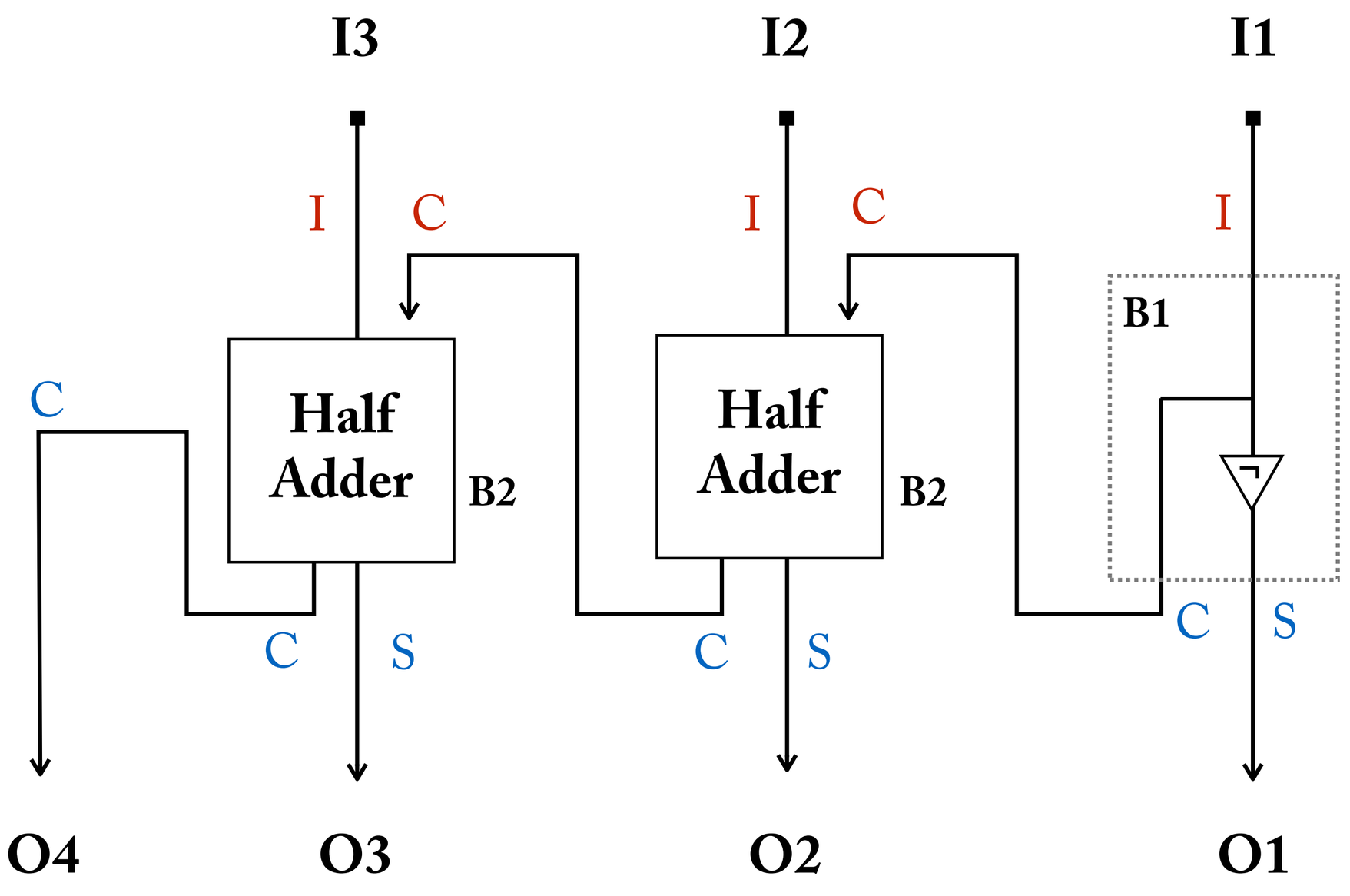
It is only *half* an adder, because it only adds two bits, rather than two bits *plus* a carry bit.

Schematically, it looks like this:



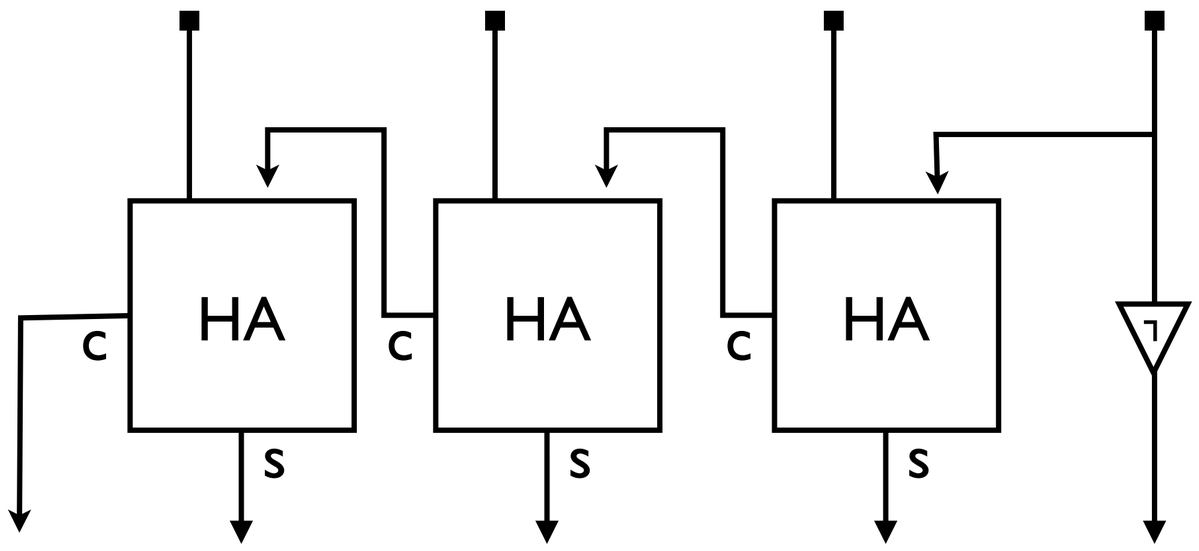
Always label your SUM and CARRY outputs!

Step 4: A Schematic Solution



10. Lessons from Binary Successor

The +1 B Combinatorial Circuit



+1[0-15] B combinatorial circuit

Lesson 1: The Physical Realization of Cognition

+1B is a blueprint for the hypothesis that an entirely physical system can perform (basic) rational operations— provided we have a way of establishing representation relations between symbols and contents. A key step in our answer to Leibniz.

> "[T]hree famous philosophical dilemmas stood in the way [of Hobbes's idea]: (i) the metaphysical problem of mind interacting with matter; (ii) the theoretical problem of explaining the relevance of meanings, without appealing to a question-begging homunculus; and (iii) the methodological issue over the empirical testability (and, hence, respectability) of "mentalistic" explanations. The computational idea can be seen as slicing through all three dilemmas at a stroke…" (Haugeland 1981, p. 34)

> "If there are no philosophical dilemmas about chess-playing computers, then why should there be any about chess-playing people – or, indeed, about human intelligence in any other form? To put it coldly: why not suppose that people *just are* computers (and send philosophy packing)?" (Haugeland 1981, p. 36)

Lesson 2: Information Processing as Functional Description

Hammers have the function of pounding nails, and hearts have the function of pumping blood. +1B has an *information processing* function, which goes beyond the way it manipulates the physical world. "M computes +1" is the informational-level description of the machine.

Lesson 3: The Design Impact of Representational Systems

Compare +1B and +1T machines: dramatically different physical functions, same information processing function. Machines and representational systems must be designed together, or co-evolve.

Lesson 4: The Causal Power of Representations

The value of a function depends on the argument: the value of +1 depends on whether you apply it to 2 or to 3, for example. In a computer, these mathematical dependencies are turned into causal dependencies. In the +1 B machine, the *symbol* for 2 *causes* the *symbol* for 3, just the way the number 2 determines the number 3 for the +1 function. A central idea in CTM is that we explain the inferential connections between ideas by the causal connections between symbols.

> "In effect, given an interpreted formal system with true axioms and truth-preserving rules, if you take care of the syntax, *the semantics will take care of itself*."
> (Haugeland 1981, p. 44)

> "An automatic formal system with an interpretation such that the semantics will take care of itself is what Daniel Dennett (1981) calls a *semantic engine*. The discovery that semantic engines are possible – that with the right kind of formal system and interpretation, a machine can handle meanings – is the basic inspiration of cognitive science and artificial intelligence."
> (Haugeland 1981, p. 45)

Lesson 5: Computing a Function vs. Implementing a Function

All things *implement* the laws of physics, but only some things *compute* the laws of physics. To compute is to *represent* a function, but not to actually conform with it. +1B can compute the laws of *adding-apples-to-a-basket*, but it does not *implement* these laws. (By the way, do the planets compute their own orbit? When humans walk, do they compute or implement their trajectory?)

Lesson 6: Computation Does Not Require Explicitly Represented Rules

+1B computes a function not by "following" a set of rules, which would require an inner-agent to do the following, but simply by *embodying* a set of rules in its functional architecture. A machine can compute a function without *understanding* that function in any way; all it has to do is *perform* that function. Compare: the cognitive capacities of insects and babies, but also most adult human cognition, outside of explicit reasoning, including perception, motor control, autonomic functioning, and so on.

> "[The rule-applier] must be regarded as a complete little person – or *homunculus* (in Latin) – inside the head, directing the thoughts like a traffic cop. The trouble is: a theory that invokes an homunculus to *explain* thinking has begged its own question, because the homunculus itself has to think, and *that* thinking has *not* been explained." (Haugeland 1981, p. 35)

Lesson 7: Competence vs. Performance

+1B shows how a physical system can have certain cognitive *capacities*— i.e. the +1B machine has the capacity to compute +1, for a range of arguments, even if it is never actually performs this operation. It also illustrates the contrast between the capacities you have *competence* for, and your actual performance. Because of entropy, friction, and the rest, how a machine actually performs can pull apart from its competence. A physical +1B machine won't compute +1 perfectly all the time. In other words, its *performance* may be flawed. We can still say that it is a +1 machine, since this is its underlying *competence*.

11. Real and Ideal Computers

1. Symbols and numbers.
2. Error and accuracy.
3. Finity and infinity.
4. Actuality and possibility.
5. Competence and performance.
6. Computing a function vs. instantiating (or satisfying) a function.

12. Introducing Turbots

Embodied Computation

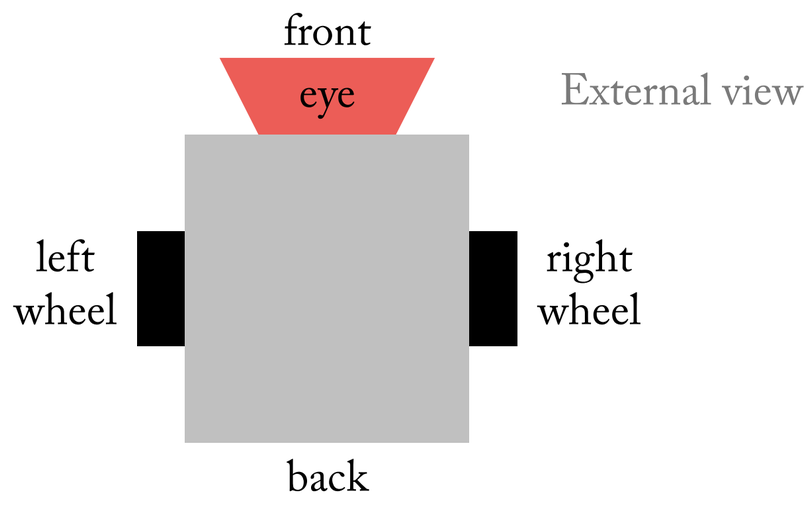
The central fact about cognition is that it allows an organism to interact meaningfully with its environment. Cognition mediates between an organism's ability to sense the environment and its ability to act on it. We can't claim to be studying cognition by only focusing on mathematical computation, even if that is an important part of the story. In fact, many believe that for a computer to exhibit *genuine* mental representation, it must be part of a fully embodied organism that causally interacts with the physical world.

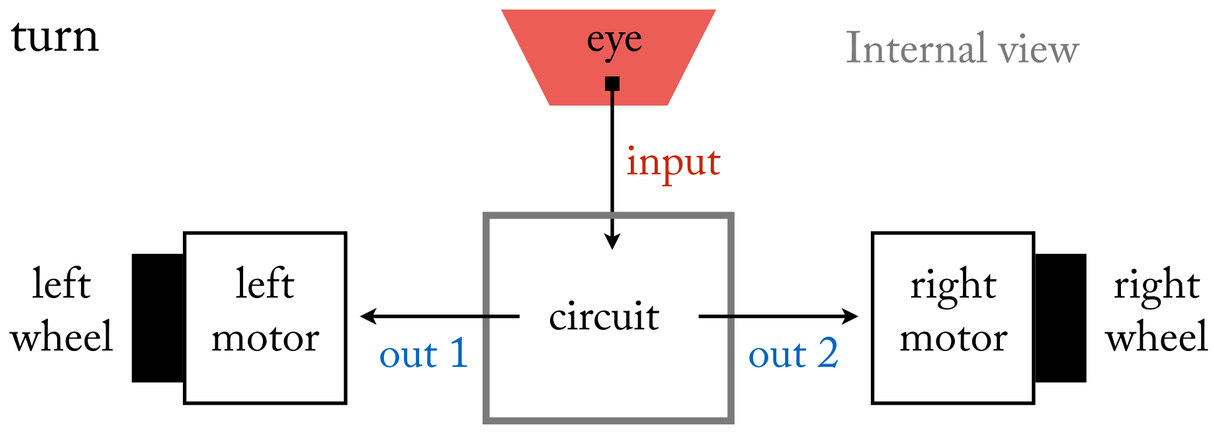
For these reasons, we will now begin to investigate computational problems of navigation for organisms capable of perception and action, alongside the more abstract, traditional study of mathematical computation.

Turbots

A **turbot** (from "Turing" + "robot") is a simple robot with a single electronic eye, two wheels powered by two motors, and an internal circuit which processes signals from the eye, and issues signals to the two motors.

For perception, turbots are limited to seeing the object immediately in front of them; for action, they can only turn left or right, go straight, or stop.



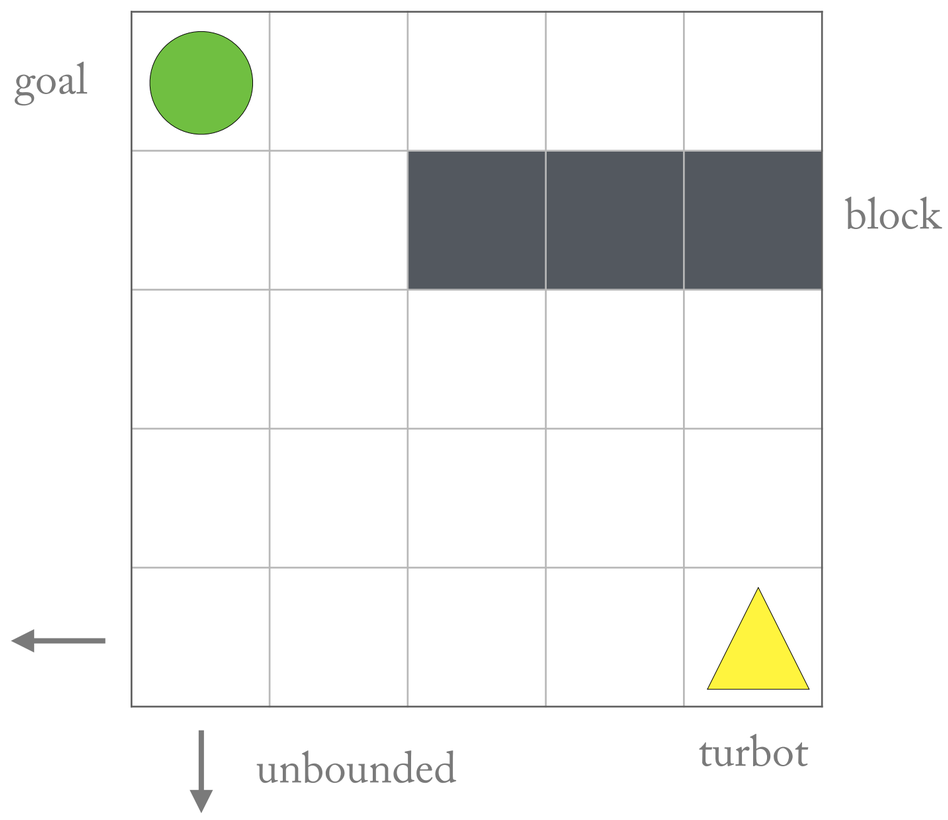


Environment

Turbots live in an unbounded grid-world that includes the turbot, unmovable blocks, and (typically) a goal.

The position and orientation of the turbot are indicated by the yellow triangle.

For navigation problems, the yellow triangle indicates the turbot's starting point. Typically, navigation is "completed," and the problem solved, when the turbot *crosses over* the goal. It does not need to stop on the goal. (Think of Pac-Man.)



Progression of Time

During each unit of time, or **cycle**, the turbot "instantly":
(1) receives a new input;
(2) processes it;
(3) produces an output;
(4) acts on that output.

At the end of each cycle the circuit is "wiped clean" of any information, but the circuit itself persists through time, and the process begins again. (By contrast, for CCs that compute mathematical functions, we only consider their operation at a single moment in time, for a single input.)

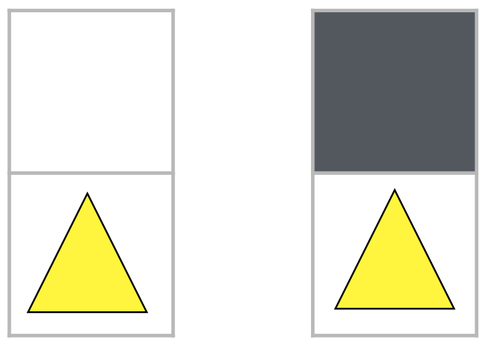
Input: Perception

The input to the turbot comes through the eye. At each cycle, the eye can only see the cell in front of the turbot at that time. And it can only detect the absence (input = 0) or presence (input = 1) of a block.

For now, turbots cannot see goals, only empty cells and blocks.

Turbot perception

empty → 0
block → 1



empty
input = 0

block
input = 1

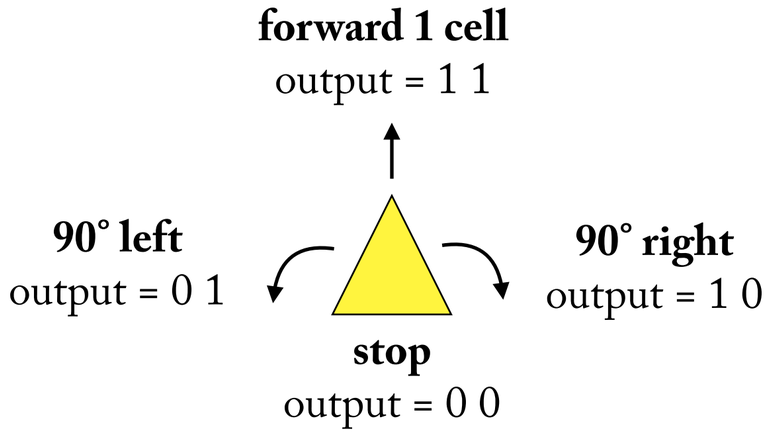
Output: Action

The output from the turbot goes through the two motors and on to the wheels. An output of 1 to a motor will turn it on for 1 cycle, while an output of 0 will turn it off for 1 cycle.

If both motors receive a 1, the turbot moves forward 1 cell. If both motors receive a 0, the turbot stops in the current cell.

If the *left* motor receives a 1 and the *right* motor receives a 0, the turbot will rotate 90° *to the right*, but it will not change cells. (This makes mechanical sense if you think about it.) The opposite is also true: if the *right* motor receives a 1 and the *left* motor a 0, the turbot as a whole will rotate 90° *to the left*.

The only way to turn 180° is to apply two 90° turns over two cycles.



Turbot action

1 1 → forward 1 cell
0 1 → 90° left
1 0 → 90° right
0 0 → stop

A Simple Example

Consider the following problem:

> Design a turbot which moves straight ahead until it reaches a block then stops.

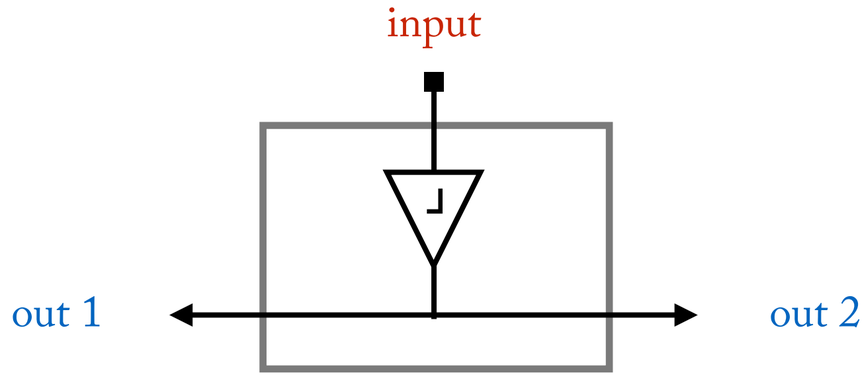
To satisfy this description, the turbot must repeatedly apply the following rule:

> If perception = **empty**, then action = **move forward**.
> If perception = **block**, then action = **stop**.

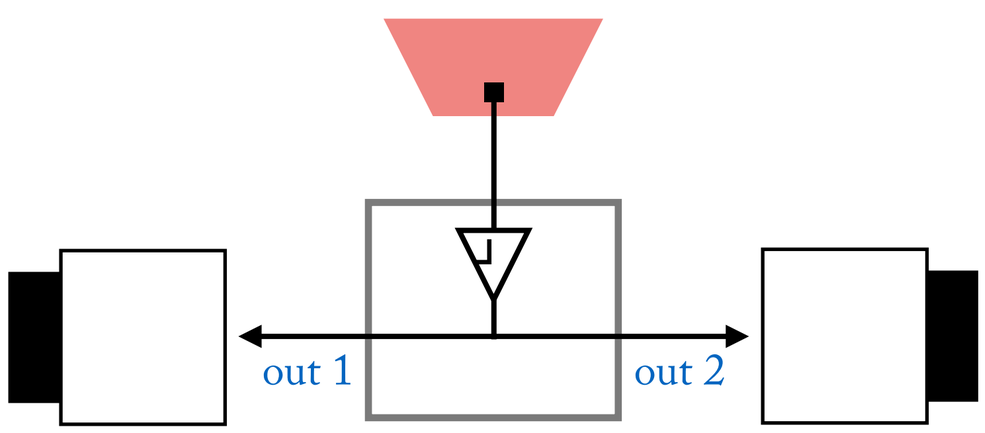
In terms of inputs and outputs:

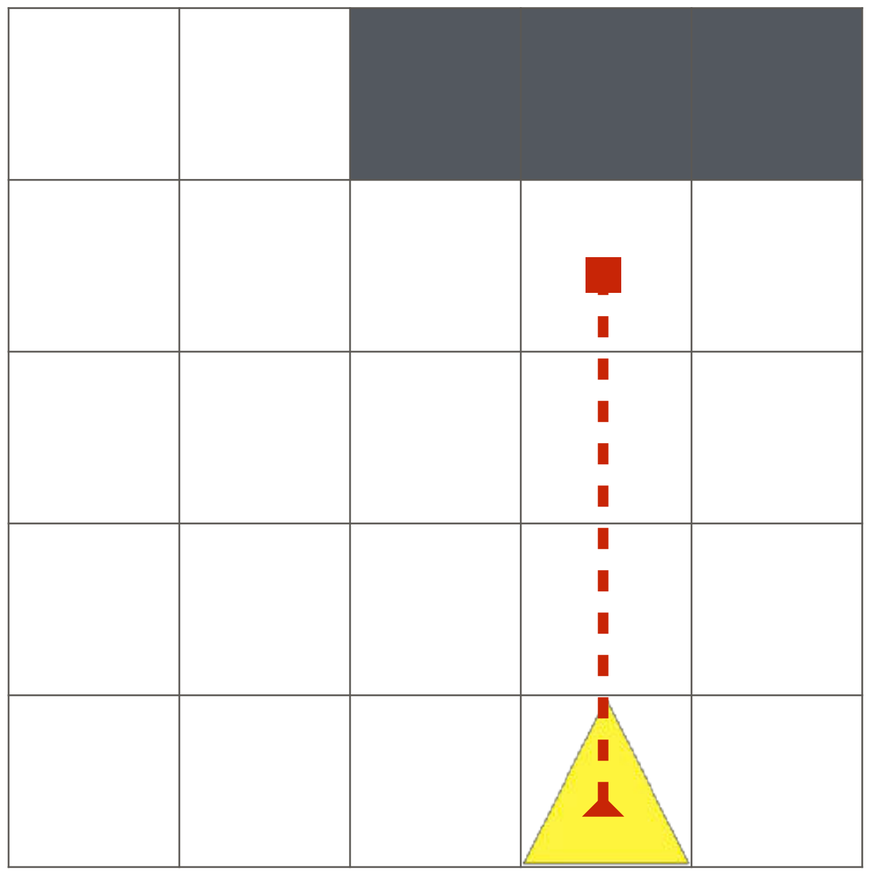
> If IN = **0**, then OUT = **1 1**.
> If IN = **1**, then OUT = **0 0**.

It is now trivial to design such a circuit:



Installed into a turbot, this circuit solves the problem we started with.





13. Thought as Symbol Processing

Quotes: Kenneth Craik, *The Nature of Explanation* (1943), ch. 5, "Hypothesis on the Nature of Thought."

Craik's Hypothesis on the Nature of Cognition

Writing in 1943, on the cusp of what would soon be recognized as the "cognitive revolution," Craik envisioned a mechanistic theory of the mind:

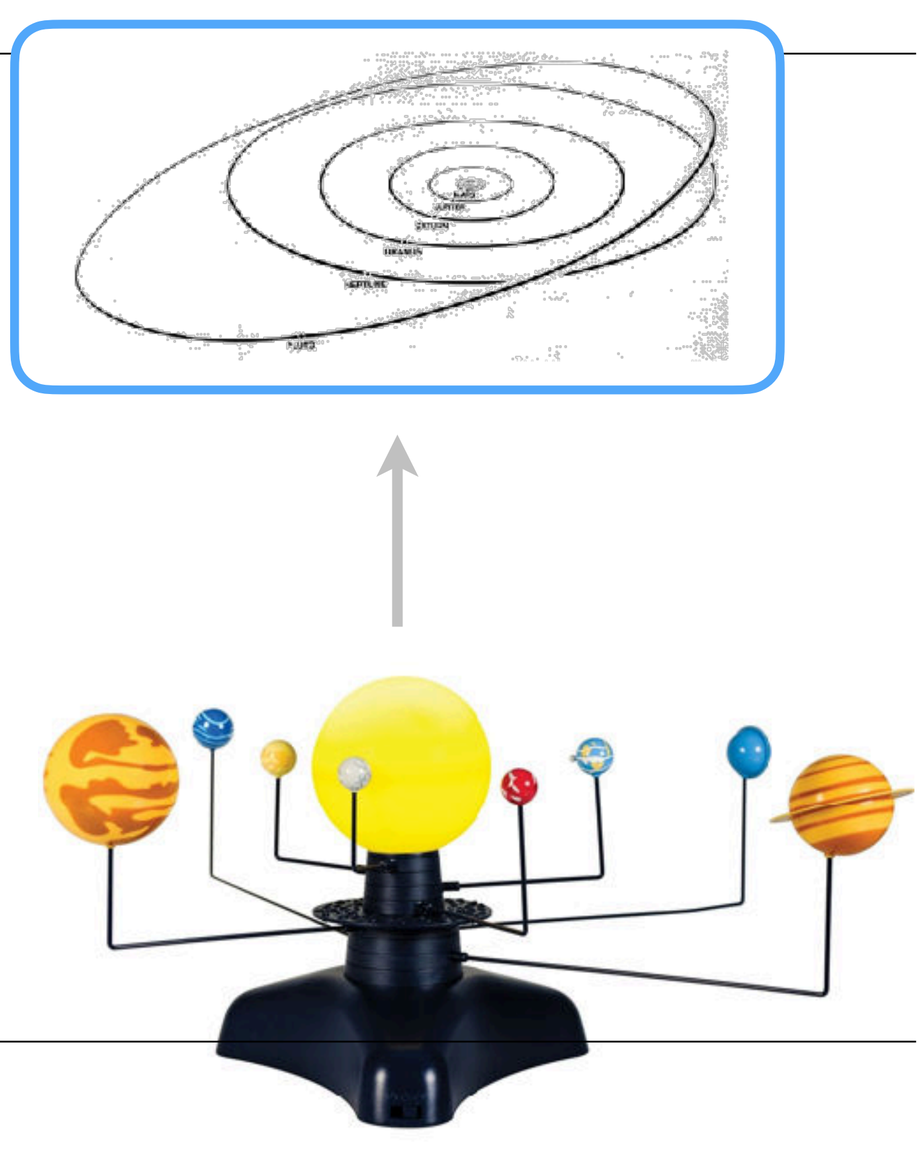
> My hypothesis then is that thought models, or parallels, reality— that its essential feature is not 'the mind', 'the self', 'sense-data', nor propositions but symbolism, and that this symbolism is largely of the same kind as that which is familiar to us in mechanical devices which aid thought and calculation. (57)

There are two parts to this hypothesis: (i) that "thought models, or parallels, reality"; and (ii) that this modeling is achieved through "symbolism" analogous to that of calculating machines.

Physical Models of Reality

Let's start with the observation that we often use physical models to help understand other phenomena. Like words or pictures these models represent external events. But they do more than that. They change and evolve to match the way the represented system changes and evolves.

> The physical process which it is desired to predict is *imitated* by some mechanical device or model which is cheaper, or quicker, or more convenient in operation. (51)



Thought as a Model of Reality

Craik proposes that thought may have a similar function of modeling external processes.

> Human thought has a definite function; it provides a convenient small-scale model of a process so that we can, for instance, design a bridge in our minds and know that it will bear a train passing over it instead of having to conduct a number of full-scale experiments; and the thinking of animals represents on a more restricted scale the ability to represent, say, danger before it comes and leads to avoidance instead of repeated bitter experience. (59)

But he notes that this can't be achieved by actually *duplicating* the external object within.

> This process of reasoning has produced a final result similar to that which might have been reached by causing the actual physical processes to occur (e.g. building the bridge haphazard and measuring its strength or compounding certain chemicals and seeing what happened); but it is also clear that this is not what has happened; the man's mind does not contain a material bridge or the required chemicals. (51)

Symbolic Models of Reality

The key insight is to realize that modeling can be achieved by "relation-structures" involving symbols.

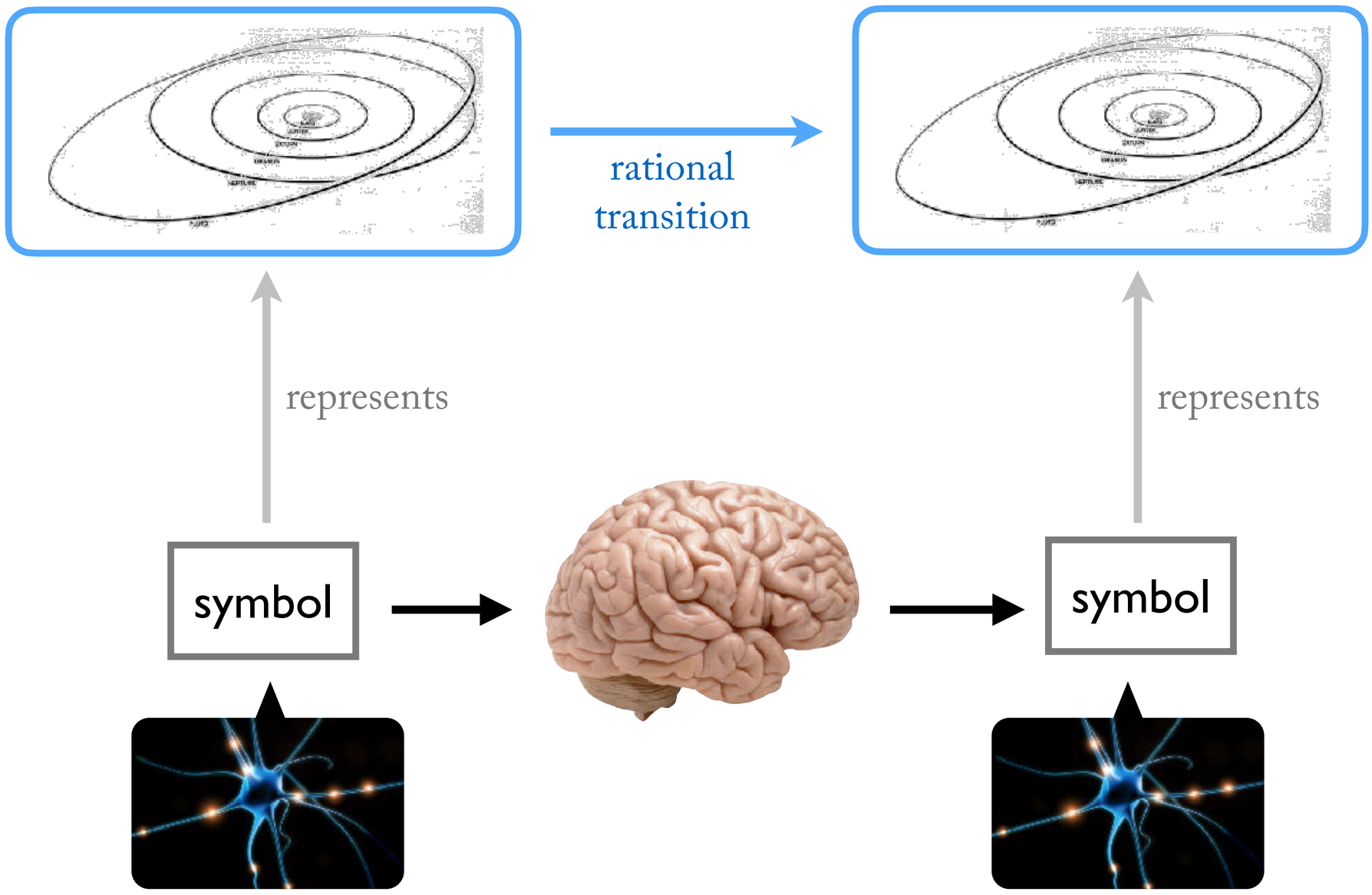
> By a model we thus mean any physical or chemical system which has a similar relation-structure to that of the process it imitates. By 'relation-structure' I do not mean some obscure non-physical entity which attends the model, but the fact that it is a physical working model which works in the same way as the process it parallels, in the aspects under consideration at any moment. Thus, the model need not resemble the real object pictorially. (51)

This leads to Craik's core hypothesis:

> The process of thought, reduced to its simplest terms, is as follows…
> (1) '**Translation**' of external process into words, numbers or other symbols,
> (2) Arrival at other symbols by a process of '**reasoning**', deduction, inference, etc., and
> (3) '**Retranslation**' of these symbols into external processes (as in building a bridge to a design) or at least recognition of the correspondence between these symbols and external events (as in realising that a prediction is fulfilled). (50; emphasis added)

And the central analogy between minds and computers:

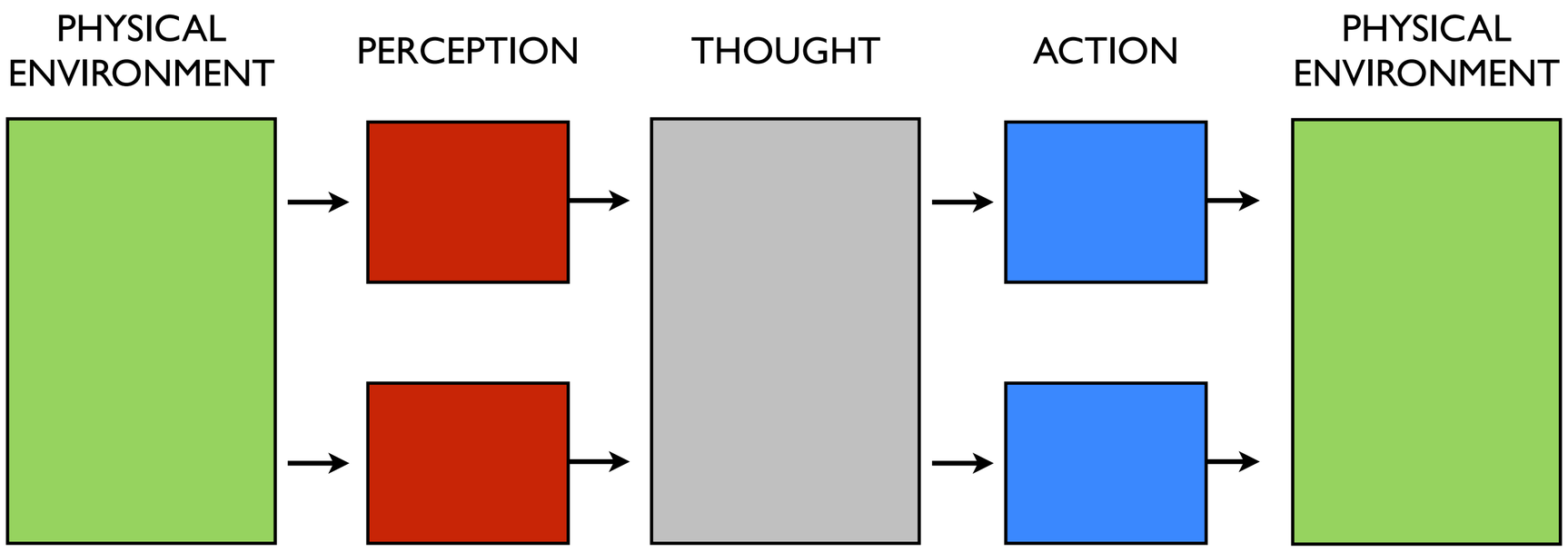
> I have tried… to indicate what I suspect to be the fundamental feature of neural machinery— its power to parallel or model external events— and have emphasised the fundamental role of this process of paralleling in calculating machines. (52)

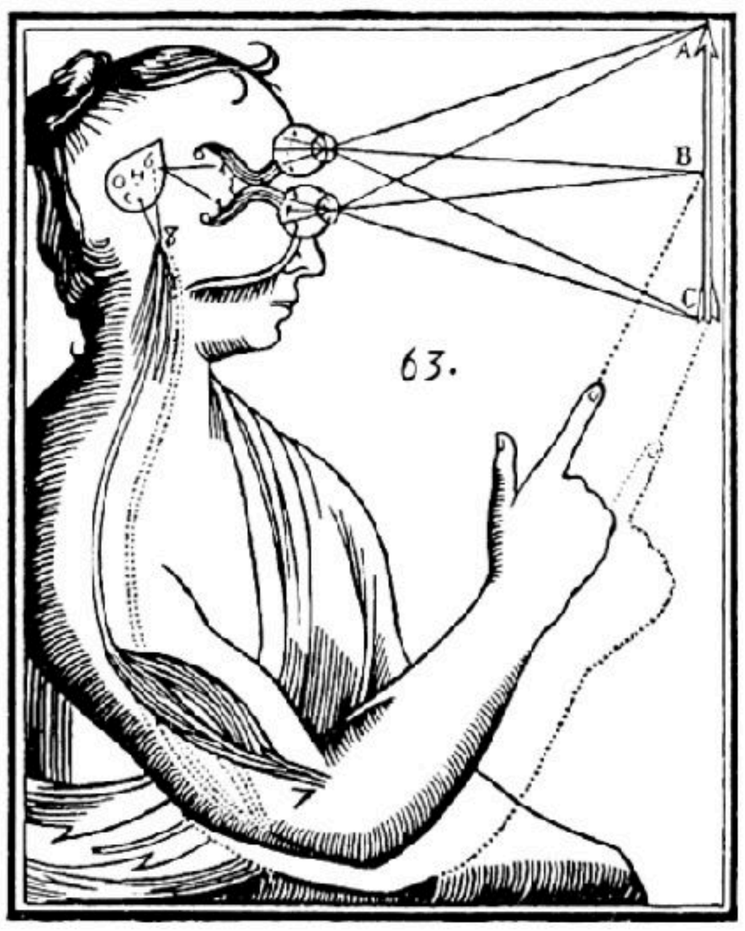


Mind as Input/Output Machine

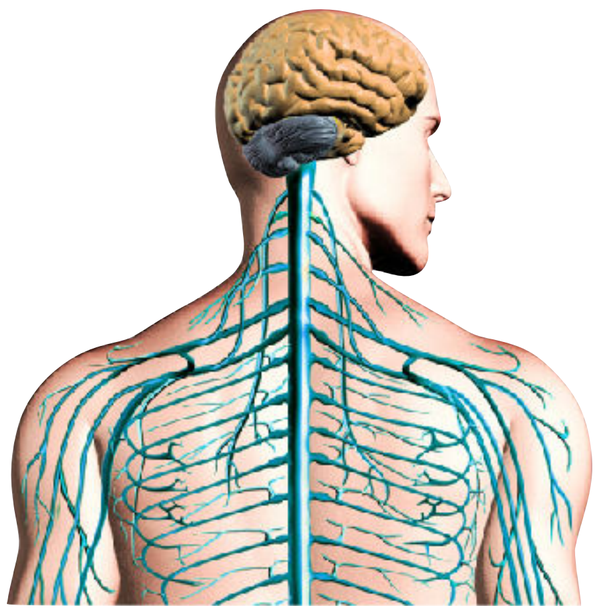
> …it does not seem overbold to consider whether the brain does not work in this way—that it imitates or models external processes. The three processes of translation, inference, and retranslation then become the translation of external events into some kind of neural patterns by stimulation of the sense-organs, the interaction and stimulation of other neural patterns as in 'association', and the excitation by these of effectors or motor organs. (53)

This three-part division of cognition is as old as Descartes, and it is the foundation of contemporary cognitive science:





For Descartes, the mind was made up of the *passions*, *thinking*, and *actions* of an immaterial soul.



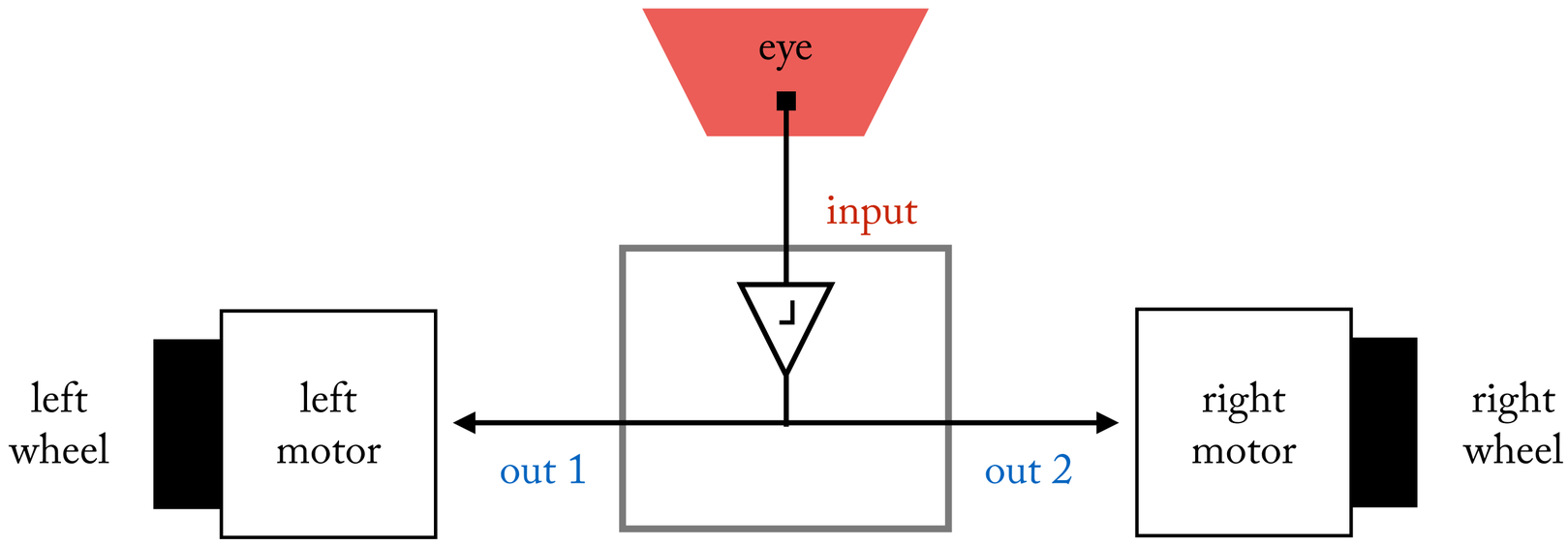
Neuroscience recognizes a central nervous system with inputs from the sensory systems and outputs to the motor systems.



Even in the humble turbot, simple perception and action abilities are mediated by logic circuits.

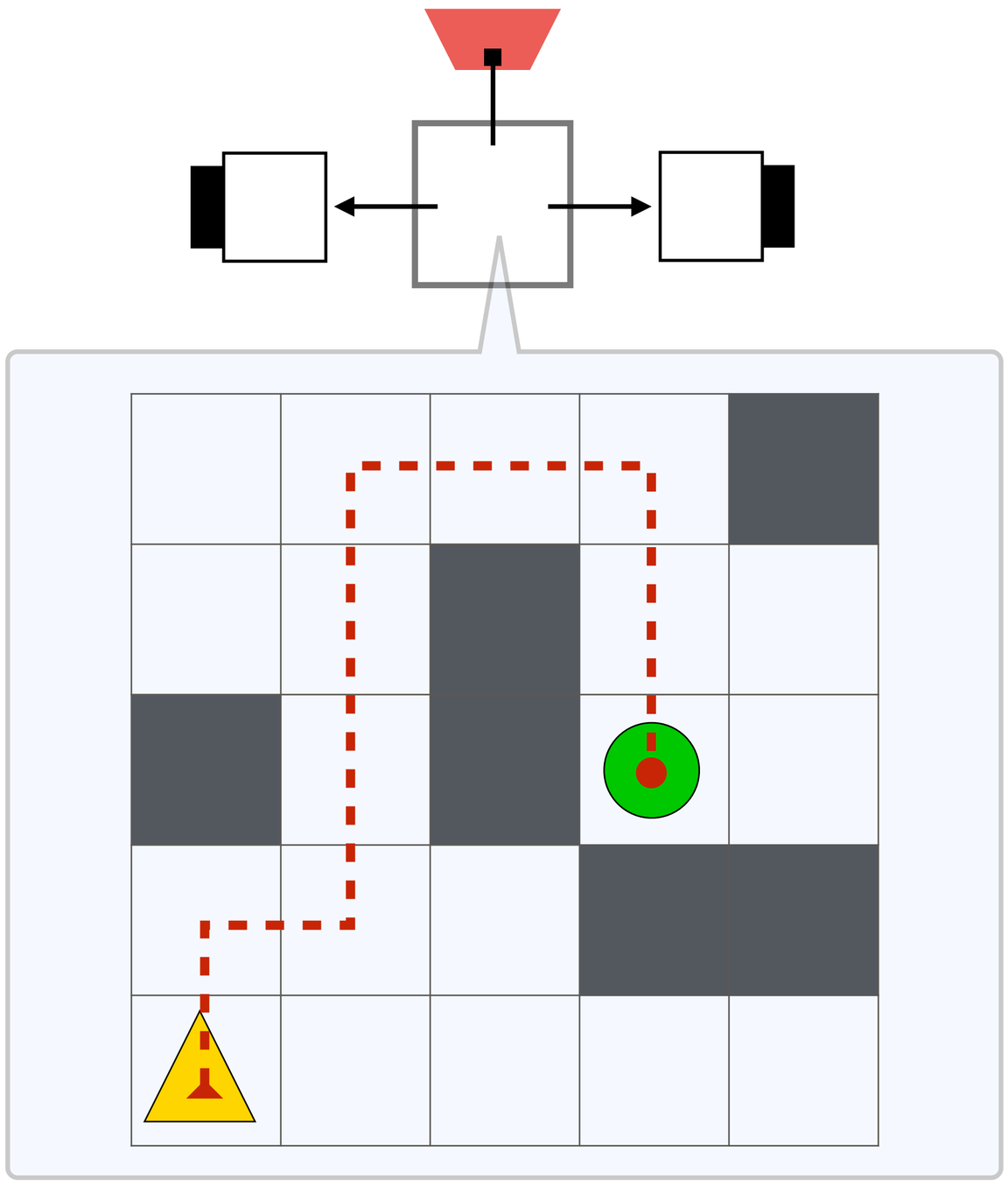
Turbots and Symbolic Modeling

By design, turbots contain the three key elements of cognition that Craik identified: (1) a capacity to perceive the external world; (2) a capacity to symbolically manipulate the inputs from perception; (3) a capacity to turn the outputs of this process into actions.



But so far, the turbots we are considering don't have anything like an "internal model". They *just react* to the environment, where their reactions are determined directly by their input and by their circuitry, but not by any internal information.

But there is no in-principle obstacle to constructing more complex turbots, ones that can learn from the environment, create rich internal models, and work out solutions to problems of navigation before physically pursuing them.



If the organism carries a 'small-scale model' of external reality and of its own possible actions within its head, it is able to try out various alternatives, conclude which is the best of them, react to future situations before they arise, utilise the knowledge of past events in dealing with the present and future, and in every way to react in a much fuller, safer, and more competent manner to the emergencies which face it. (61)

14. Artificial and Natural Computers

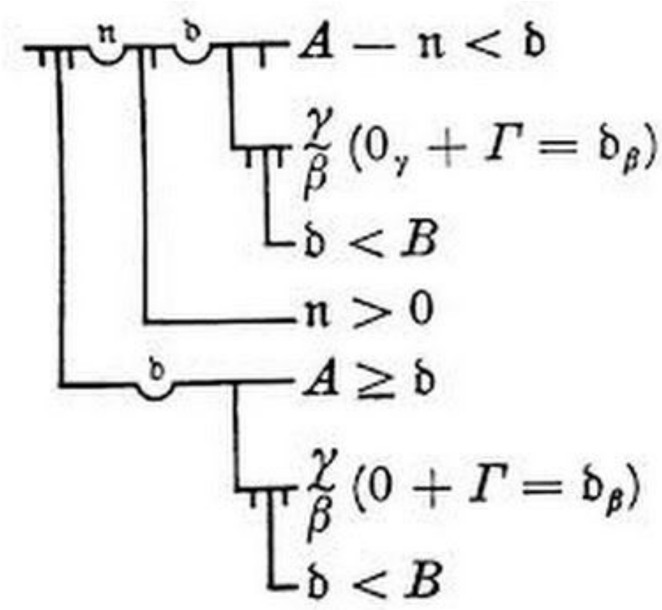
The Invention of Computers

Why create a computer? Why isn't human cognition enough? Why appeal to computation at all? Speed. Accuracy. Reliability. Scalability. Cost. Humans have developed automated computing machines for the same reason we have created any number of automated devices, from the egg beater to the steam engine.

Leibniz himself imagined a machine that would be faster and more reliable than human calculators.

> "If controversies were to arise, there would be no more need of disputation between two philosophers than between two accountants. For it would suffice to take their pencils in their hands, and say to each other: *Calculemus*—Let us calculate."
>
> — Leibniz, 1684

Meanwhile, logicians from Aristotle to Frege attempted to formalize the laws of thought.



Frege's 1879 "Begriffsschrift" or *Concept-script*.

In the 20th century, the bridge was finally made between logic and physical systems. The promise of human-run computation was finally achieved.

Table I. Analogue Between the Calculus of Propositions and the Symbolic Relay Analysis

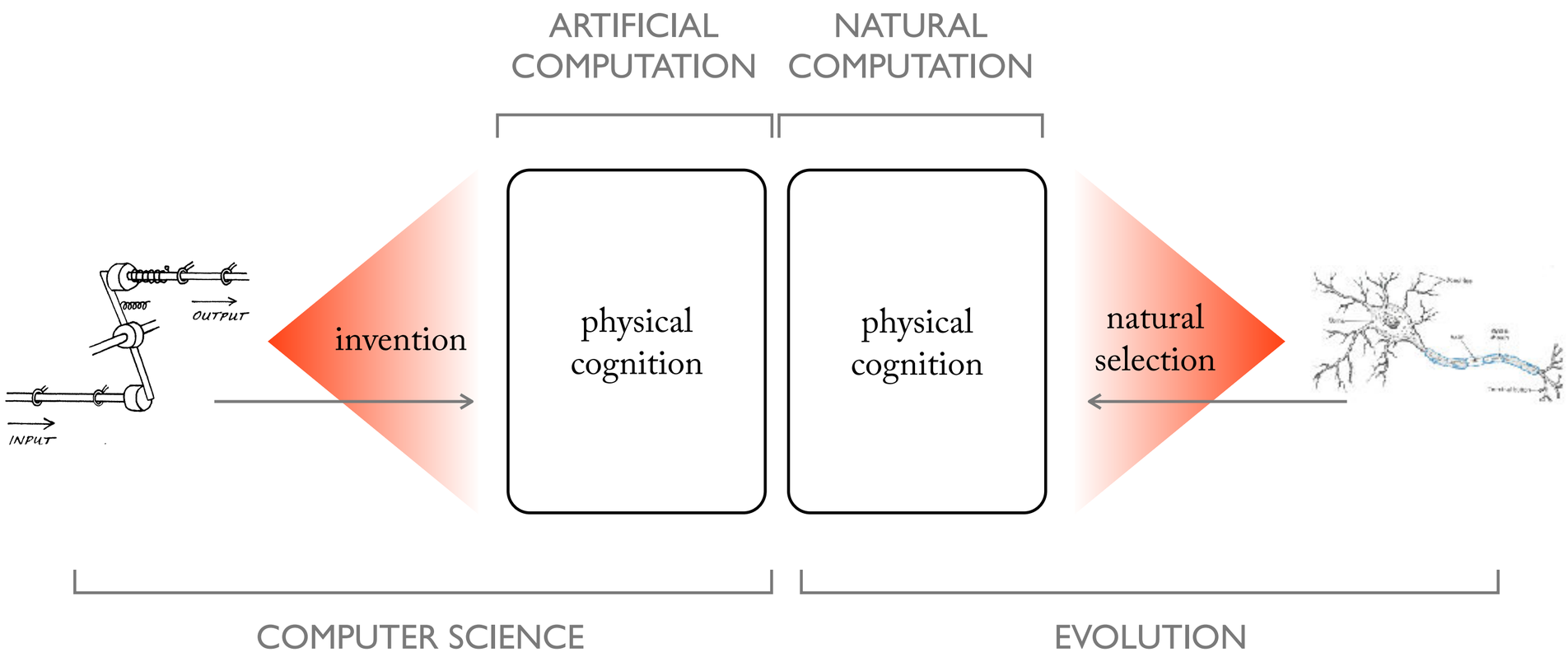
Symbol	Interpretation in Relay Circuits	Interpretation in the Calculus of Propositions
X	The circuit X	The proposition X
0	The circuit is closed	The proposition is false
1	The circuit is open	The proposition is true
$X + Y$	The series connection of circuits X and Y	The proposition which is true if either X or Y is true
$X\ Y$	The parallel connection of circuits X and Y	The proposition which is true if both X and Y are true
X'	The circuit which is open when X is closed and closed when X is open	The contradictory of proposition X
=	The circuits open and close simultaneously	Each proposition implies the other

Shannon 1938 "A symbolic analysis of relay and switching circuits"

> "Logic began as a way to understand the laws of thought. It then helped create machines that could reason according to the rules of deductive logic. Today, deductive and inductive logic are being combined to create machines that both reason and learn. What began, in Boole's words, with an investigation "concerning the nature and constitution of the human mind," could result in the creation of new minds—artificial minds—that might someday match or even exceed our own." Dixon

Computer Science vs Nature

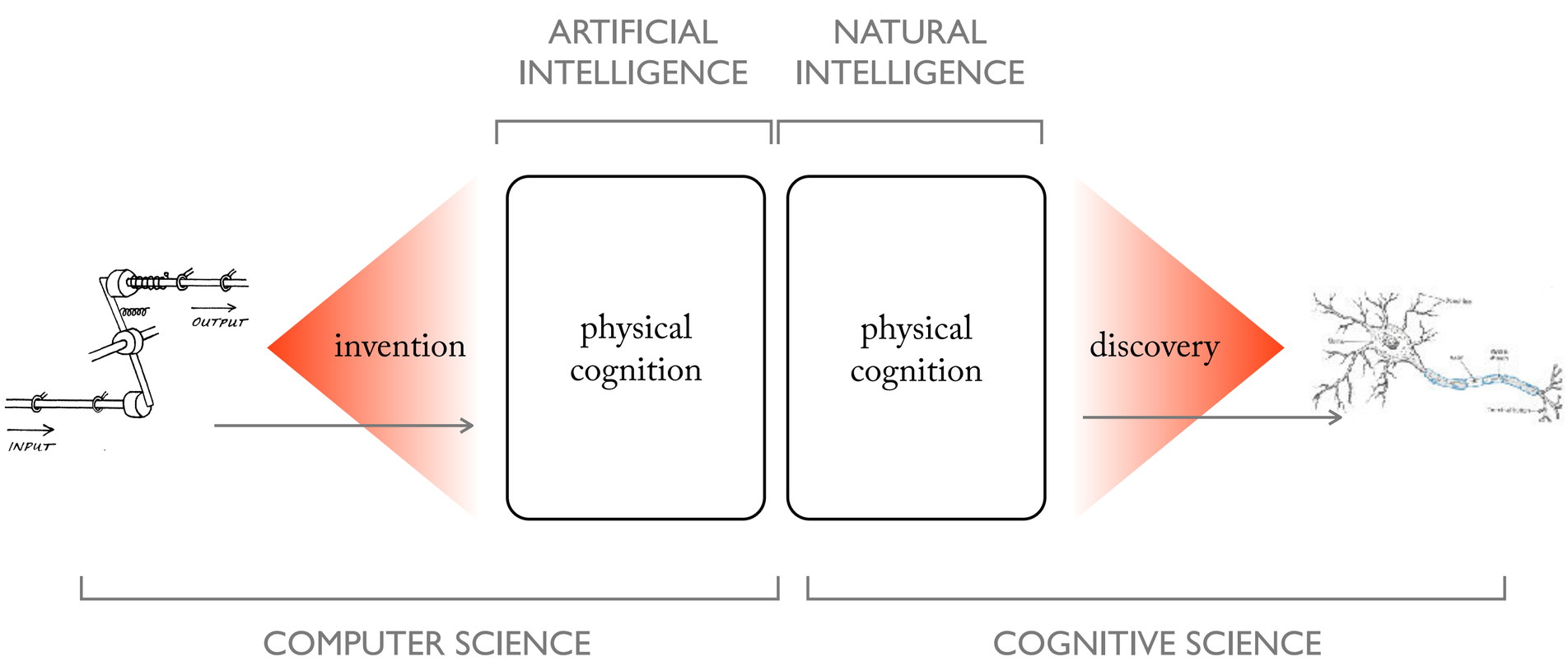
Human Computer Science and Nature represent two processes by which physical cognition has developed.



These two processes give rise to **artificial computers** and **natural computers** respectively.

Computer Science vs Cognitive Science

Roughly speaking, Computer Science and Cognitive Science attack the problem from opposite ends. Computer Science seeks to create unknown artificial intelligence from known physical parts. Cognitive Science seeks to explain known human intelligence from unknown physical parts.

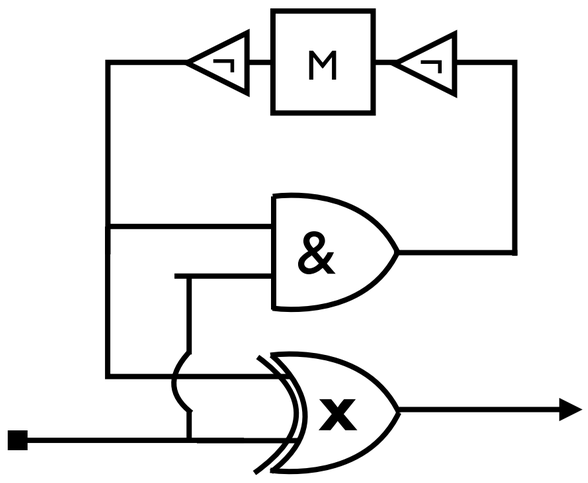


"Psychology is engineering in reverse. In forward-engineering, one designs a machine to do something; in reverse-engineering, one figures out what a machine was designed to do." Pinker (1997, 21)

Making Minds

In this class, we are straddling the divide between computer science and cognitive science, artificial and natural. We are using artificial computers as a model for what natural selection could have produced. And we are using the invented machines of computer science as a model for the discovered machines of cognitive science.

Artificial and Natural Computers





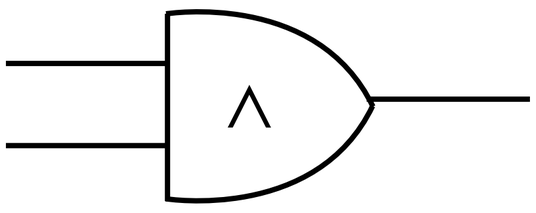
Artificial Computers

Designed. Artificial computers are designed by humans.

Assigned functions. The purpose or function of an artificial computer is what it was assigned by its human designer or user.

User interpretation. The meanings of symbols in an artificial computer are (+/-) determined by the interpretive choices of a human user.

Hardware. Modern artificial computers are built from digital logic gates in an electrical medium.



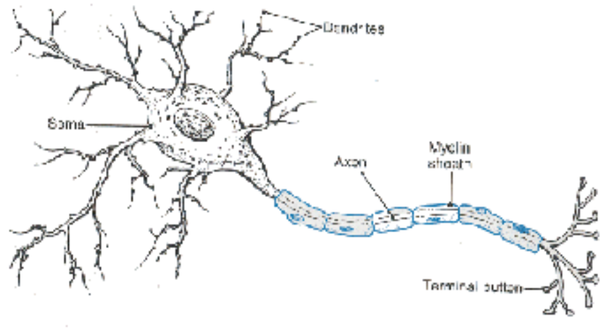
Natural Computers

Evolved. Natural computers are the product of natural selection.

Selected functions. The function of a natural computer is what it was selected to do as a result of an evolutionary process.

Natural representation. The meanings of symbols in a natural computer are determined by the organism's causal interactions with the environment.

Wetware. Natural computers are built from neurons in an electro-chemical medium.



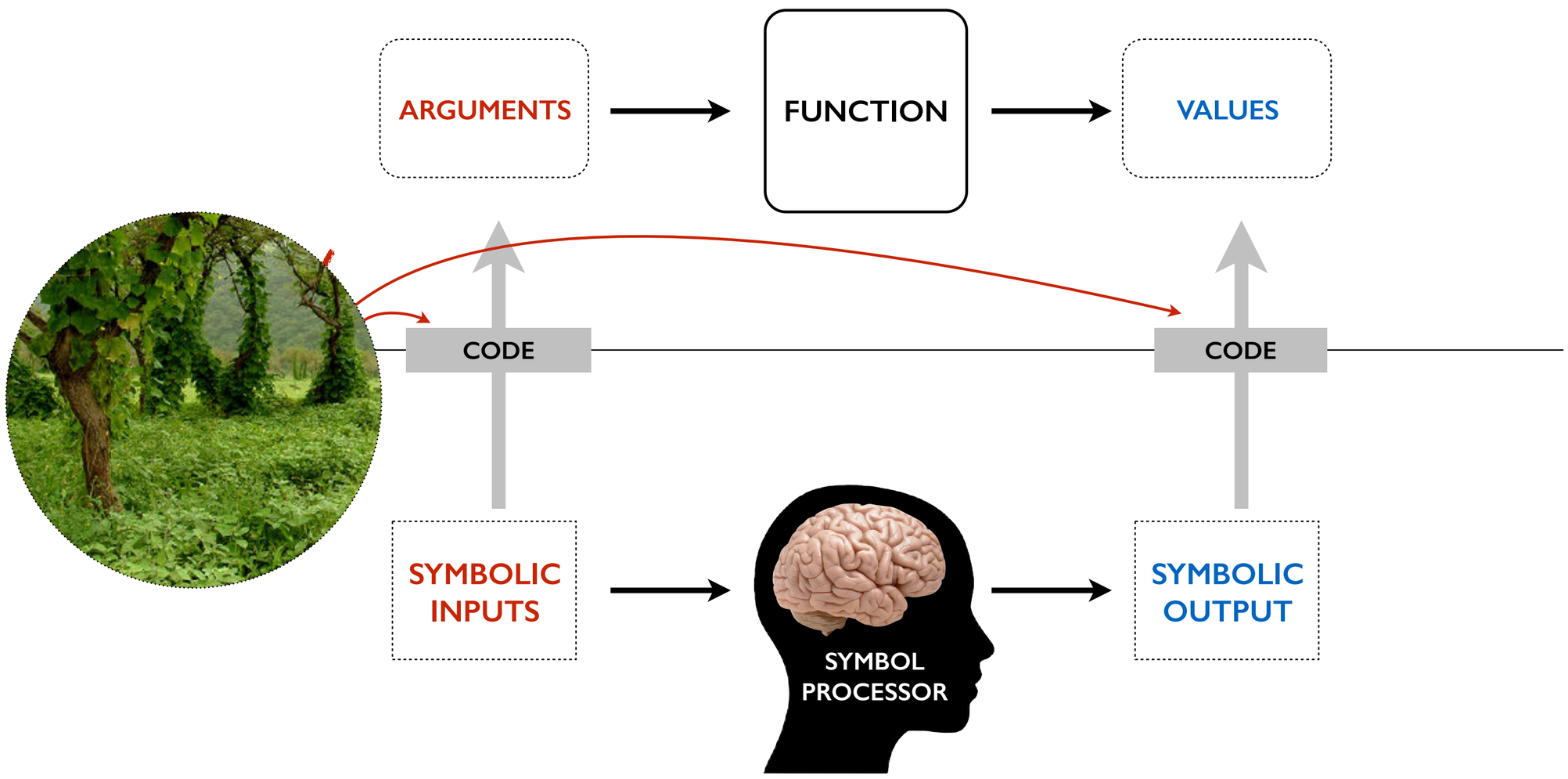
"The mind is a system of organs of computation, designed by natural selection… The mind is what the brain does; specifically, the brain processes information, and thinking is a kind of computation."
Pinker (1997, 21)

Original vs Derived Intentionality

Original intentionality. Symbols in an artificial computer are given their contents by stipulation or convention. But what gives the contents to the symbols in a natural computer? The computer concept doesn't resolve this at all, but without content there is no representation, and no computation.

From plants to informavores: plants have evolved to be responsive to information in the environment, but they do not need to store or process that information. More complex creatures take information in from the environment, process it, and act on the basis of this processing. For these creatures, information-processing is a biological function.

Externalism: the philosophical idea that the content of your mental states depends in part on the internal state of your brain (what symbols you are processing), but also on your relationship to the external environment (how those symbols carry external information).



15. The Computational Theory of Mind

CTM Defined

CTM v.1: "The central tenet of the theory is that a mind is a computer." (McLaughlin 2004)

Not a metaphor: the brain is literally a computer.
The mind is a computer just the way the heart is a pump. That is its actual biological function. But this is a philosophical conjecture. In cognitive science, computationalism is treated more like a methodological assumption than a world-view.

The brain is a natural computer.
The technical concept of a "computer" is that of a system whose function is to perform cognitive tasks (rational transitions) that relate cognitive contents by mechanically manipulating symbols which function to represent those contents— i.e. an information processor. This definition is neutral about the physical composition of the system or its history. The brain is a computer generated by natural selection.

Is it the brain or the mind?
Really, this is a claim about the brain, since computers are *embodied* systems. Instead, the mind is better thought of as what the brain *does*. Perhaps we should say, the mind *emerges* from a computer (the brain).

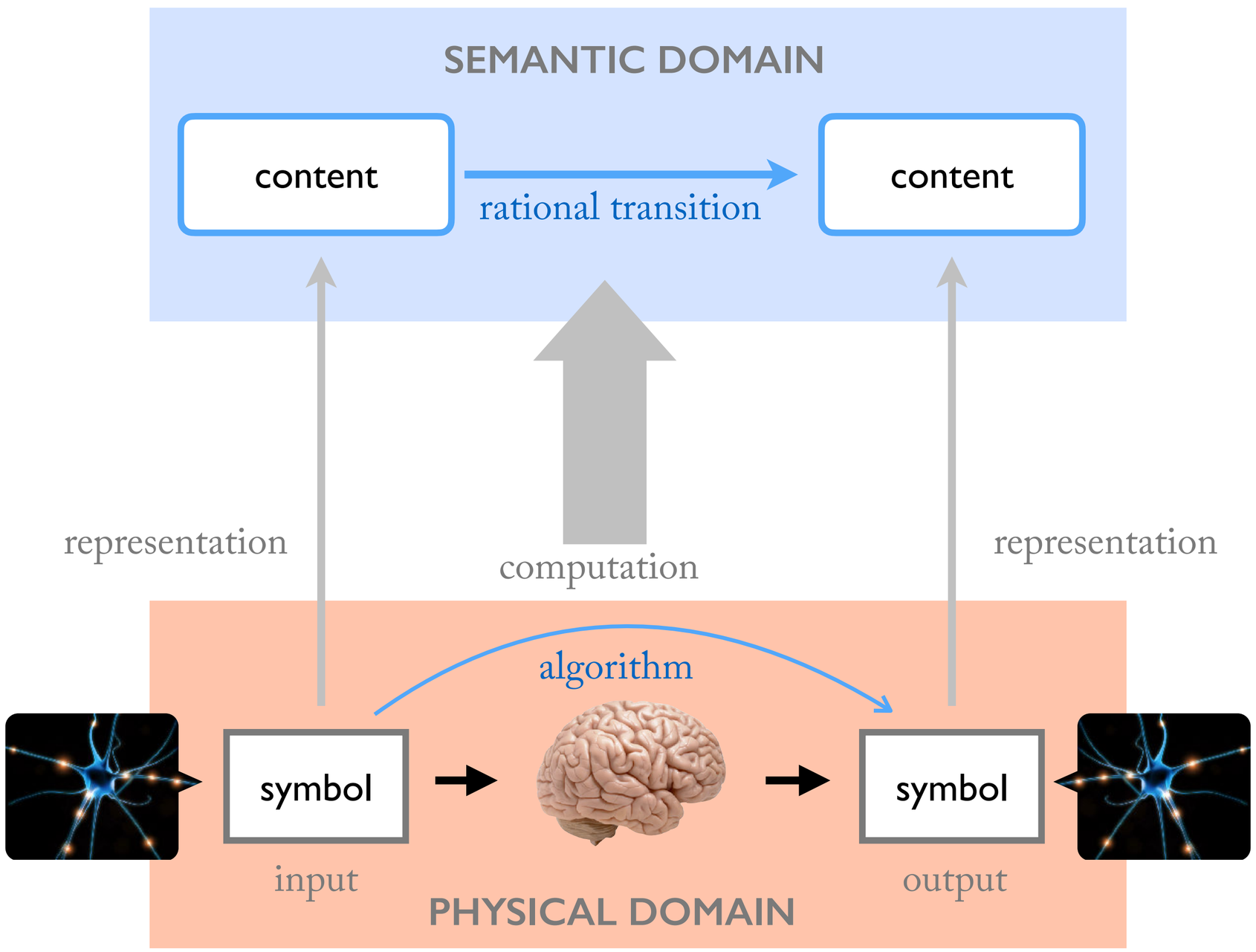
Is it a *computer* or a *computer program*?
If the brain is a *computer*, perhaps the mind is a *computer program*? This might be half right, in the sense that a computer executes (or performs) a computer program. But as we'll see, the concept of a "program" refers specifically to instructions that are distinct from the hardware they are stored in. But much of the mind emerges from biological hardware. The more general concept is that of an *algorithm*.

Is this a claim about all aspects of the mind, including consciousness and emotion?
While consciousness may be a computational state of the brain, there are few computationalists who believe that computation can *explain* phenomenology. In the end, computationalism is really about the cognitive mind. Emotions are debated, but many see emotions as important computational states which give us dispositions to act.

What is meant by "the mind" anyway?
Given that we have many different kinds of mental states, which are constantly changing over time?the concept of *mind* is not a particularly clear one. It's probably better to talk about mental *capacities*, mental *states*, and mental *processes*. (The mind, roughly, is the unity of these elements.) The idea is that mental states are computational states of the brain, and mental processes are computational processes of the brain.

How could the brain function to be a computer, since no one designed a brain?
The brain wasn't designed, but it does have an overall *biological* function, and its parts have many specific biological functions. States of the brain function to carry information about the world (to represent); and structures in the brain function to process these states (or, compute).

Computationalism v. 2: the brain is a natural computer, or information processor. The biological function of the brain is to perform a range of cognitive functions by carrying out mechanical transformations on physical symbols which compute those functions. Cognitive states are computational states of the brain. Cognitive processes are computational processes of the brain.



Computational Theory of Mind: the brain computes cognitive functions by manipulating symbolic representations in the brain according to biologically grounded algorithms.

Cognition beyond Combinatorial Circuits

1. **Multi-functionality.** Natural cognitive systems compute many different functions. Our CC machines computes only one.

2. **Learning.** Natural cognitive systems change their behavior in response to experience. The behavior of our CC machines is fixed for eternity.

3. **Non-arithmetical functions.** Natural cognitive systems perform non-arithmetical functions, including decision making, action-planning, prediction, navigation. What can a CC do beyond +1B?

4. **Logical and probabilistic inference.** Among non-arithmetical functions some employ language-like reasoning, especially logic and inference. +1B is stuck in a world of integers; can we extend CC's to logic?

5. **Perception and action.** Natural cognitive systems exist within an environment. They gather information from that environment (perception), and they change that environment (action). +1B has no obvious causal connection to the natural order. Can CC's perform perception?

6. **Biological basis.** Brains are built from neurons, not logic gates; neurons are put together to form neural nets, not combinatorial circuits. So CC's fails to capture the physical and functional basis of natural computation.

7. **Emotion and consciousness.** CCs are incapable of emotion or consciousness. (Right?) It is not clear, from the current standpoint, what would even be required to achieve this.

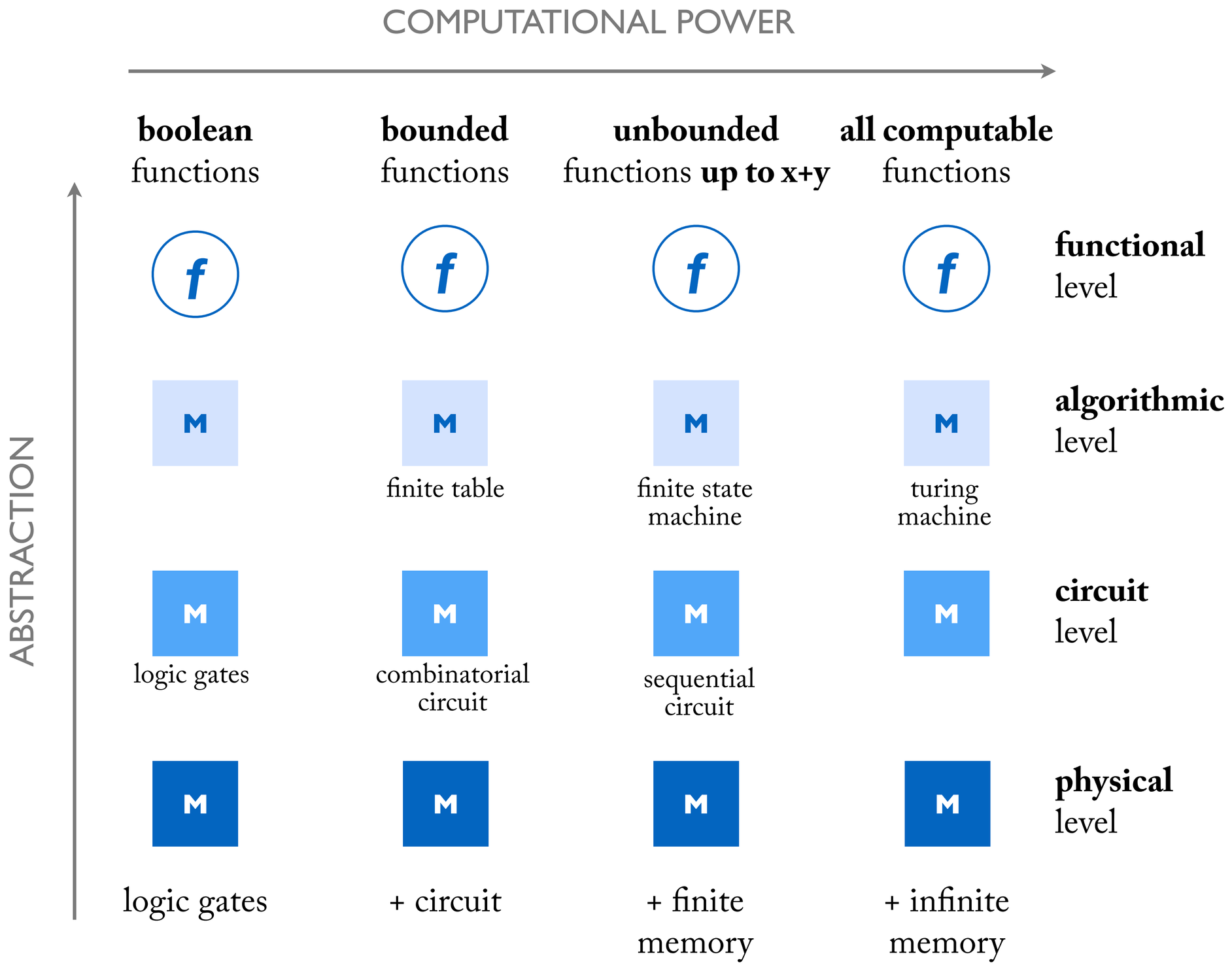
8. **Original functionality.** Natural cognitive systems have the biological function of performing certain information-processing tasks, as a result of natural selection. The +1B machine's only function was stipulated in a problem set.

- CTM is a conjecture, not a conclusion.

Unit 2

LEVELS

In Unit 2, we explore the dimensions of computation. We'll look at levels of abstraction and levels of computational power.



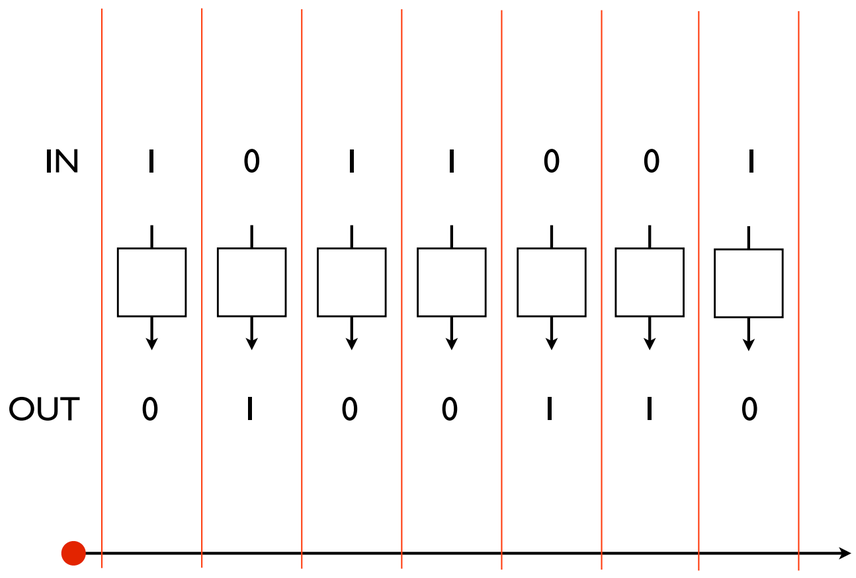
16. Memory

Harnessing the Power of Time

At the end of Unit 1, we saw some of the limitations of combinatorial circuits. For one thing, the size of the inputs they can process is limited by the size of the circuits. In a small and finite amount of space, only small and finite numerals can be processed. For another, combinatorial circuits always respond in the same way to the same input, no matter what has happened in the past. They are incapable of learning.

To overcome both problems, we must find a way to harness the power of time. If we can look back in time, we can condition our behavior on what has happened in the past. And we can access the unlimited store of time available to us for the purposes of computation. Memory, of course, is what will allow our little computers to master time.

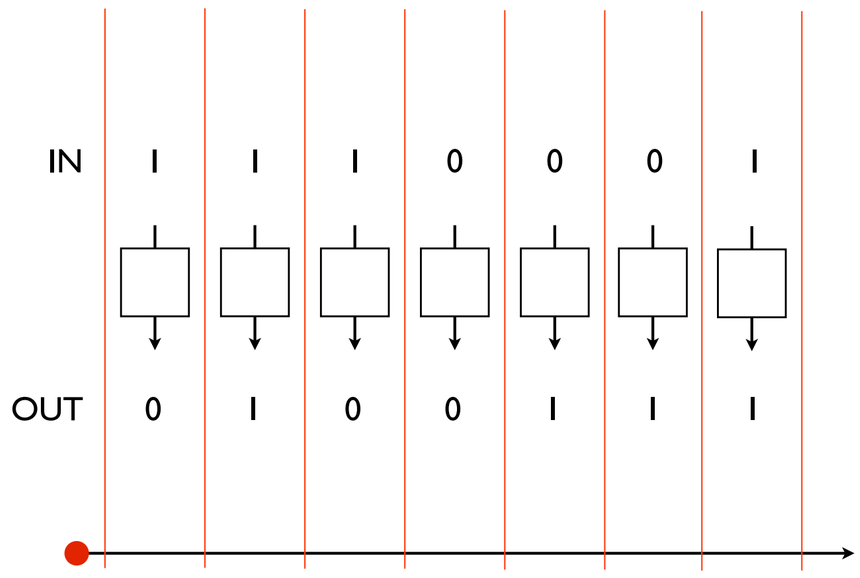
History Independent Response



This machines always responds to the same input in the same way, independent of its past responses.

No memory required.

History Dependent Response



This machines responds to the same input in different ways, depending on its past responses.

Memory required.

Memory

Memory is **information carried forward in time**. The chalkboard is a form of memory; marks are carried forward in time until they are erased. A seesaw is a primitive form of memory too; it stores information about which end was pushed down most recently.



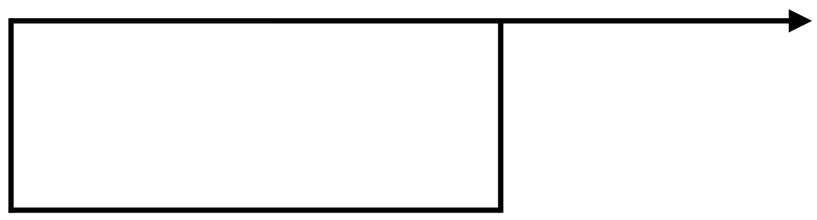
17. Physical Memory

Constructing a Memory Register

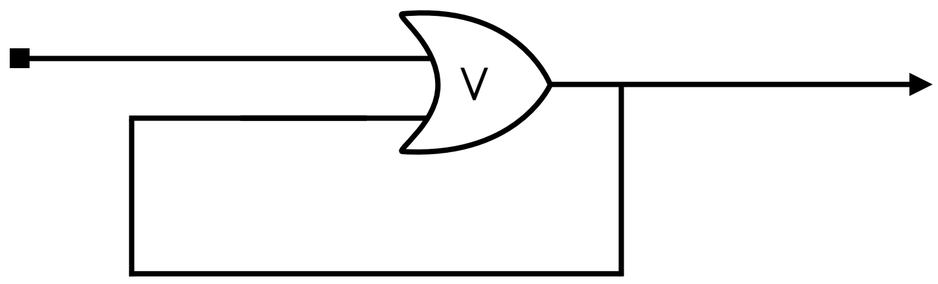
We can easily carry information forward through time in a stable medium like chalk on a chalkboard. But how could we store information in a dynamic medium like the flow of electric current, or electro-chemical interactions of neurons? How can we get the information stored in an electric current to "sit still"? We'll now explore how memory can be implemented in an electronic circuit. The details here won't matter much for the overall narrative, but if we are out to demonstrate the possibility of physical cognition, we better prove to ourselves first that memory itself can can physically implemented.

Warning! In this section I'm going to use the familiar-looking circuit diagrams to represent **electronic circuits**, and not **logic circuits**. This means that 1's and 0's represent states of an electric current (**on** and **off**), and lines represent **wires**, not just information pathways. In an electronic circuit, the resting state for all wires is off, so the default input signal to any logic gate is 0. Also, in electronic circuit, you can create loops, though their behavior can be unpredictable! I've abstracted away from voltage source and ground.

The core idea is that information could be stored by locking an electrical current into a **loop**, which would sustain it over time. The circuit below embodies this idea. The signal stored in the loop is being repeatedly delivered to the output wire at right.

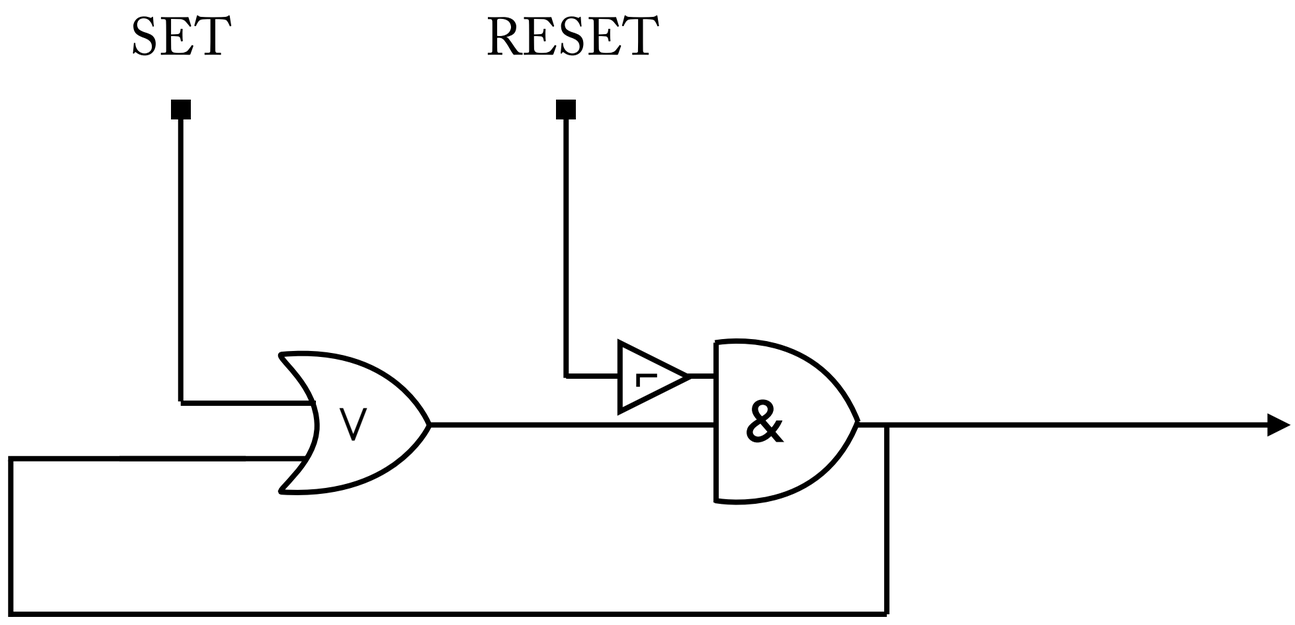


But how do we get information into the loop in the first place? A circuit like the one below does the trick. As long as the input signal is 0, the output is 0. But as soon as the circuit receives a single 1, it will output 1's forever. This is a form of memory— it carries information forward in time about whether a 1 has ever been inputted.



But it is not very useful. It's like a seesaw that always gets stuck. We want a memory that can store whatever information we like, the way a blackboard allows us to do! What we need is a way of selectively breaking the loop, and resetting the circuit at will. We want something that, at one time, can take a bit and keep reproducing it (like the circuit above), but at another time, can also start afresh if its convenient.

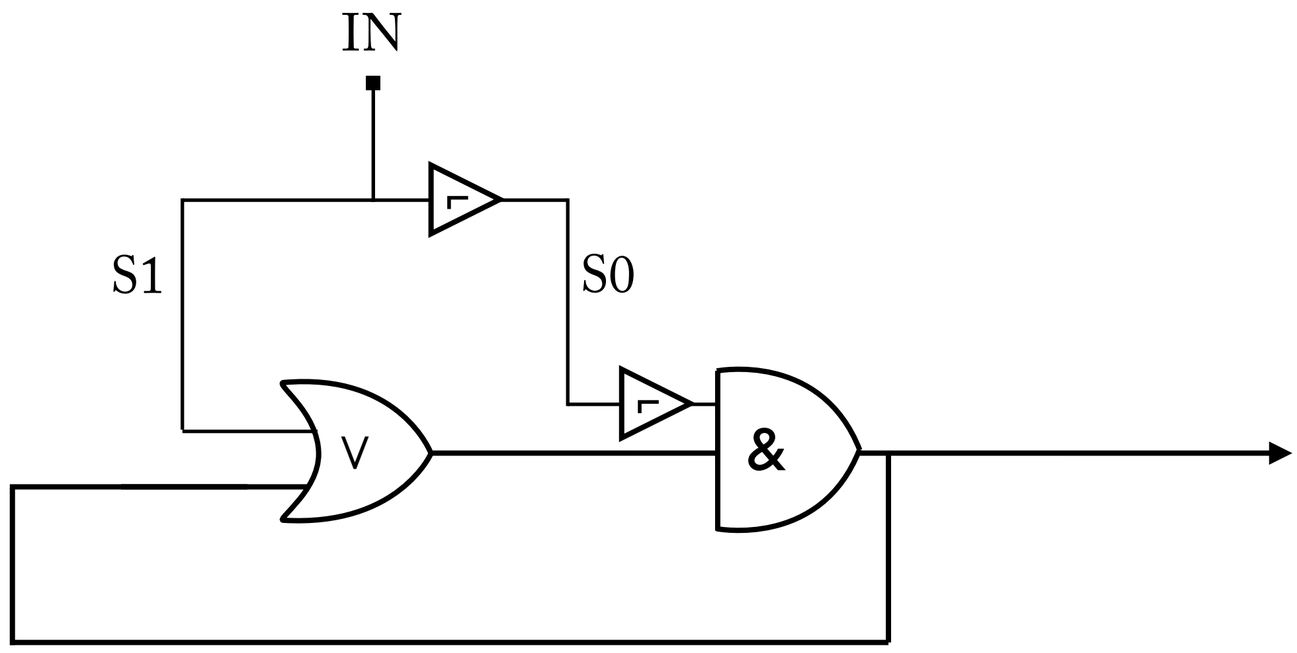
Here is an idea: introduce an AND-gate after the OR-gate. That way we can block the loop when we want to. For conceptual clarity, we'll add a NOT-gate just before the AND-gate.



To think about this circuit, let's say that, by default, RESET=0 and SET=0. In the initial "resting state", the circuit as a whole will store a 0. If at some point SET=1 for a moment, then the circuit will switch over into an "active state", in which case it stores a 1, even when SET=0 again. When you are ready to return to the resting set, let RESET=1 for a moment.

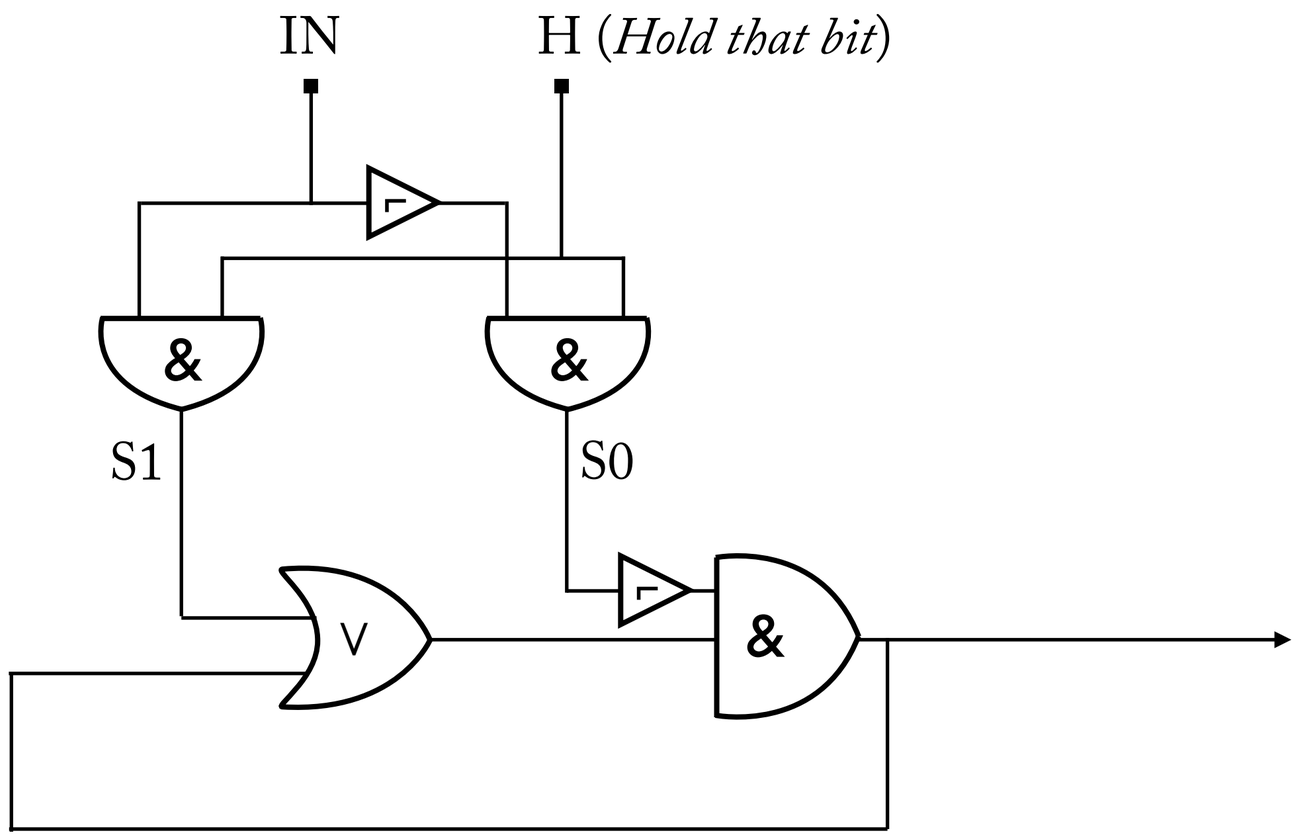
In a way, the resting state stores a 0, and the active state stores a 1. The glimmer of real memory! Can we use the two different states to store an arbitrary bit? Suppose we had 1's trigger the active state, and 0's trigger the resting state?

The natural next step is to hookup an input wire to the S1/S0 wires so that 1's activate the S1 side (and not the S0 side) and 0's activate the S0 side (and not the S1 side).



This is a good start, but our resulting circuit doesn't actually remember anything. The problem is that there is always *something* coming along as input (even if it's just the OFF signal).

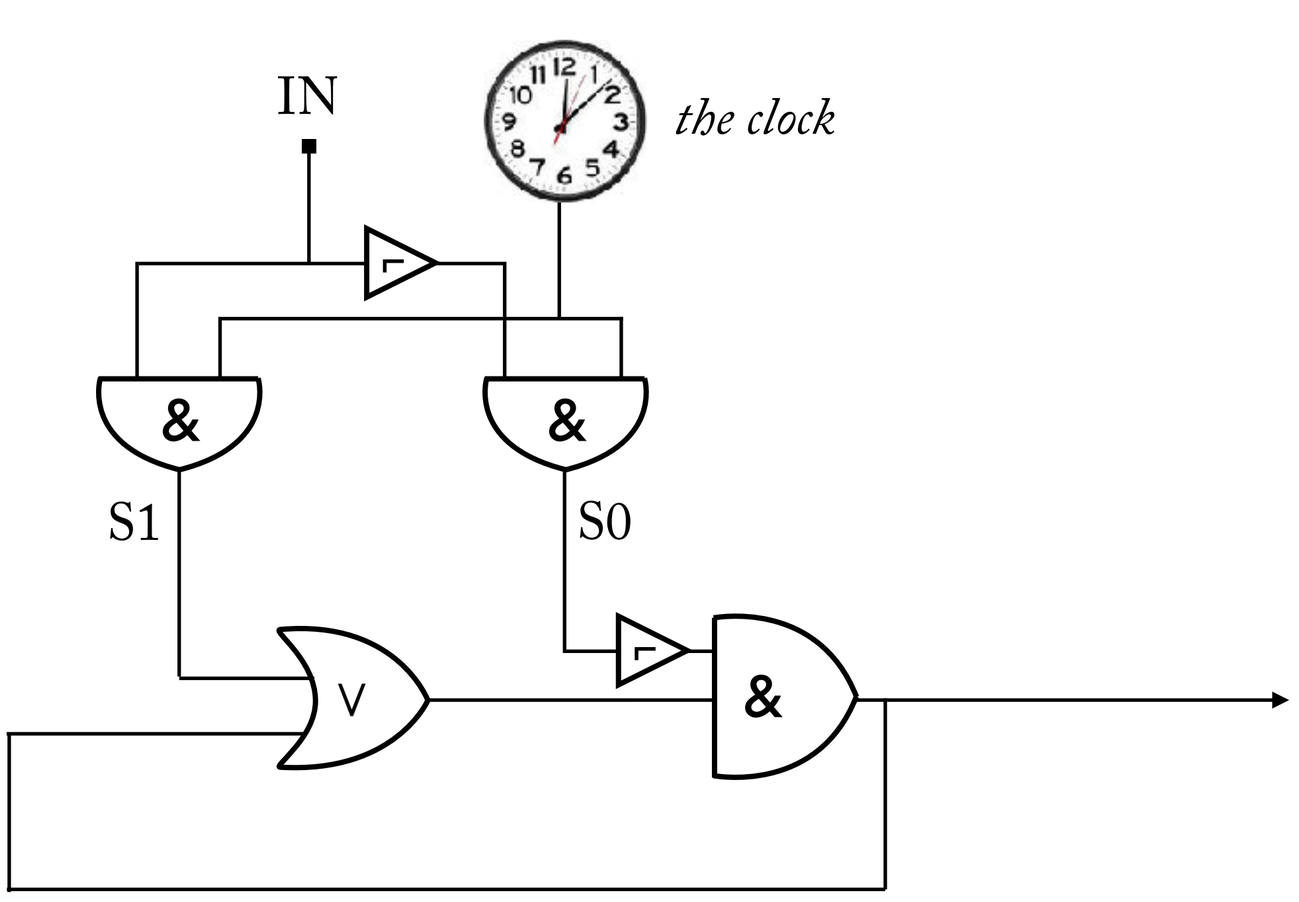
So we are going introduce a control mechanism which we'll call "Hold that Bit" (H). The idea is that you activate H when you want to capture and store the current input bit, but if you are happy with the bit you've got, then leave H off. This circuit is known as a **flip-flop.**



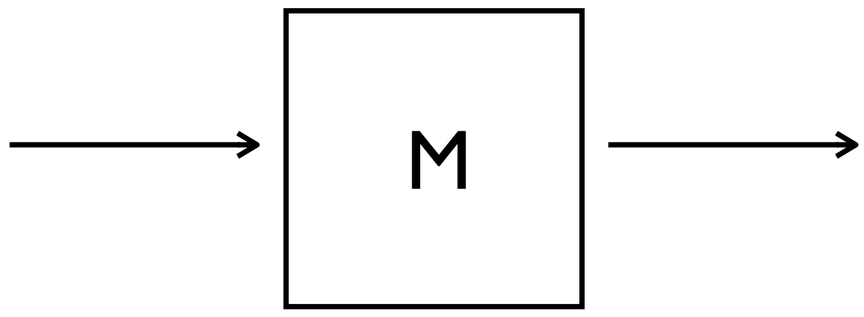
Our memory is like a box. Sending a 1 through H is equivalent to opening the box; whatever the current input bit is, that bit is stored in the box. Sending a 0 to H is equivalent to closing the box; the current input bit is now irrelevant, only what's in the box matters.



There's one last step. After all, when does H get activated? Does the user somehow have to turn on each memory when they want to use it? No. For this we'll use **a clock**. A clock in this sense doesn't *store* the time, instead it marks out **ticks** at regular intervals, like a metronome. We'll connect the clock up directly to H. The idea is that, at each tick, H is activated, and the loop is either set to 1 or set to 0; after each tick, H is deactivated and the loop continues to store and output whatever bit it was last set to.



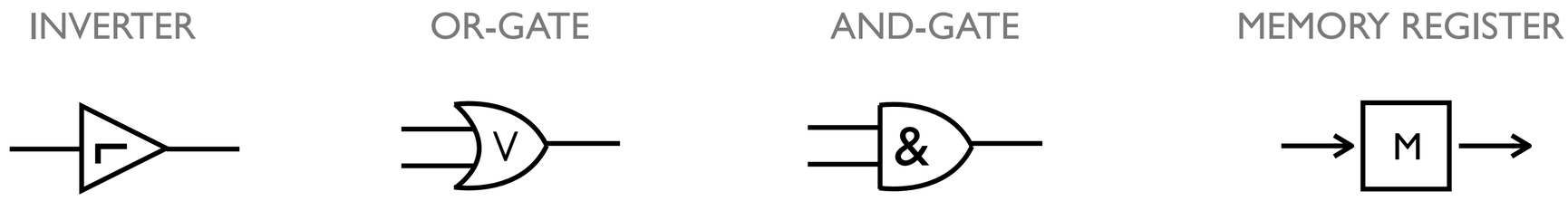
The result is what we'll call a **memory register**: a type of self-contained circuits that can store a single bit. In the future, we'll want to abstract away from all this physical circuitry, in order to focus on the logic. So I'll think of each memory as 1-input 1-output box which outputs a bit at the beginning of each clock cycle and inputs a new bit at the end of each clock cycle. We'll hide the input from the clock, with the assumption that the clock is hooked up in the same way to every memory register.



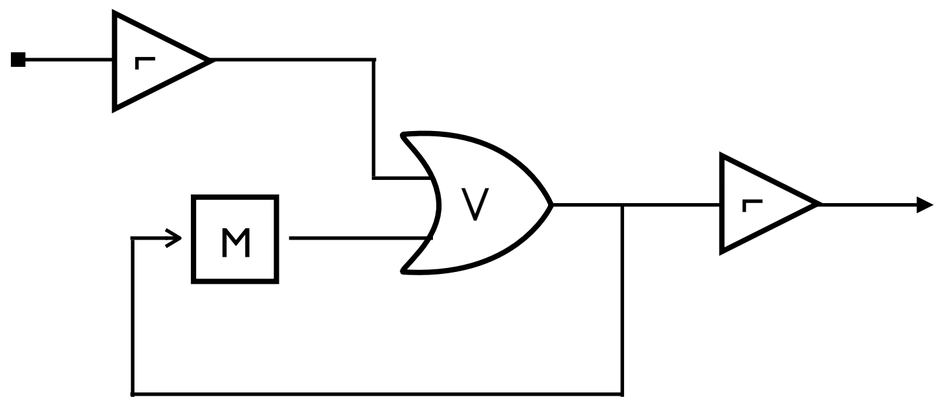
18. Machines in Time: Sequential Circuits

Sequential Circuits = Combinatorial Circuits + Memory

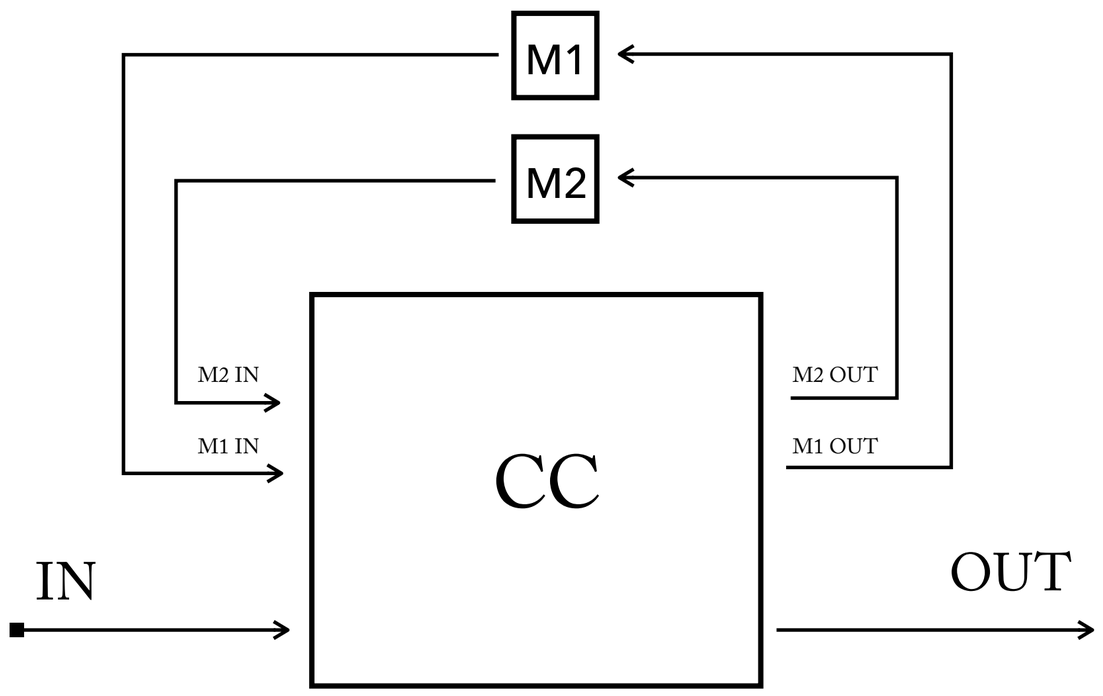
A **sequential circuit** (SC) is a combinatorial circuit combined with a finite number of memory registers. Our circuits now consists of four basic parts.



The structure of a sequential circuit is just like a combinatorial circuit, except that **loops are allowed**, but **only when they are interrupted by a memory register.** Like this:

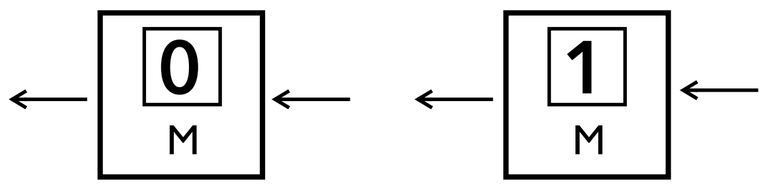


In a **standard layout,** the memory registers are kept separate from the combinatorial circuit. The inputs to the circuit as a whole are aligned with inputs to the CC from memory; the outputs of the circuit as a whole are aligned with the output from the CC to memory. This design that makes the conceptual relationship between CC and memory visually clear. (You don't have to use standard layout when designing circuits.)

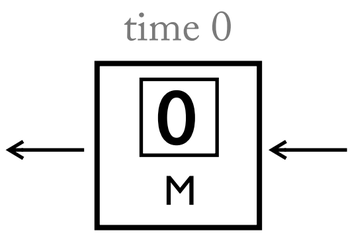


Memory Registers

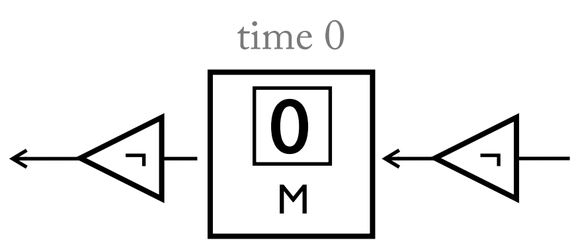
Each memory register can store exactly one 1-bit.



We'll assume our memory registers are **0-first**:
at time 0 they always begins by storing a 0.



But you can build a **functional 1-first** memory with a circuit that uses a 0-fist memory, but simulates the behavior of 1-first memory. It outputs a 1 on the first cycle



The Clock

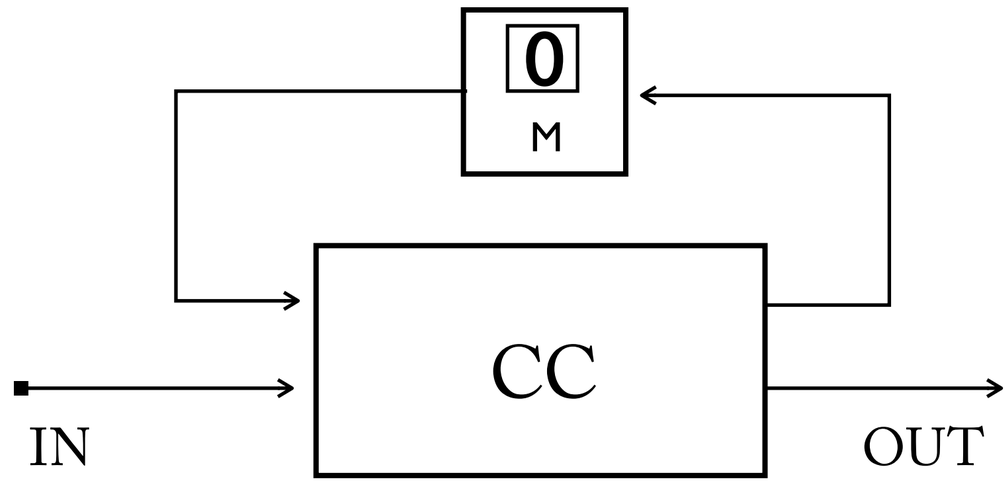
Machine time is governed by an abstract clock, which goes through discreet cycles. At the beginning of each cycle, inputs are read, and output are written or displayed. As before, within a cycle, the machine works instantaneously.



What makes memory registers *memory*— in other words, what makes them carry information forward through time— is that they preserve the bit they carry after each cycle of the clock.

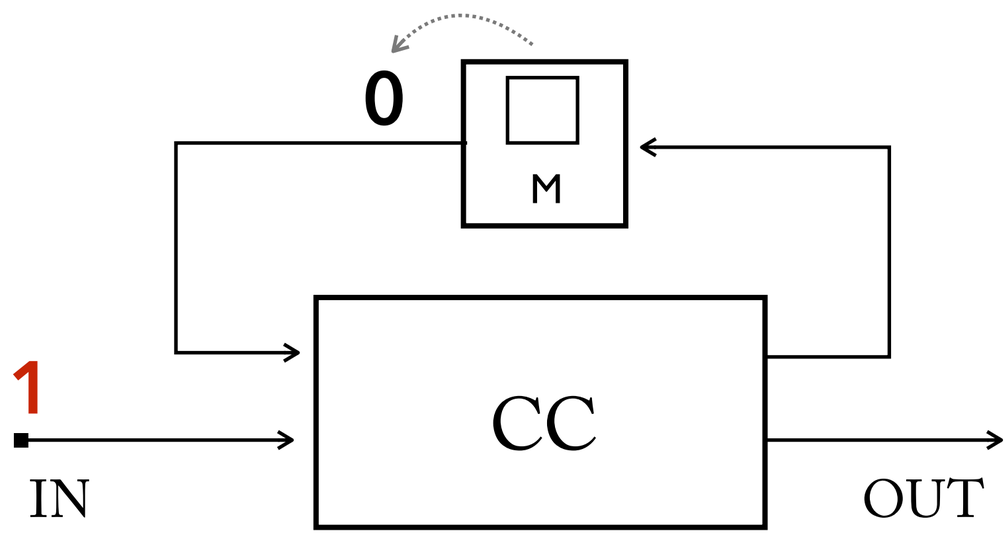
Cycles of a Sequential Circuit

1. In the beginning, the circuit is quiet and a bit is stored in each memory register.



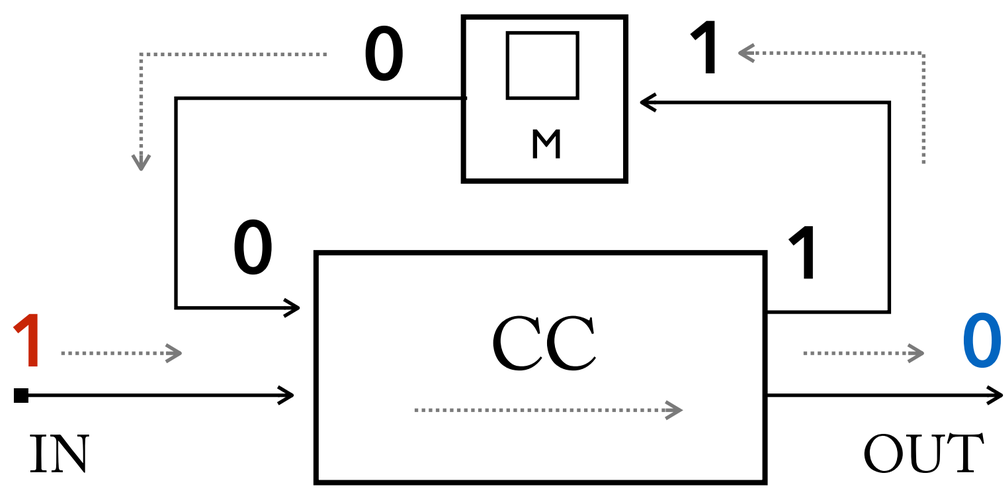
---

2. Input is received from each local input and from each memory register.



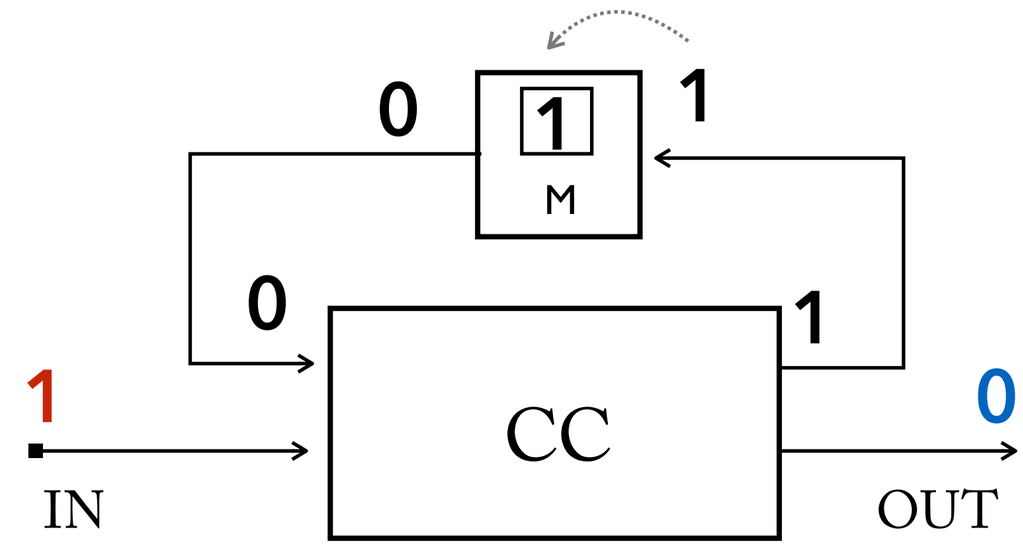
---

3. Run the circuit, produce outputs.



---

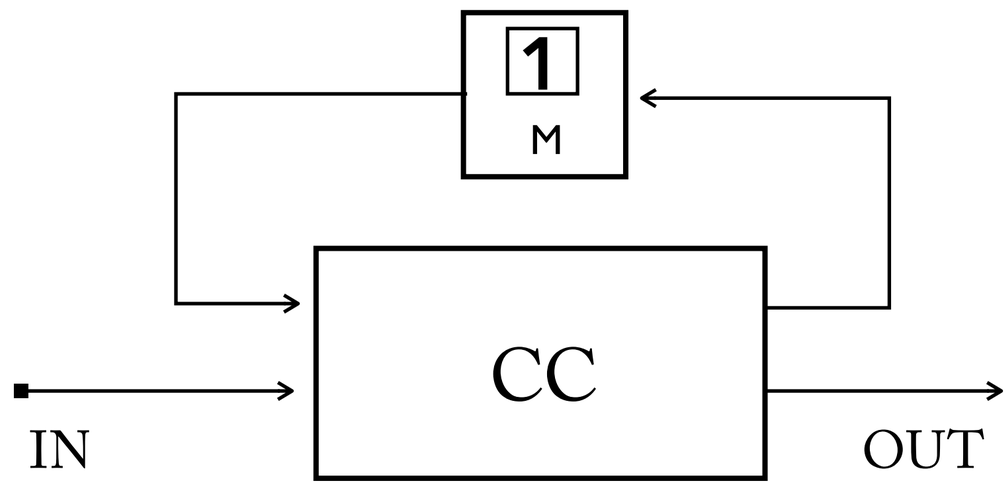
4. Record new bits into memory registers.



---

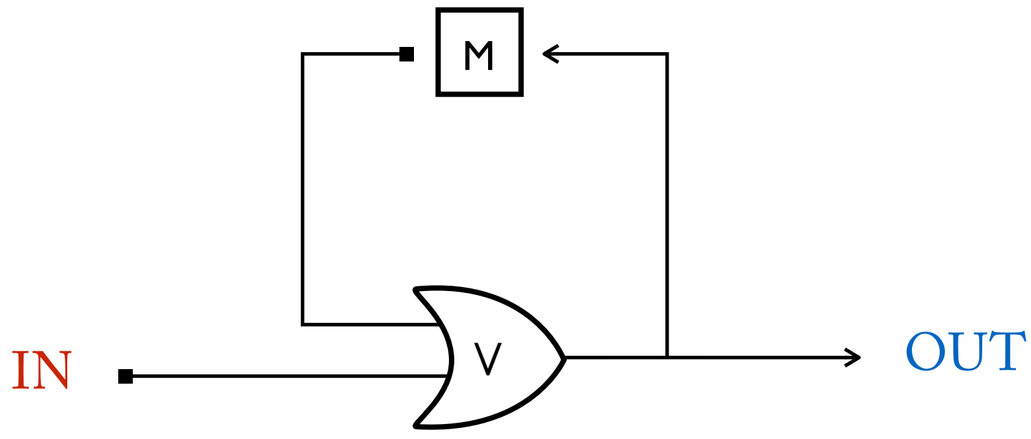
5. Clear the circuit of all information except for the memory circuits.

Initiate the next cycle.



Memoirs of a Sequential Circuit

To keep track of the behavior of a sequential circuit, we'll typically have to keep *two* kinds of tables: one **global input-output table**, and a set of **sequential local input-output tables.**



Global input-output table

IN	OUT
000	000
001	111
010	
100	100
100	
101	111
…	…

Sequential local input-output tables

…	t5	t4	t3	t2	t1	
…	0	0	0	0	0	IN
…	0	0	0	0	0	OUT

…	t5	t4	t3	t2	t1	
…	0	0	0	0	1	IN
…	1	1	1	1	1	OUT

…	t5	t4	t3	t2	t1	
…	0	0	0	1	1	IN
…	1	1	1	1	1	OUT

…	t5	t4	t3	t2	t1	
…	0	0	1	0	1	IN
…	1	1	1	1	1	OUT

What does this circuit *do*? At which point in the sequence does this machine exhibit distinctively sequential history-dependent behavior?

In-Class Problems

Design SC's which solve the following problems.

1. **Temporal AND:**
   Output = 1 iff the current input = 1 **and** the last input = 1.

2. **Temporal OR:**
   Output = 1 iff the current input =1 **or** the last input = 1.

3. **Trip-wire**
   Output = 0 iff there is or ever was an input = 0.

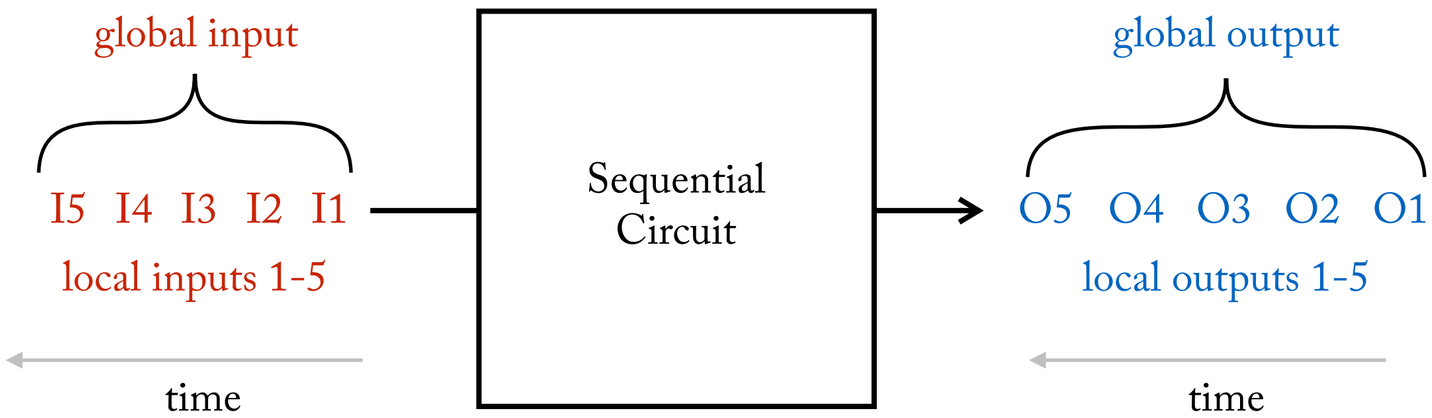
4. **Pattern Finder:**
   Output = 1 as long as the input alternates 1's and 0's (e.g. "101010…"). If the pattern is ever broken, output = 0 forever after.

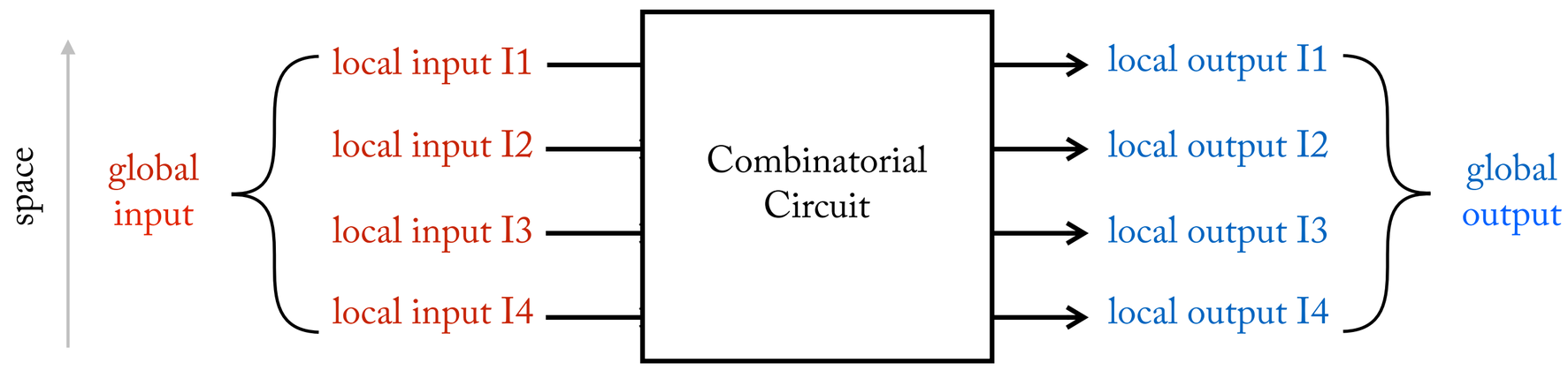
5. **ON/OFF switch:**
   Starts in the OFF state. If you push a button, it goes ON. If you push the button again, it goes OFF. (**Hint:** think in tables.)

19. Computing in Time

Inputs and Outputs

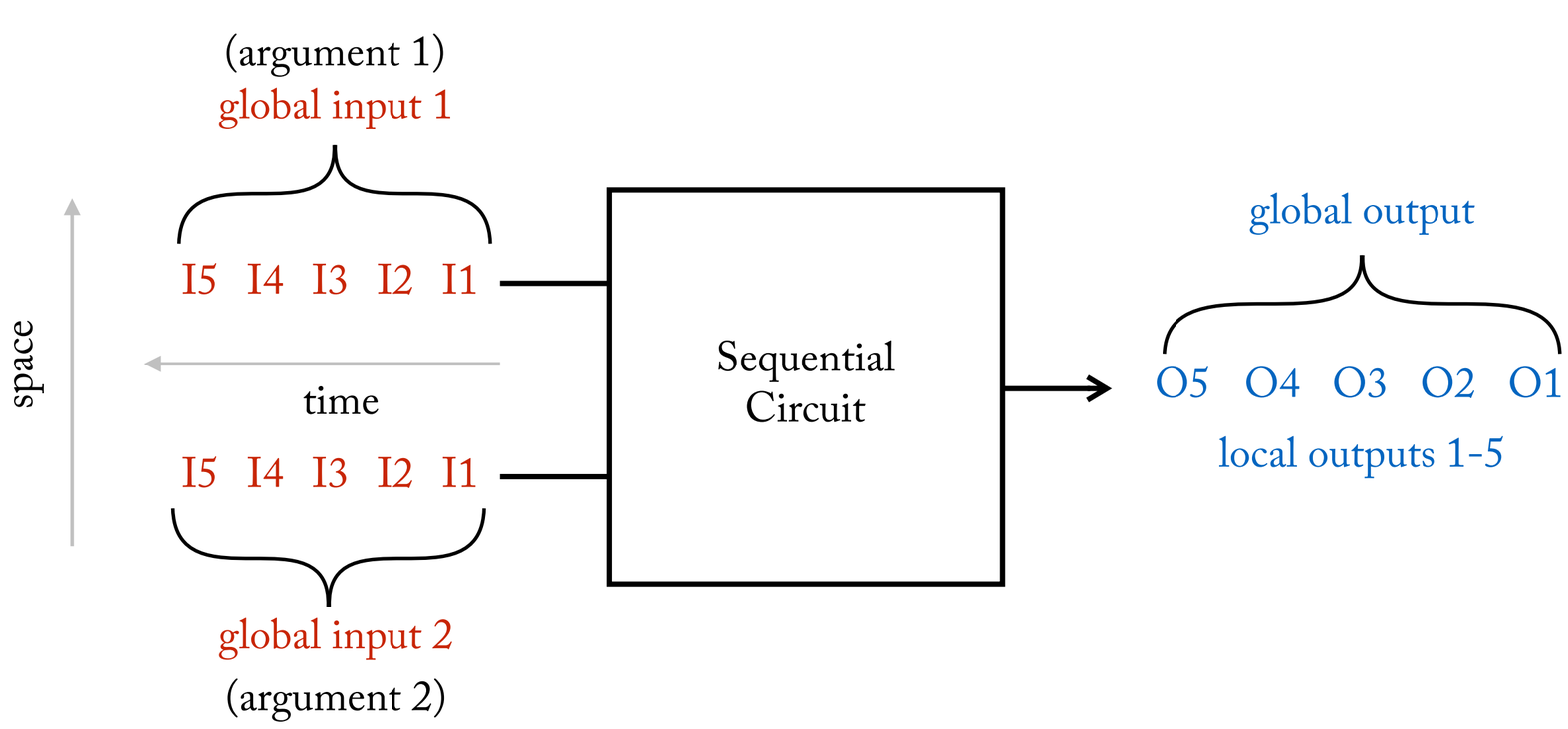
We now introduce the concept of **input at a time**, as well as the overall **input over time**. The input at a time corresponds to an SC's **local input**, the input over time corresponds to an SC's **global input**. So, for SC's, arguments and values are represented by complex symbols over time.



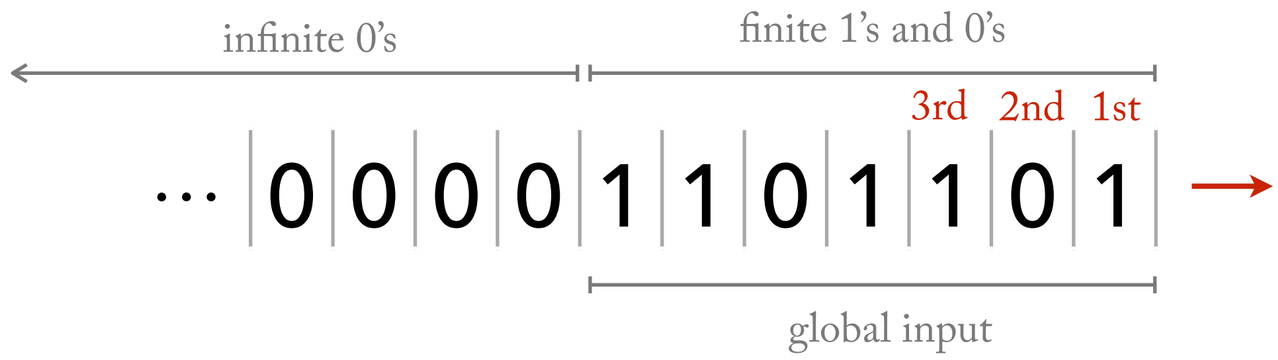


SC's do with time what CC's do with space. The major difference is that space is finite, but time is (for all practical purposes) unlimited.

By the way: SC's can have more than one global input. When performing computation, each argument-place in the computed function corresponds to a distinct global input in the circuit.

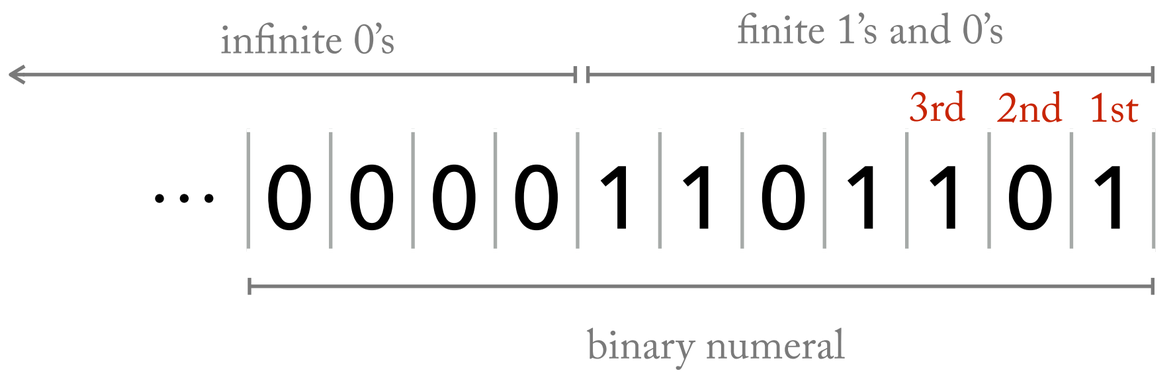


Computing with Sequential Circuits



On any one occasion, we'll imagine that an SC goes on forever. It receives an infinite number of digits over an infinite amount of time. We'll assume that the input always begins with a finite number of 1's and 0's followed by an infinite number of 0's.

When an SC computes a numerical function, the global inputs and outputs are those finite sequences of local inputs/outputs at the point when all remaining local inputs *and* outputs are 0's.



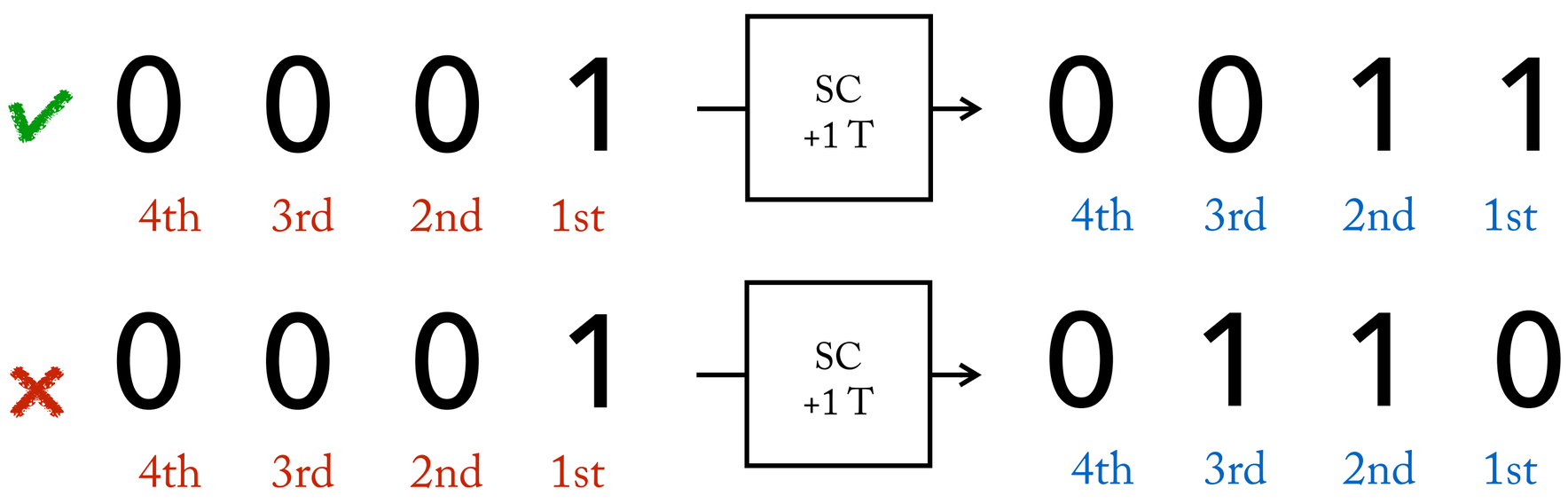
Numerals in Tally:

- Entered as a finite number of consecutive 1's followed by an infinite number of 0's. E.g. 00001, 00011, 00111, 01111, etc.
- 0 will count as a numeral in Tally (representing zero).

Numerals in Binary:

- Made up of a finite number of 0's and 1's followed by an infinite number of 0's.

Computing a function with SC's is just like computing a function with CC's. Only now, the inputs arrive in temporal order, rather than spatial order.



+1 T: One More Time with Feeling….

Remember our temporal input-output table for +1 T. It's a temporal version of the +1 T CC. It always produces a 1 first and then delays every other bit by 1 cycle.

+1 T where x=0

t5	t4	t3	t2	t1	
…	0	0	0	0	IN
…	0	0	0	1	OUT

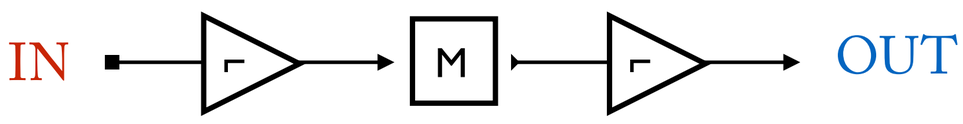
+1 T where x=2

t5	t4	t3	t2	t1	
…	0	0	1	1	IN
…	0	1	1	1	OUT

We can always delay a bit by 1 cycle by introducing a memory in the middle of a wire.



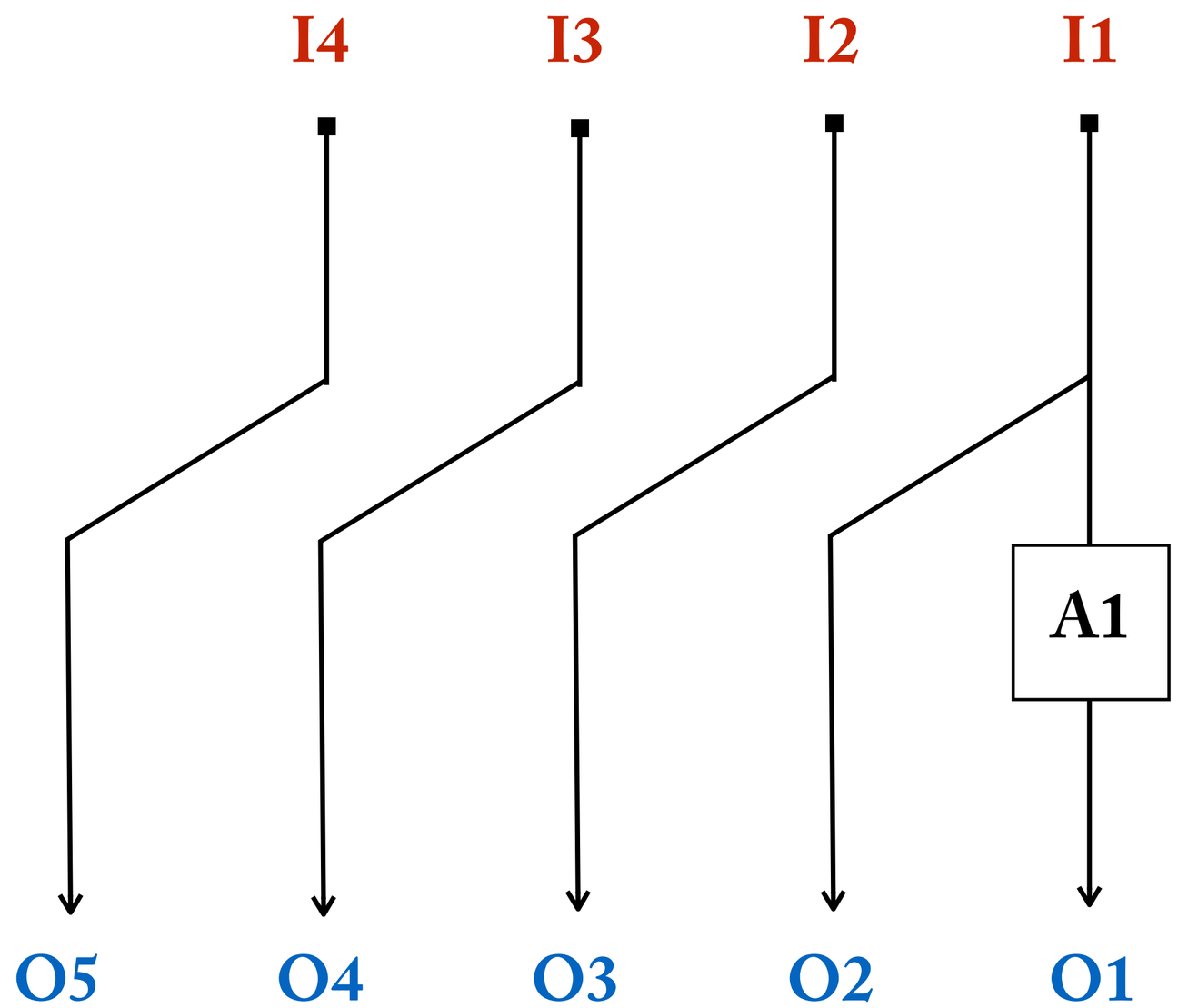
Of course for +1 T, we need the first bit to be a 1. So then it would make sense to use a **1-first memory**, and use *that* to delay the incoming bit.



Voila! +1 T SC

If you squint in just the right way, you should be able to see that our solution for SC +1 T is the same as our solution to CC +1 T.

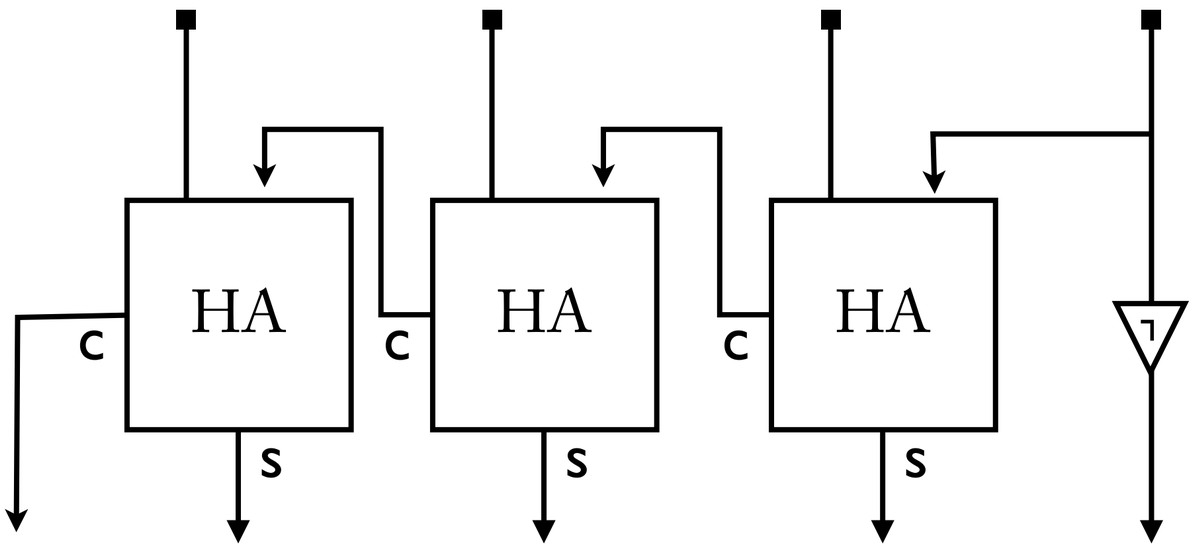
The CC just pushed each bit one position over in *space*. The SC pushes each bit one position over in *time*.



Binary +1

How do you compute Binary +1? Intuitively, by computing +1 on the first digit, outputting the sum, then adding the carry to the second digit, outputting the sum, then adding the carry to the third digit, outputting the sum, and so on…

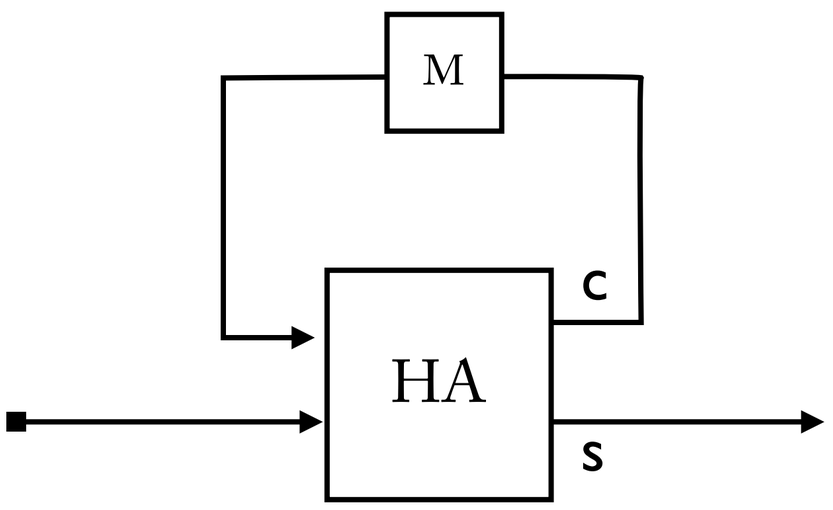
We've already developed a sub circuit for adding a carry to an input, outputting the sum, and computing the next carry. This is the half-adder! Before, we strung half-adder's together *in space*, so that the carry from one was the input to the next.



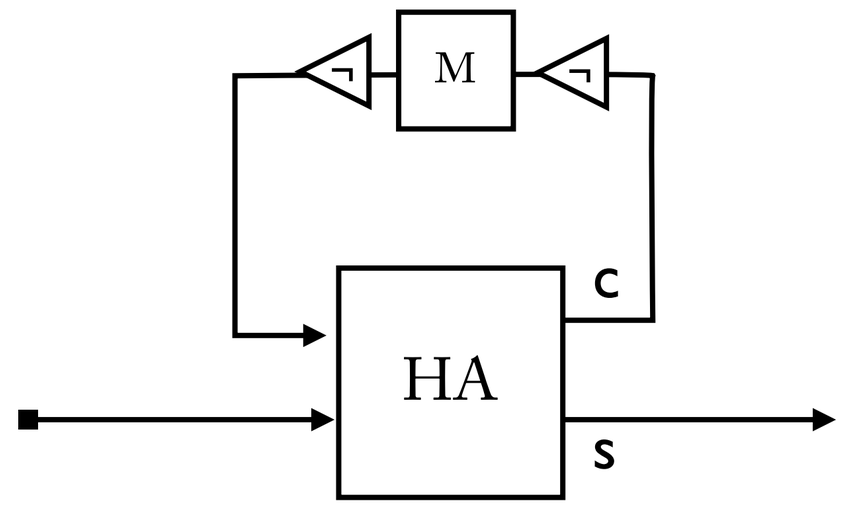
+1 B CC

Now we are going to string half-adder's together *in time*. The idea is that, at each cycle, you store the carry digit in memory, and then use that memory as input on the next cycle. Something like the circuit at right.

The only problem with this is, it doesn't have any mechanism for adding *the first 1*. (Remember how we handled this with inverters for the combinatorial circuit?)



One way to solve this problem would be to start the memory off with a 1 in it— but then have it behave like a normal memory after that. Then the memory will push a 1 out on the first cycle, this will get inputted into the half-adder, and the result will be effectively adding 1 to your binary digit. This is exactly wha the function 1-first memory does. Here you go:



+1 B SC

Prove to yourself that this works! And meditate (deeply) on the relationship between this binary +1 machine, and our old combinatorial binary +1 machine. What one does in space, the other does in time!

In-Class Problems

Design SC's which solve the following problems.

1. **+2 T**

2. **+2 B**
   Challenge: what is the smallest circuit, in terms of numbers of logic gates, that you can think of that computes this function?

3. **2*x* B**

4. **2*x*+1 B**

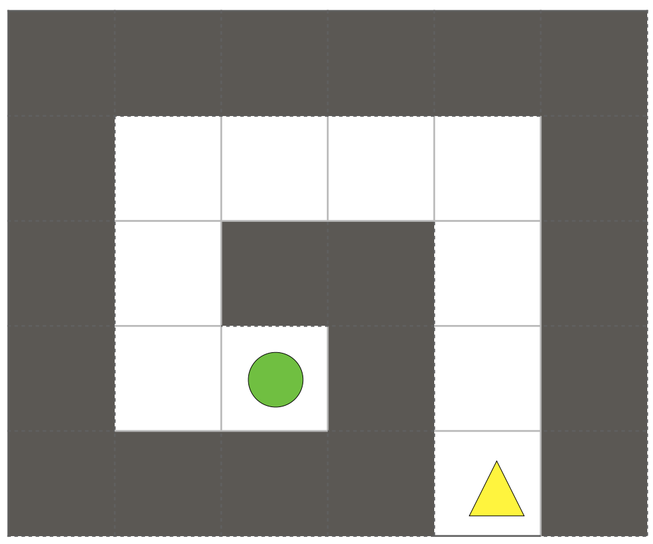
5. **Come up with an SC that computes a two-place function.** Be creative— it doesn't have to be a familiar function; define your own if you like.

6. **Pattern Finder 2**
   Output = 1 as long as current digit and the previous 3 conform to the pattern "1100" or "0011".

20. Computational Power

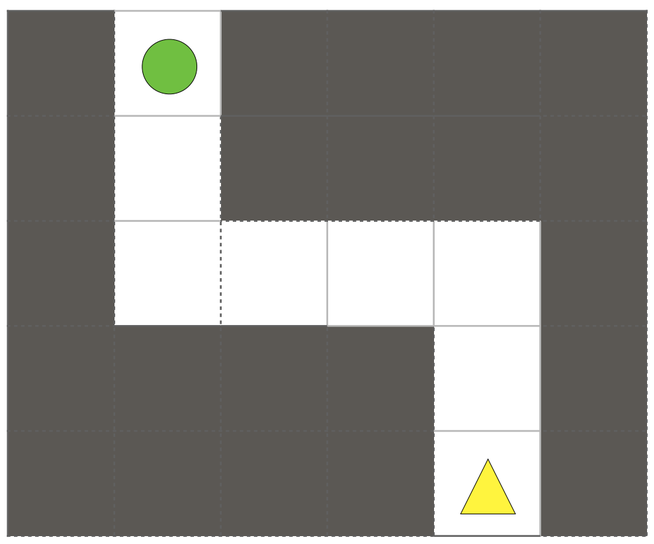
Differences in Computational Power: CCs vs SCs

Spiral



CC computable.
SC computable.

Zig-Zag



Not CC computable.
SC computable.

Differences in Power

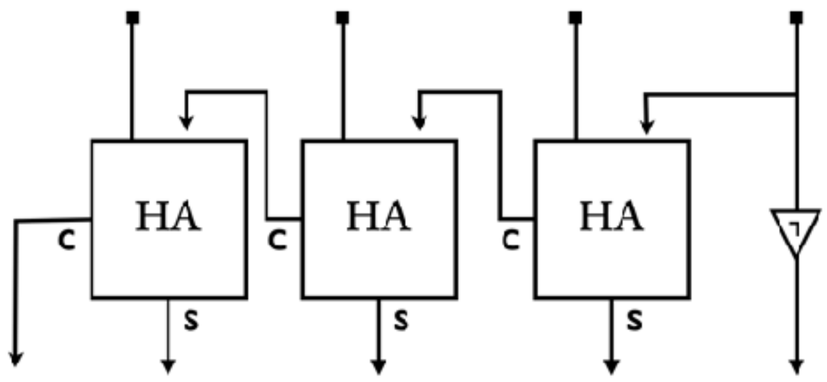
Zig-Zag is not CC computable because it requires different responses to the same input at different times. But CC's are history-independent..

Zig-Zag is SC computable, because SC memory allows the machine to enter into different input-output states over time, depending on its history.

The Memory Moral

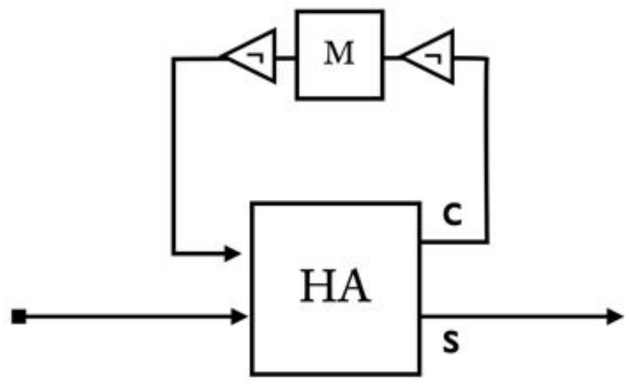
Memory allows a machine to display flexible responses to inputs. As a consequence, memory increases computational power.

Binary +1 [0-15]



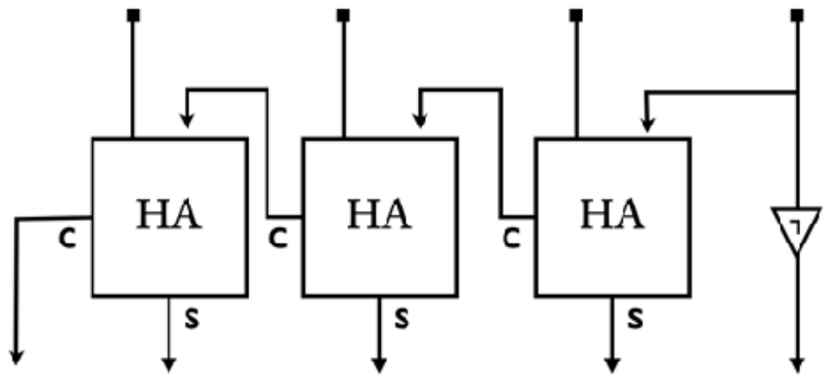
CC computable.
SC computable.

Binary +1



Not CC computable.
SC computable.

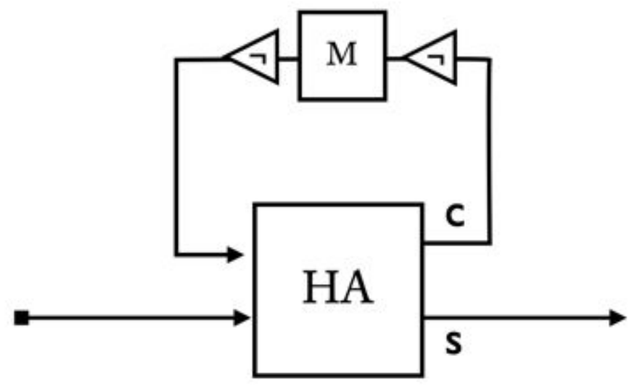
CC Powers and Limitations



CC Binary +1 [0-15]

- CC's have no memory or history responsiveness: they always respond in the same way to the same input.
- The input size for any given CC is fixed by its internal mechanisms.
- CC's can compute *any* bounded function.
- CC's compute *only* bounded functions.
- Practically: the higher the bound, the bigger the machine.

SC Powers and Limitations



SC Binary +1

- SC's contain memory and are history responsive: they can respond in different ways to the same input.
- They input size for any given SC is the infinite (or extendable) dimension of time.
- SC's can compute *any* bounded function.
- SC's compute *some* unbounded functions.
- Practically: for many functions, the higher the bound, the machine stays the same. (For other functions, this isn't true.)

"Infinite ends, finite means."

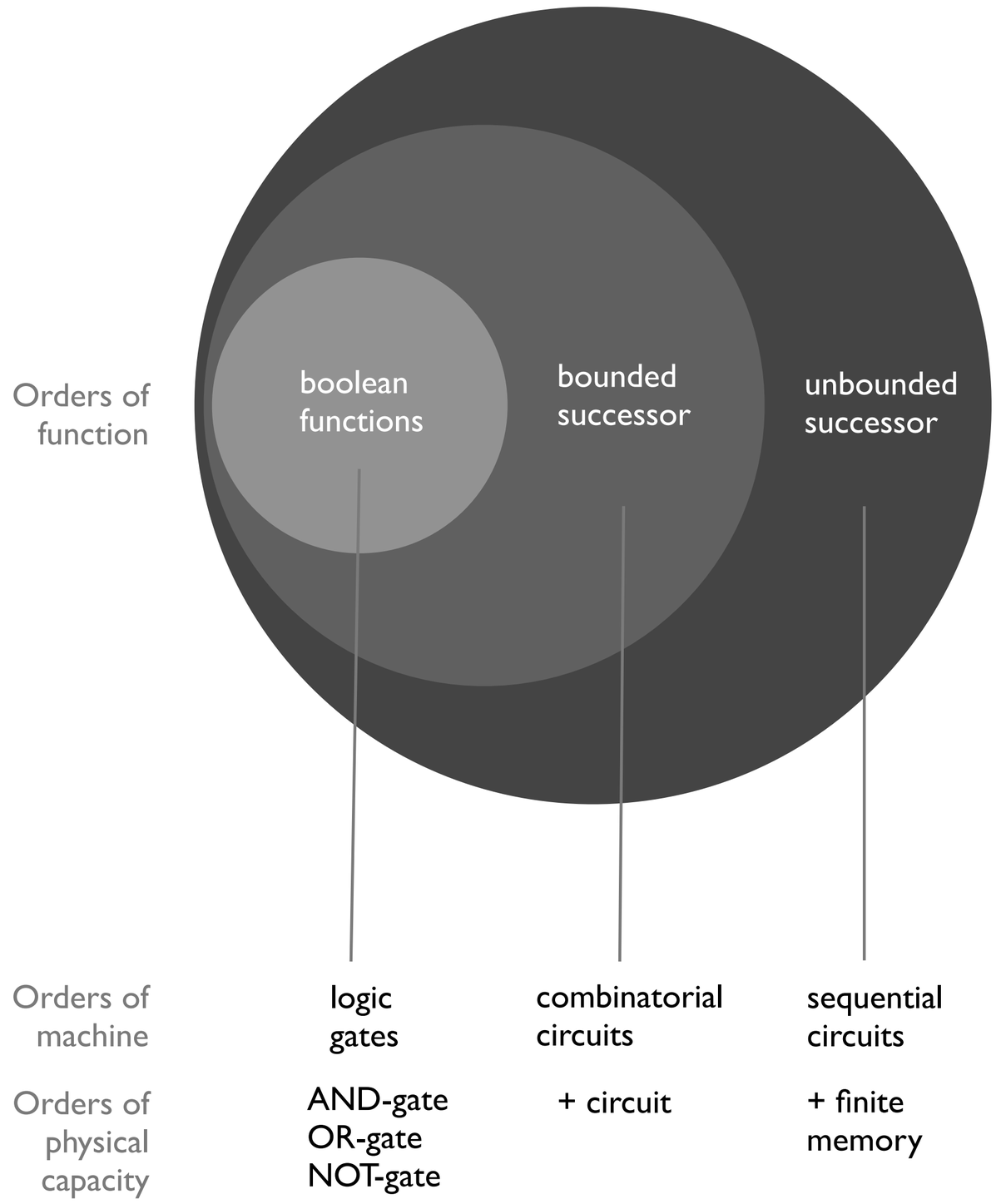
CC Implementation

- Fast.
- Limited in scope.
- The larger the problem, the larger the machine.
- Good for problems with fixed size.
- Compare: perception.

SC Implementation

- Slow.
- Unlimited in scope.
- The larger the problem, the machine stays the same.
- Good for problems with unpredictable size.
- Compare: thought.

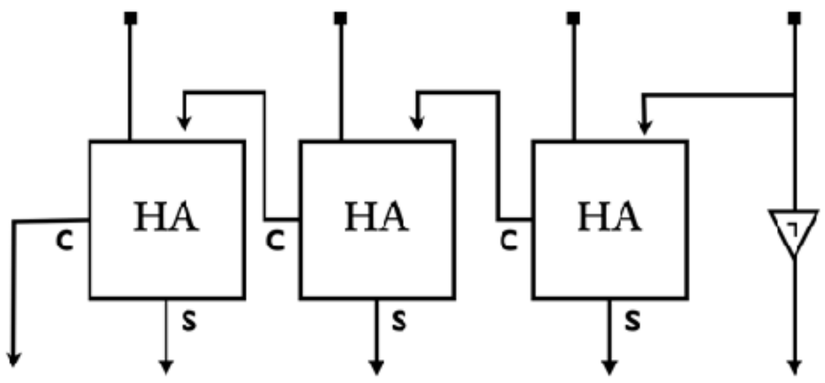
Orders of Computational Power



Definition: Computational Power
A machine M1 has greater computational power than a machine M2 iff

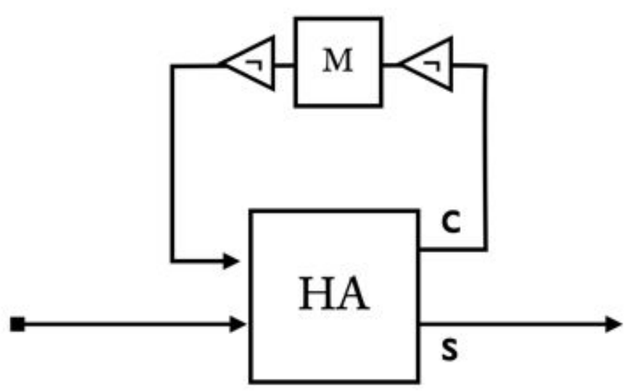
(a) for any function *f* and any code *C*:
if M1 can compute *f* under C, then M2 can compute *f* under C

(b) there some function *f* and code *C* such that:
M1 can compute *f* under *C*, but *M2* cannot compute *f* under *C*.



Binary +1 [0-15]

CC computable.
SC computable.

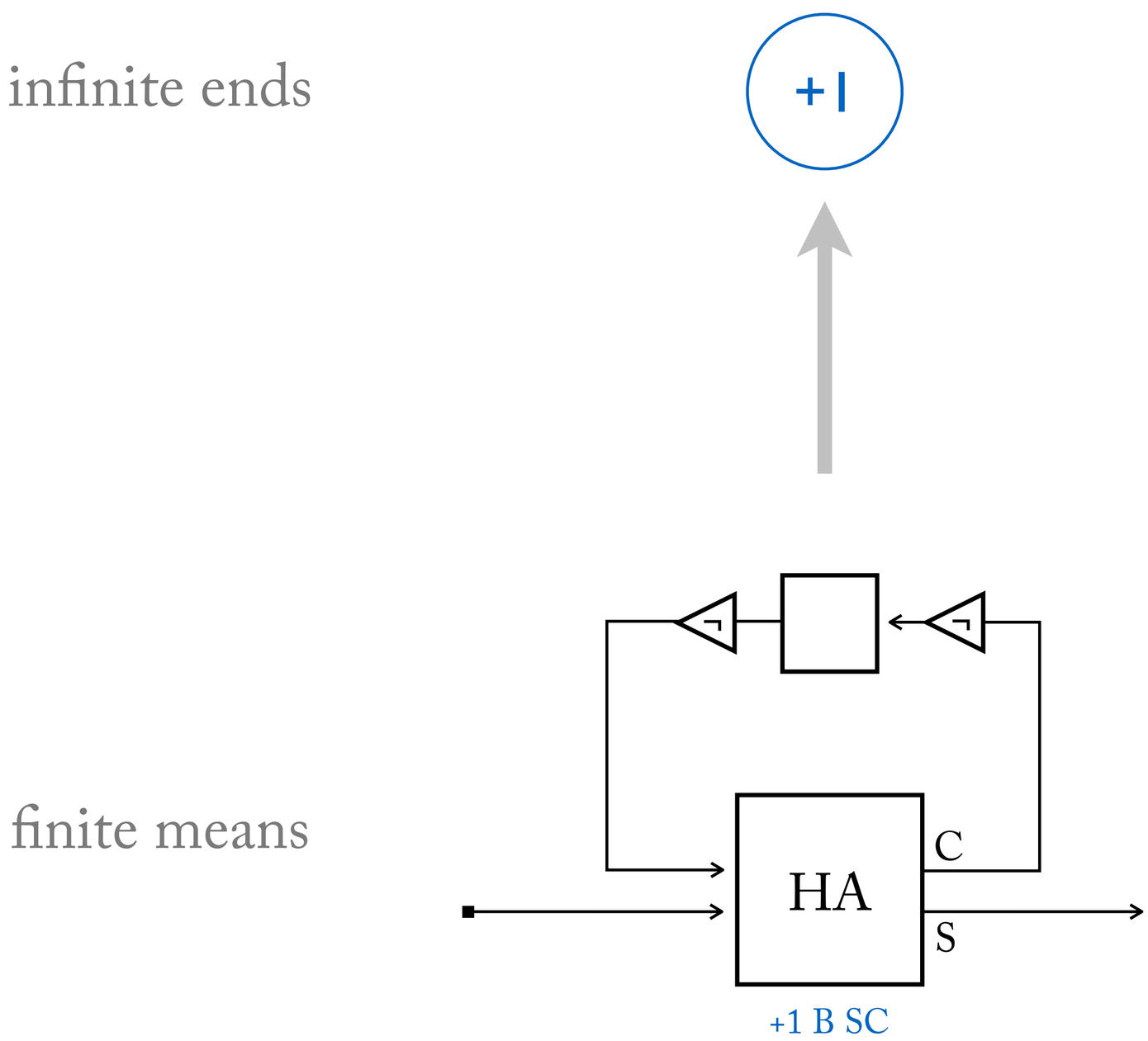


Binary +1

Not CC computable.
SC computable.

Memory and Cognitive Power

> For language is quite peculiarly confronted by an an unending and truly boundless domain, the essence of all that can be thought. It must therefore make infinite employment of finite means, and is able to do so through the power who produces identity of language and thought.
>
> — Humbolt, *On Language*, 1836



What allows SC's to overcome their own finite physical limitations? They exploit the potential infinity of time.

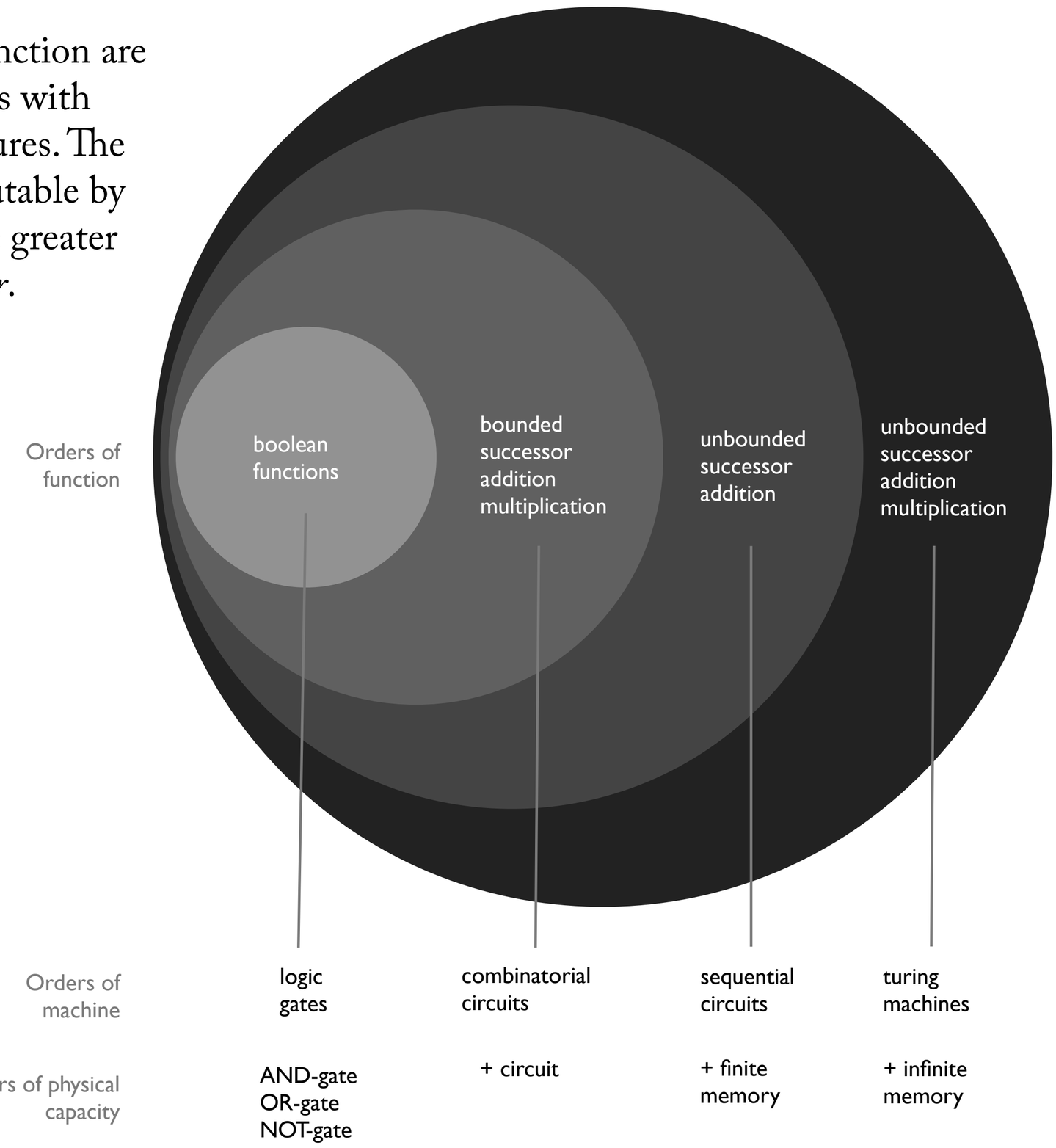
Memory isn't just necessary for achieving infinite potential. It seems to be necessary for *coherence* too.

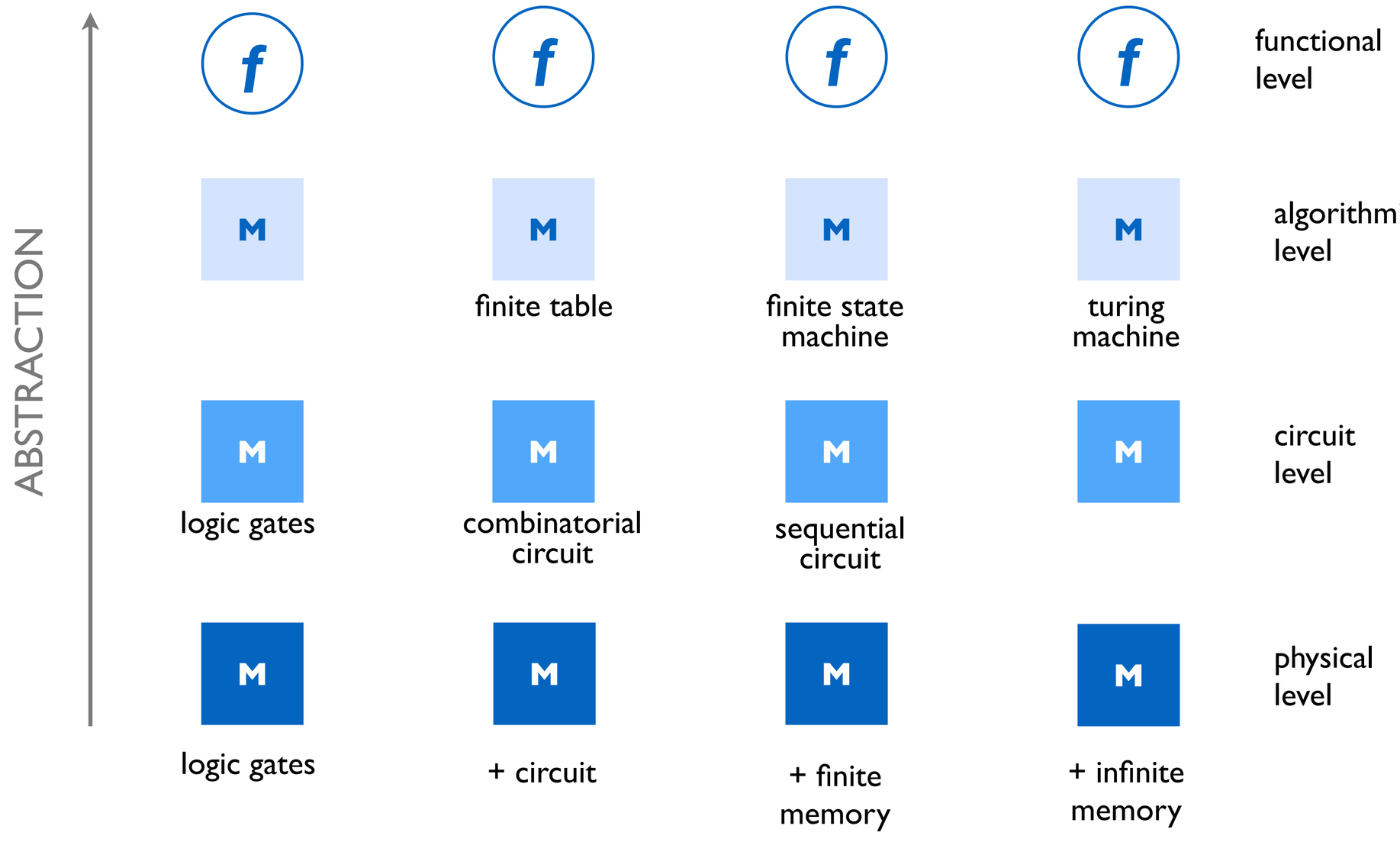
Language. I'd like to buy a carne asada taco.

Argument. I think, therefore I am.

A Map of Power and Abstraction

Different classes of function are computed by machines with different physical features. The more functions computable by a kind of machine, the greater its *computational power*.





21. The Method of State Analysis

Solving SC Computation Problems

The way that circuits with memory work can be unintuitive, and solving SC computation problems is sometimes difficult. What follows is a step-by-step method for turning SC problems into discrete sequences of CC problems. To do this, we'll have to conceptualize SCs in terms of abstract *states*— a concept will examine in greater depth in the coming chapters. Analyzing SC's in terms of states gives you a distinctive understanding of their core mechanism.

Histories and States

Over the course of the history of a machine, it will enter into various fixed states, that often repeat. Consider various sequences for the SC tripwire, and its two characteristic states.

…	t5	t4	t3	t2	t1	
…	0	0	1	0	0	IN
…	1	1	1	0	0	OUT

tripped state un-tripped state

…	t5	t4	t3	t2	t1	
…	0	0	1	1	0	IN
…	1	1	1	1	0	OUT

tripped state un-tripped state

These enduring states can be further analyzed into instantaneous states of the machine that repeat in time.

…	t5	t4	t3	t2	t1	
…	0	0	1	1	0	IN
…	1	1	1	1	0	OUT

tripped state tripped state tripped state tripped state un-tripped state

The idea behind state analysis is to describe a sequential circuit in terms of the characteristic inputs and outputs of a state over time, and the conditions under which one state leads to another, or simply repeats.

Example 1: Trip-Wire

Design a **trip-wire** which sets off an "alarm" (outputs constant 1's) as soon as it reads a 1 for the first time.

Step 1: Sequential input-output table. Start by making a left-to-right table of local inputs and outputs over time. (A top-to-bottom table will also work, but the horizontal layout will be helpful when we get to numerical computation.). You may want to make a few of these, for variation.

One sequence…

…	t5	t4	t3	t2	t1	
…	0	0	0	0	0	IN
…	0	0	0	0	0	OUT

Another sequence…

…	t5	t4	t3	t2	t1	
…	0	0	1	0	0	IN
…	1	1	1	0	0	OUT

Yet another…

…	t5	t4	t3	t2	t1	
…	0	0	1	1	0	IN
…	1	1	1	1	0	OUT

Step 2: State description. Consider how many "states" the circuit has, or overall ways of responding to input and output. (We'll define this idea more precisely later.) In this case, the machine has two states— before the first 1, and after the first 1. Write these down in words.

State 0

If IN=0, OUT= 0. Repeat State 0.

If IN=1, OUT=1. Go to State 1.

State 1

If IN=0, OUT= 1. Repeat State 1.

If IN=1, OUT=1. Repeat State 1.

As a rule, start with State 0, and count up from there. In general, each state description will have the following schematic form:

State X

If IN=A, then OUT= B. Go to state Y.

If IN=C, then OUT=D. Go to State Z.

Step 3: State tables. Next, make construct an input-output table for each state. But when you do this, include extra input and output columns for memory, M_{IN} and M_{OUT}. M_{IN} is the bit coming *in* from memory, and M_{OUT} is to the bit going *out* into memory.

A single memory register has two states (0 and 1). So for the current problem, we can assume that State 0 is defined by a 0 in memory, and State 1 is defined by a 1 in memory. Since M_{IN} is the digit coming in from memory at the beginning of a new cycle, State 0 corresponds to $M_{IN} = 0$, and State 1 corresponds to $M_{IN} = 1$. In the same way, an instruction to move State 0 corresponds to $M_{OUT} = 0$, and an instruction to move State 1 corresponds to $M_{OUT} = 1$.

State 0

If IN=0, OUT= 0. Repeat State 0.

If IN=1, OUT=1. Go to State 1.

State 1

If IN=0, OUT= 1. Repeat State 1.

If IN=1, OUT=1. Repeat State 1.

State 0

M_{IN}	IN	OUT	M_{OUT}
0	0	0	0
0	1	1	1

State 1

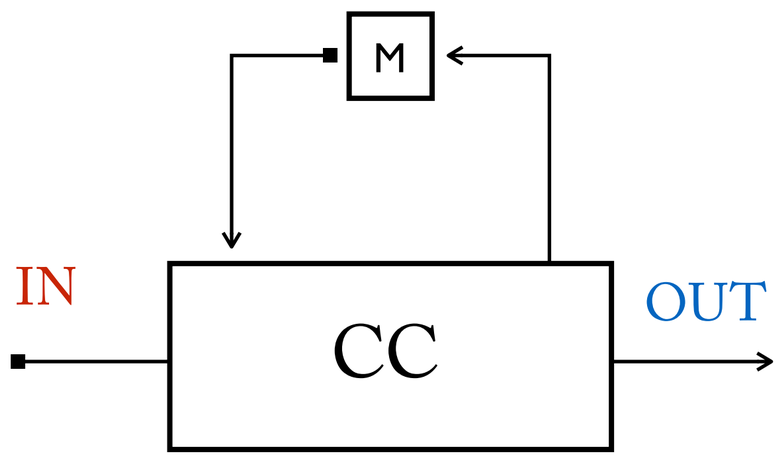
M_{IN}	IN	OUT	M_{OUT}
1	0	1	1
1	1	1	1

Step 4: CC design. Assemble these states into a single input output table, and design a CC which computes this table. And design a CC to solve it.

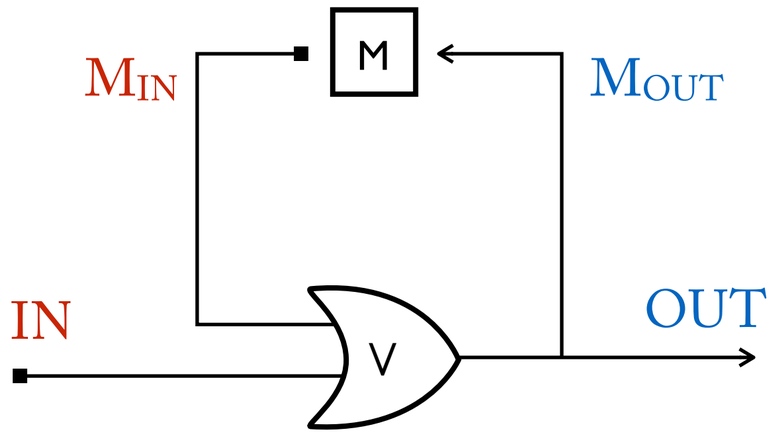
M_{IN}	IN	OUT	M_{OUT}
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1



Step 5: SC design. Convert your CC design into a SC, follwing the schematic layout of an SC:



SC schematic layout



SC tripwire

+1 T: Step by Step

Design an SC which computes +1 T.

Step 1. Start by making a global-input output table, the way you might for any numerical function.

IN	OUT
0	1
1	11
11	111
111	1111
…	…

Remember you can always add 0's to the left, without changing the global inputs and outputs.

IN	OUT
0000	0001
0001	0011
0011	0111
0111	1111
…	…

To get a sense for what's going on here, make a left-to-right table of local inputs and outputs over time. You will want to look at a few different variations. Key cases to always check for are x=0 and x≠0.

+1 T where x=0

t5	t4	t3	t2	t1	
…	0	0	0	0	IN
…	0	0	0	1	OUT

+1 T where x=2

t5	t4	t3	t2	t1	
…	0	0	1	1	IN
…	0	1	1	1	OUT

Step 3 & 4. Divide the machine up into states. This step isn't formulaic; you'll have to think about how the computation intuitively works. Then make an I/O table for each state.

State 0

If IN=0, OUT= 1. Go to State 1.
If IN=1, OUT=1. Repeat State 0.

State 1

If IN=0, OUT= 0. Repeat State 1.
If IN=1… **never happens!**

State 0

I	M→	O	→M
0	0	1	1
1	0	1	0

State 1

I	M→	O	→M
0	1	0	1
1	1		

Now combine these two state descriptions so it looks like a familiar input-output table:

I	M→	O	→M
0	0	1	1
1	0	1	0
0	1	0	1
1	1		

Step 5. Design a circuit which handles the table from Step 4.

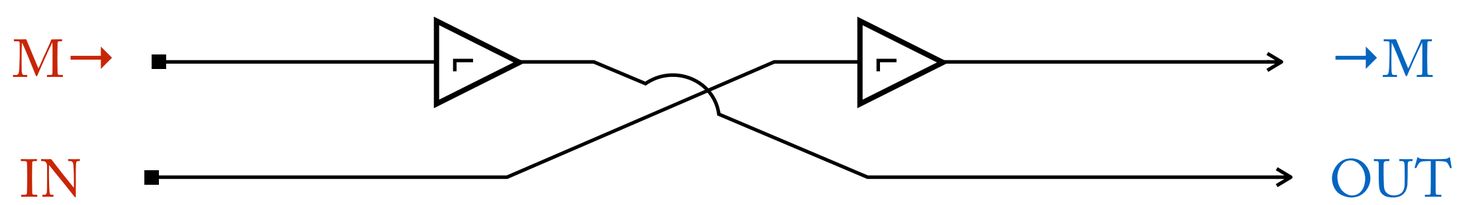
I	M→	O	→M
0	0	1	1
1	0	1	0
0	1	0	1
1	1		

Typically it's useful to solve for the O and →M columns separately. Here I'd note two patterns:

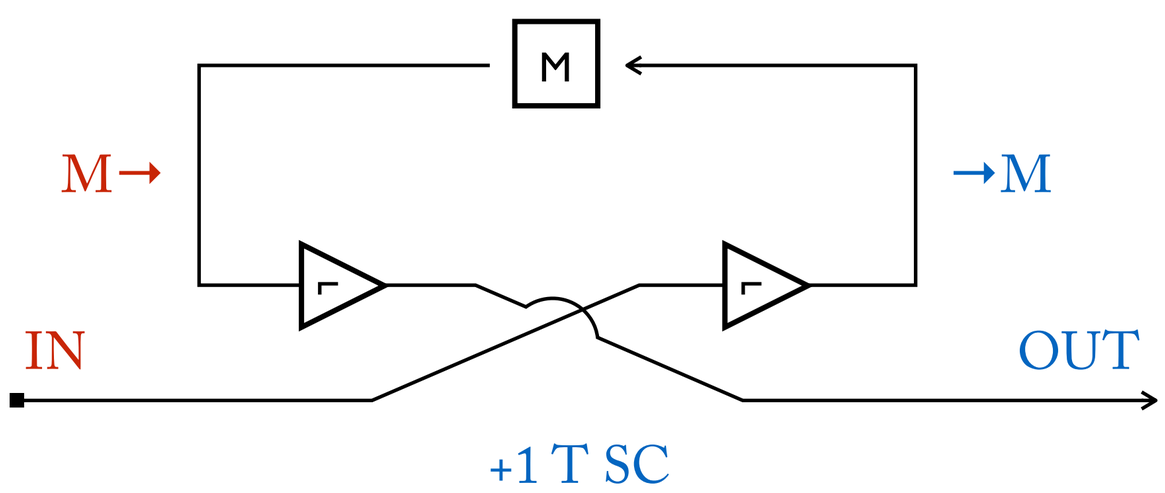
(1) Inverting the M→ value always gives you the O value.

(2) Inverting the I value always gives you the →M value.

We can draw this up as a simple circuit:



We now interpret "M→" as *reading from memory*, and we interpret "→M" as *writing to memory*. The result is our final circuit.



22. State Abstraction

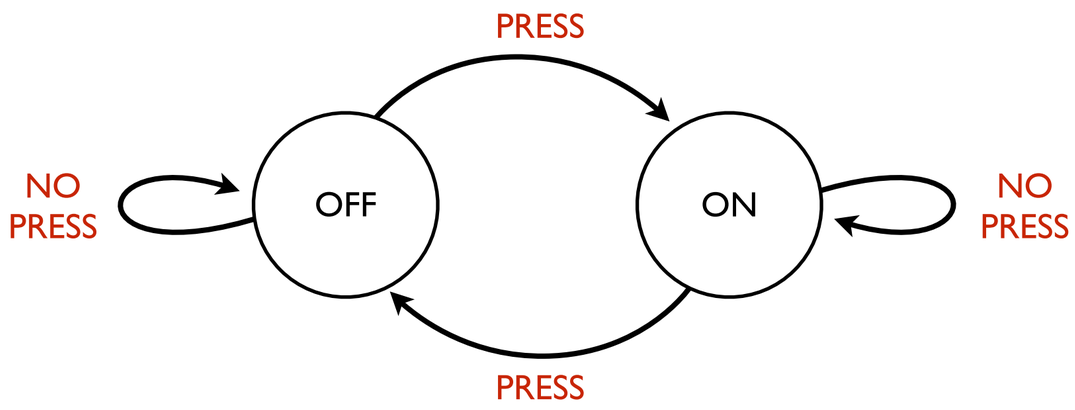
The Concept of State

The concept of a state is at once one of the most profound and one of the most trivial ideas we will explore in this class. On the trivial end, identifying a machine's states is little more than some tedious book-keeping. (And that'll be the focus of this lesson.)

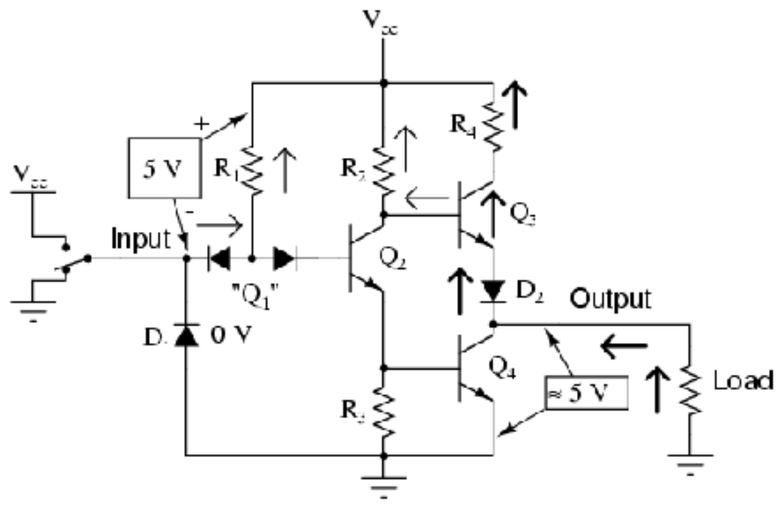
On the profound end, once defined, the concept of a machine state provides us with fundamental insights about the relationship between low-level physics and high-level cognition. The philosopher Hillary Putnam even thought that the concept of machine states solved (or dissolved!) the mind-body problem. You'll have to judge for yourself.

A **state** is the overall configuration of a given system. Given an input, a system's state determines its current output and what state it will be in next. **State abstraction** leaves aside particular details of circuitry, and describes system purely in terms of inputs, outputs, and state transitions.

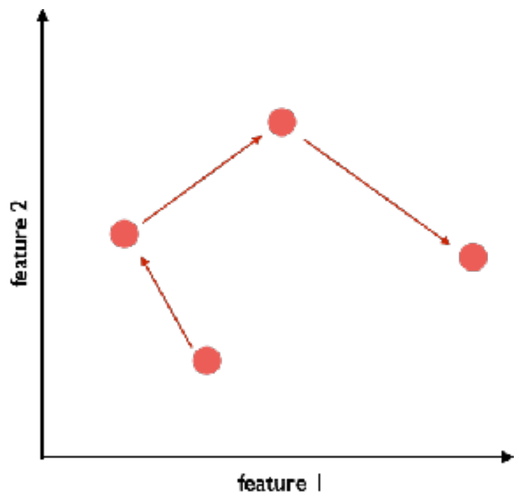
For example, here is a diagram which gives the state abstraction of the on-off switch. It describes the machine entirely in terms of its overall dispositions to produce inputs in response to outputs.



ON/OFF Switch



Whereas a machine as a complicated internal configuration at any given point…

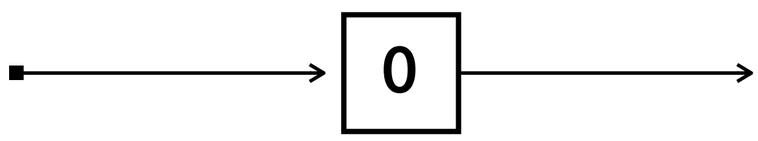


We can imagine a **state space** of all possible internal configurations. Each state is a point in this space.

States of an SC

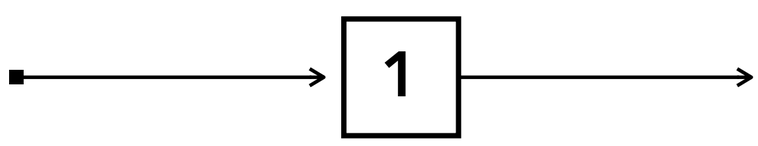
In the case of sequential circuits, a state is the total "input-output disposition" of the machine at a given moment. It includes how the machine will respond to all possible inputs at a time. It also includes what states the current state will lead to. It **doesn't** include the actual circuity of the machine.

The total input-output disposition of a machine is determined by the internal circuitry of the machine and the configuration of its memory registers at a time. All machines start in the "initial state," where every memory register is set to 0. This determines one set of responses to possible inputs. Other settings of the memory registers result in different responses to possible inputs, hence different states.



State 0

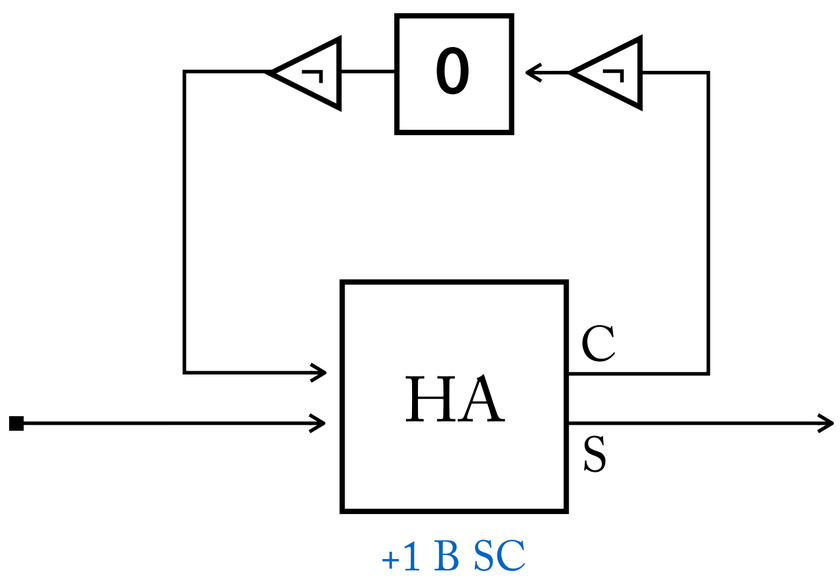
IN	OUT
1	0
0	0



State 1

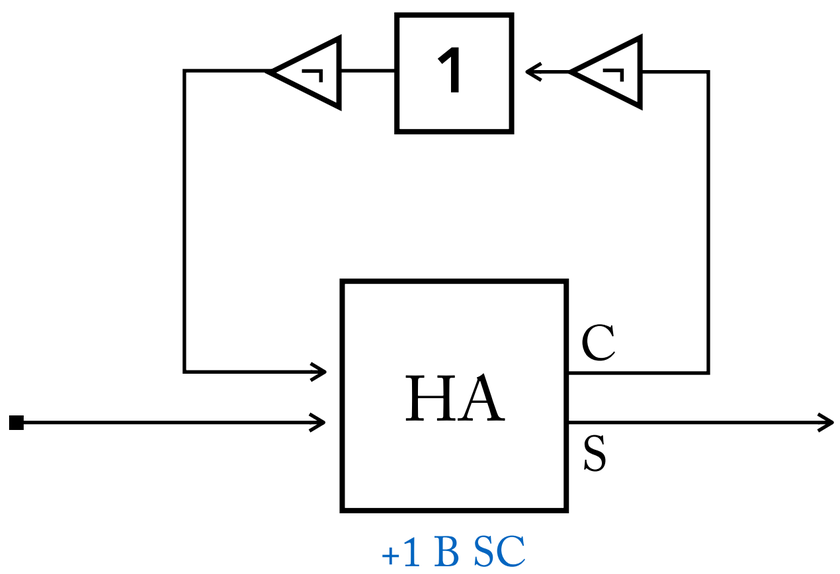
IN	OUT
1	1
0	1

Here's a more interesting example:



State 0

IN	OUT
1	0
0	1



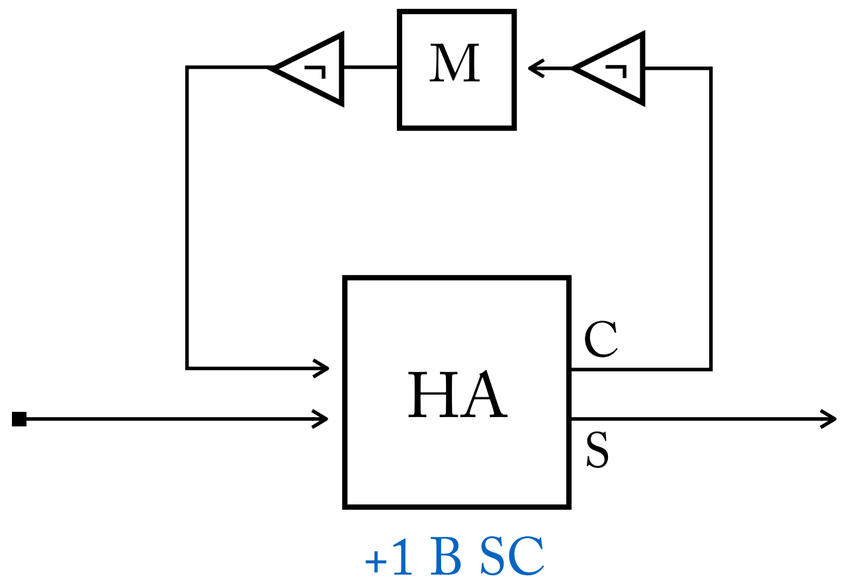
State 1

IN	OUT
1	1
0	0

But there is more to a state that just input and output. There is also its **relation to other states**…

State Tables

Step 0
Design a sequential circuit which computes a specific function.



Step 1
Make a table of all possible inputs from memory and inputs from the user.

If there are more memory registers, or more inputs from the user, there will be more rows in the table.

M→	IN	OUT	→M
0	0	1	1
0	1	0	0
1	0	0	1
1	1	1	1

Step 2
Fill out the table with all of the outputs to the user and all of the outputs to memory.

Step 3
Add a column at the left for "**current state**" and a column on the right for "**next state**".

Think of the input from M→ as describing the current *total memory configuration* of the machine.

CURRENT STATE	M→	IN	OUT	→M	NEXT STATE
S_0	0	0	1	1	A
S_0	0	1	0	0	S_0
A	1	0	0	1	A
A	1	1	1	1	A

Step 4
For each distinct possible configuration of memory, enter a distinct state name in the "**current state**" column. For the configuration where all memories are set to 0, that state should be called S_0. After that you can call the states anything you like.

Step 5
Using the state names chosen in Step 4, fill out the "**next state**" column. The value of →M determines the next value of M→, which determines the identity of the next state.

Step 6
Merge the cells in the "**current state**" column that correspond to the same state.

CURRENT STATE	M→	IN	OUT	→M	NEXT STATE
S_0	0	0	1	1	A
	0	1	0	0	S_0
A	1	0	0	1	A
	1	1	1	1	A

Step 7
Erase the memory columns.

CURRENT STATE	IN	OUT	NEXT STATE
S_0	0	1	A
	1	0	S_0
A	0	0	A
	1	1	A

… and you've got yourself a **state table**!

Step 8 (Optional)
Eliminate unused states: eliminate any state that either:
(1) cannot be reached by finite steps, beginning with S_0;
(2) will not be reached by finite steps, beginning with S_0, given the representational code (binary, tally) intended for this machine.

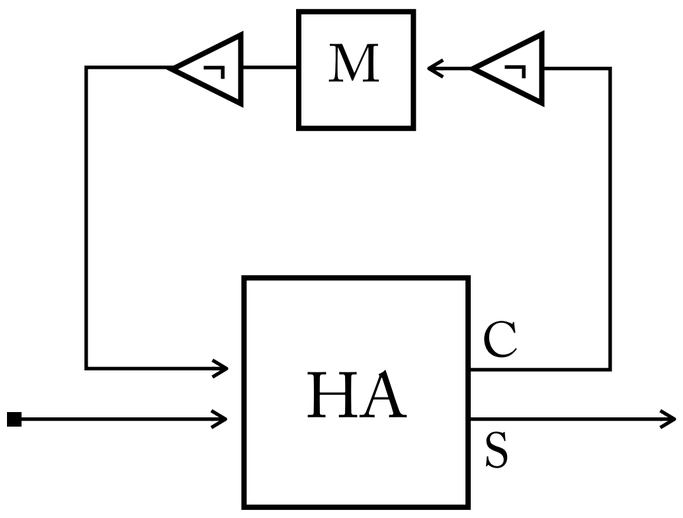
Note: this step is not *strictly* required, but it is best practice.

For purposes of illustration, here is a table for a machine with two memory registers, before the memory columns have been erased.

STATE	M1	M2	IN	OUT	NEXT
S_0	0	0	1	1	S_0
			0	1	A
A	0	1	1	0	S_0
			0	1	C
B	1	0	1	0	B
			0	0	C
B	1	1	1	1	C
			0	1	S_0

The grey row is eliminated by Step 8(1).

From State Tables to State Diagrams



+1 B SC

+1 B SC determines the following state table.

STATE	IN	OUT	NEXT
S_0	0	1	A
	1	0	S_0
A	0	0	A
	1	1	A

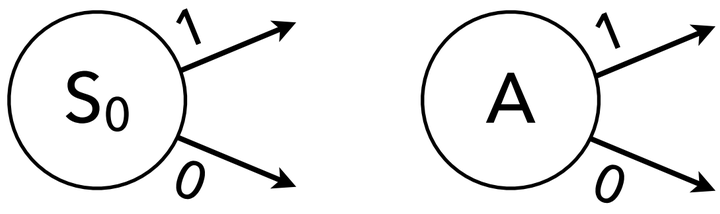
+1 B State Table

To construct a state diagram from a state table…

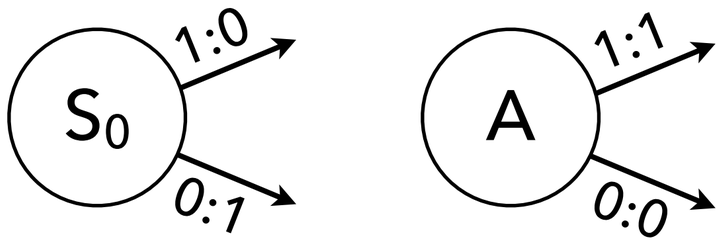
Step 1
For each state of the machine draw a circle, and label it with the name of the state.



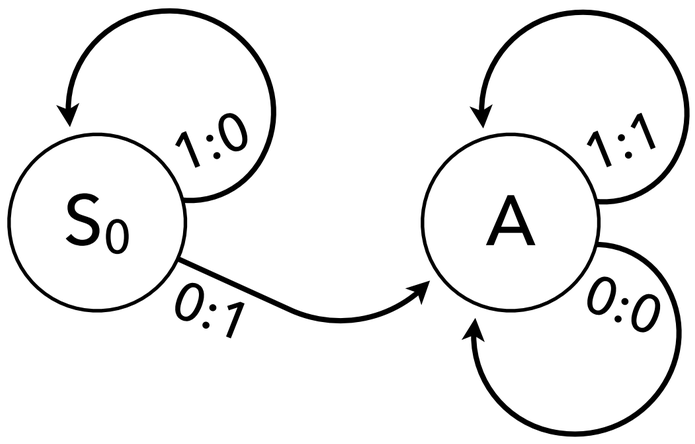
Step 2
For each circle draw up to two arrows exiting it, one labeled "1", the other "0" corresponding to the two possible inputs for that state.



Step 3
For each arrow, add the notation "input : output", corresponding to the input and output associated with the given state transition.



Step 4
For those inputs which lead to a next state, connect the arrow to that state.

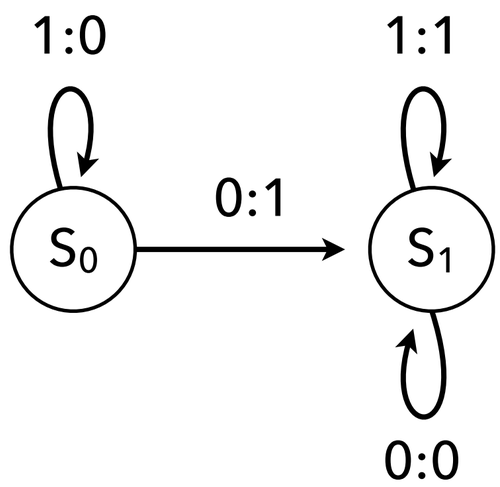


Note: you'll often find diagrams with some of the I/O arrows omitted because they don't arise in computation for a given problem.

STATE	IN	OUT	NEXT
S_0	0	1	A
	1	0	S_0
A	0	0	A
	1	1	A

+1 B State Table

equivalent descriptions



+1 B State Diagram

23. The Concept of Abstraction

Abstraction, the Very Idea

Abstraction is the transition from a more specific (**low level**) description of some system to a more general (**high level**) description of that system . Abstraction always involves both a loss of information and a gain in generality. But abstractions are not arbitrary deletions of detail. Instead, they capture **"real patterns"** in lower-level phenomena.

For example, consider the following two systems made up of circles.



We can describe each system in terms of the relative location of colored **circles**. Or we can describe each as conforming to the shape of a **square**.



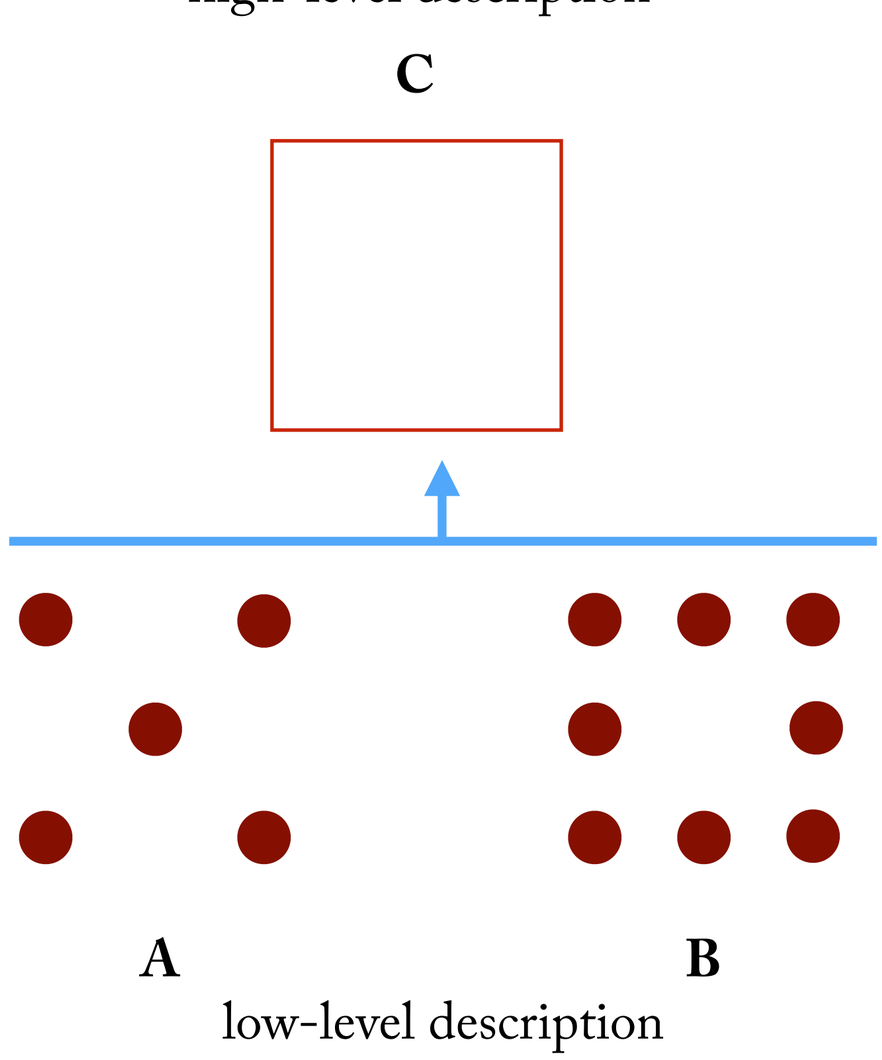
The relationship between the lower level and higher level descriptions here has many of the signature feature of abstraction in general.

Multiple realizability is the hallmark of abstraction. A description X is multiply realizable iff there is more than one (more) concrete description that can realize X. C is realized by both A and B.

Loss of detail. The high-level description C leaves out the composition and distribution of the circles that make up A and B.

Addition of deep structure. C explicitly introduces a level of structure, a "real pattern," which is present in both A and B, but explicit in neither.

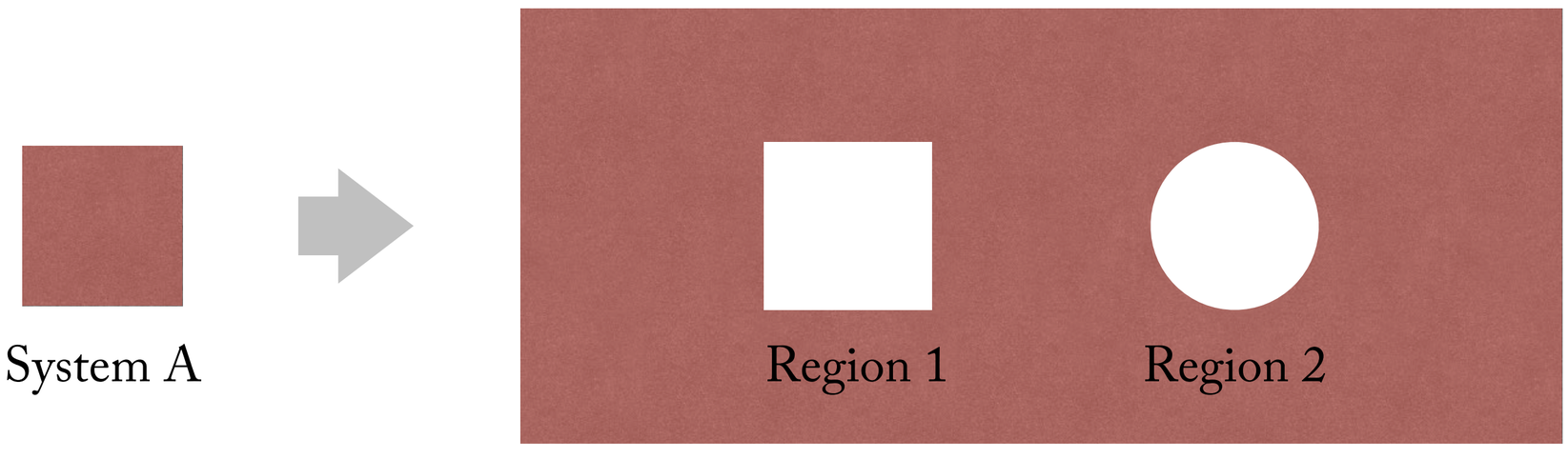
Autonomy. We can understand and reason about a system as an instance of C, without knowledge of whether it is A or B.



Abstraction as Explanation

Putnam's puzzle (from "Philosophy and our Mental Life," 1975):

> "Suppose we have a very simple physical system - a board in which there are two holes, a circle one inch in diameter and a square one inch high, and a cubical peg one-sixteenth of an inch less than one inch high. We have the following very simple fact to explain:
>
> *the peg passes through the square hole, and it does not pass through the round hole.*" (p. 295)



Consider two possible explanations of this fact.

> **Explanation 1:** microphysical deduction.
> System A is rigid lattice of atoms where atom a_1 is in position k_1 with features f_1, a_2 is in position k_2 with features f_2, … (and so on for every atom in A). It follows from the laws of quantum mechanics that, among all possible trajectories of A through space-time, A passes through R1 in some, but never passes through R2
>
> **Explanation 2:** geometrical deduction.
> A is a square with sides just under 1 inch across, and diagonal just under $\sqrt{2}$ inches. Since R1 is a square shape with sides 1 inch across, A passes through R1. Since R2 is 1 inch across at its widest point, and A is $\sqrt{2}$ inches across at its widest point, and $\sqrt{2}>1$, A does not pass through R2.
>
> **Verdict:** clearly E2 is the superior explanation; it's not clear that E1 is an explanation at all. But why? (p. 295)

In defense of the high-level explanation: **relevance.**

- "In this explanation certain *relevant structural features of the situation* are brought out."
- "*The ultimate constituents don't matter…only the higher level structure matters.*"
- "The explanation at the higher level brings out the relevant geometrical relationships. The lower level explanation conceals those laws. Also notice that the higher level explanation applies to a much more interesting class of systems."

In defense of the high-level explanation: **generality.**

- "The same explanation will go in any world (whatever the microstructure) in which those *higher level structural features* are present. In that sense *this explanation is autonomous*."
- "The higher level explanation is far more general, which is why it is *explanatory*."
- "An explanation is superior if it is more general."

The Moral: abstraction plays an essential role in any successful scientific explanation.

Functional Abstraction

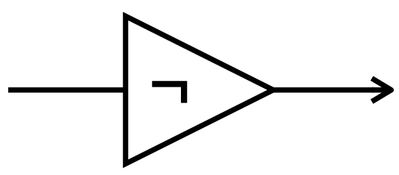
There are many different kinds of abstraction. We may describe a planet as an irregularly shaped solid, as a ball, or as point. We may describe a person as loose affiliation of particles, as a coordinated system of cells, as an animal, or as a rational agent.

In cognitive science and computational theory, **functional abstraction** dominates. This is a kind of abstraction that focuses in on a system's *function* or *purpose*, and abstracts away from details of how it achieves that function. Still there are many kinds of functional abstraction. We've already encountered several!

Functional Abstraction 1: Input-Output Abstraction

Logic gates themselves can be thought of as functional abstractions which make types of inputs and outputs relevant and let everything else fade away.

They abstract away from the internal description of a machine by specifying only inputs and outputs. And they abstract away from the external description by specifying inputs and outputs only as "0" and "1".



A NOT-gate.

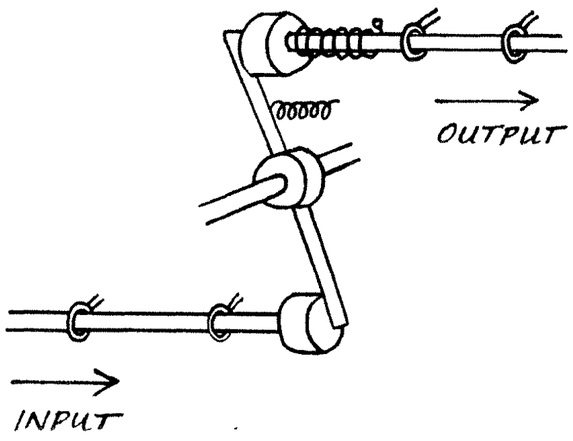
IN	OUT
0	1
1	1

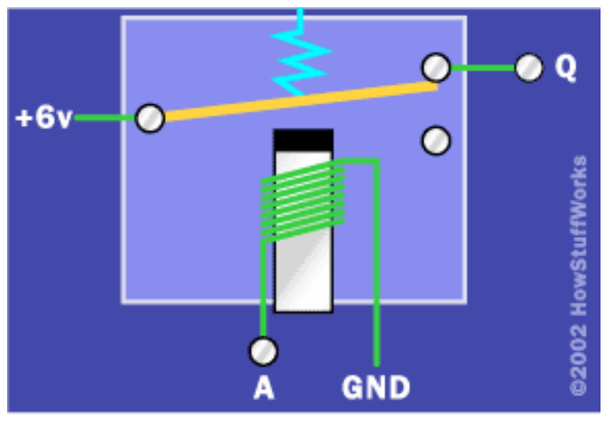
Abstraction over external mechanics.

IN	OUT
→	←
←	→

IN	OUT
⚡	⚡
⚡	⚡

Abstraction over internal mechanics.

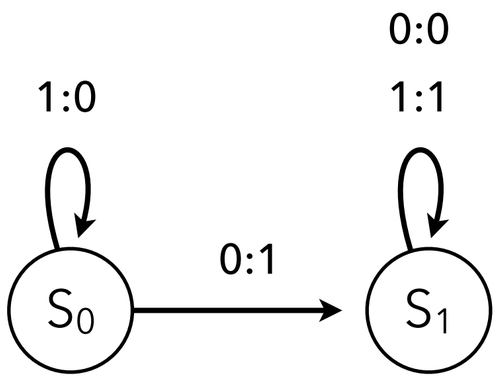




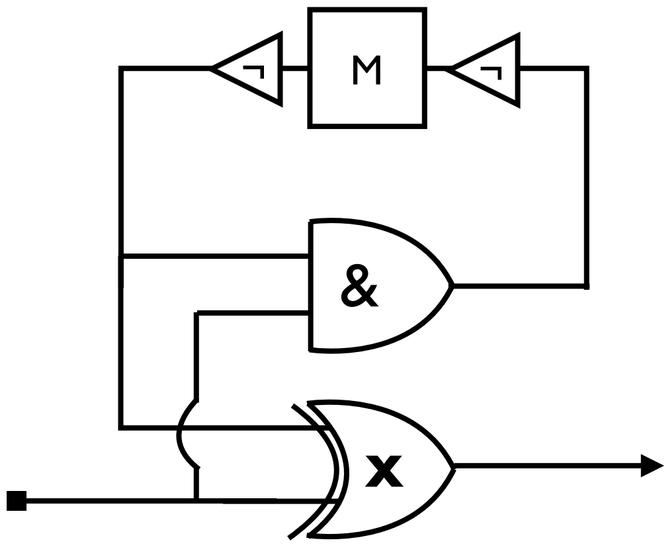
Physical implementations of a NOT-gate.

Functional Abstraction II: Piecewise Abstraction

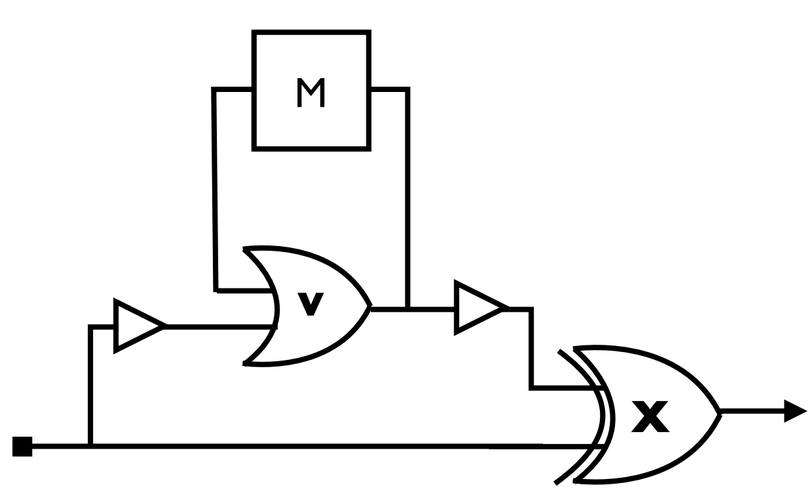
Functional Abstraction III: State Abstraction



+1 B FSM



+1 B SC



+1 B SC

Abstraction over internal machinery.

In state abstraction we find all of the central hall marks of abstraction itself.

Multiple realizability: Multiple SC's realize the same FSM.

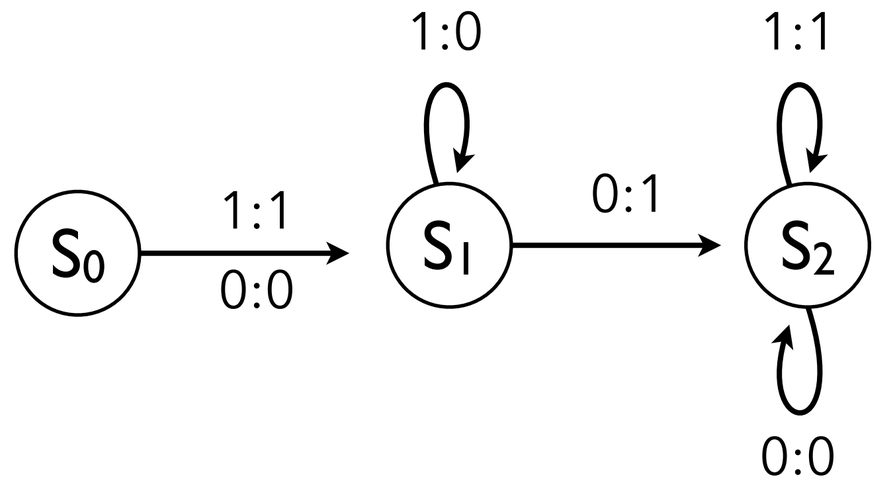
Loss of detail. The FSM leaves out details of circuitry.

Addition of deep structure. The FSM reveals the inner procedure that each SC is following.

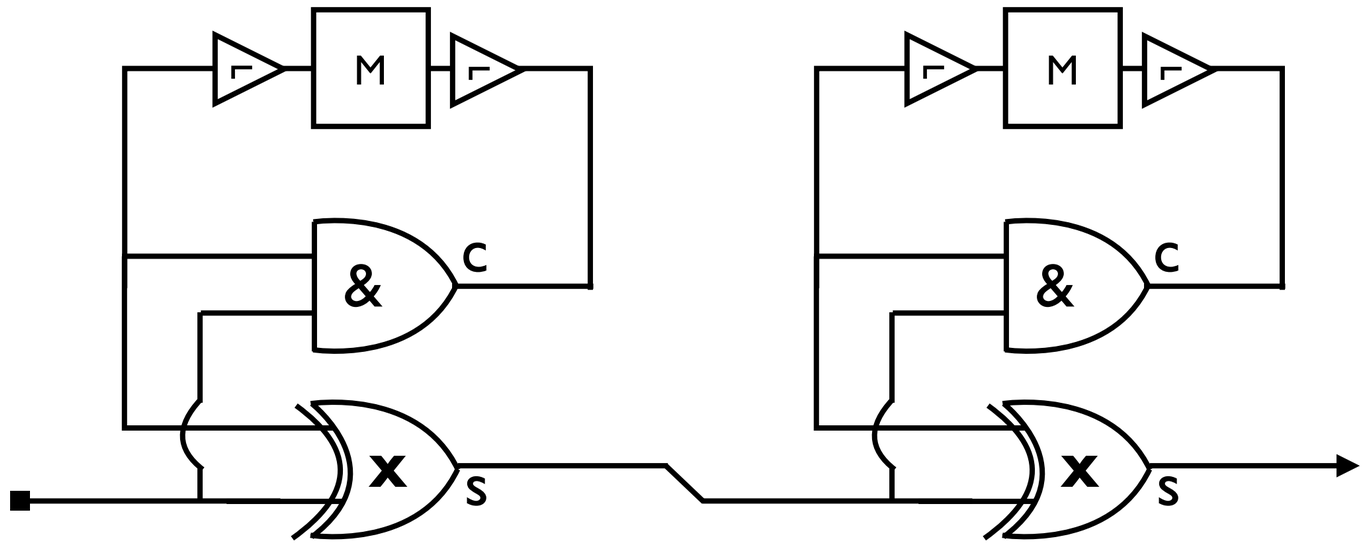
Autonomy. We can understand and reason about an FSM without understanding SC's, and visa versa.

State Machines and Sequential Circuits

FSMs describe pathways through overall configurations of the system. As the system changes, it flows through the state diagram. Each node represents a *temporal stage* in the machine's history.



SCs describe pathways of information. Information flows through the circuit as computation is carried out. Each nodes represents a *spatial* part of the machine.



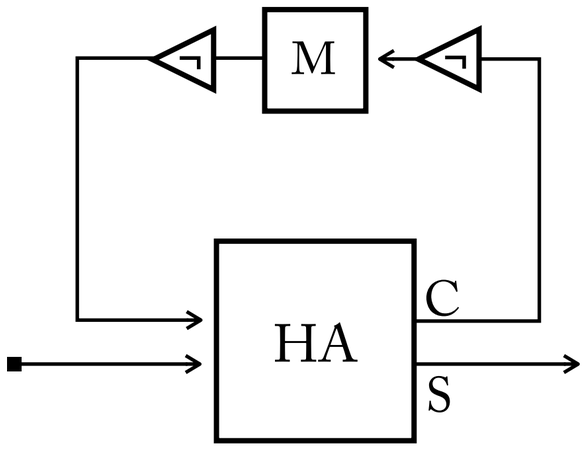
Both information flow diagram and state flow diagram describe systems, but in different ways.

We can think of a human as consisting of a complex circuit. Or we can think of them as implementing a complex algorithm and moving from state to state. Here we see the seeds of the contrast between neuroscience and psychology.

State abstraction marks an important kind of independence from the physical level. They are the first indication that computations can be carried out by neurons.

State Abstraction and the Mind-Body Problem

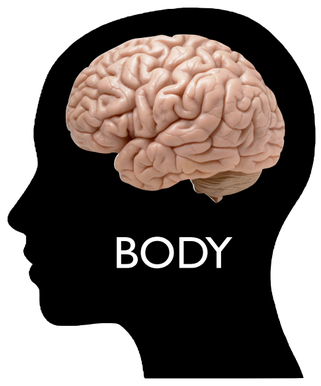
Consider now two very different descriptions of the same machine:

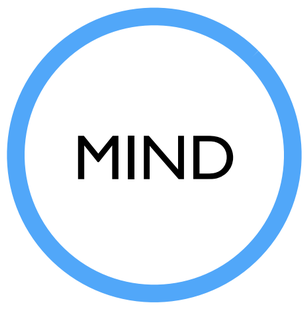


+1 B SC

STATE	IN	OUT	NEXT
S_0	0	1	A
	1	0	S_0
A	0	0	A
	1	1	A

+1 B State Table





Putnam (1967) "The Nature of Mental States" argued that state abstraction was the key to unlocking the mind-body problem. He thought the SC-State Table relationship was essentially equivalent to the body-mind relationship.

Since there is nothing mysterious about the way that SC's determine State Tables, there should be nothing mysterious about body-mind relationship. The implicit idea is that *mental states* are just *machine states*, for the right sort of human machine.

"I shall, in short, argue that pain is not a brain state, in the sense of a physical-chemical state of the brain (or even the whole nervous system), but another *kind* of state entirely. I propose the hypothesis that pain, or the state of being in pain, is a functional state of a whole organism." (Putnam 1967)

"Even though octopus and mammal are examples of parallel (rather than sequential) evolution, for example, virtually identical structures (physically speaking) have evolved in the eye of the octopus and in the eye of the mammal, notwithstanding the fact that this organ has evolved from different kinds of cells in the two cases. Thus it is at least possible that parallel evolution, all over the universe, might *always* lead to *one and the same* physical 'correlate' of pain… Thus any animal that we count as capable of these various states will at least seem to have a certain rough kind of functional organization."

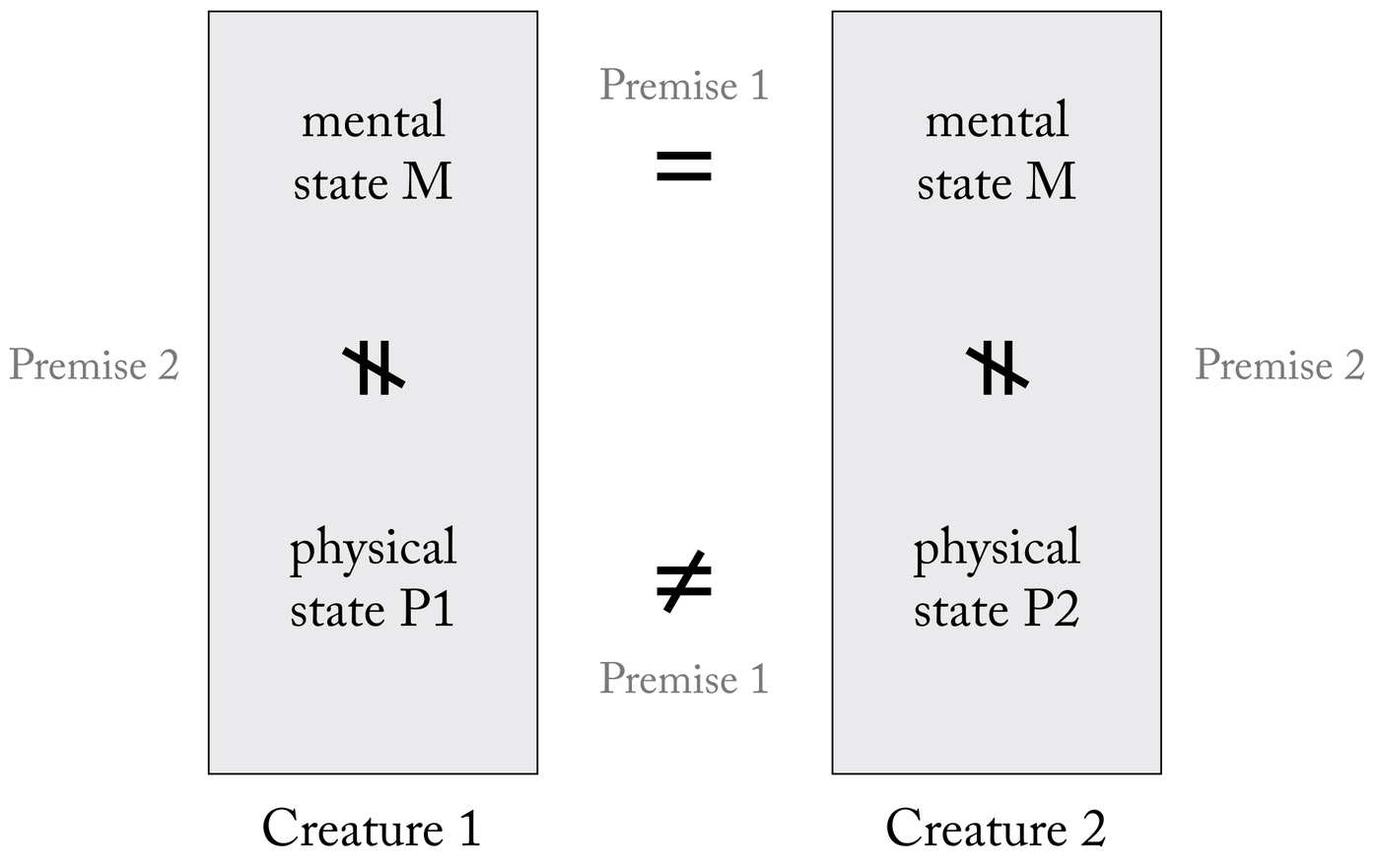
An Argument for Dualism?

1. In many cases, the same mental state can be physically realized in different ways by different creatures. Examples: beliefs, thoughts, perceptions, emotions.

2. Identity is transitive. Therefore: the mental states from Premise 1 cannot be identical with the physical states they are realized by.

3. According to physicalism, mental states are identical with physical states.

4. Therefore, physicalism is false.

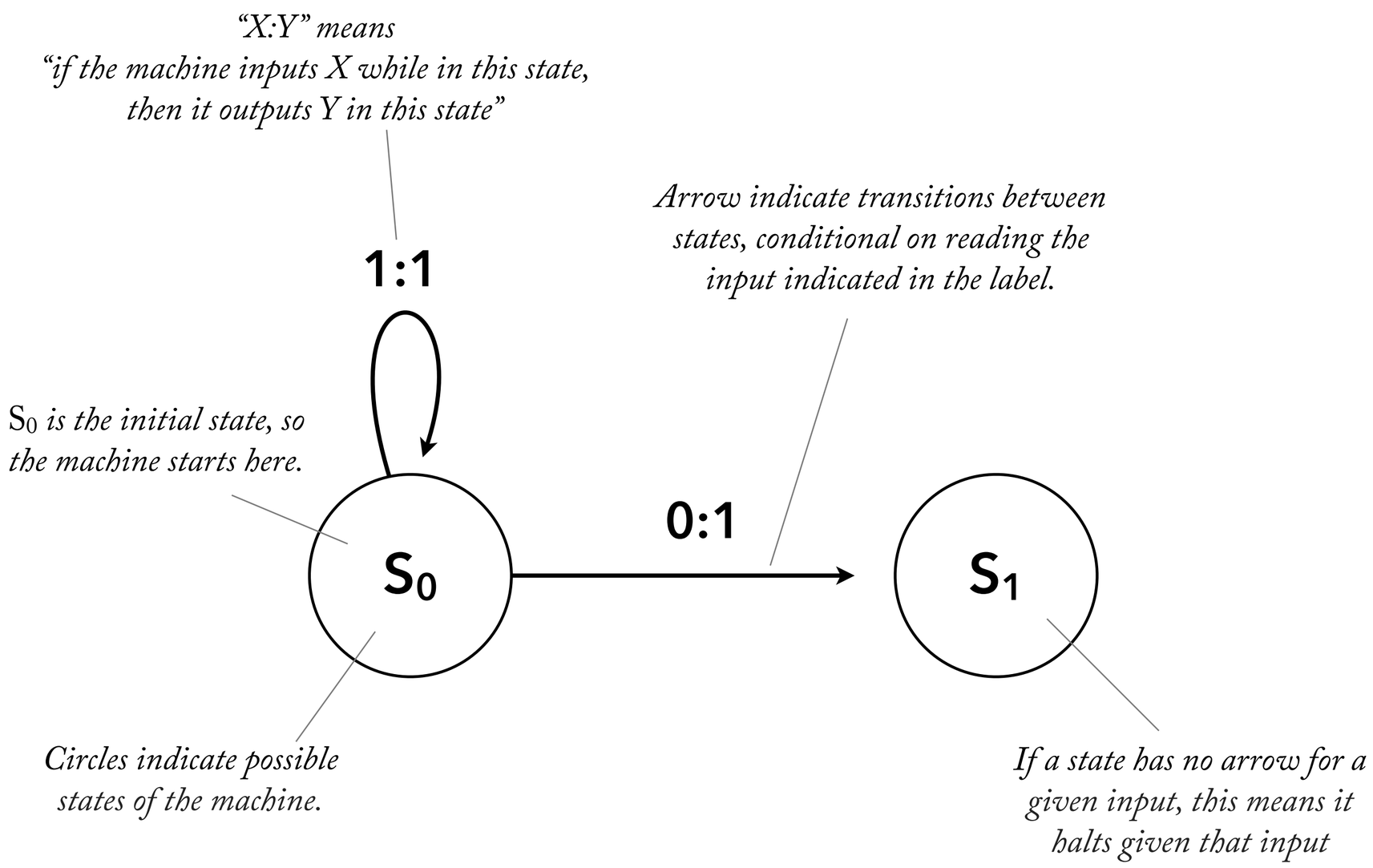


"Anyone who wishes, then, to argue on this basis for the existence of the soul will have to be prepared to hug the souls of Turing machines to his philosophical bosom!" (Putnam 1960)

24. Finite State Machines

Reading FSM Diagrams

You can get a feel for how FSM diagrams work by studying the following simple example:



This diagram represents the following rule:

In state S_0,
if input = 1, then output = 1, and return to S_0;
if input = 0, then output = 1, and move to S_1.

In state S1, halt on any input.

Constructing FSM Diagrams

FSM diagrams can be constructed from the following elements.

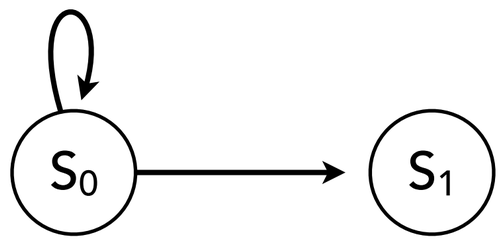
Nodes: circles associated with names.

- Each node represents a state.
- The initial state is always named "S_0".



Links: one-directional arrows connecting circles to circles.

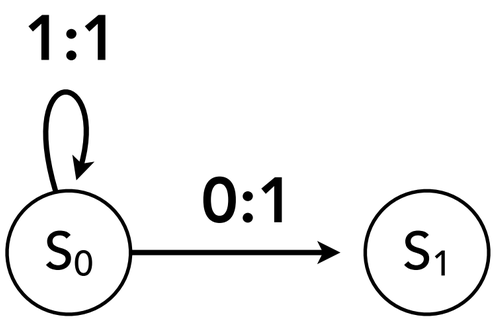
- There are at most two lines leaving each circle.*
- There may be zero lines leaving a circle.
- Circles can be connected to themselves.
- The lines can cross.



*More precisely, there are as many lines leaving each circle as possible inputs. If the machine can read *n* symbols, there will be up to *n* links for each node.

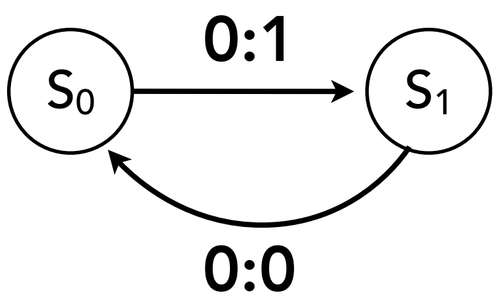
Labels: each line is labeled with with an expression of the form "*input* : *output*".

- Possible inputs: 1, 0
- Possible outputs: 1, 0.
- **Note!** For each circle, there can be at most one arrows labeled with input 1, and one arrows labeled with input 0.
- A common variation writes the output where we've put the state name, and labels the line only with the input.



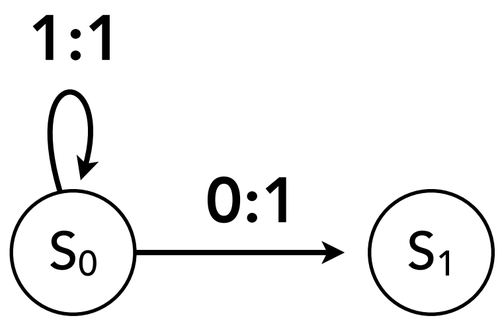
Loops: any kind of loop is acceptable:

- Loops from a state to itself.
- Loops among a groups of states.



Halting:

- A machine halts in a given state, given an input, if there are no arrows leaving that state for that input.
- If there are *no* lines leaving a state, the machine will halt in that state no matter what.



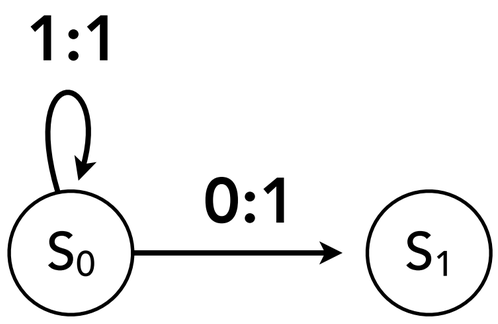
Computing with Finite State Machines

Computation for FSMs is defined in the same way as it is for SCs. Local inputs and outputs are processed one bit at a time, and the global input and output corresponds to the sequences of inputs and outputs which is reached when all further input and output bits will be 0s.

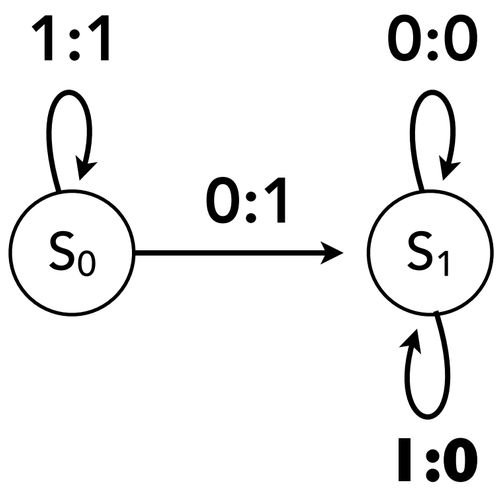
Note on halting:

- A machine **halts** in a given state, given an input, if there are no arrows leaving that state for that input.
- When computing a function, an FSM should never halt. Instead, it should end computation by outputting 0's indefinitely.

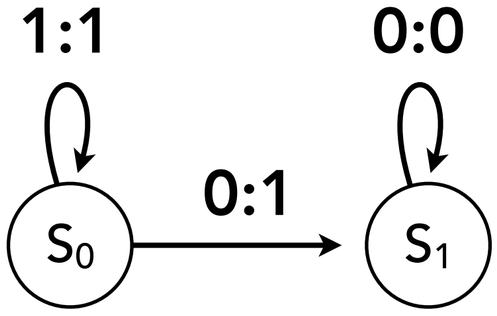
So **don't** design your FSM this way.



Do ensure that the final state produces a continuous output of 0's:



To compute tally successor…

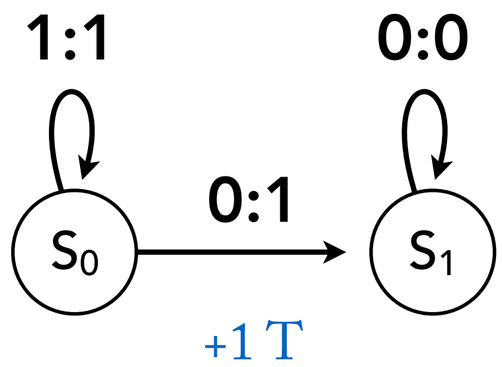


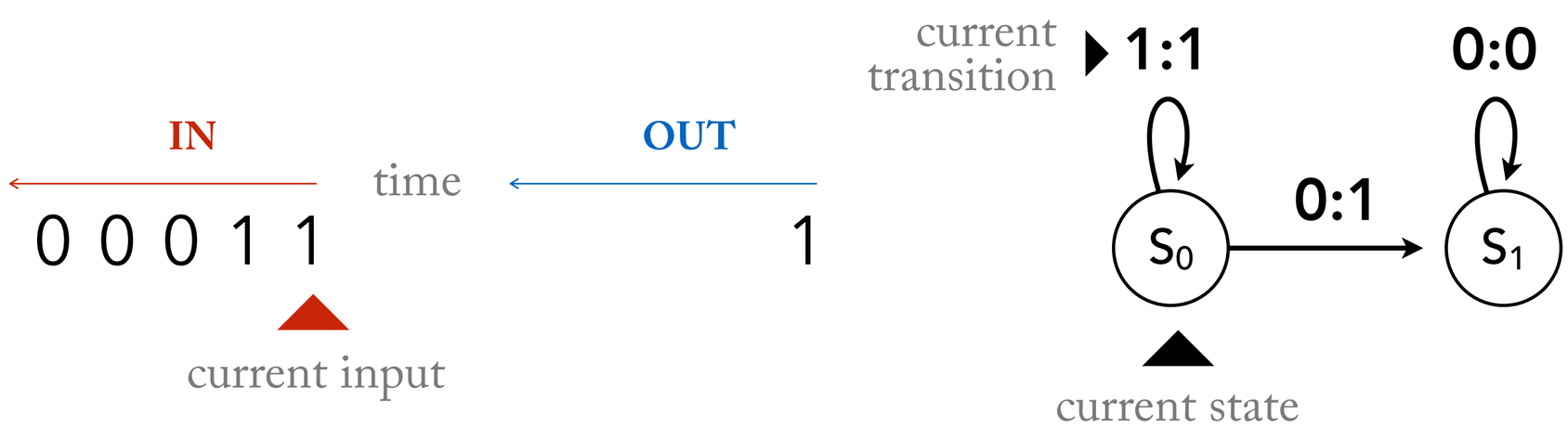
+1 T

Example: +1 T

This FSM at right computes successor in tally.

The timeline below shows the step-by-step progression of the machine for a sample input.



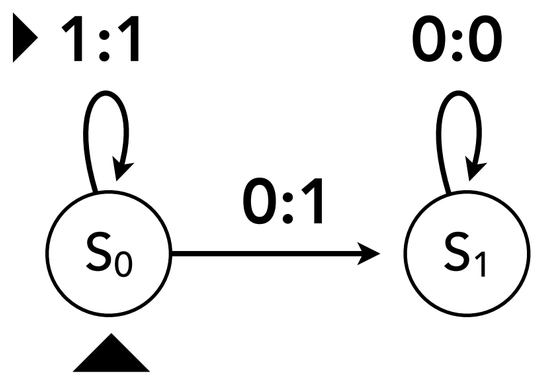


IN

0 0 0 1 1

OUT

1 1

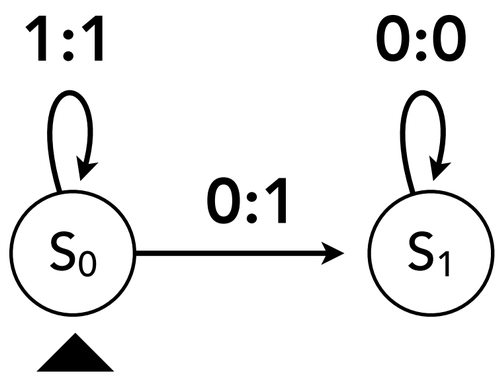


IN

0 0 0 1 1

OUT

1 1 1

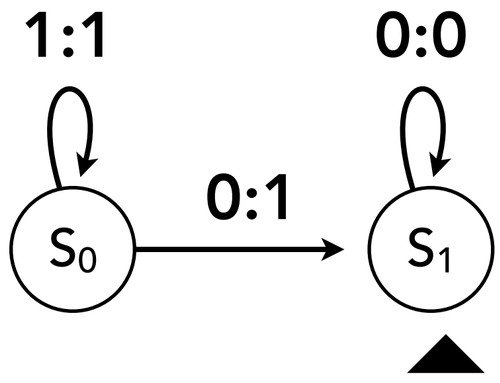


IN

0 0 0 1 1

OUT

0 1 1 1

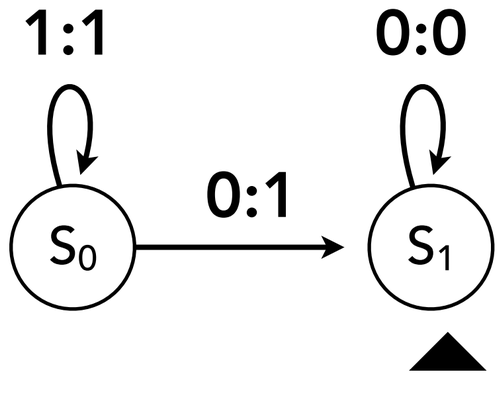


IN

0 0 0 1 1

OUT

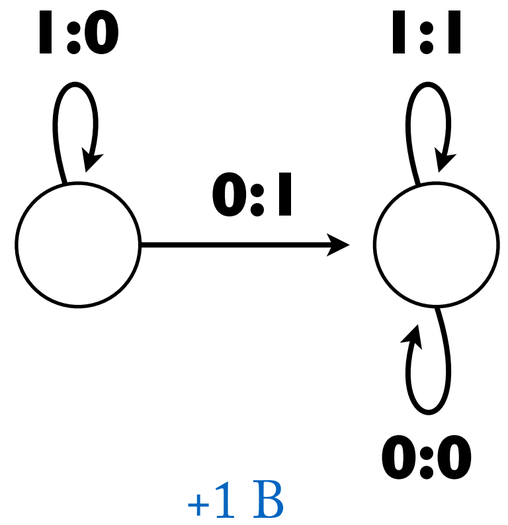
0 0 1 1 1



Example: +1 B

This FSM at right computes successor in binary.

The timeline below shows the step-by-step progression of the machine for a sample input.



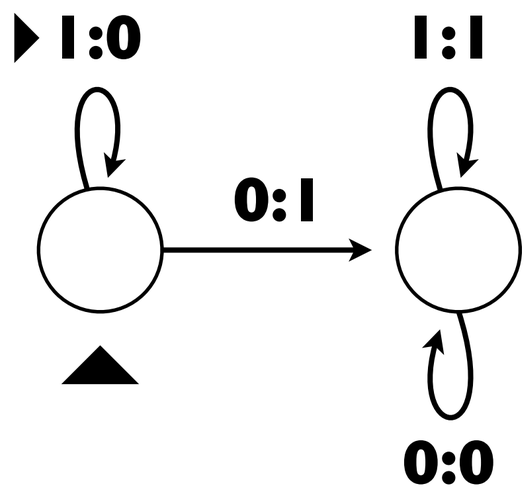
+1 B

IN

0 0 1 1

OUT

0

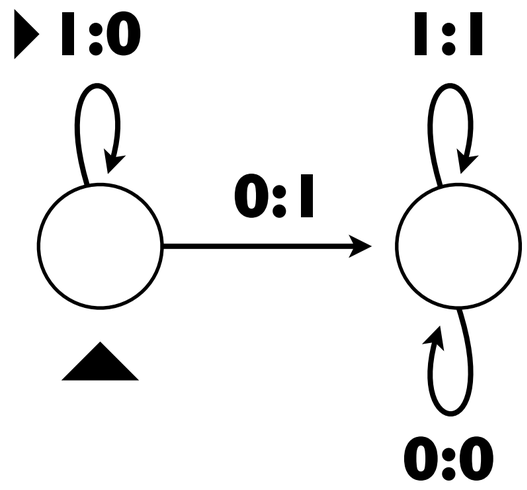


IN

0 0 1 1

OUT

0 0

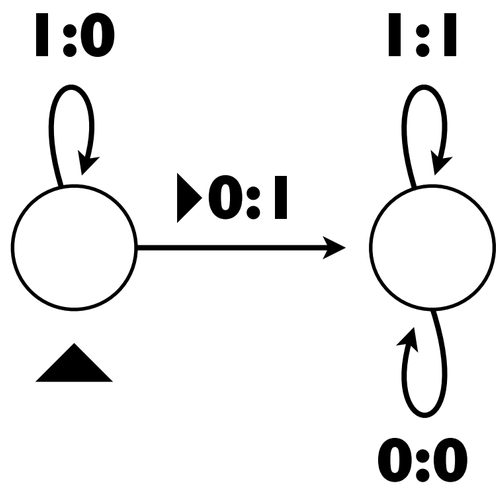


IN

0 0 1 1

OUT

1 0 0

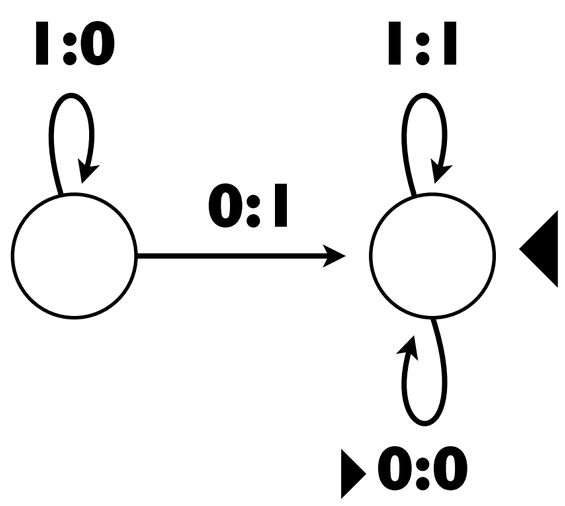


IN

0 0 1 1

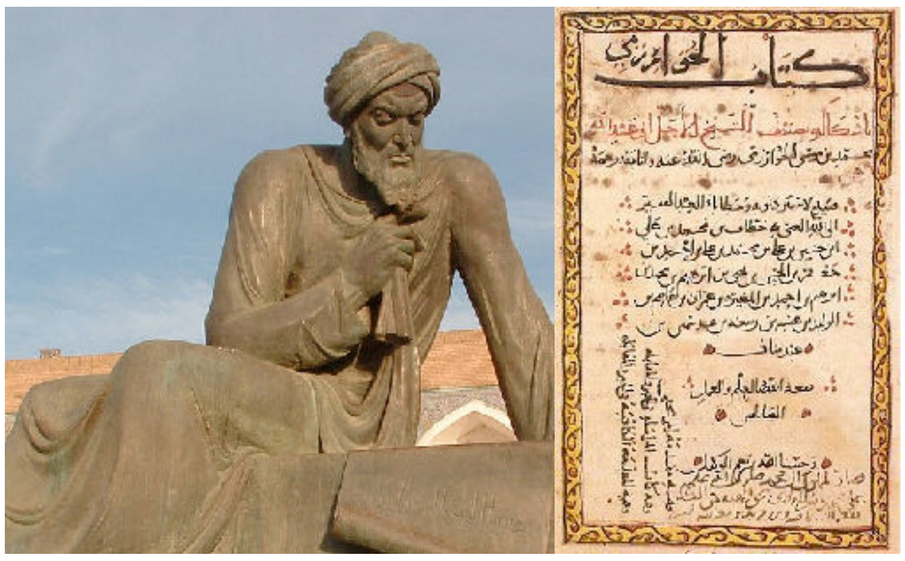
OUT

0 1 0 0



25. What Is an Algorithm?

The Algorithm Concept



Muhammad ibn Mūsā al-Khwārizmī (708-850), Baghdad



Ada Lovelace (1815-1852), London



Alan Turing (1912-1954), Wilmslow, England

An **algorithm** is a *procedure* such that:

1. The procedure can be divided into finitely many steps.
2. The transition from one step to another is completely determinate.

A **program** is a *description* of an algorithm in a given computational language. But a machine can **implement** an algorithm in its physical circuitry, without a program. This is the case of finite state machines that are based on sequential circuits.

Steps in the algorithm are the same as states in an FSM. States and state transitions are the basic units out of which algorithms are built.

To do *X*

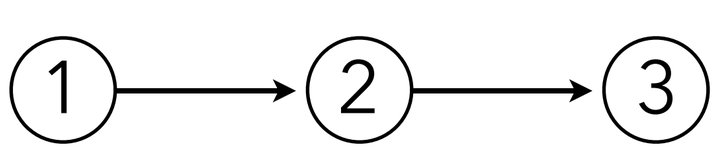
1. [redacted]
2. [redacted]
3. [redacted]

To do *X*

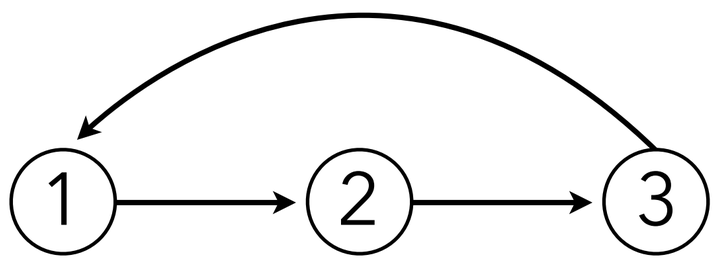
1. [redacted]
2. [redacted]
3. [redacted]

and repeat State 1.

Running an algorithm on a given sequence of inputs involves a certain number of transitions. This is the total number of state transitions that occur until the input is fully processed.



In this case, there are only three states, and only three possible steps.



In this case, there are only three states, but potentially infinite number of transitions.

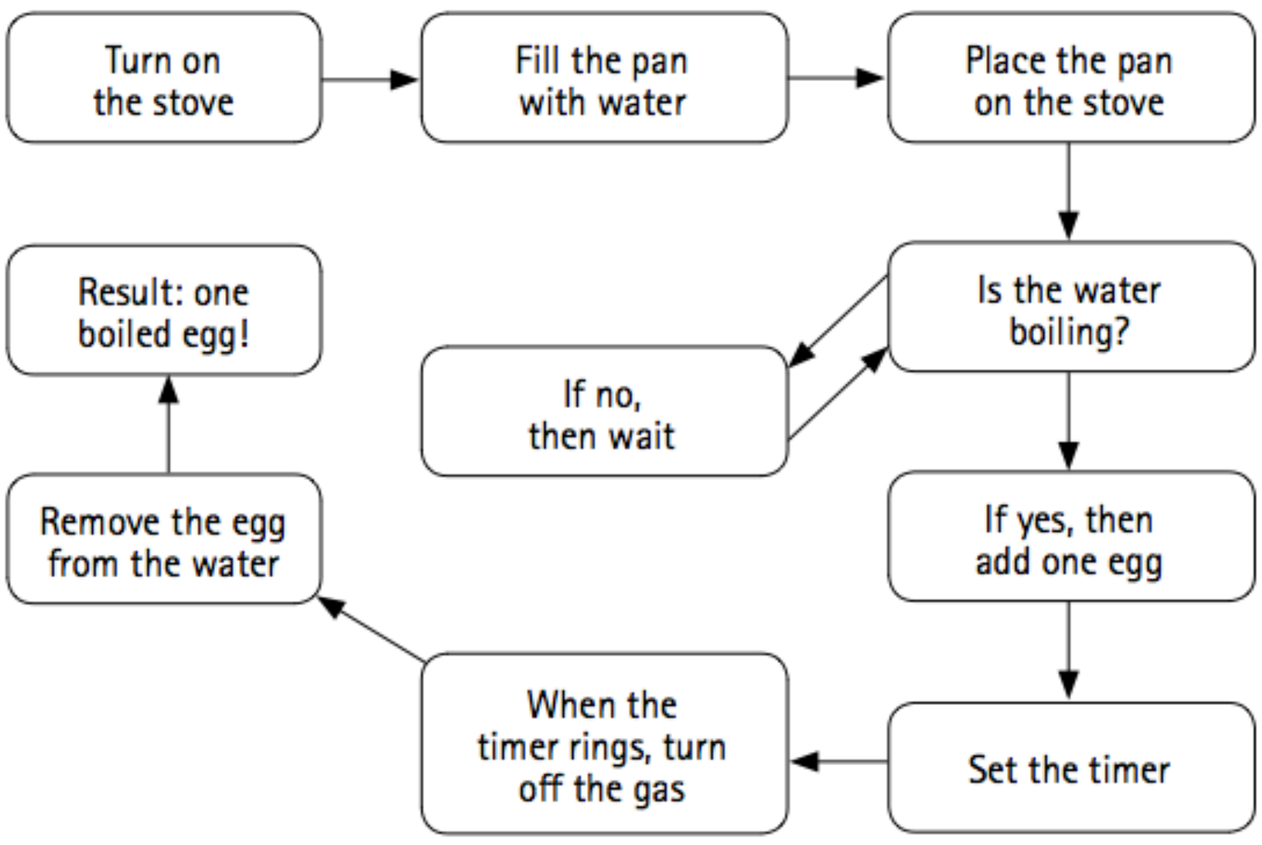
Algorithms and Local Logics

Following an algorithm succeeds as an evolutionary strategy to the extant that it can mirror a relevant process in the world.

Perhaps an ideal being could simulate everything in the world by implementing algorithms that perfectly implemented the laws of physics. The laws of physics are a **global logic**, a set of rules that describes all situations.

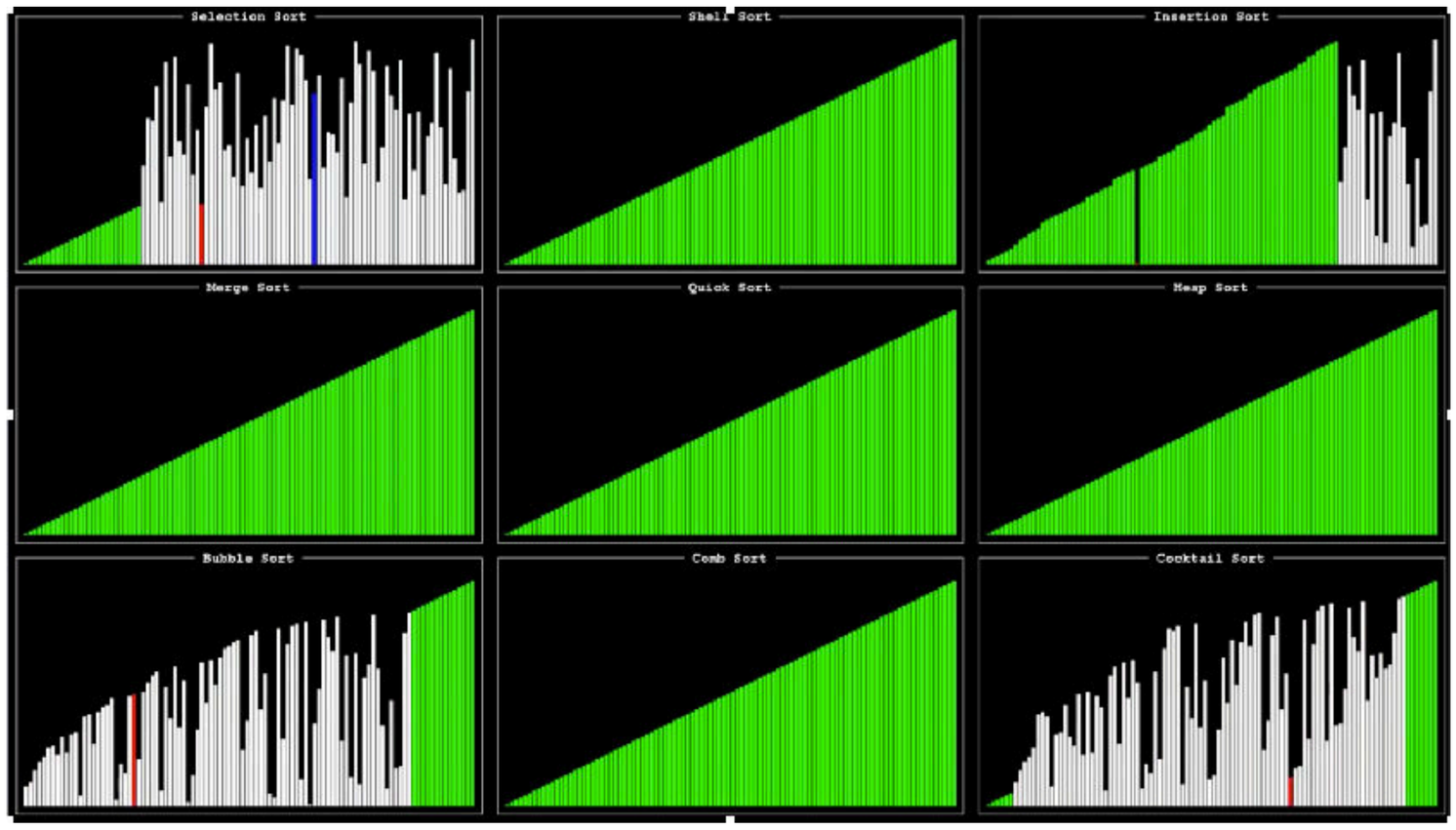
We succeed to the extent that we can identify constrained situations governed by **local logics**. Examples of local logics are the …. laws of psychology, heuristics.

1 Turn on the stove
2 Fill the pan with water
3 Place the pan on the stove
4 When the water boils, add one egg, and set the timer
5 When the timer rings, turn off the gas
6 Remove the egg from the water
7 Result: one boiled egg.



Algorithms at Work

Sorting



Math

How to Do Long Addition

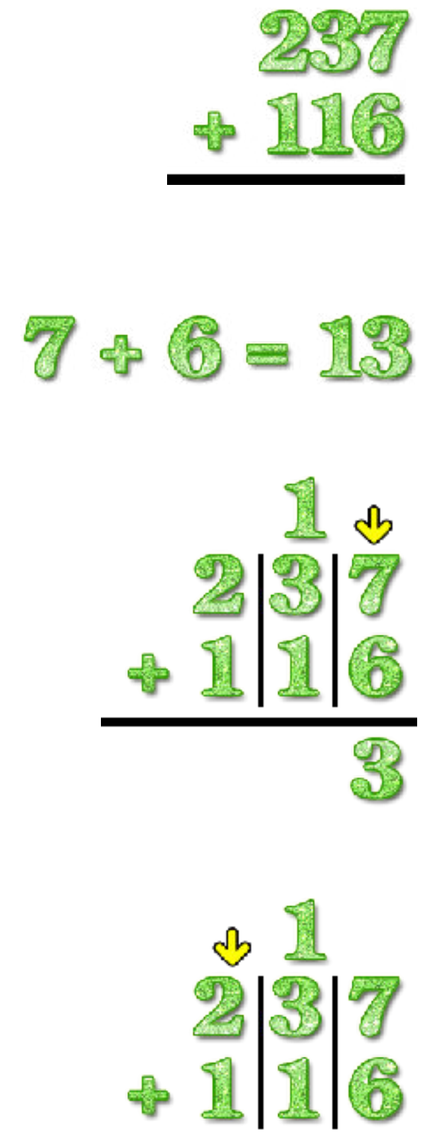
Step 1
Write the numbers you wish to add, one underneath the other.

Step 2
Add up the numbers in the right-most column.

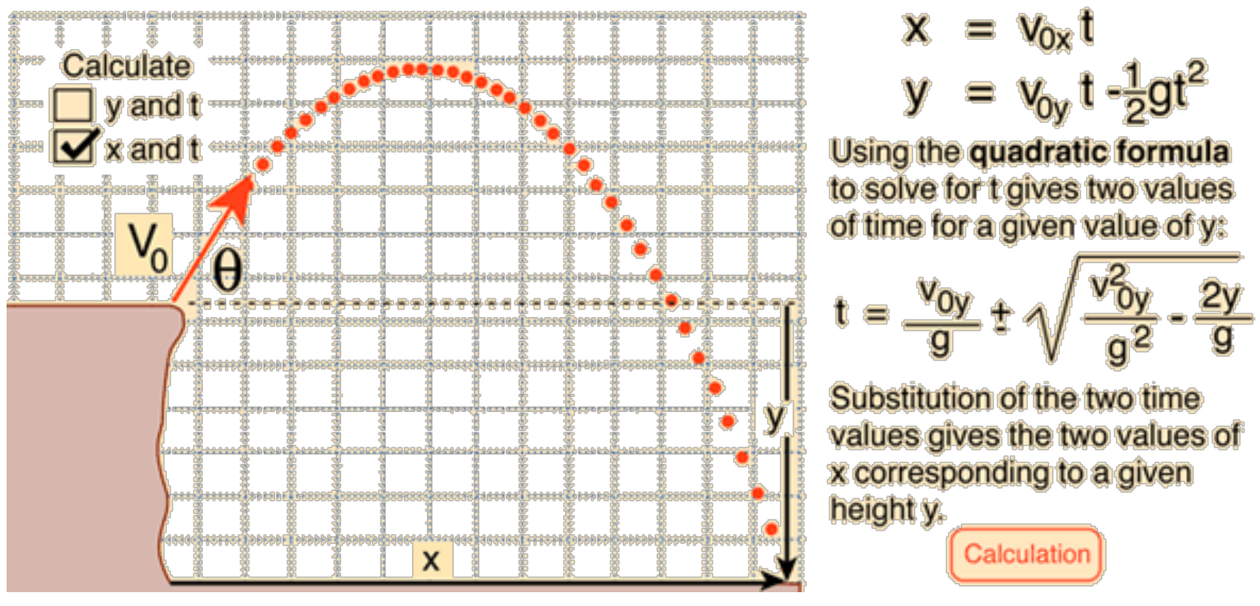
Step 3
Check if the answer from the previous Step is 9 or less:
- If Yes, write the number below the column and move to Step 4.
- If No, the answer will have two digits.
  - Write the digit on the right below the column.
  - Write the digit on the left above the column to the left of the current column. This is carrying. Move to Step 4.

Step 4
Move to the column to the left. Add up the numbers in this column, including any numbers above the two lines.

Step 5
Go to Step 3 and repeat until all columns have been added.



Physics



$$x = v_{0x} t$$

$$y = v_{0y} t - \frac{1}{2} g t^2$$

Using the **quadratic formula** to solve for t gives two values of time for a given value of y:

$$t = \frac{v_{0y}}{g} \pm \sqrt{\frac{v_{0y}^2}{g^2} - \frac{2y}{g}}$$

Substitution of the two time values gives the two values of x corresponding to a given height y.

Calculation

26. Levels of Abstraction

The Concept of Level



Descriptions of complex systems can be divided into **levels**.

Upper levels are **abstractions** of lower levels.
Lower levels are **realizations** of upper levels.

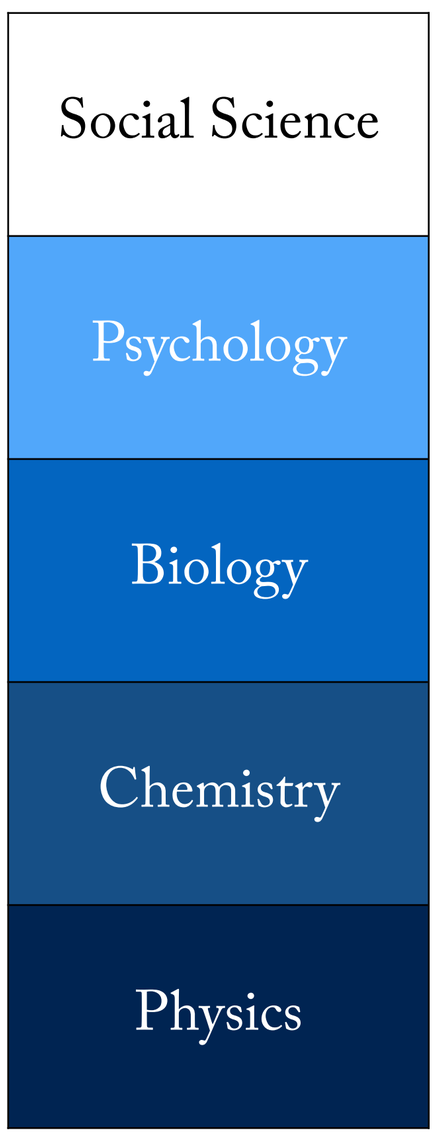
Upper levels capture **real patterns** that emerge from lower levels.

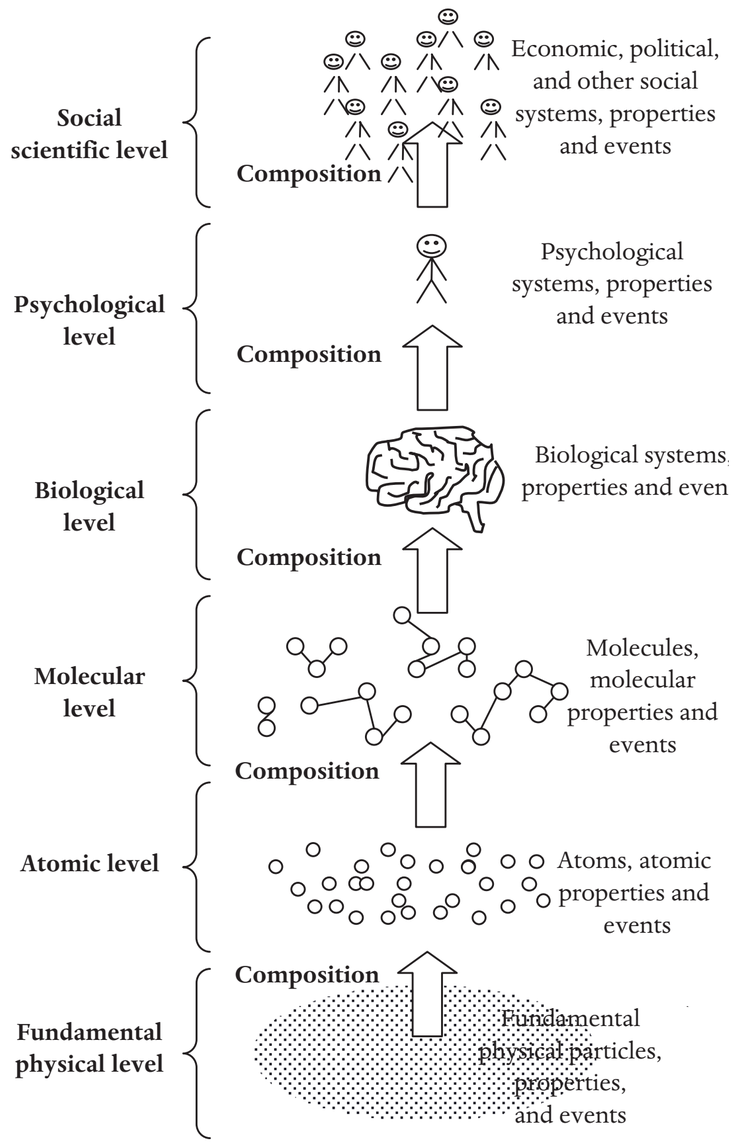
Levels in Nature

The natural sciences as a whole are organized into **levels of abstraction**, also known as **levels of explanation.**

Every level is **necessary**. A complete explanation of reality requires the contributions of all levels, not just the bottom level of physics.

Every level is **autonomous**. Levels of explanation can be studied at least in part without knowledge of the levels above and below.

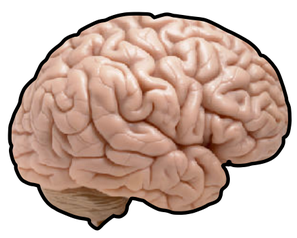




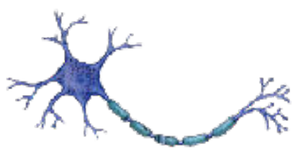
Levels in the Nervous System

The brain itself is a complex system which must be studied at many levels simultaneously.

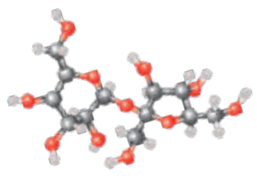
> "If one hopes to achieve a full understanding of a system as complicated as a nervous system… then one must be prepared to contemplate different kinds of explanation at different levels of description that are linked, at least in principle, into a cohesive whole, even if linking the levels in complete detail is impractical." (Marr 1982)



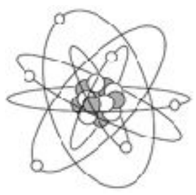
Neural network



Biology



Chemistry



Physics

Marr's Three Levels

Marr proposed an influential three-level view of cognitive science where each level aims to answer a different kind of explanatory question.

Computational theory	Representation and algorithm	Hardware implementation
What is the goal of the computation, why is it appropriate, and what is the logic of the strategy by which it can be carried out?	How can this computational theory be implemented? In particular, what is the representation for the input and output, and what is the algorithm for the transformation?	How can the representation and algorithm be realized physically?

Figure 1–4. The three levels at which any machine carrying out an information-processing task must be understood.

An Example: The Humble Cash Register

Computational Level

"The most abstract [level] is the level of what the device does and why. What it does is arithmetic, so our first task is to master the theory of addition. Addition is a mapping, usually denoted by + from pairs of numbers into single numbers."

Algorithmic Level

"In order that a process shall actually run, however, one has to realize it in some way and therefore choose a representation for the entities that the process manipulates. The second level of the analysis of a process, therefore involves choosing two things: (1) a *representation* for the input and the for the output of the process and (2) a *algorithm* by which the transformation may actually be accomplished… If the first of our levels specifies what and why, this second level specifies *how*. For addition, we much choose Arabic numerals for the representations, and for the algorithm we could follow the usual rules about adding the least significant digits first and "carrying" if the sum exceeds 9.

Hardware Level

"This brings us to the third level, that of the device in which the process is to be realized physically… The child who methodically adds two numbers from right to left, carrying a digit when necessary, may be using the same algorithm that is implemented by the wires and transistors of the cash register in the neighbor-hood supermarket, but the physical realization of the algorithm is quite different in these two cases.



Two features of levels:

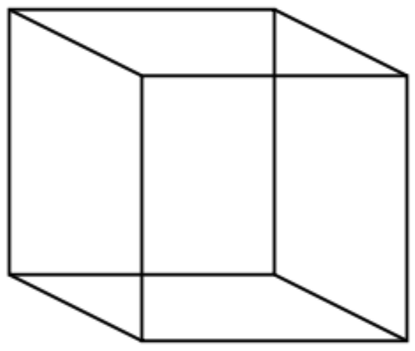
Multiple Realizability: upper levels can be realized in multiple ways at lower levels.

Methodological Independence: one can gain knowledge about different levels more or less independently.

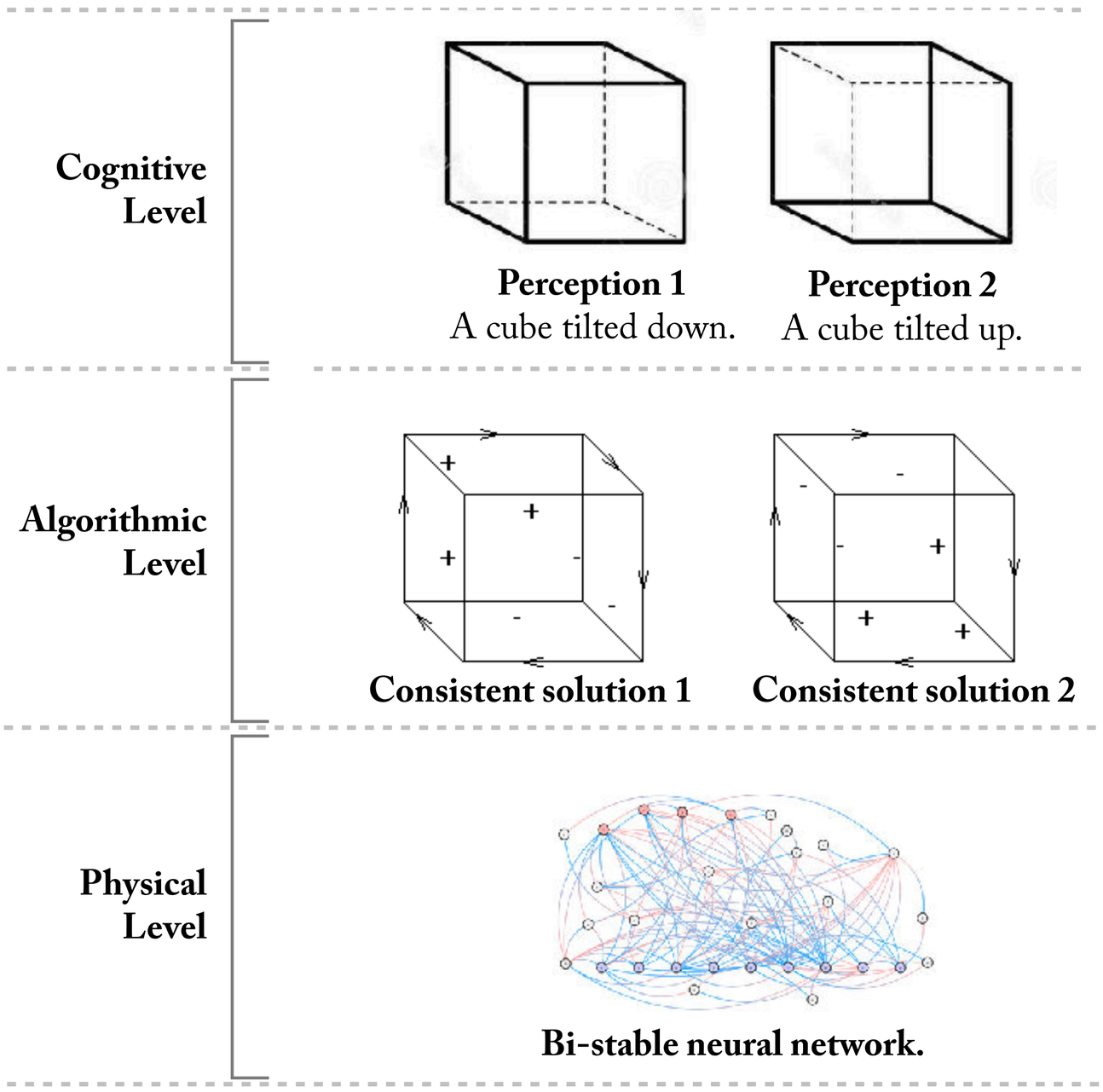
The Cognitive-Ecological Level

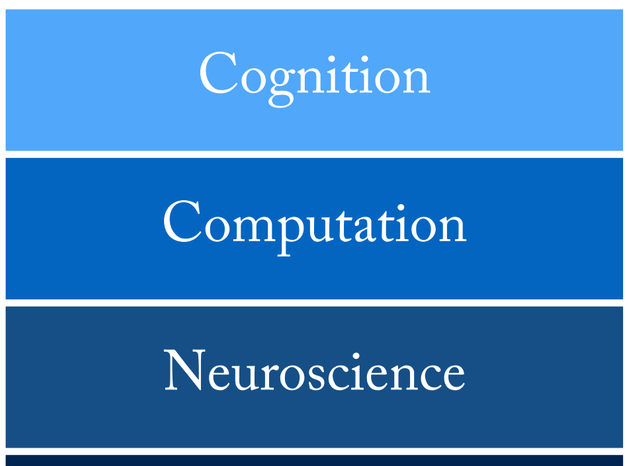
Marr called the top level of his hierarchy the "computational" level, with an eye towards *what* is computed. But this is confusing to us, since "computation" evokes algorithms and circuits. Instead, I'll call the top level the **cognitive-ecological level.** It is the level where the **cognitive task** is specified and where the **ecological rationale** for performing that task is explained.

> "It is the top level, the level of computational theory, which is critically important from an information-processing point of view. The reason for this is that the nature of the computations that underlie perception depends more upon the computational problems that have to be solved than upon the particular hardware in which their solutions are implemented. To phrase the matter another way, an algorithm is likely to be understood more readily by understanding the nature of the problem being solved than by examining the mechanism (and the hardware) in which it is embodied."



"The ambiguity of the Necker cube seems to demand a different kind of explanation. To be sure, part of the explanation of its perceptual reversal must have to do with a bistable neural network (that is, one with two distinct stable states) somewhere inside the brain, but few would feel satisfied by an account that failed to mention the existence of two different but perfectly plausible three-dimensional interpretations of this two-dimensional image."





The Physicalist Worldview

Marr's "computational" level is the level at which we ascribe content and rationality to an agent. Modern physicalists take this idea as the basis for their theory of the mind: cognition emerges from computation, just as biology emerges from chemistry. Mental states are entirely natural phenomena, just like cells or molecules. The natural world is a unified hierarchy of levels of abstraction.

Levels of Abstraction

Multiple Realizability

COGNITIVE-ECOLOGICAL LEVEL

"At one extreme, the top level, is the abstract computational theory of the device, in which the performance of the device is characterized as a mapping from one kind of information to another, the abstract properties of this mapping are defined precisely, and its appropriateness and adequacy for the task at hand are demonstrated."

+1

"If one hopes to achieve a full understanding of a system as complicated as a nervous system… then one must be prepared to contemplate different kinds of explanation at different levels of description that are linked, at least in principle, into a cohesive whole."

Functions

Semantic Abstraction

ALGORITHMIC LEVEL

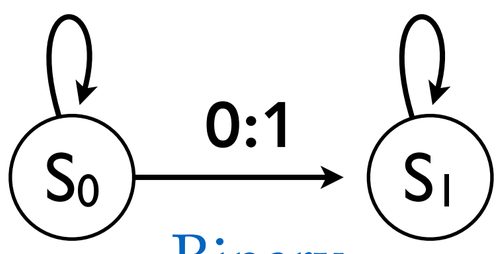
"The second level of analysis of a process, therefore, involves choosing two things: (1) a representation for the input and for the output of the process and (2) an algorithm by which the transformation may actually be accomplished."

1:0

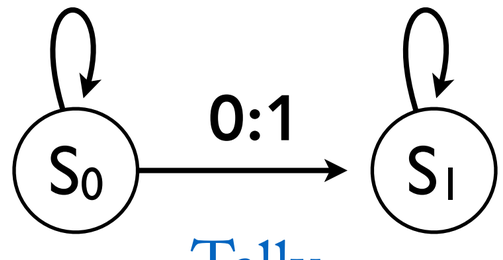
0:0
1:1

1:1

0:0



Binary

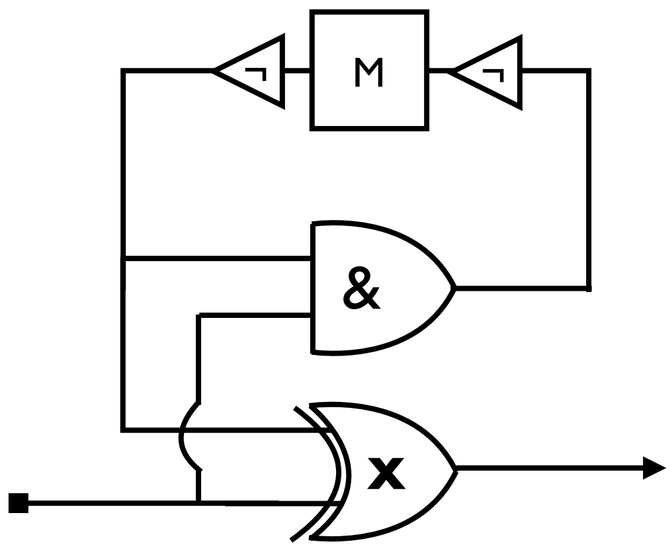


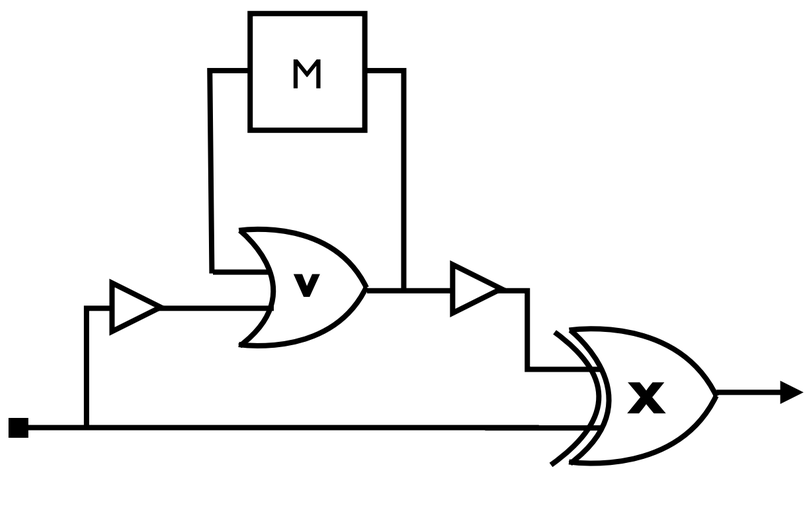
Tally

State Machines

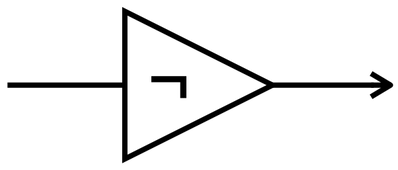
State Abstraction

MECHANICAL LEVEL





Logic Circuits

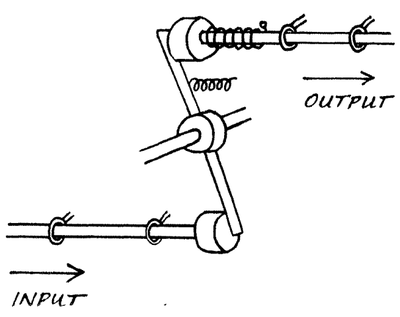


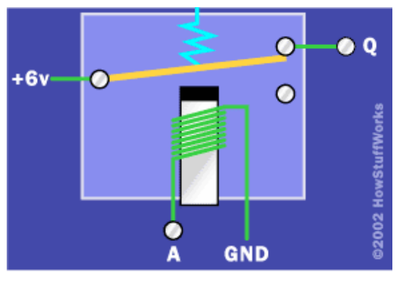
Logic Gates

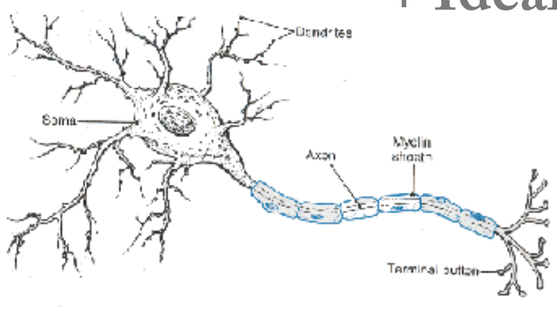
Mechanical Abstraction

+ Idealization

PHYSICAL LEVEL





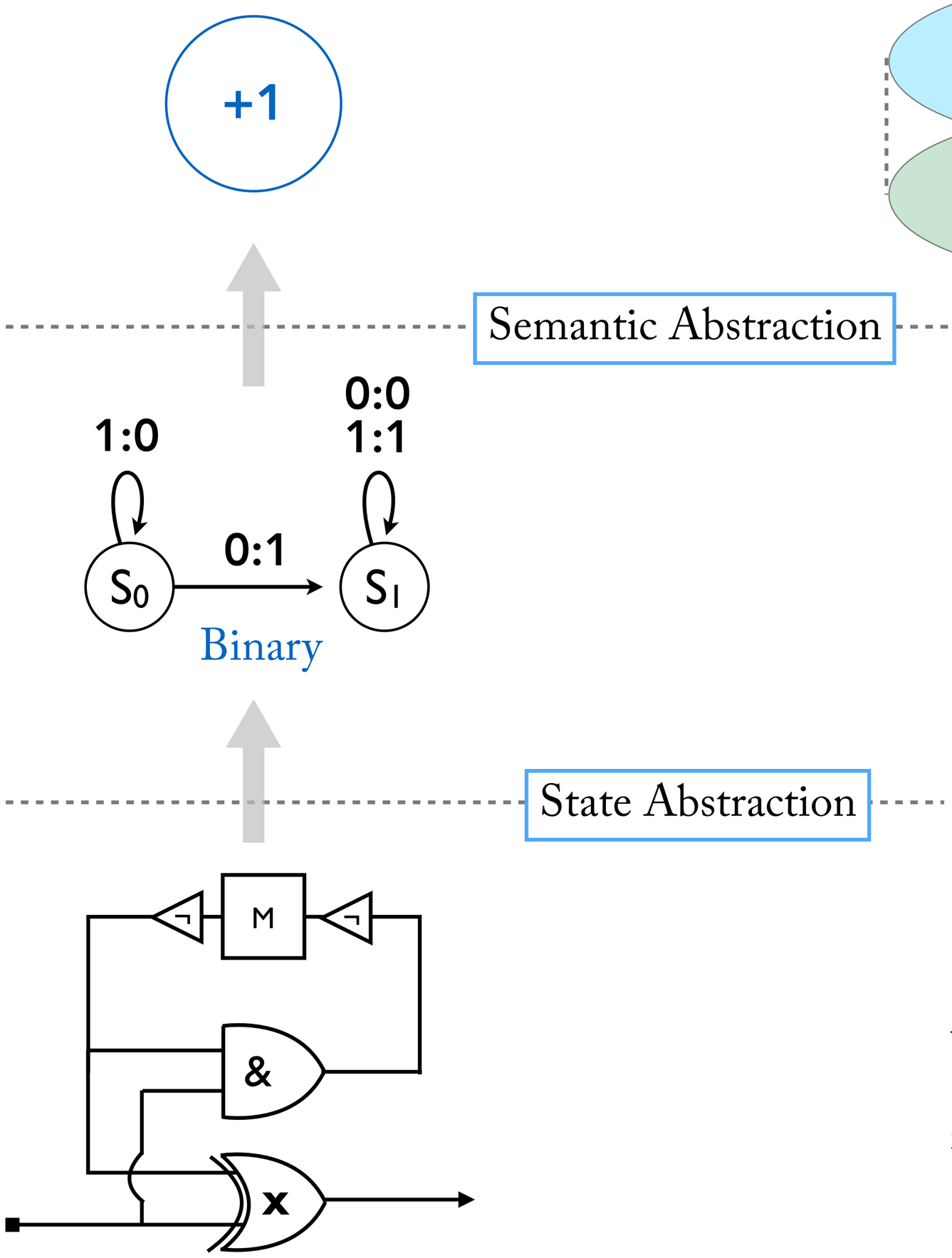


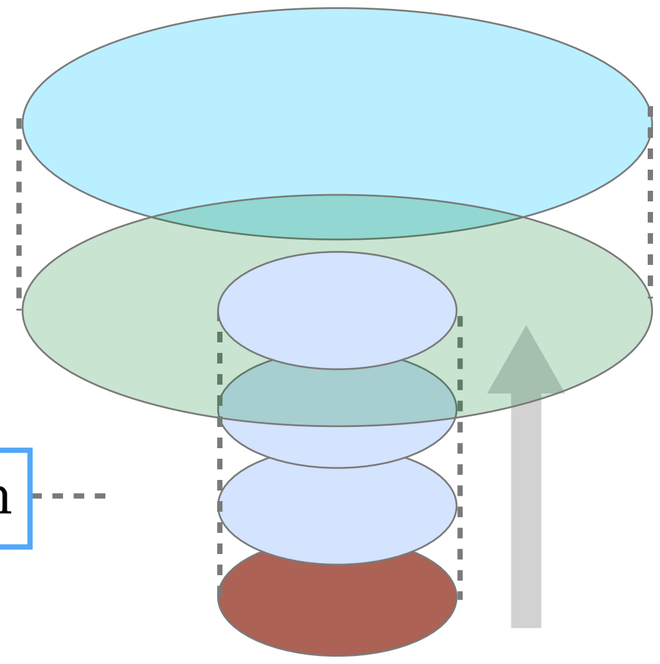
Physical Machines

"How can the representation and algorithm be realized physically?"

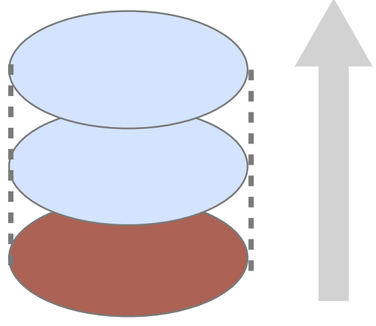
Semantic Abstraction

What are the conditions when we would say an organism computes +1? When it counts some *features of its environment*, e.g. the number of fruits in the tree or the number of steps back to the nest.





Outward abstraction abstraction over the informational relations between a system and the environment.



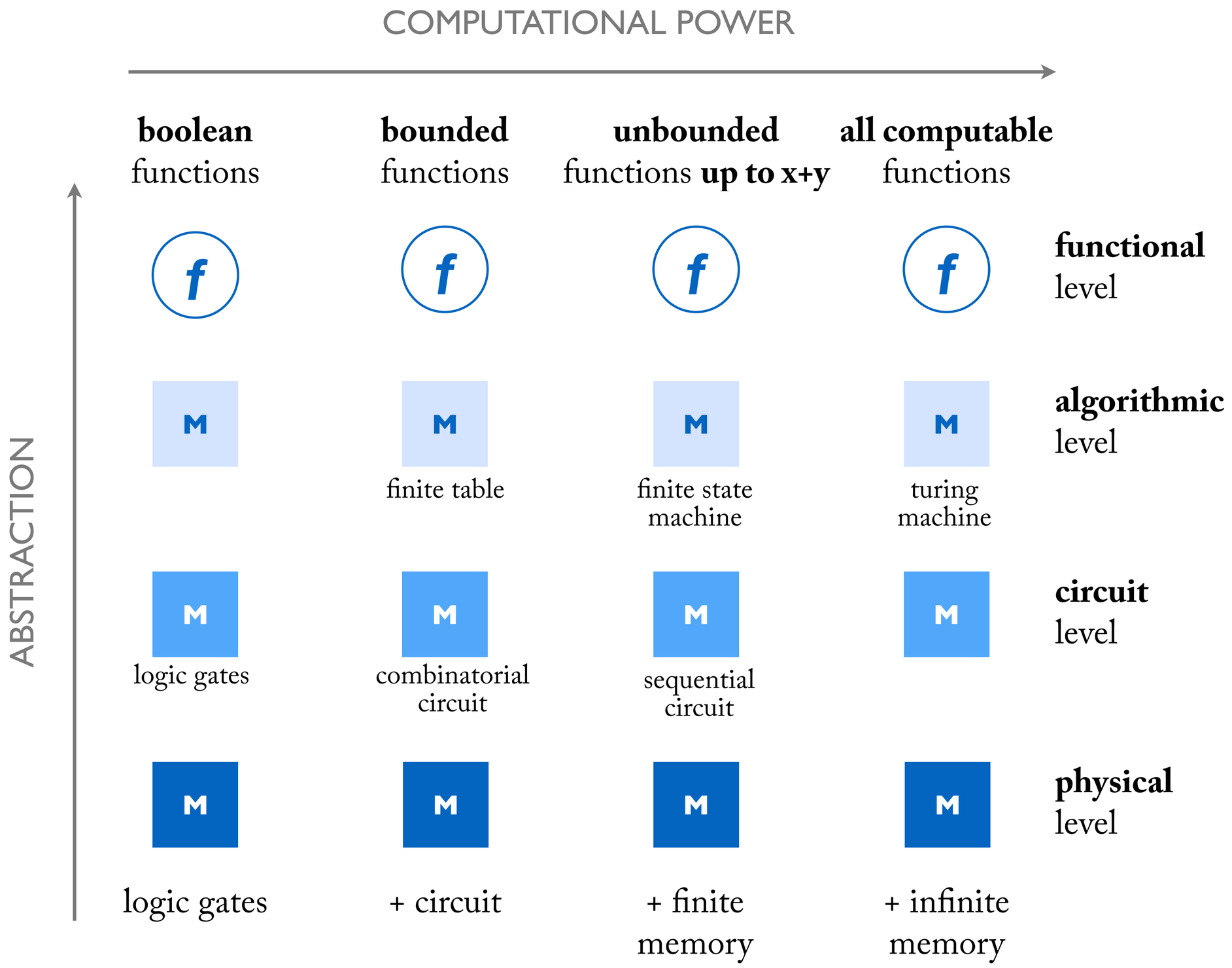
Upward abstraction: abstraction over a system's internal mechanisms.

The central idea here is that semantic description is just another form of functional abstraction. This brings representation and rationality into dialogue with the rest of scientific explanation.

Rational transition as abstraction. To say that a machine computes +1 is to say that, under functional abstraction, it can be characterized in terms of the +1 function. Computation is not like a metaphysical chain between the machine and the function.

Content as abstraction. The same is true for representation. Representation isn't an abstract chain (to be broken or maintained) between a symbol and its content. Content is a way of abstractly characterizing a symbol. We can describe the same physical system as a pattern of electric pulses, as the bit sequence 101, or as the number five. Each belongs to a different level of description.

In Unit 2, we've explored levels of computation: levels of abstraction and levels of computational power.



Unit 3

COGNITION

27. Turing Machines

Turing's Idea

Turing's problem: can all mathematical proofs be described mechanically? What kind of mechanical tools would be needed to generate any given proof?

This problem arose because of Turing's interest in the **Entscheidungsproblem**, or decision problem: the question of whether there is a general algorithm for determining whether any given mathematical claim is provable.

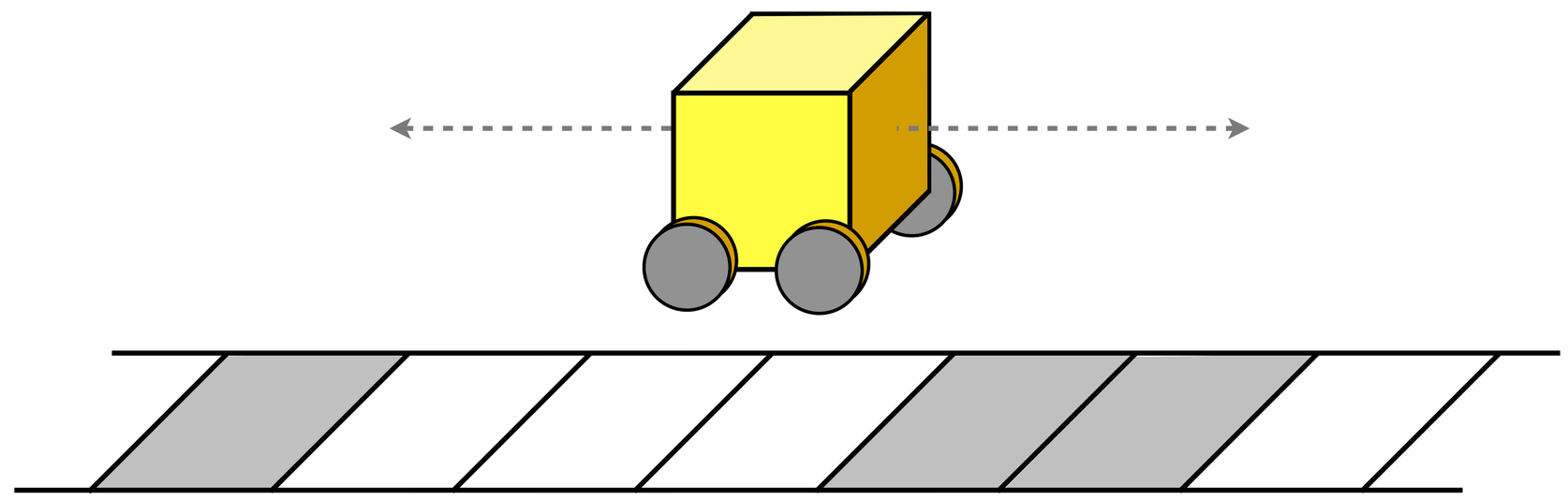
Turing reasoned that the tools you need to solve any given math problem are:

- A piece of paper.
- A stock of mathematical symbols.
- A pencil that writes.
- A human controller.

Now simplify each of these to their simplest mechanical form:

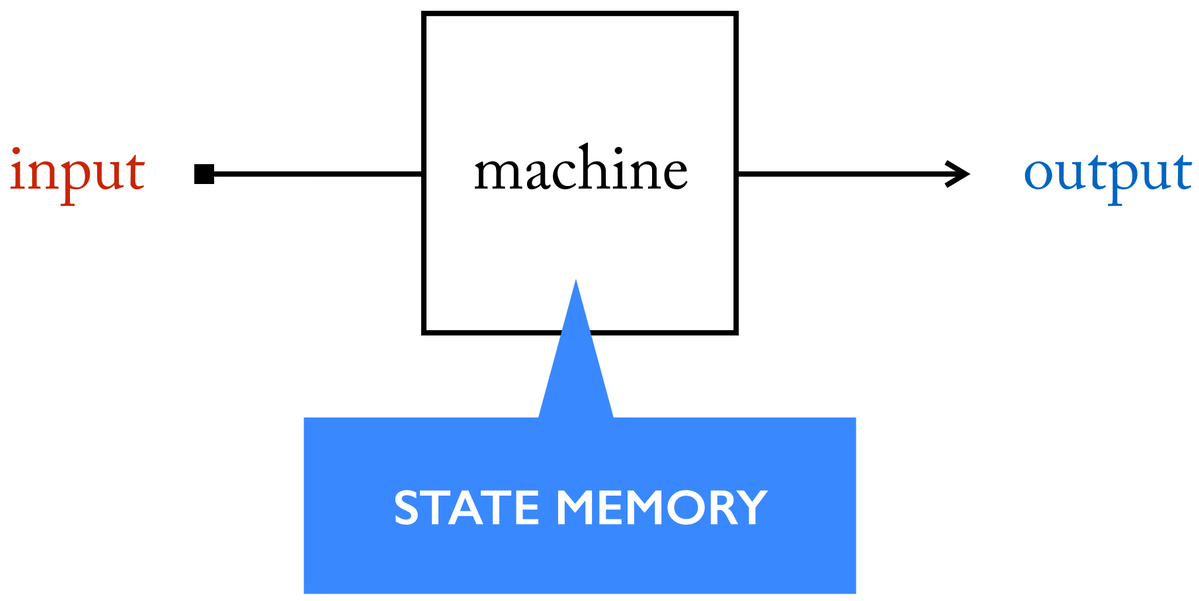
- The paper becomes a 1-dimensional tape.
- The mathematical symbols become 0's and 1's.
- The actions of the pencil become write a 0, write a 1, move left, move right.
- The controller becomes a finite state machine that can read 0's and 1's,

A machine which computes in this way is a Turing Machine (TM).

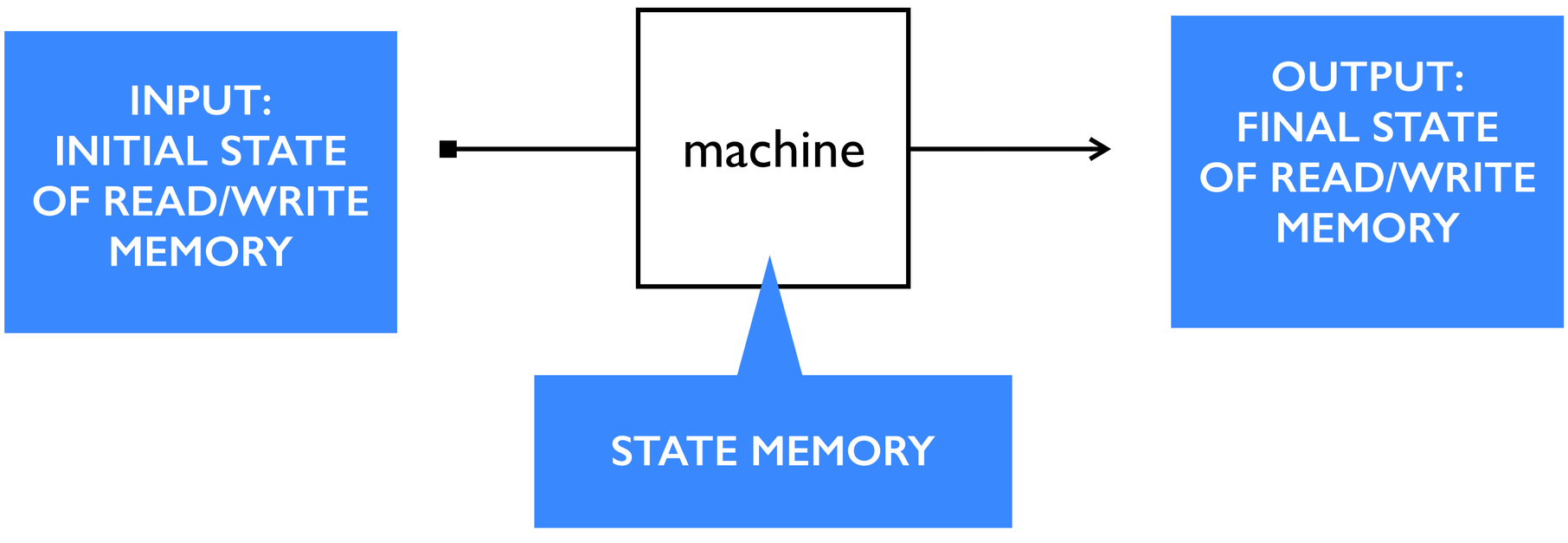


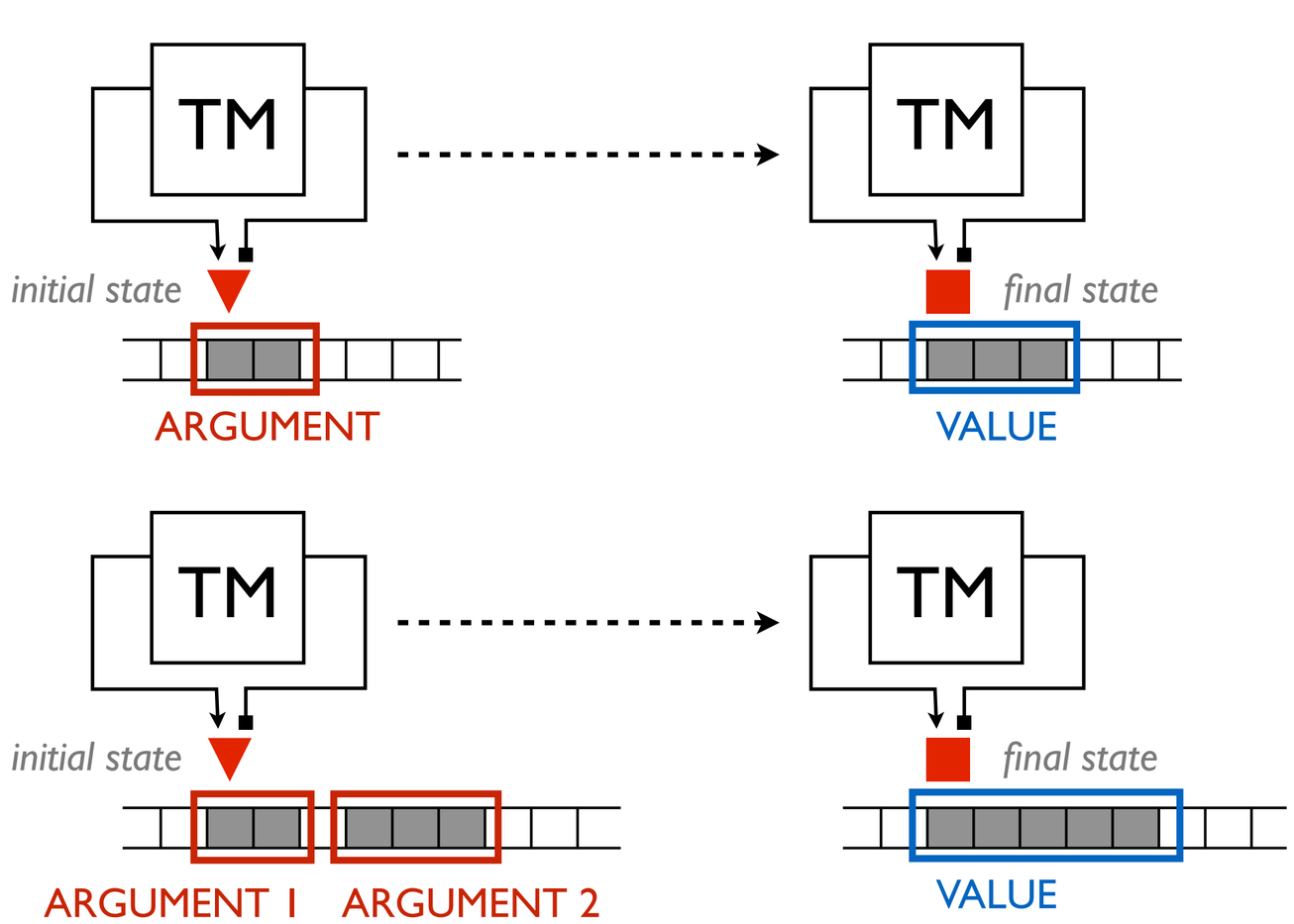
Computing with Memory

FSM: State Memory.
State memory is: (1) short-term; (2) fixed by the architecture of the system.

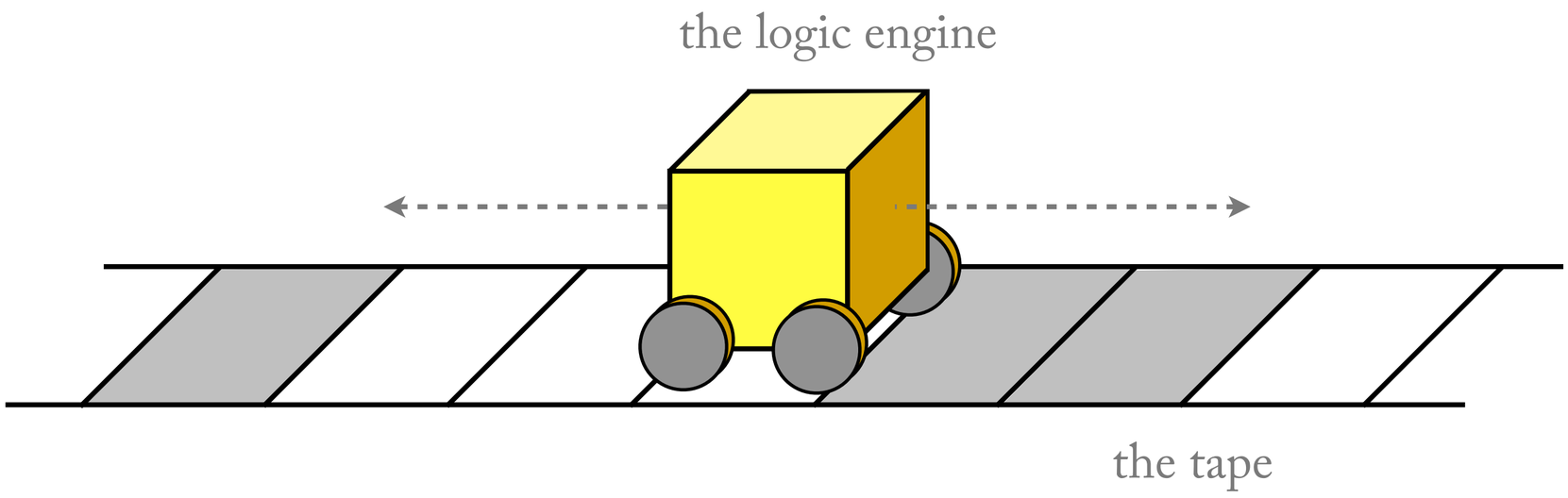


TM: State Memory + Read/Write Memory
Read/write memory is: (1) stable; (2) changeable; (3) extendable within a system.





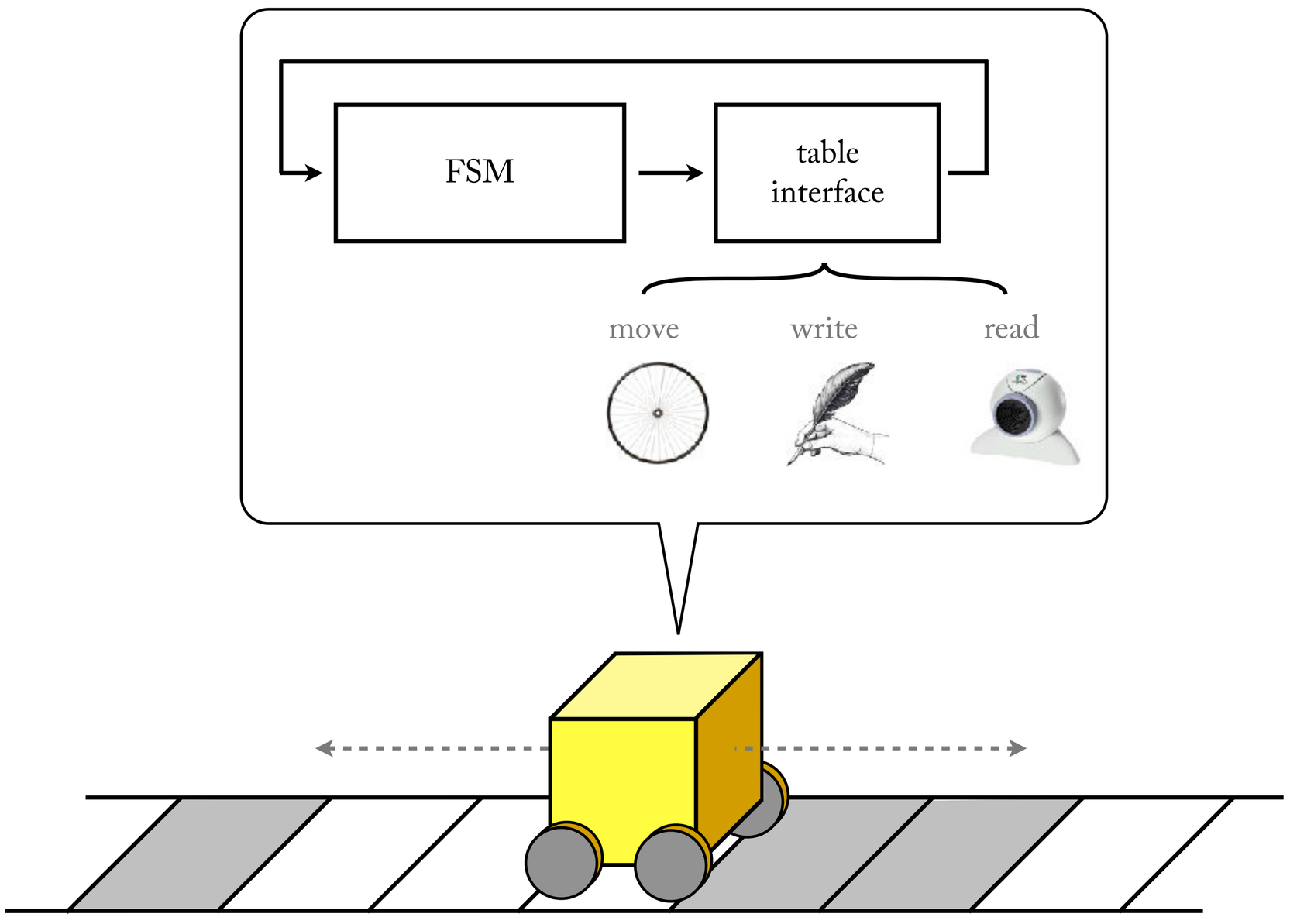
Turing Machine Architecture



A Turing Machine has a two-part architecture:
(i) **A logic engine** (a finite state machine).
(ii) **A read-write memory** (a tape).

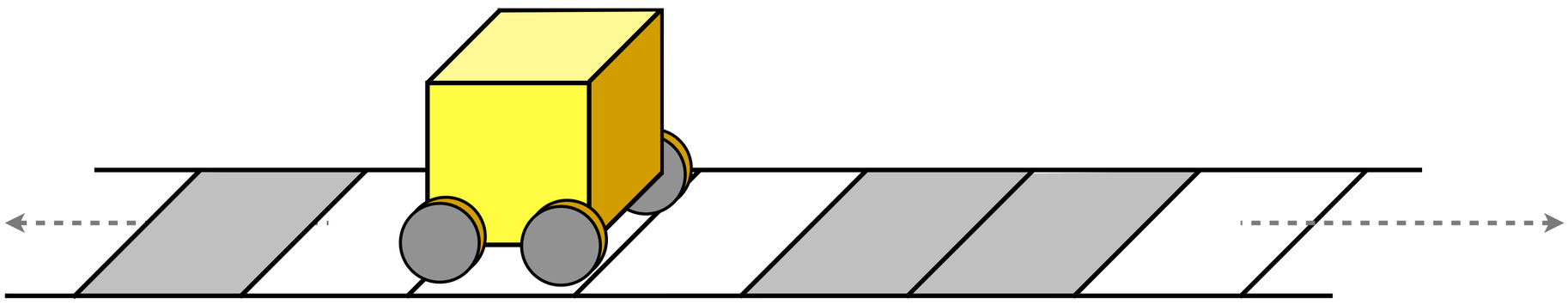
The logic engine is a finite state machine, realized by a sequential circuit. The tape is an infinite strip of graph paper covered with 0's and 1's which the logic engine reads from and writes onto.

In order to work on the tape, the FSM inside the TM is connected to a tape-interface. This interface allows the FSM to move left and right on the tape, write onto it, and read from it. The whole system is connected together in a loop so that the machine calculates, then interacts with the tape, then repeats.



Each reading of the tape becomes the input to the circuit. Each output of the circuit becomes an action of the machine on the tape. The behavior of the machine as a whole depends on both the content of the tape and the content of the memory registers in the sequential circuit.

The Tape



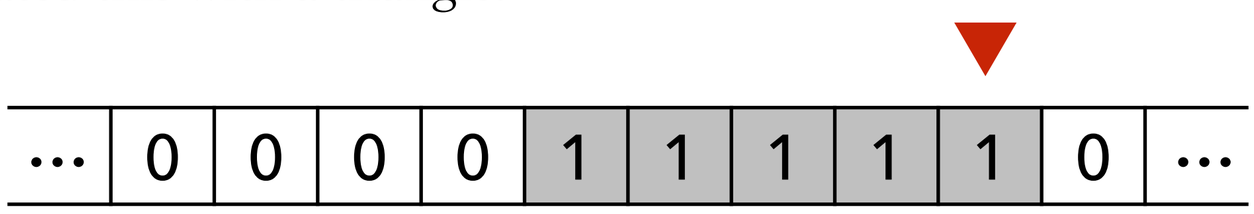
The **turing tape** has the following properties:

- The tape is infinitely long.
- It is divided into cells.
- Every cell is either filled-in (1) or empty (0).
- There are **infinitely** many empty cells (0's) on the tape.
- There are **finitely** many filled-in cells (1's) on the tape.

Machine-Tape Interface

At any given time, the machine occupies exactly one cell of the tape. This is called the **position** of the machine at that time.

We'll indicated this with a triangle:



The machine's abilities are very limited. It can only perform "1-cell" operations: it can only write on, move across, or read one cell at a time.

Actions

The machine can only perform 1-cell actions:

1. **R** = move right one cell.
2. **L** = move left one cell.
3. **1** = make the current cell filled-in.
4. **0** = make the current cell empty.
5. **Halt**

Perceptions

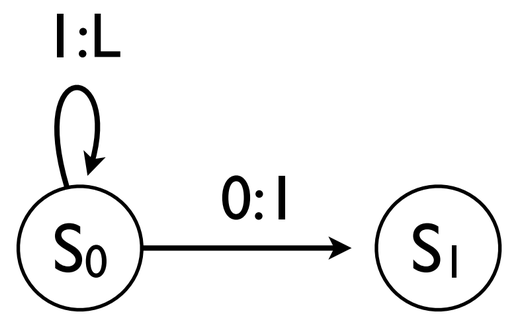
The machine can read only two 1-cell states of the tape:

1. **1** = the current cell is filled-in.
2. **0** = the current cell is empty.

Turing Machine Diagrams

TM diagrams are just like FSM diagrams, where the possible inputs are (for now) 1 or 0, and the possible outputs are R, L, 1, or 0.

This FSM can be realized by a sequential circuit with one input, and several outputs which control a motor.



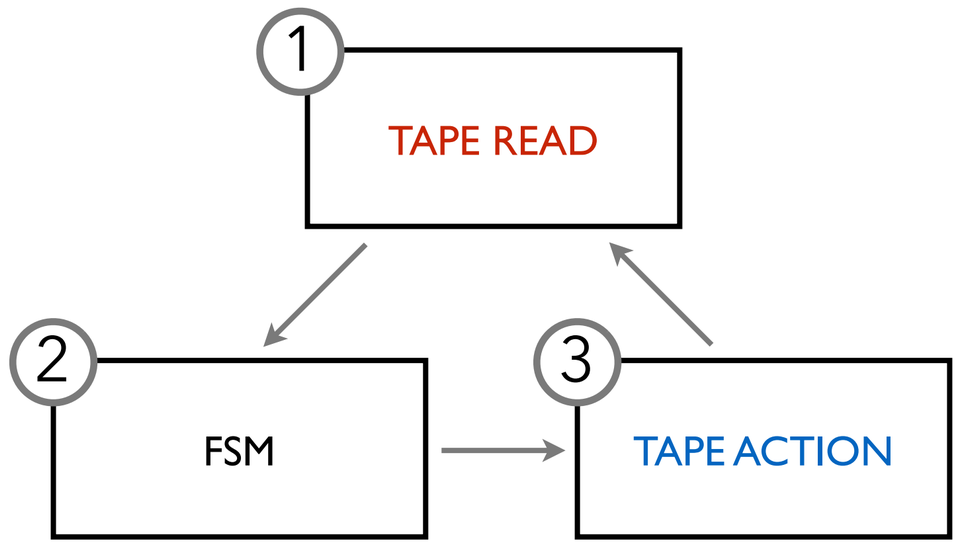
This machine above reads 1's and moves left, until it reads a 0, then writes a 1.

Operation in Time

Order of operations

At each time cycle, the machine undergoes a sequence of actions:

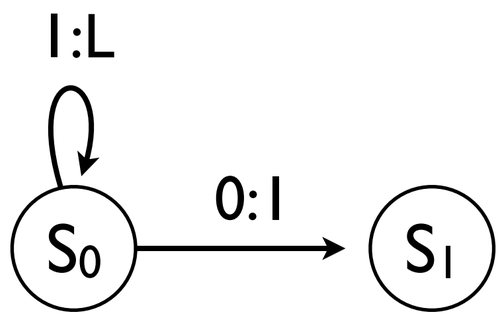
1. Read tape
2. Run circuit
3. Execute tape action (or halt)
4. Repeat



Halting:

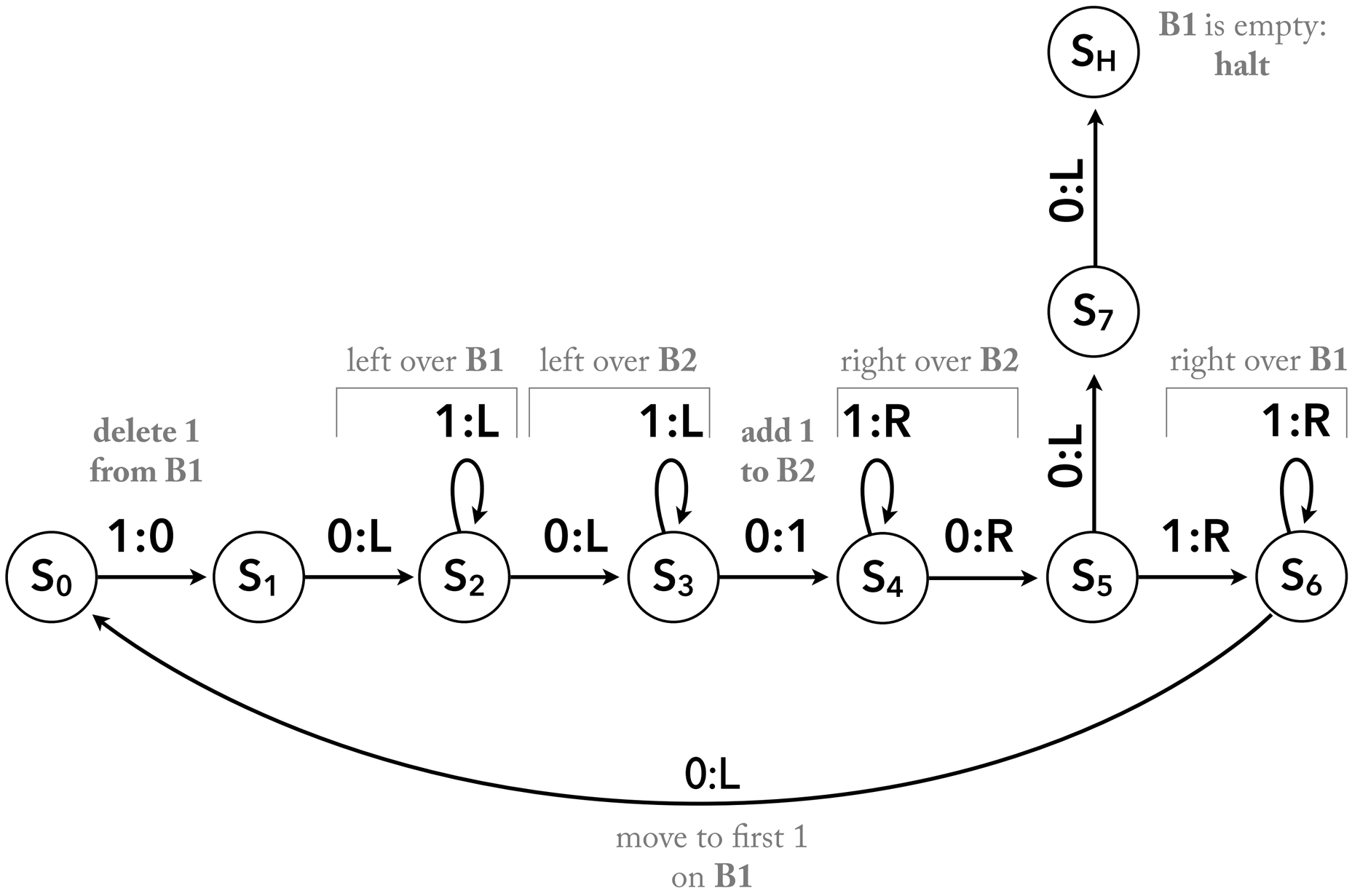
When a machine halts, computation ends. There is no going back.

- Machines do not have to halt. (They can go on forever.)
- A machine halts in a given state, if it is given an input and there are no arrows leaving that state for that input.
- If there are no lines leaving a state, the machine will halt in that state.

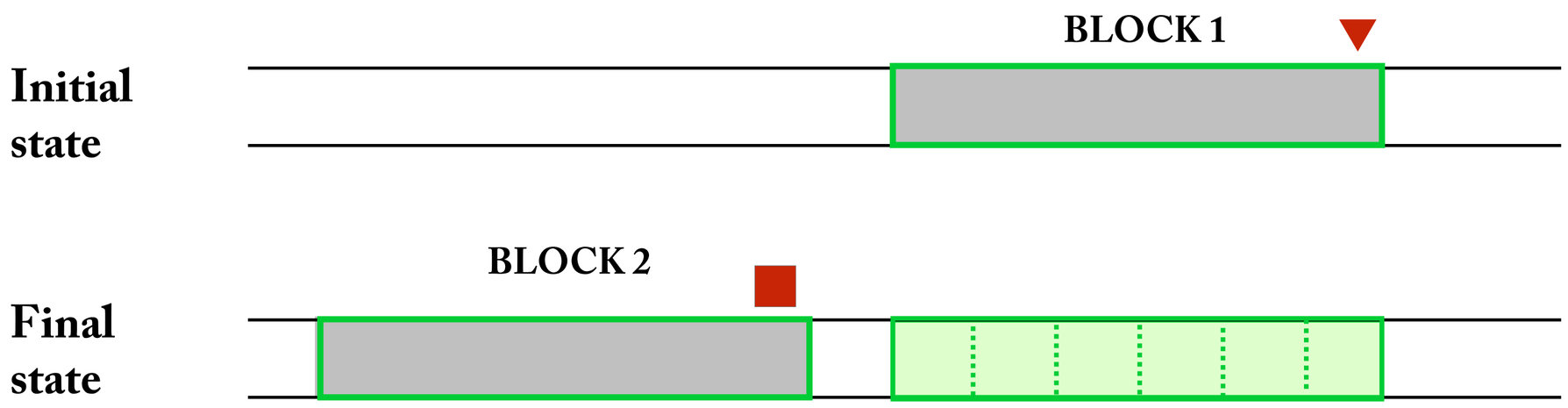


This machine halts in State 1.

Example: Cut and Paste



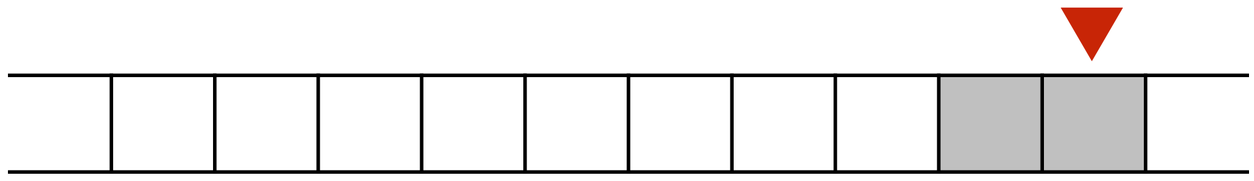
The goal is to shift a block of 1's from the Block 1 position to the Block 2 position, where Block 2 is to the left of the original Block 1 position, separated by 1 cell.



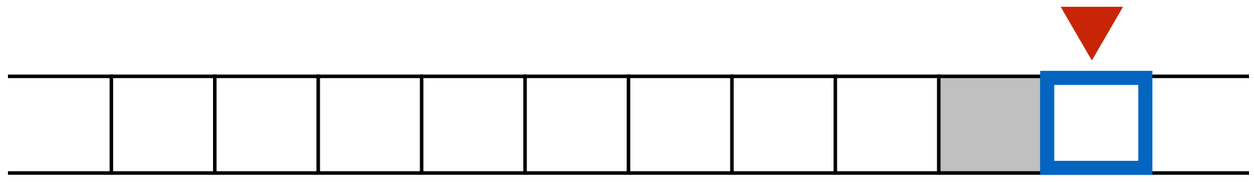
Cut-and-paste:

a) Remove the first 1 at the right edge of B1.
b) Move to the left edge of B1, then move two cells to the left.
c) Skip left over any 1's in B2 (initially there are none), and add a 1 at the end of B2.
d) Go back to B1.
e) Check if it is empty.
   If it is empty, move to B2 and halt.
   If it is not empty, go to the right edge of B1 and return to (a).

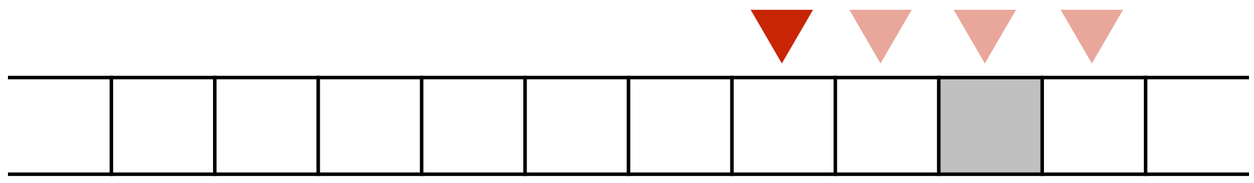
Standard position.



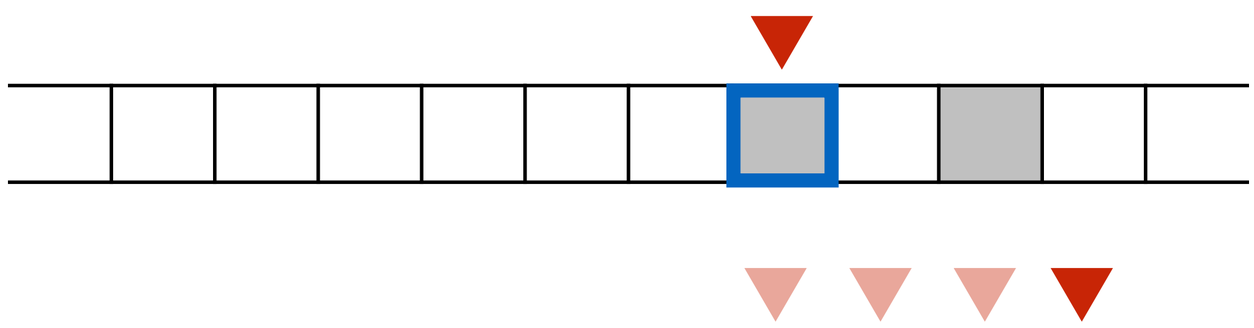
Delete 1 from B1.



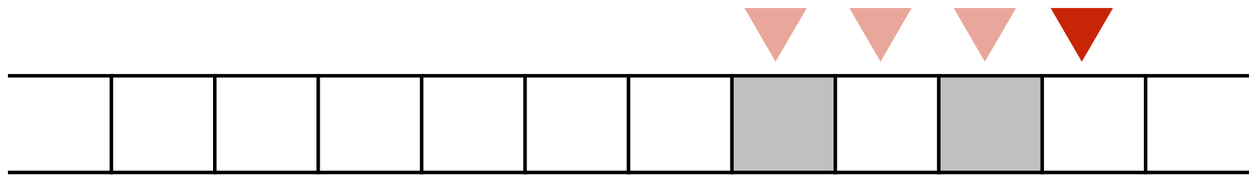
Move left to B2.



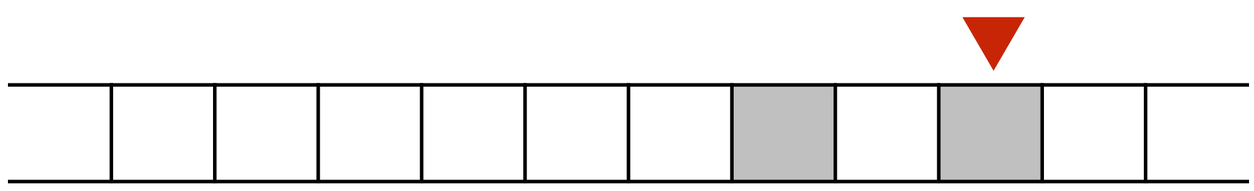
Add 1 to B2.



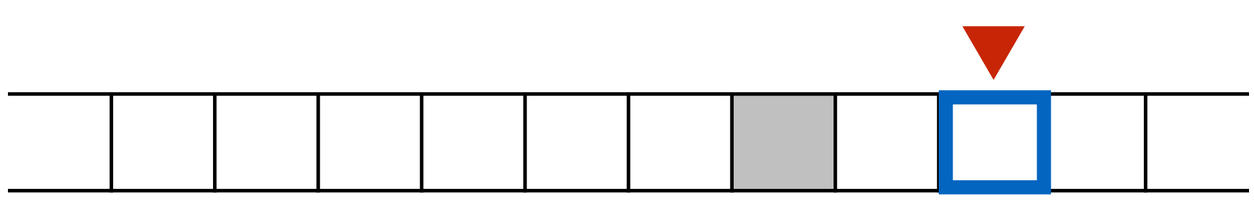
Move right to B1.



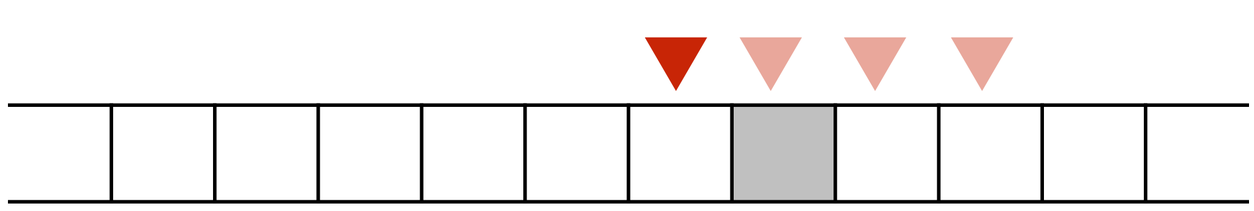
Move to first 1 of B1.



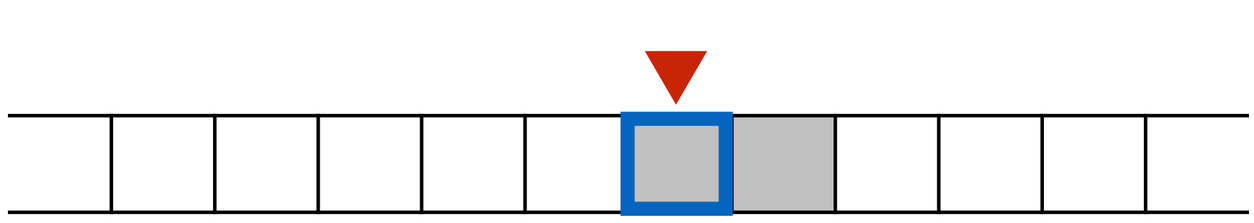
Delete 1 from B1.



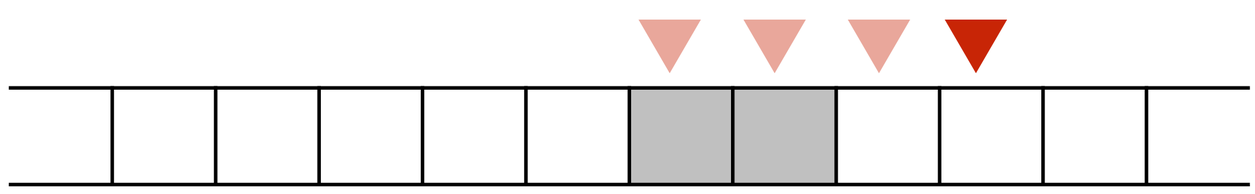
Move left to B2.



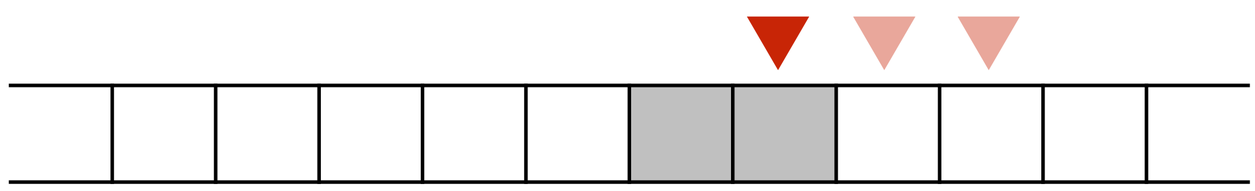
Add 1 to B2.



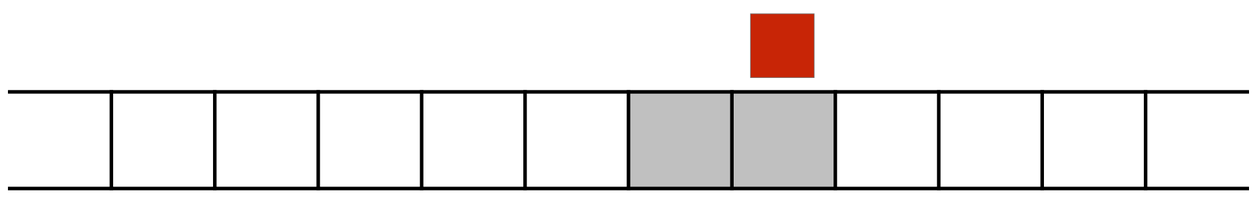
Move right to B1.



If B1 is empty, move to B2.



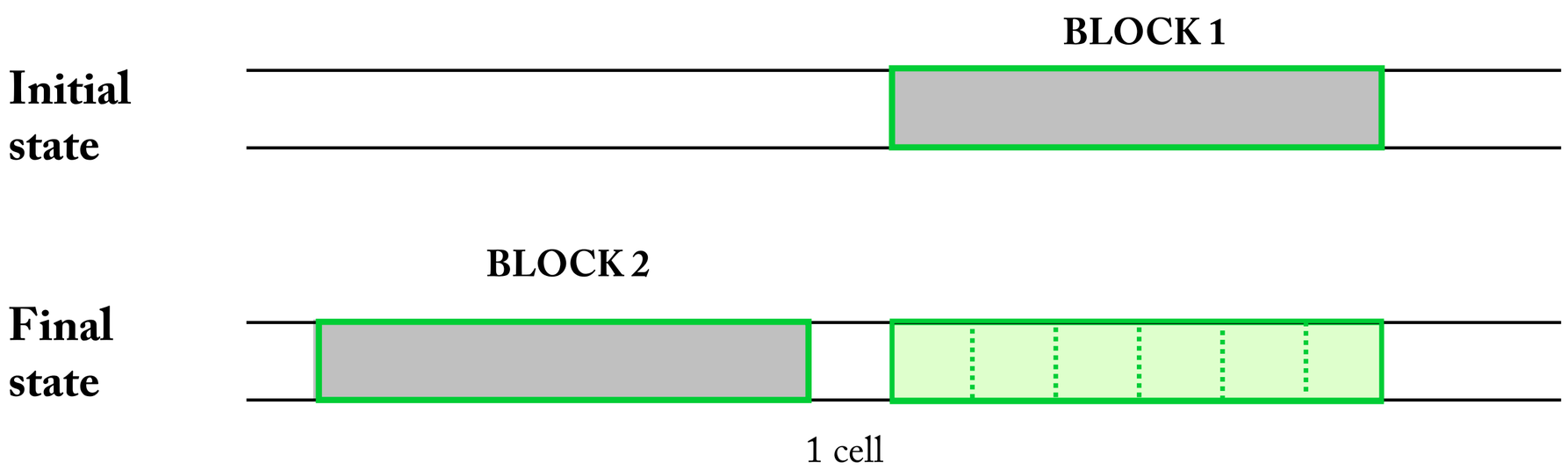
Halt.



In-Class Problems

Start on the right most cell (the "right edge") of the block in all problems.

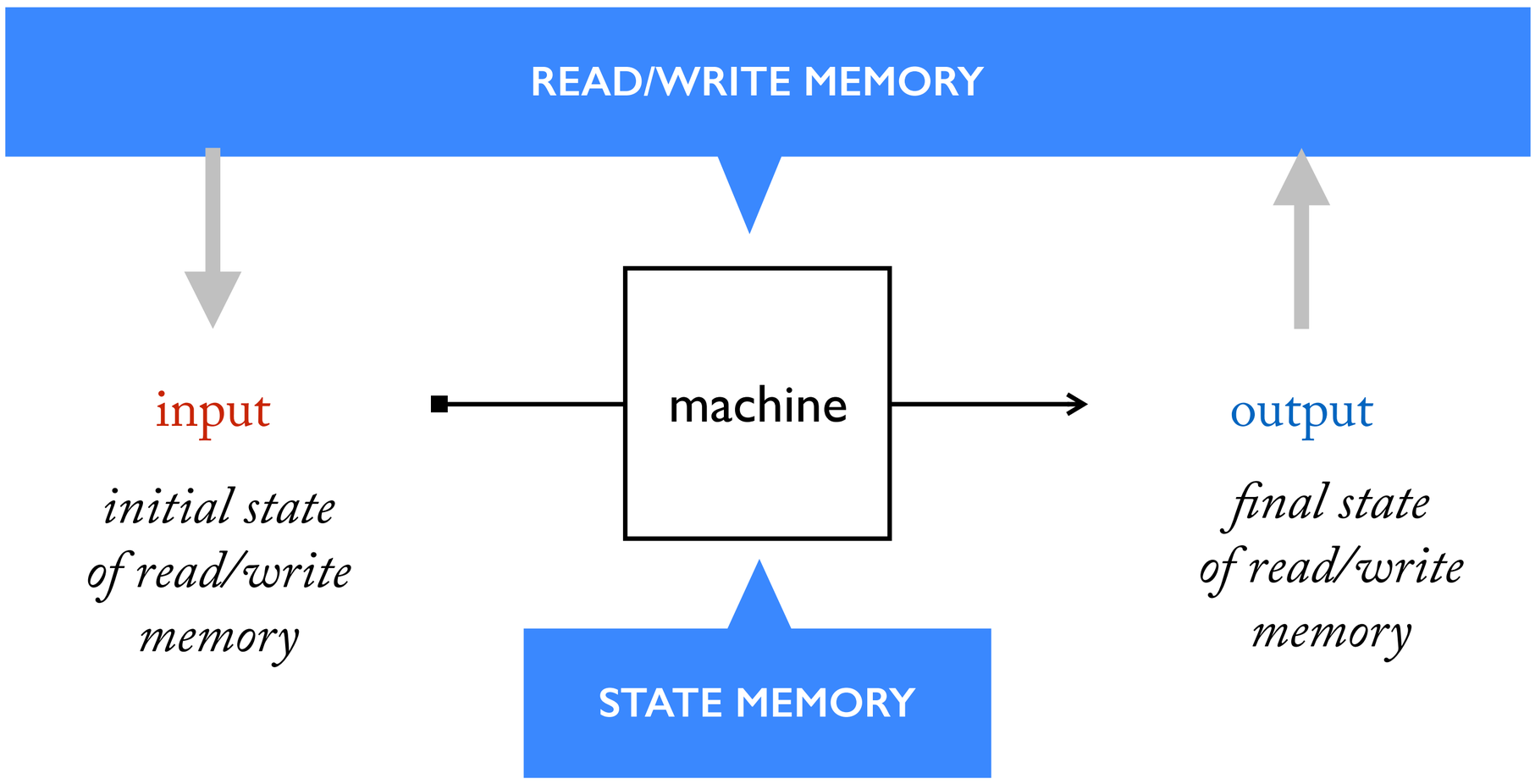
1. **To the left gap:** Move left over a block of 1's, and halt on the first blank space to the left.
2. **To the left gap and back:** Move left over a block of 1's until you reach the gap, then return to the starting point and halt.
3. **To the left gap and +1:** Move left over a block of 1's, add a 1 at the end, then halt .
4. **To the left gap, +1, and back:** Move left over a block of 1's, add a 1 at the end, then return to the starting point and halt.
5. **Infinite expander:** Add a 1 at the left end of a block, then add a 1 at the right end of a block, and repeat forever.
6. **Cut-and-paste:** shift a block of 1's from the Block 1 position to the Block 2 position, where Block 2 is to the left of the original Block 1 position, separated by 1 cell.



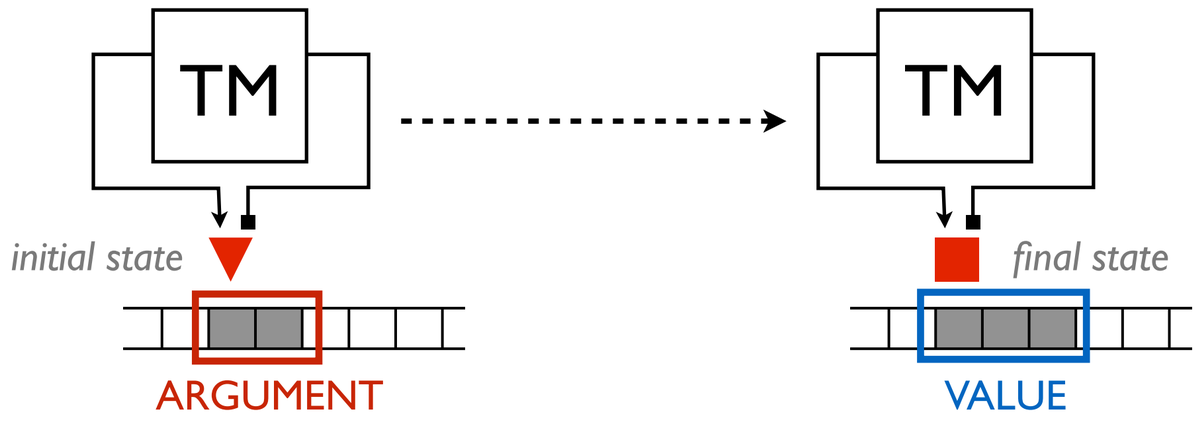
28. Computing with Turing Machines

Computing with Read/Write Memory

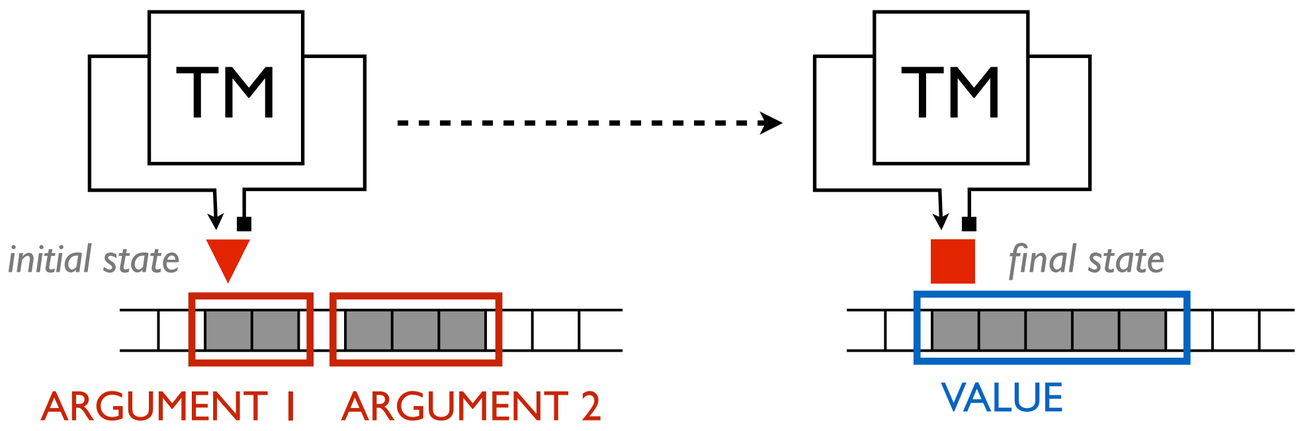
For TMs, the read/write memory serves a dual role. It functions as the input and output of the machine. But it also functions as "scratch paper" that the machine uses in the course of executing its algorithm.



Computing a one place function:



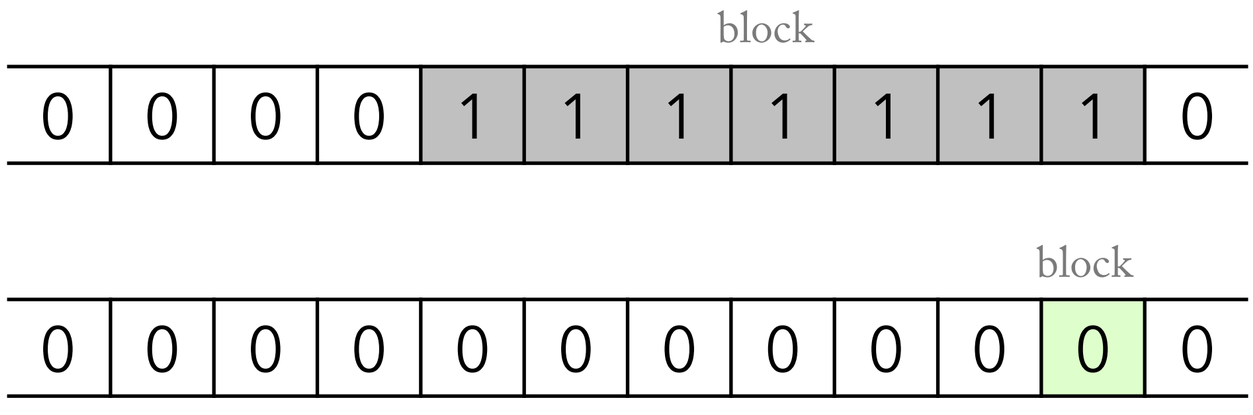
Computing a two place function:



Computing in Tally with TMs

Definition of a tally block

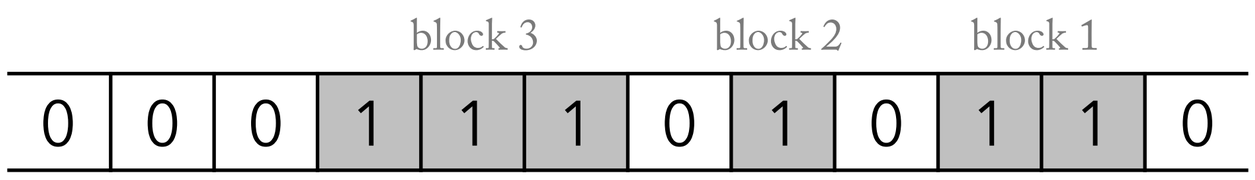
A tally block is either (a) one or more consecutive 1's; or (b) a single empty cell, that serve to represent the argument to a function.



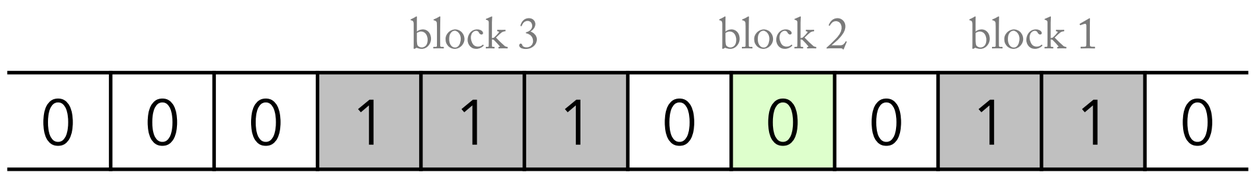
Definition of a tally input

The initial state of a tape counts as **an input** for computing a function *f* iff:

Condition 1: the tape contains a series of blocks, each separated by a single 0.



If a block is empty, it still takes up one cell.

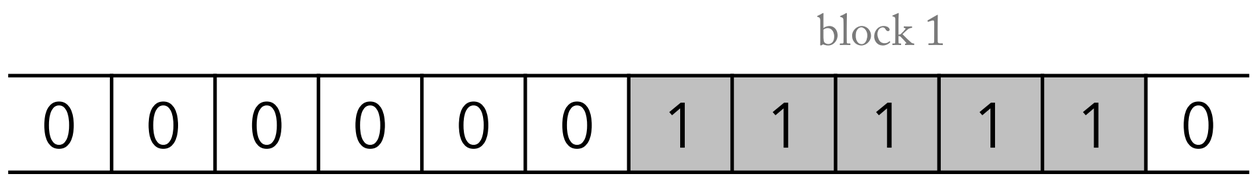


The initial blocks may be located anywhere on the tape.

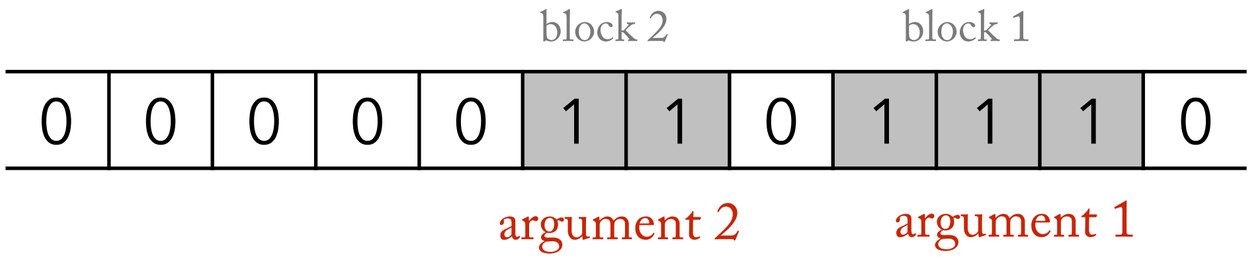
Condition 2: there are as many blocks on the tape as there are arguments for the function being computed.

Note: if the machine in question is a sub-routine of a larger machine, there may be other blocks on the tape that are irrelevant to its operation.

$f(x) = x + 1$

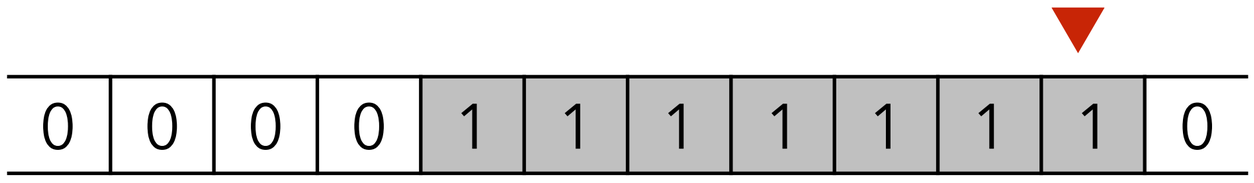


$f(x,y) = x + y$

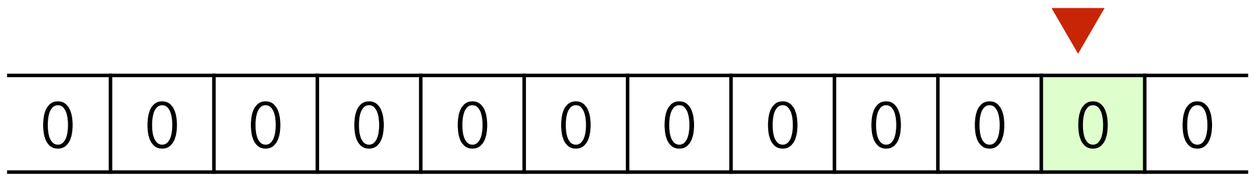


Counting from the right, the first block on the tape corresponds to the first argument of the function computed, the second block corresponds to the second argument, and so on.

Condition 3: the machine starts in **standard position.** For tally, standard position is defined as **the right most cell of the right most block**.



Even if the block is empty.



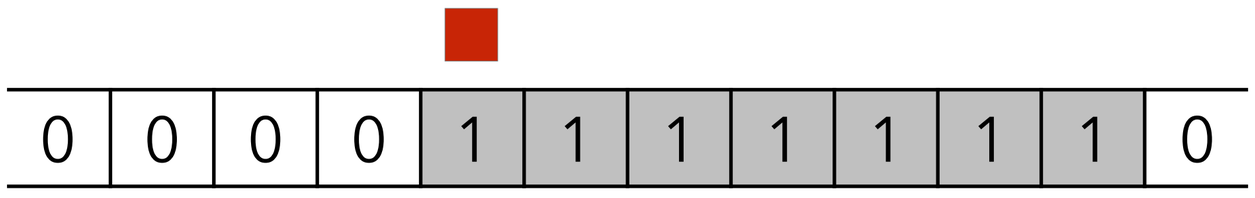
Note: If all blocks are empty, it doesn't matter where on the tape the machine starts.

Definition of a tally output

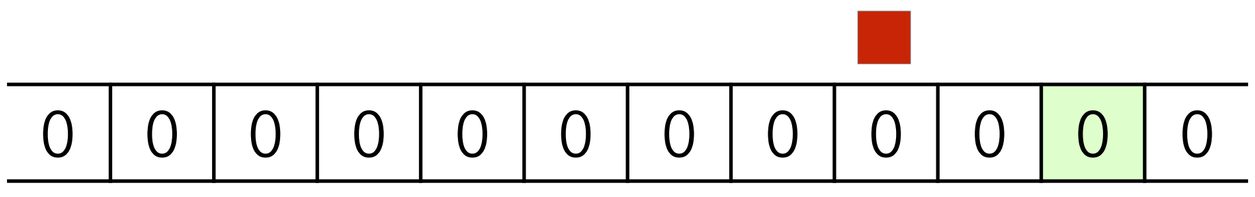
The initial state of a tape counts as **an output** for computing a function *f* iff:

Condition 1: the machine halts at some point.

Condition 2: when the machine halts, there is only a single block left on the tape.



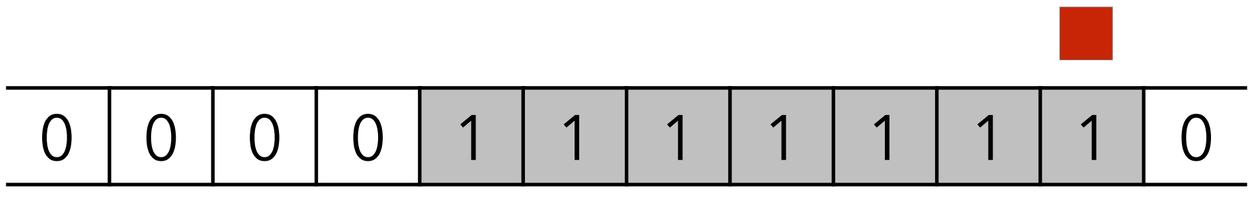
The final block may be empty:



The final block may be located anywhere on the tape.

In some circumstances, a third condition will be added. You can assume it does *not* apply, unless explicitly required.

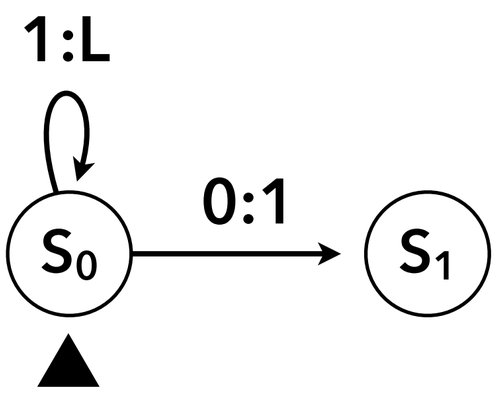
(Condition 3) The machine halts in standard position.



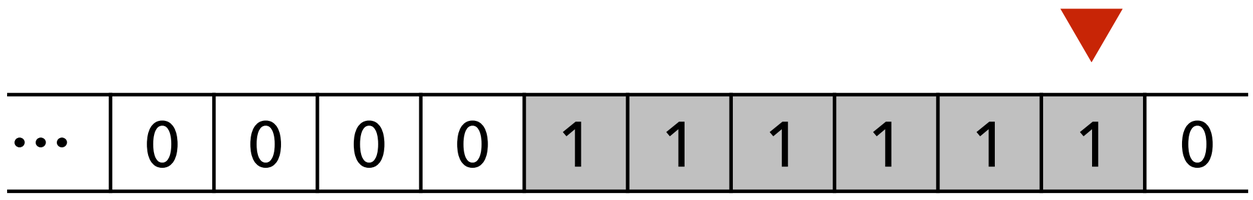
Note: standard position here is the right most cell of the right most block. It is *not* necessarily the same standard position you started in.

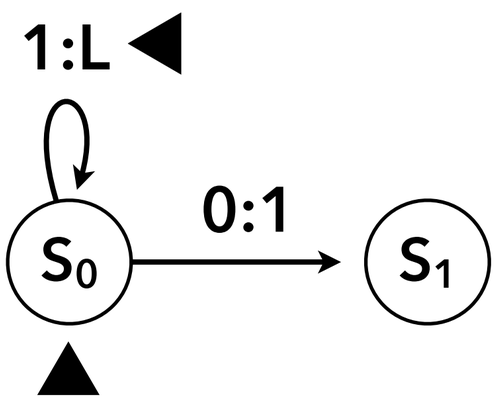
Note: if the final block is empty, then Condition 3 is satisfied by halting anywhere.

Example: +1 T

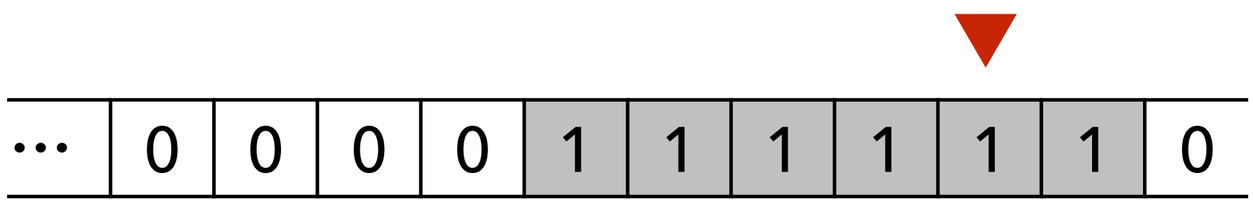


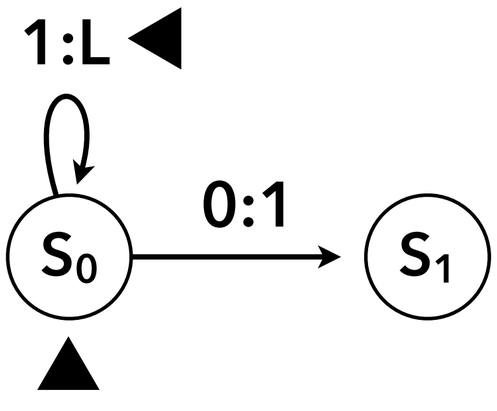
Start with S_0 in the machine, and standard position on the tape.



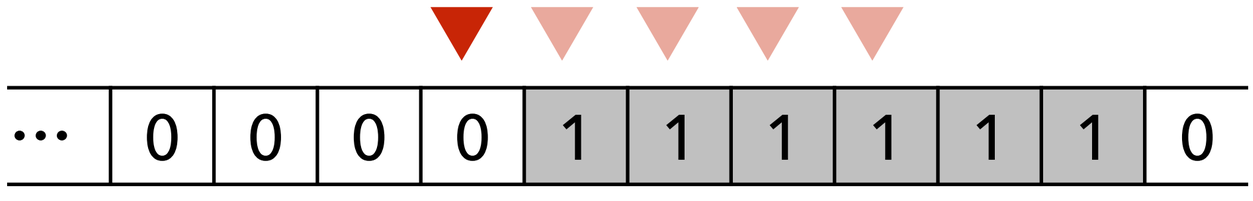


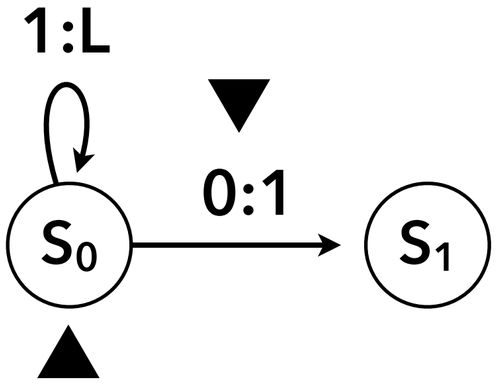
If you read a 1, move left one cell.



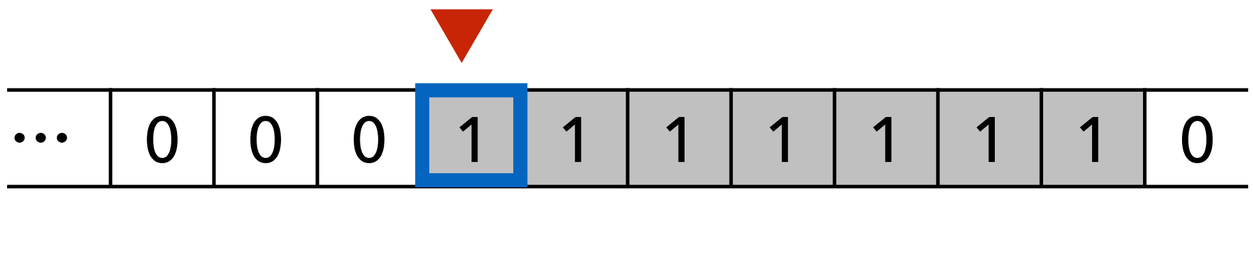


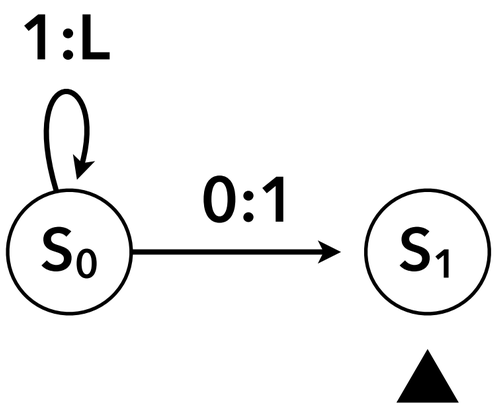
This operation repeats five more times….



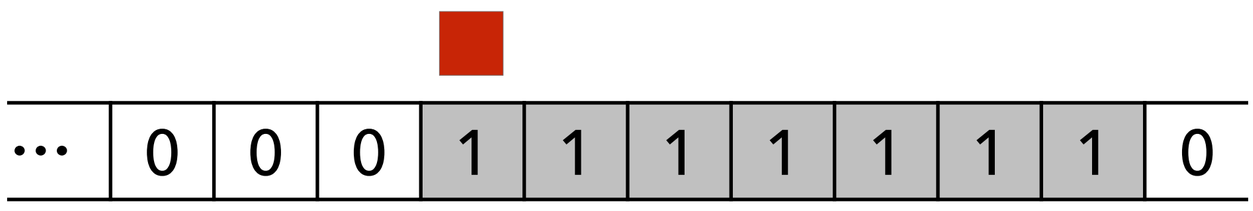


If you read a 0, write a 1.





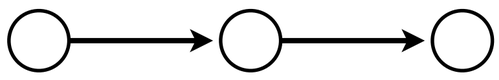
There are no conditions for exiting S_1, so the machine halts.



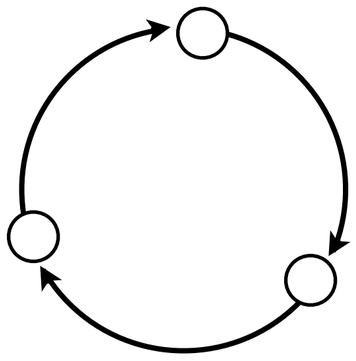
29. Recursion

The Concept of Recursion

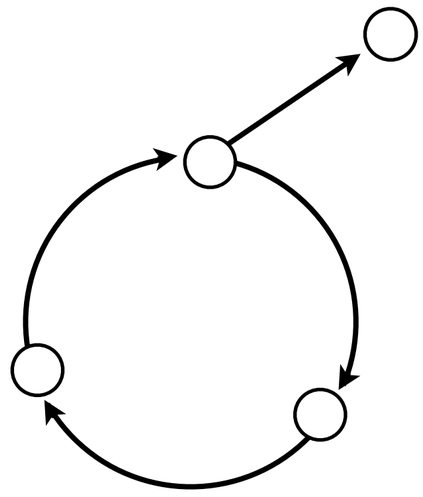
A recursive algorithm repeats some process with slight modifications until some goal is reached. "Recursive" means repeating. Recursive algorithms occur not only in computation, but also in natural processes.



A finite sequence of states.



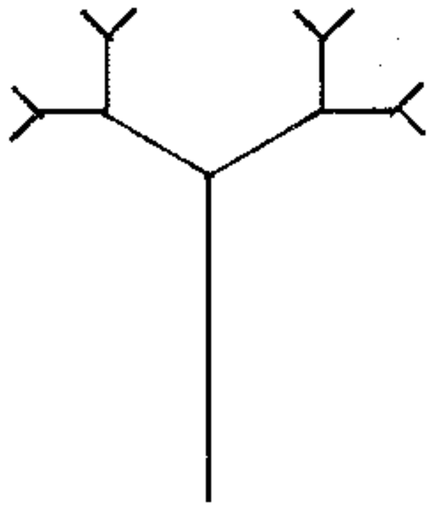
A loop of states which repeat infinitely.



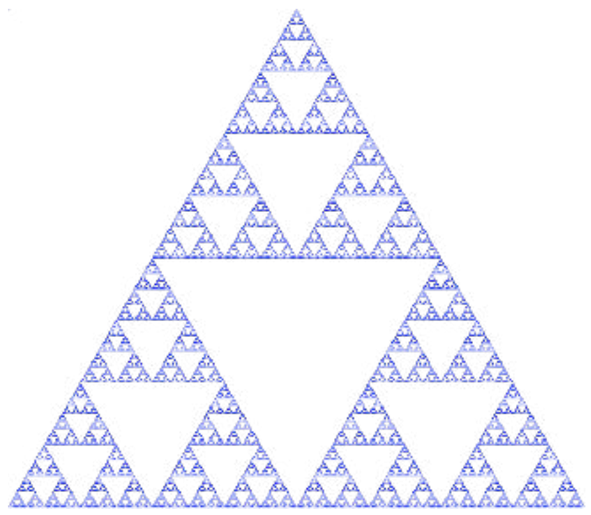
A loop which repeats a finite number of times until an exit condition is reached.

Recursive algorithms can be used to generate drawings with elements that repeat, but also change a little bit, after each loop.

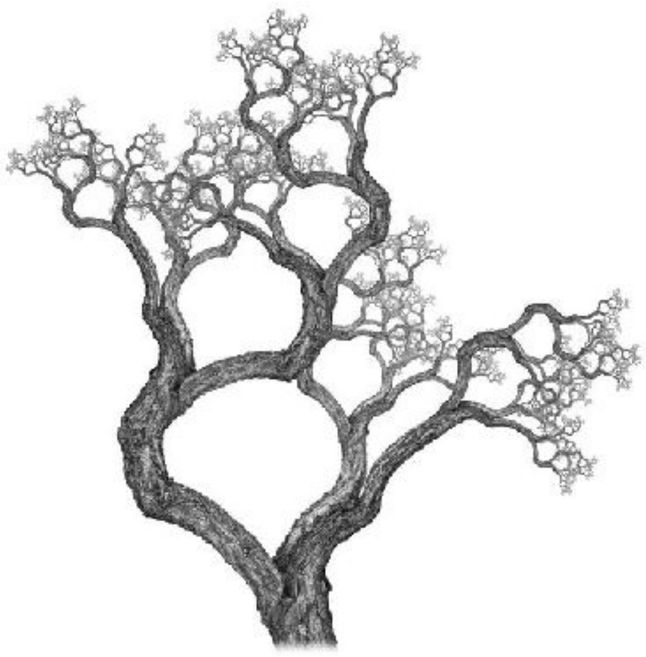
```
TO TREE :SIZE
FORWARD :SIZE
IF :SIZE <1 STOP ELSE TWO-
TREES SIZE/2
BACK :SIZE
END
```



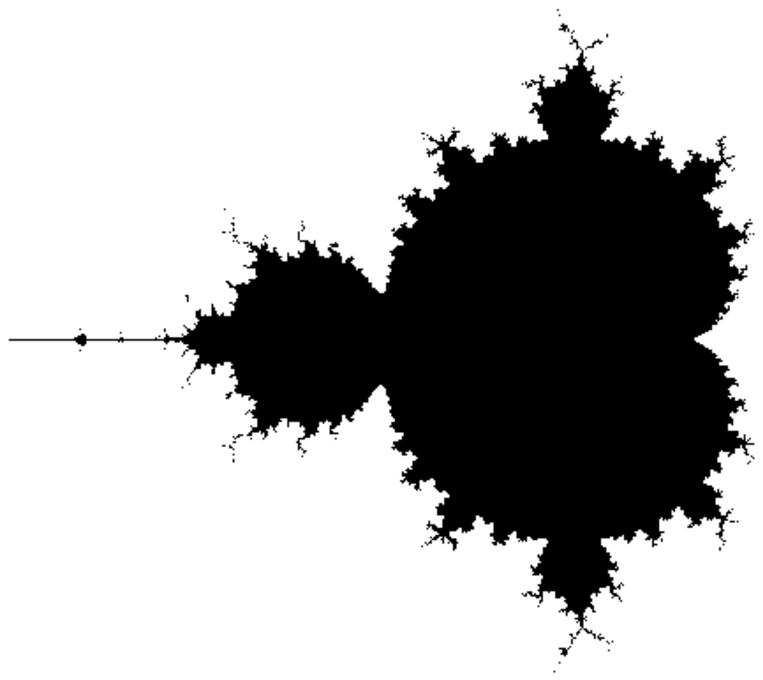
This program written in LOGO describes an algorithm for drawing simple trees.



How would you generate Sierpinksi's triangle?



The same technique can be used to draw more realistic-looking trees…

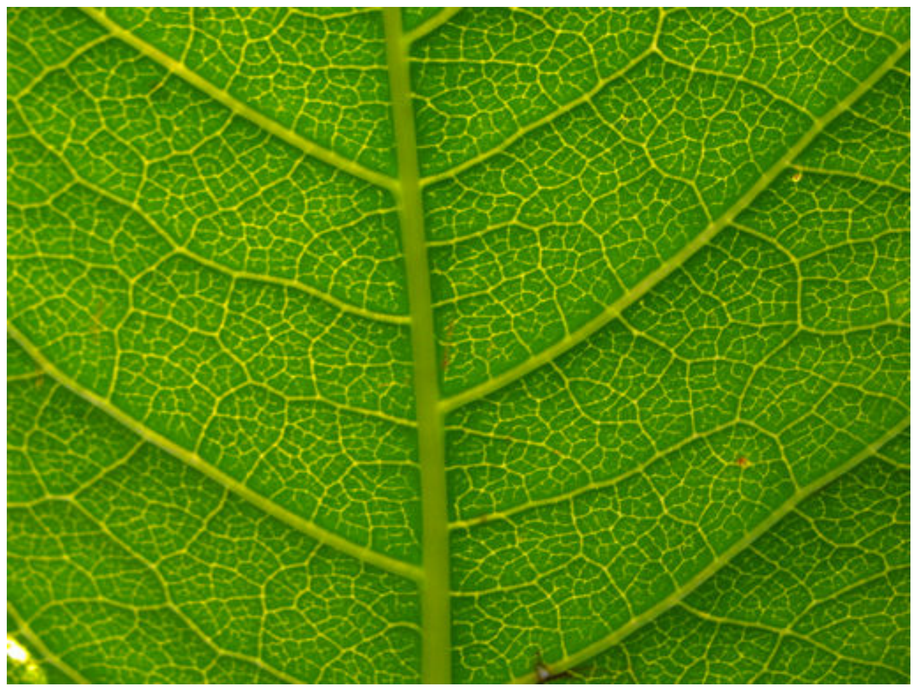


… or bizarre and complex fractals.

Recursion in Nature

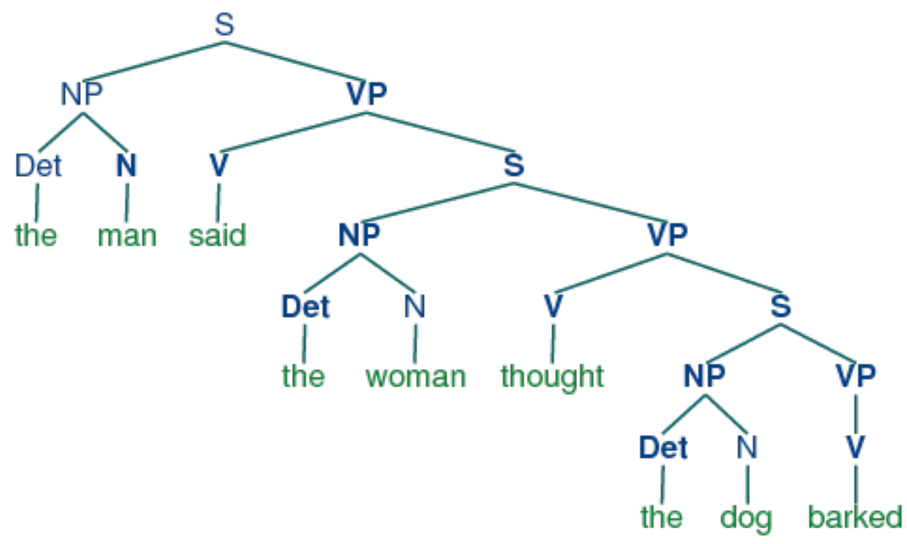
The natural world is filled with recursive structures. Consider a tree: how are the branches generated? How is each leaf on a branch generated? How is each cell is a leaf generated?







Cell growth works by repeating the same kind of operation over and over, with slight modifications made for a single kind of structure (an organ, a leaf), and larger modifications.



Famously, language is also thought to have a recursive structure. Sentences are formed by recursively combining parts of speech.

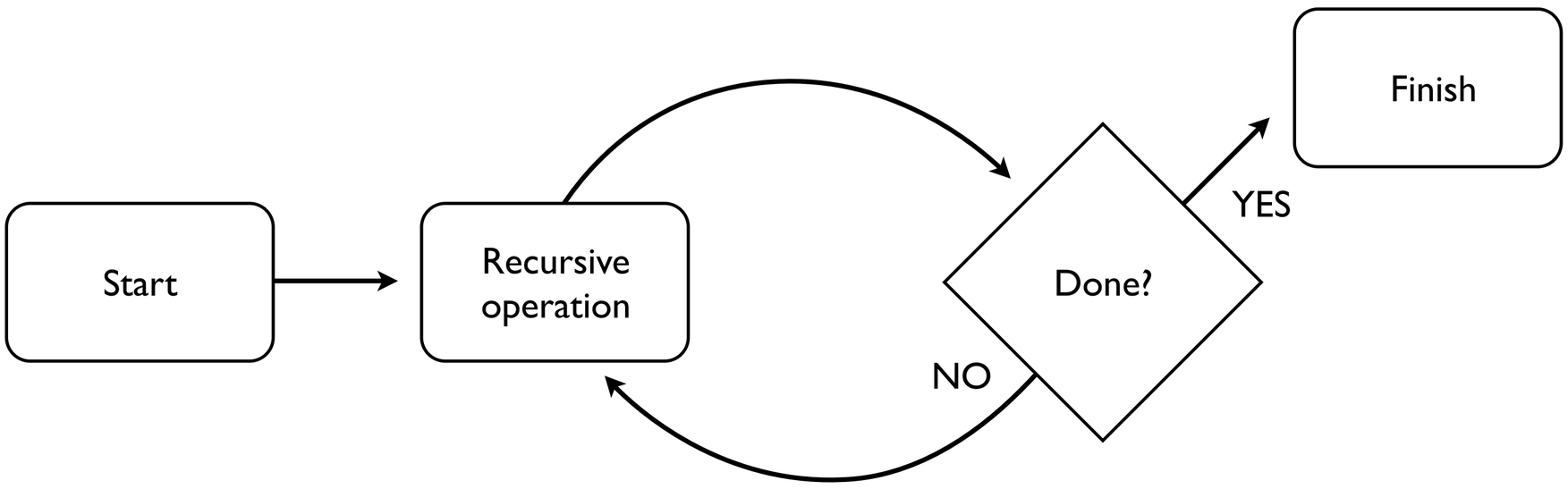
Both cellular growth and sentence structures are driven by recursive operations. But do they have the same root cause?

Perhaps language is recursive *because* biological growth is recursive. Somehow the biological structure of the brain requires it. Perhaps. A more plausible alternative is that both recursive for the same reason. Recursive algorithms are the central mechanism for generating indefinite complexity from a finite base of instructions.

The same pressures which made biology recursive are also what make language recursive, and it is the same pressure which makes computer programs recursive. All face the same kinds of ecological pressure: infinite ends, finite means.

The Architecture of Recursion

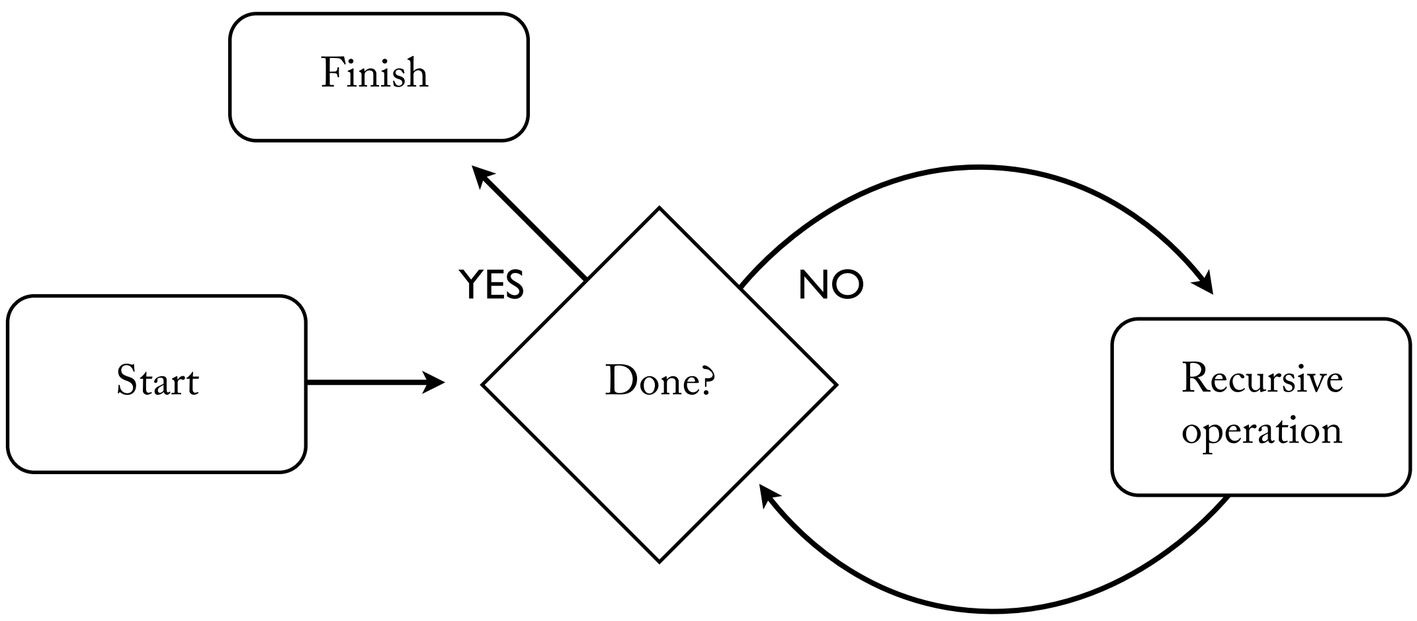
Recursive algorithms have a characteristic architecture. Here is a typical design:



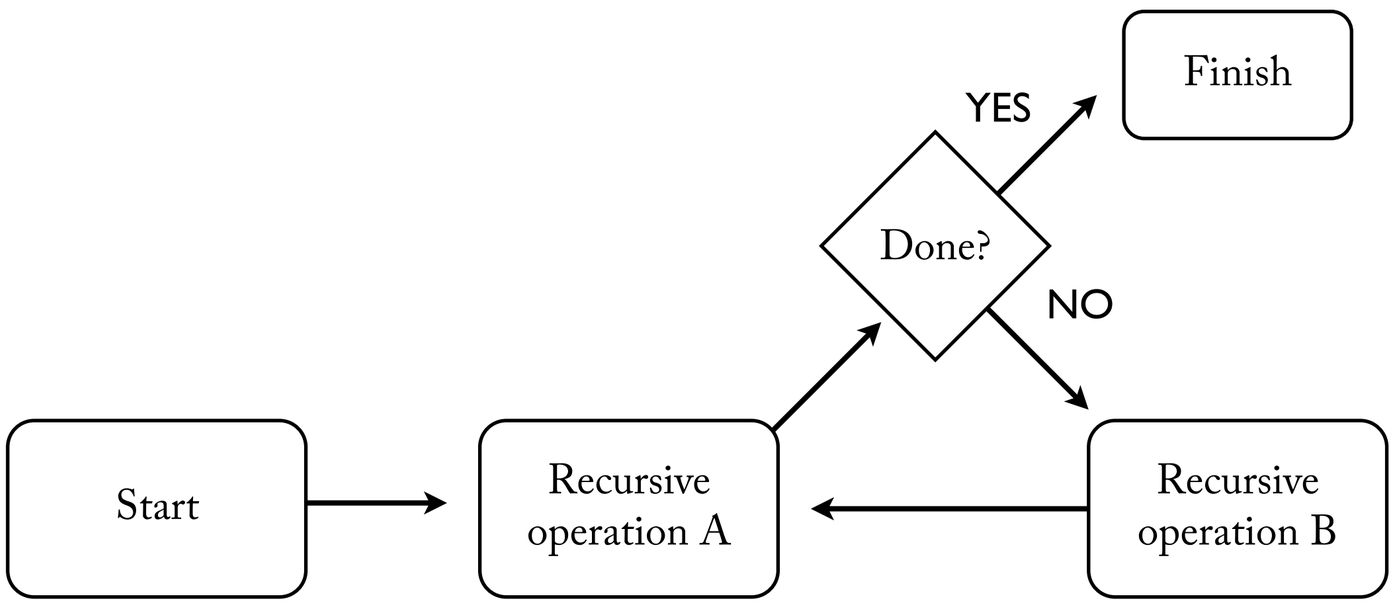
A recursive algorithm has three key elements:

(1) a **recursive operation** which is a repeated;
(2) a way to **test** whether the process should be repeated again;
(3) an **exit operation** when the process is complete.

The three elements may appear in nearly any order. You'll have to find the order best suited for the problem.



Often the recursive operation will itself involve several discrete operations, and these can be re-arranged as well. The underlying conceptual structure is the same.



Note: these diagrams are **flow charts**, a common system for describing algorithms. See the Section "Flow Charts" for more about creating and reading these diagrams.

Case Study: 2*x* T

The goal is to design a TM computes $2x$ in Tally. For present purposes assume $x \geq 1$. Here it is instructive to remember what multiplication really means. Multiplying x by 2 is equivalent to adding 2 to 0, x times. For example:

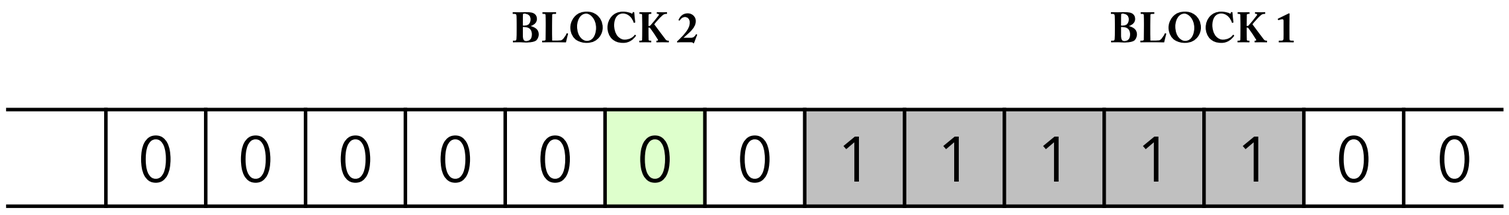
$$\begin{aligned} 2 \bullet 3 &= 0 + 2 \\ &= 2 + 2 \\ &= 4 + 2 \\ &= 6 \end{aligned}$$

3 times

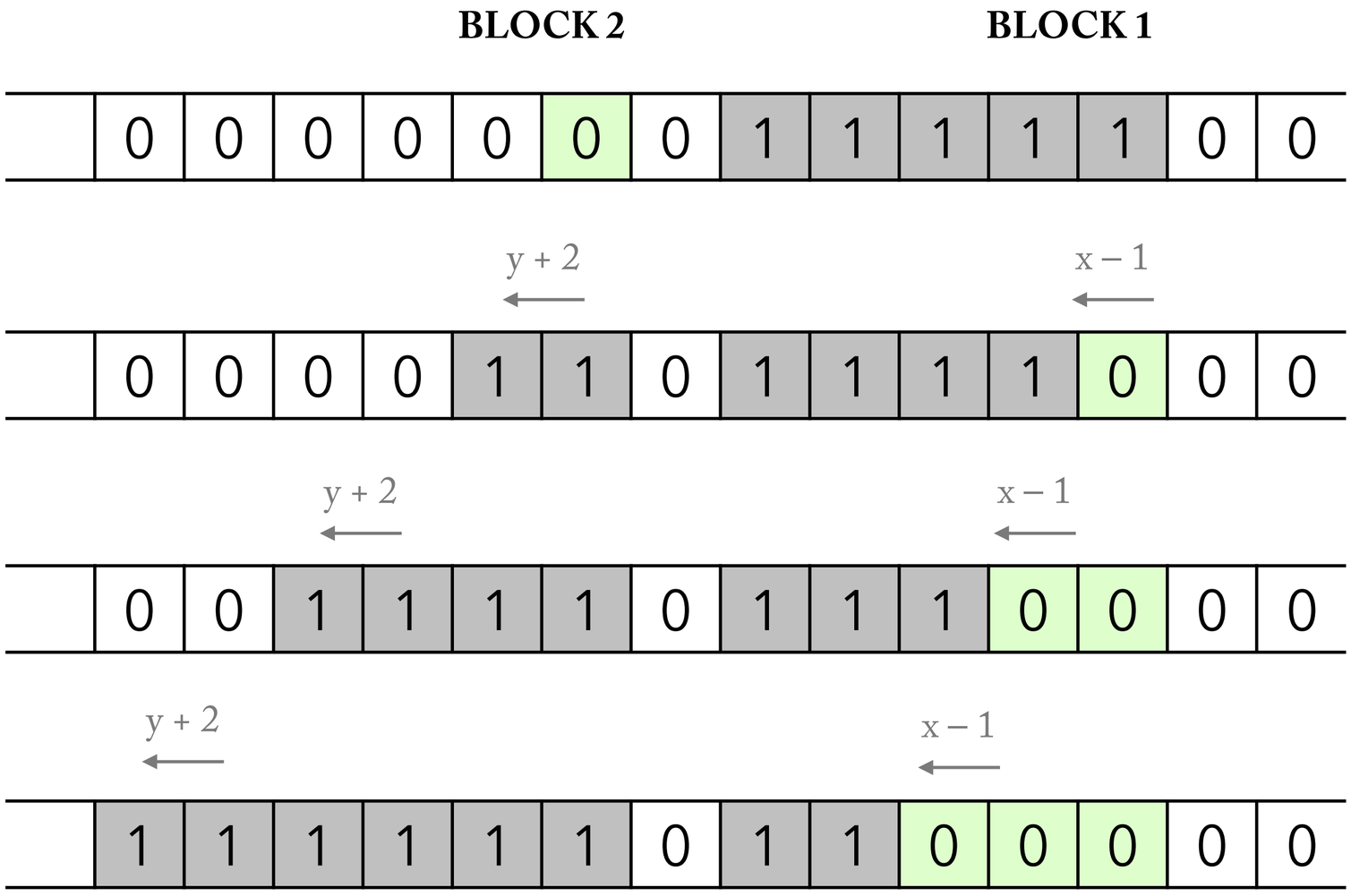
$$\begin{aligned} 2 \bullet 5 &= 0 + 2 \\ &= 2 + 2 \\ &= 4 + 2 \\ &= 6 + 2 \\ &= 8 + 2 \\ &= 10 \end{aligned}$$

5 times

To approach this problem, let us imagine two blocks on the tape. Block 1 (B1) represents the argument x. Block 2 (B2) represents the value $2x$. B1 starts with a value, but B2 starts empty.

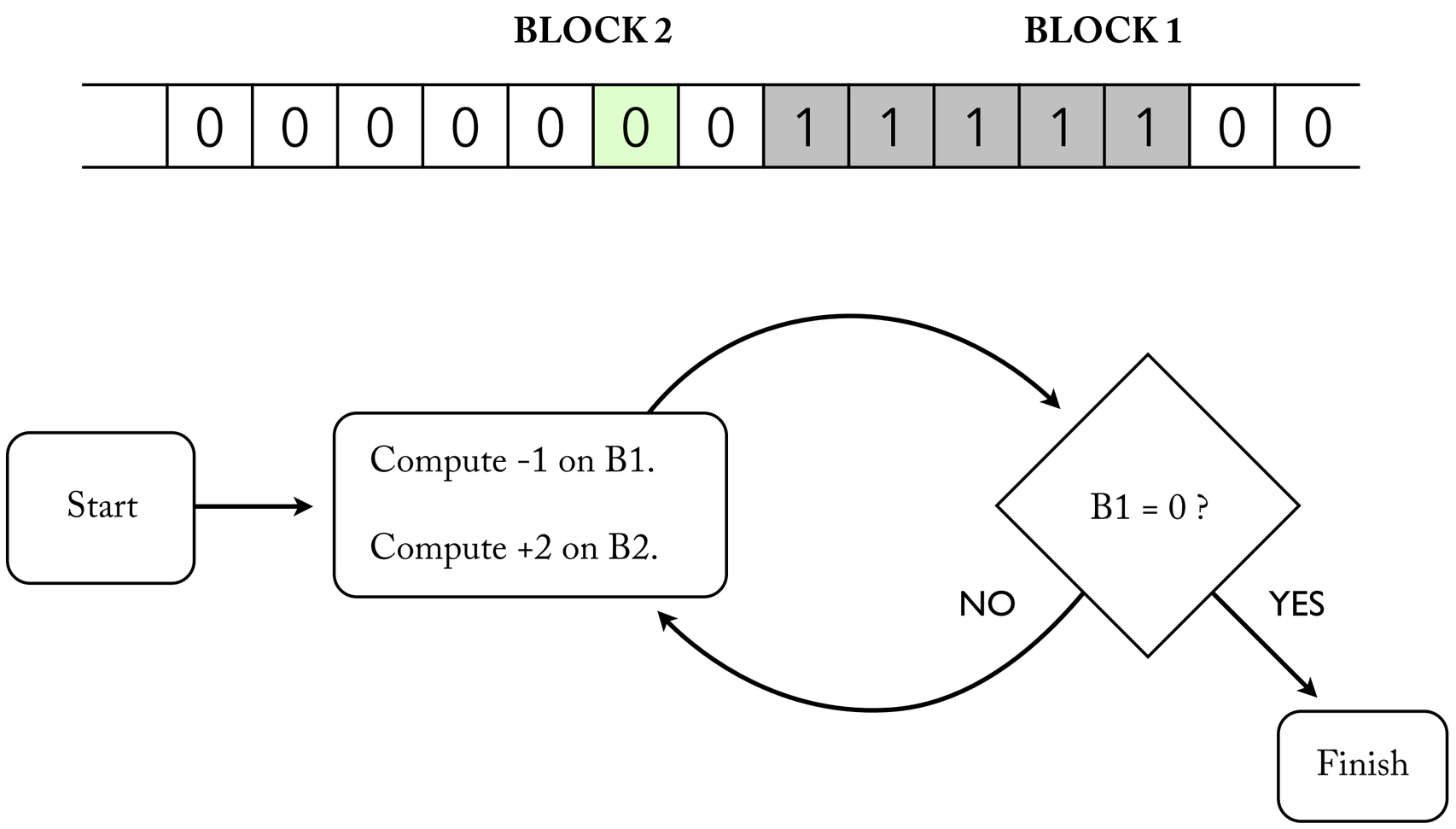


The strategy is this: let us repeatedly remove 1 from B1, and each time we do so, add 2 to B2. In other words, each time we compute x-1 on B1, we'll then compute y+2 on B2. When B1 is empty, B2 should have twice as many cells filled as B1 did originally.

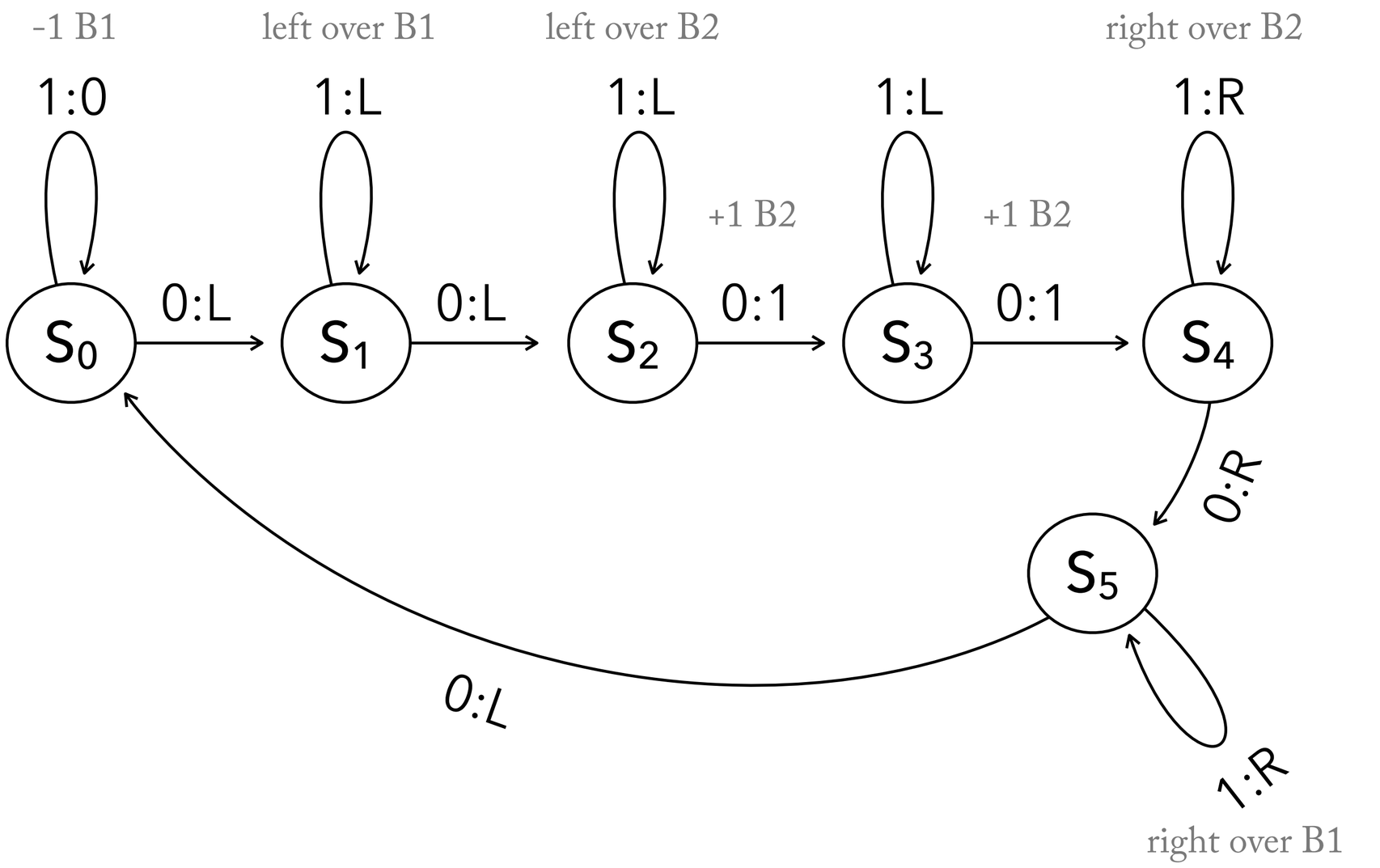


Here B1 functions as **counter**: it tells us how far we are into the comptuation. B2 functions as the **answer.** Nearly all recursive algorithms will include both elements. **Note:** here I remove 1 from the right side of my counter, but the left side would work just as well.

Before drawing up our TM, let us convert our strategy into an explicit flow chart:

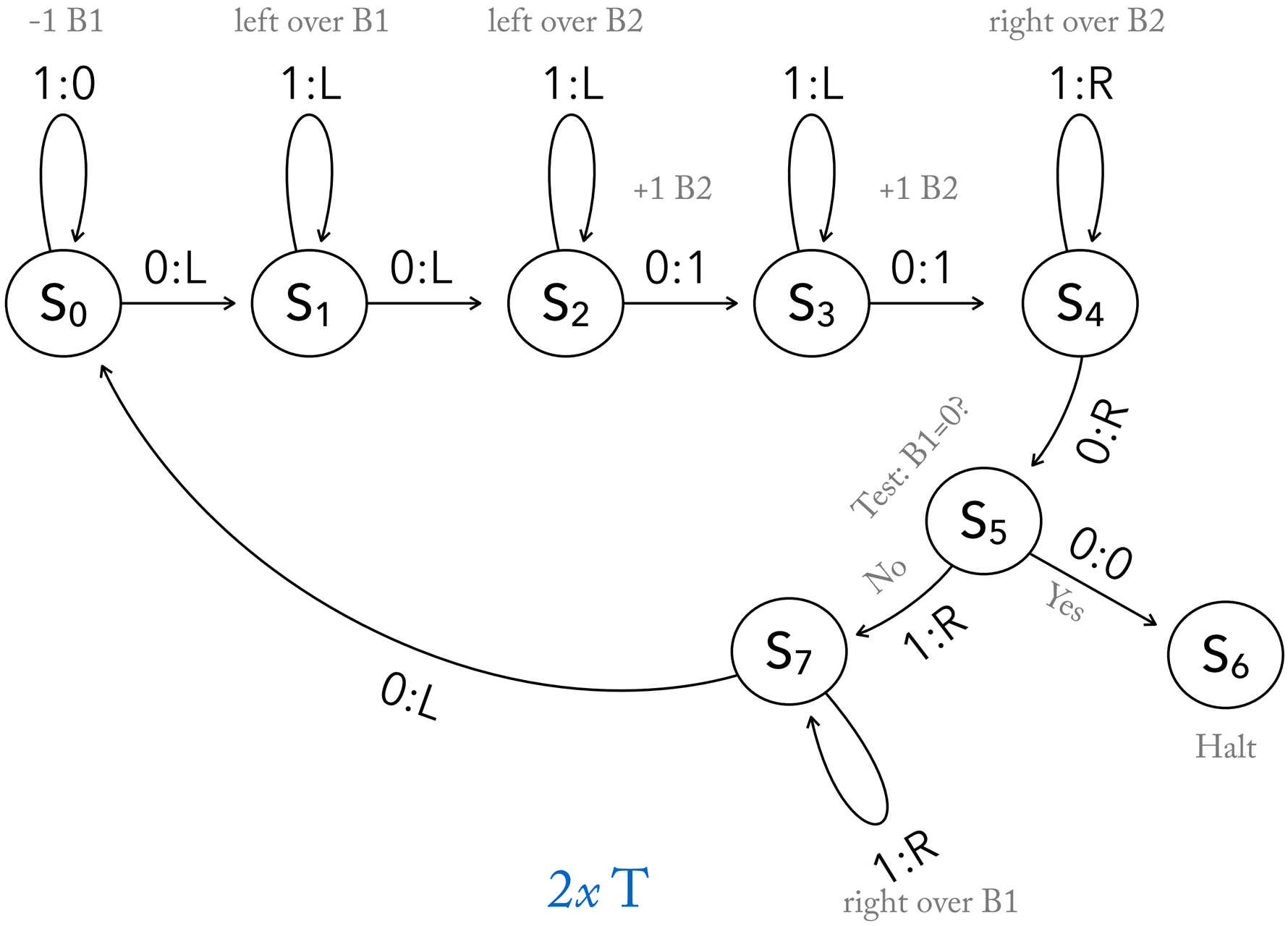


To turn this design into a TM, a good starting place is just to put the critical elements into a loop: (i) -1 from B1, (ii) +1 to B2, (ii) +1 to B2.



The problem with this design is that it contains no **test** or **exit operation**. As a result, it loops indefinitely. Can you predict what will happen to this machine after B1 is empty?

Finally, here is a fully annotated *2x* Tally TM. It's just like the machine above, but builds in a test to see when B1 is empty, and an explicit exit condition if it is.



Thinking through *x*+*y* T

How can we break addition up into a simple, repeating operation? Remember that addition is just successor applied over and over

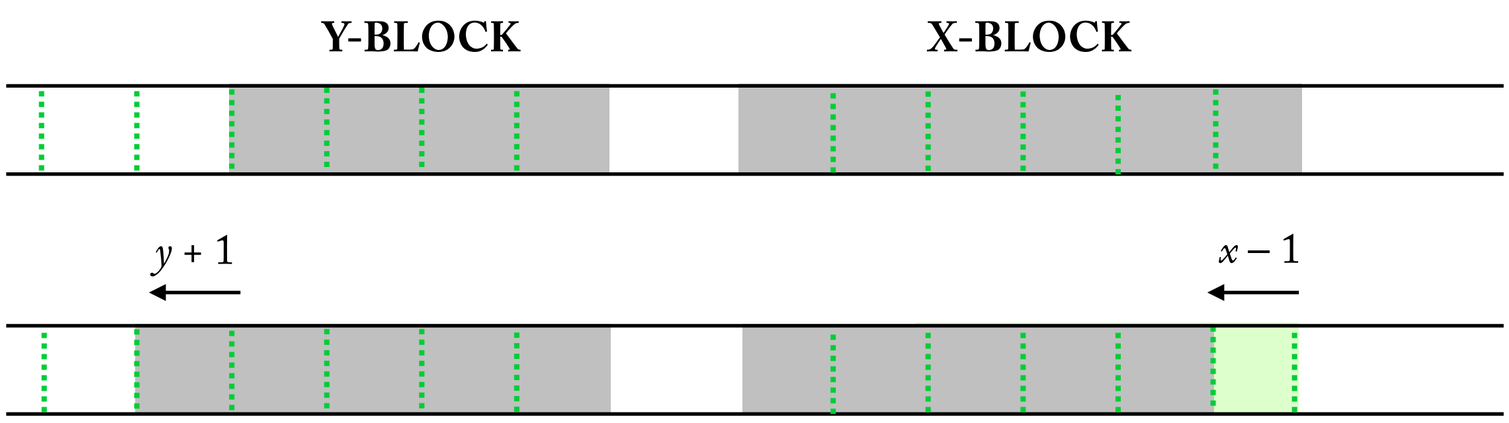
$$\begin{aligned} 2 + 3 &= 2 + 1 \\ &= 3 + 1 \\ &= 4 + 1 \\ &= 5 \end{aligned}$$

3 times

$$\begin{aligned} 4 + 5 &= 4 + 1 \\ &= 5 + 1 \\ &= 6 + 1 \\ &= 7 + 1 \\ &= 8 + 1 \\ &= 9 \end{aligned}$$

5 times

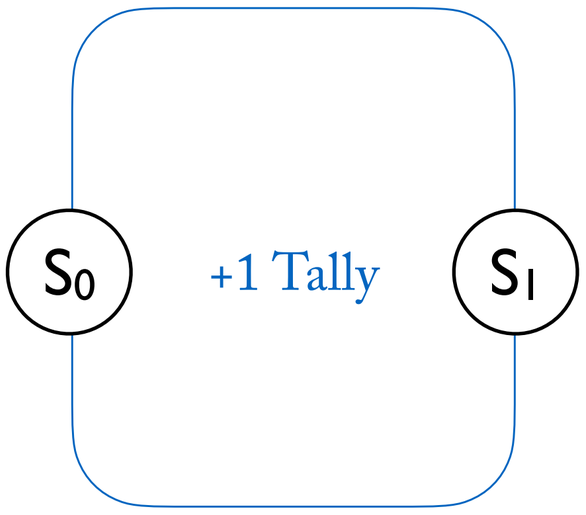
The next step is to work through a strategy. Here's a start…



30. Turing Machine Abstraction and Composition

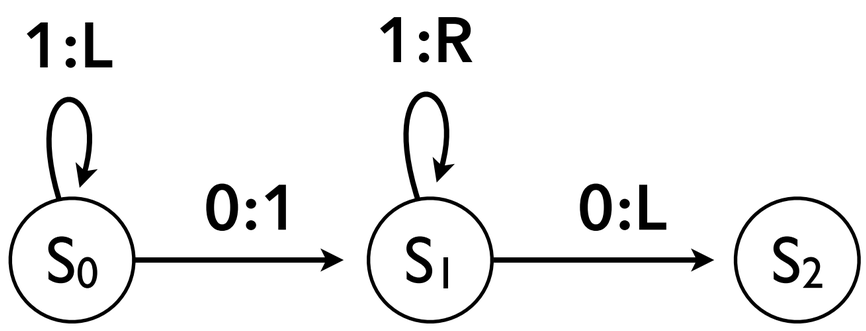
Functional Abstraction for Subroutines

We can apply functional abstraction to TM's which compute specific functions.

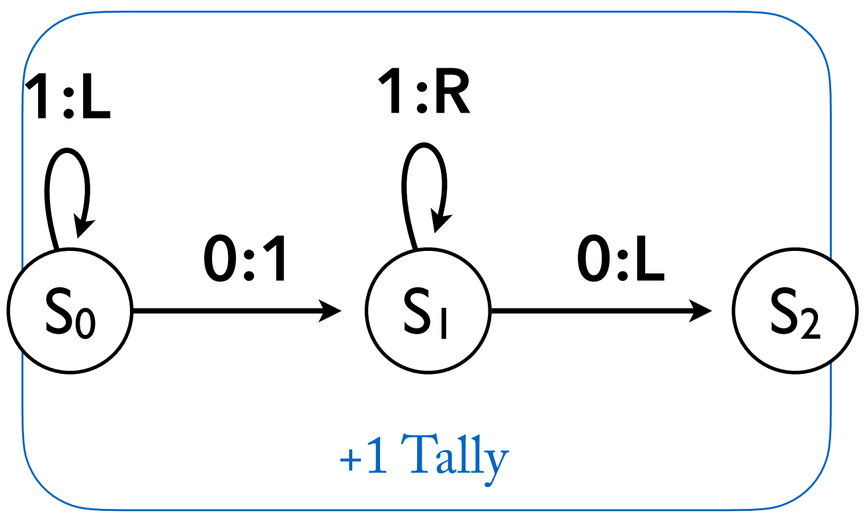


This boxed TM starts in S_0 and halts in S_1. By the time it halts, it will have computed +1 T.

Boxed TMs will be easier to combine into larger machines if they start *and* finish in standard position. For example, this TM computes +1 T and returns to standard position:



We can box this machine it by drawing a frame around it that intersects only with initial state and the halting state:



Then we can kick away the internal states:

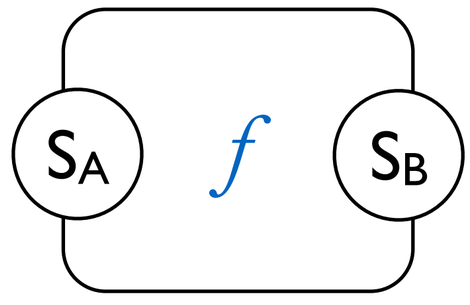


Constraints on Boxing

To prevent technical errors, we have to impose a few constraints on when a boxed machine may be introduced.

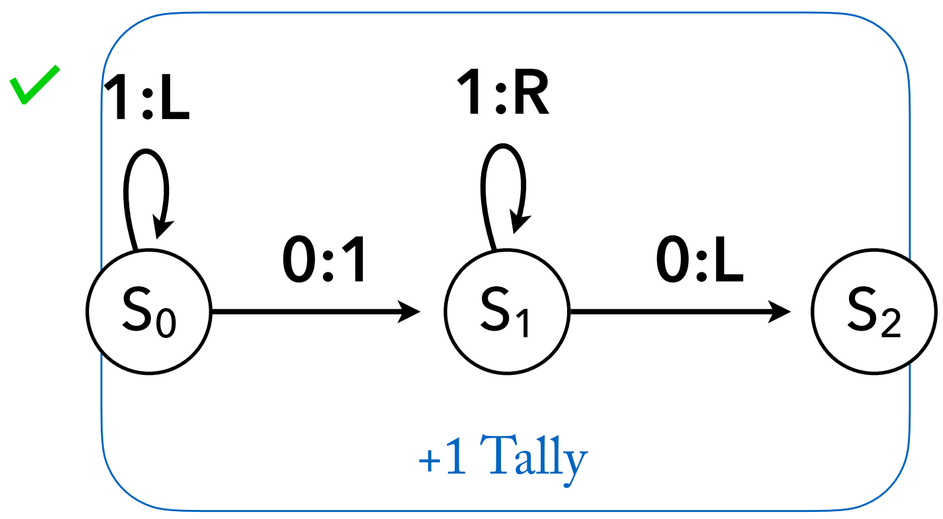
A machine M can be **boxed** only when:

1. M computes a function *f*.
2. S_A is the **initial state** of the machine.
3. S_B is the only **terminal node** of the machine:
   a) M halts in standard position;
   b) M halts in S_B;
   c) M halts in no other state (given the task);
   d) there are no transitions from S_B to other internal states of M.

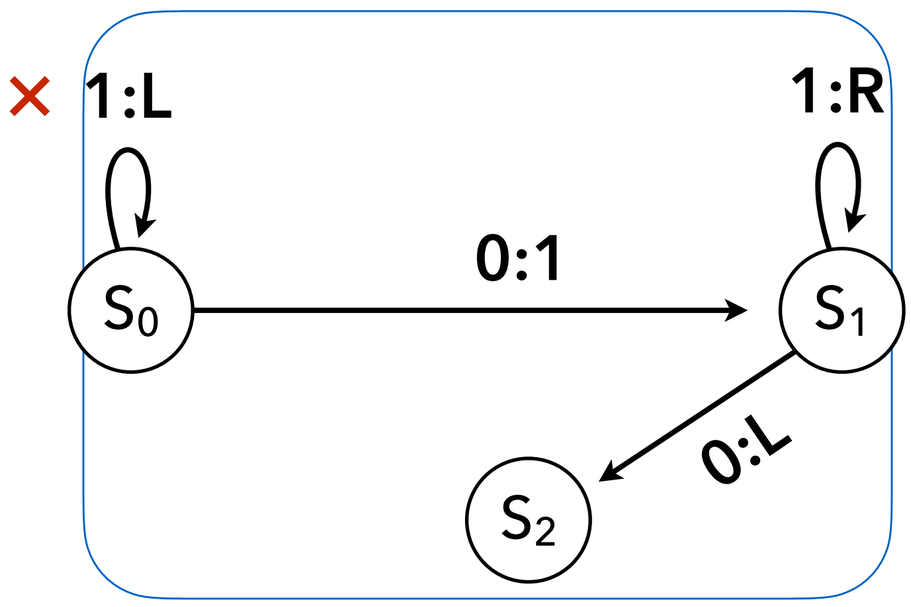


A boxed machine M can only be used in a larger circuit when there are no transitions from S_A to any states outside of M.

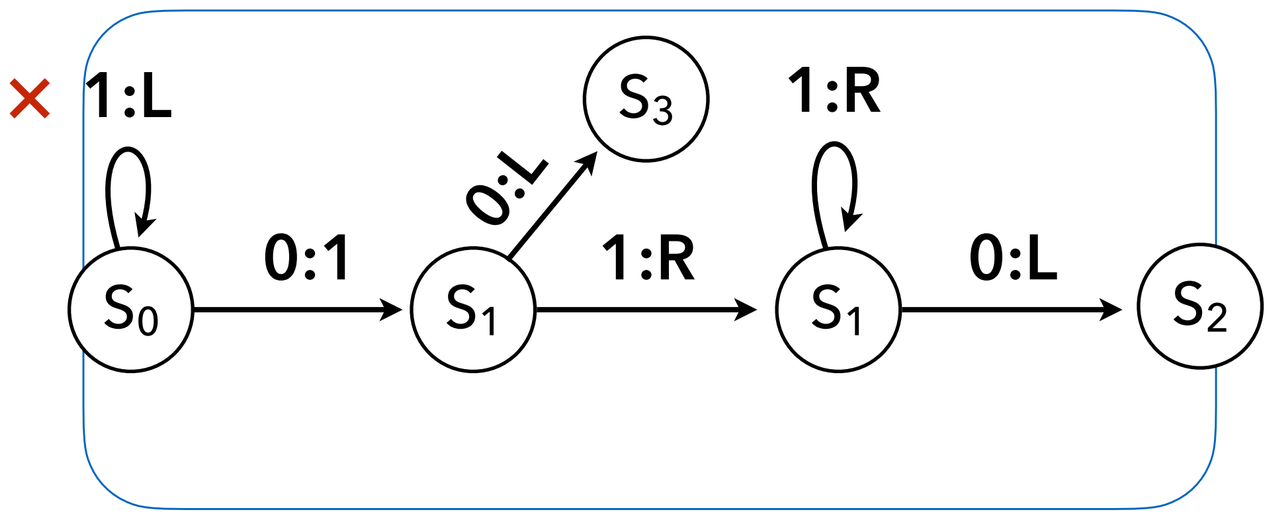
So this is right:



But this is wrong:

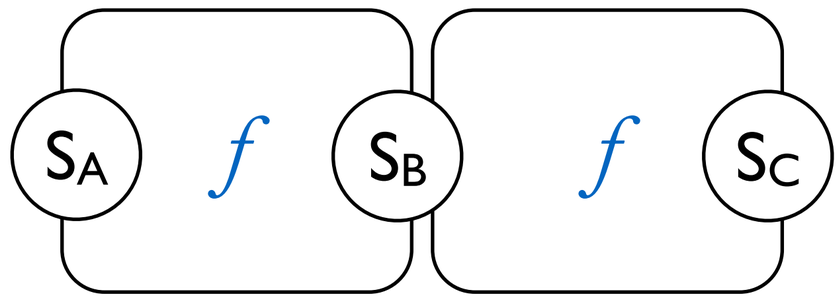


And this is wrong:

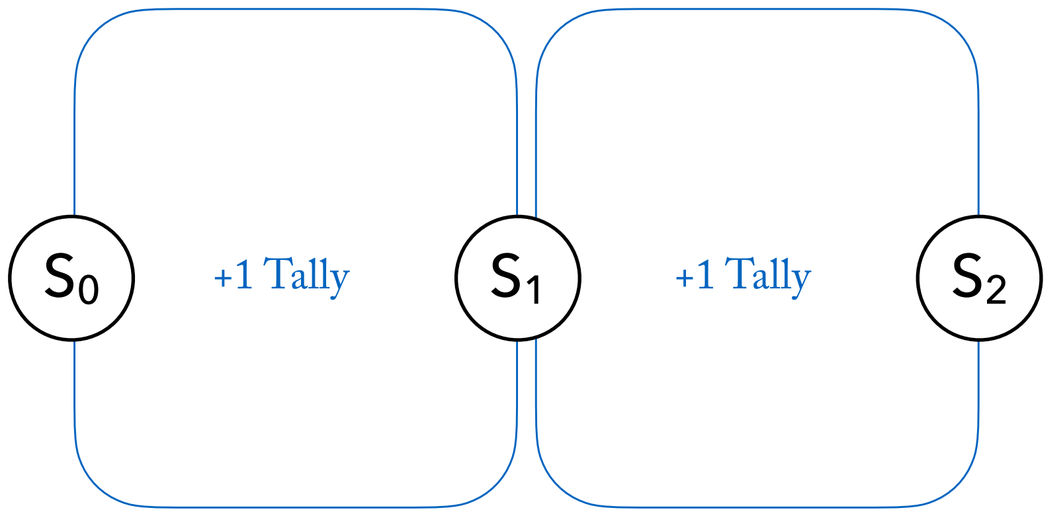


Function Composition

Boxed machines can be composed together to form larger machines:

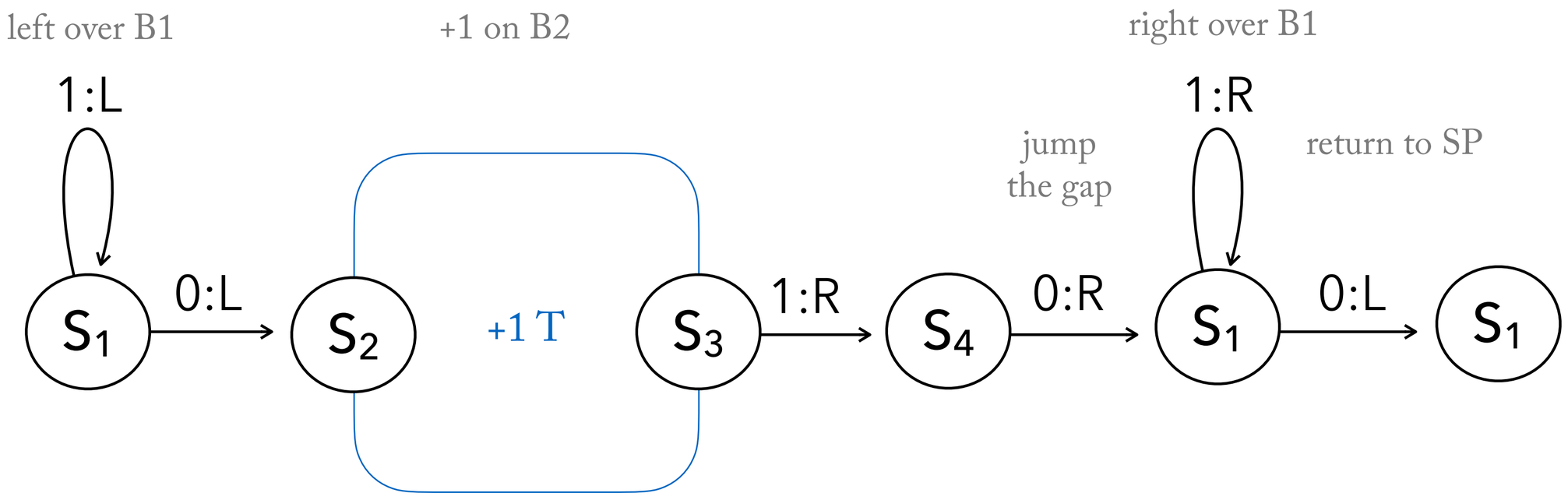


For example, this machine computes +2 T.



Boxed machines can be integrated with unboxed elements as well.

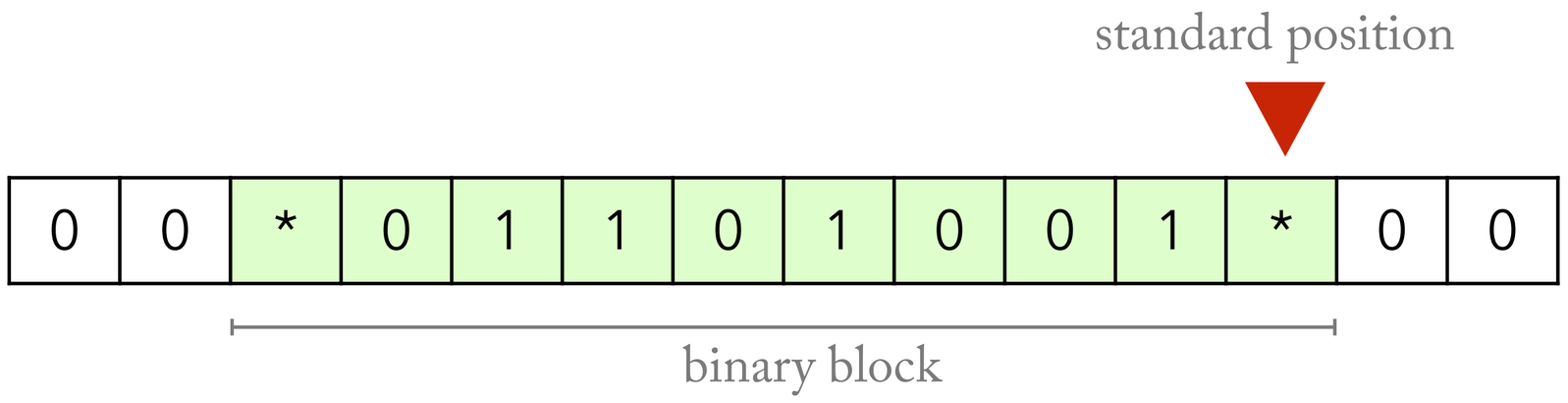
This machines moves left over a first block and then performs +1 T on a second block, then returns to standard position.



31. Binary Turing Machines

Set-Up

- We now allow three symbols: 1, 0, *
- Turing Machines can read and write all three symbols, so up to *three* arrows can leave each state.
- A **binary block** is a sequence of one or more 1's and 0's, flanked on both sides by *'s:



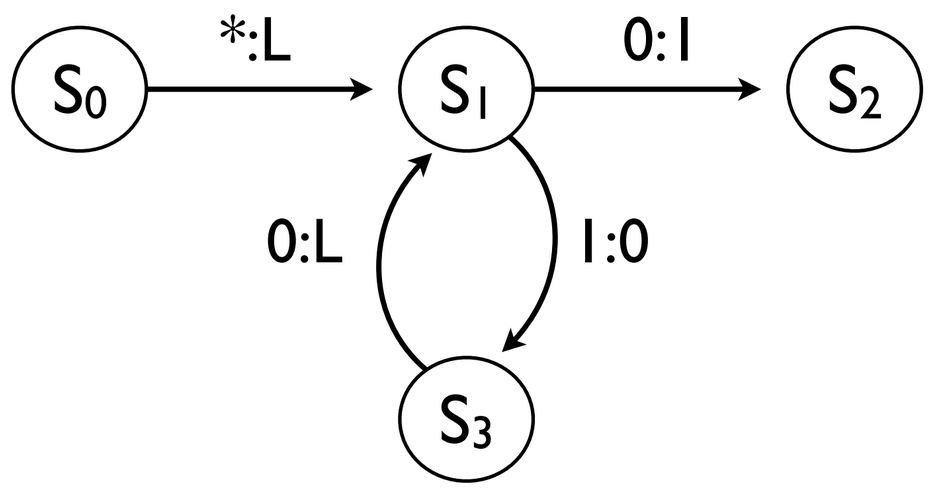
- **Standard position** is defined as the right-most * of the right-most block

---

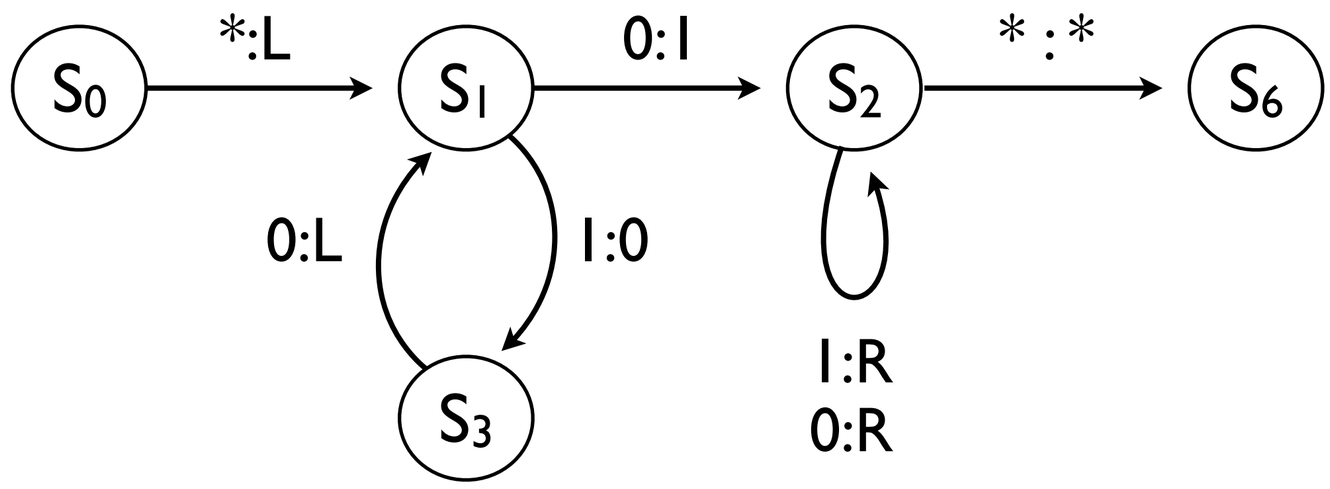
Binary +1

This machine computes the core of Binary +1.

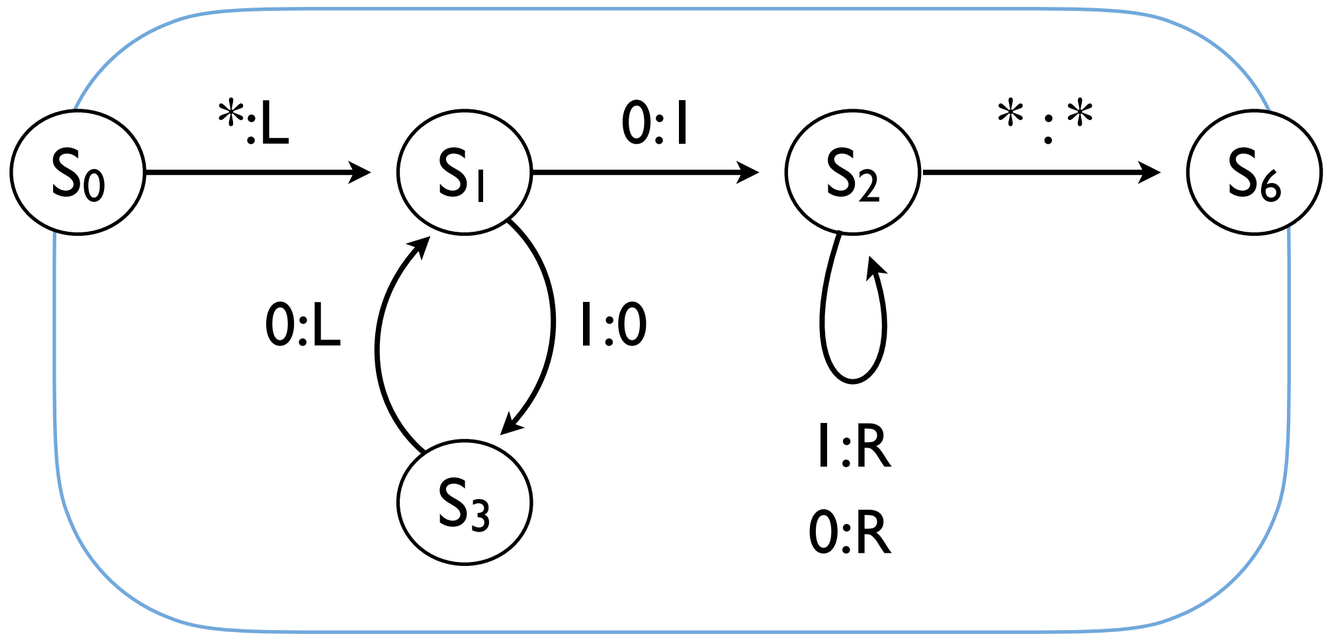
But it doesn't deal with the case where the input's are all 1's, and the left-most * must be overwritten with a 1.



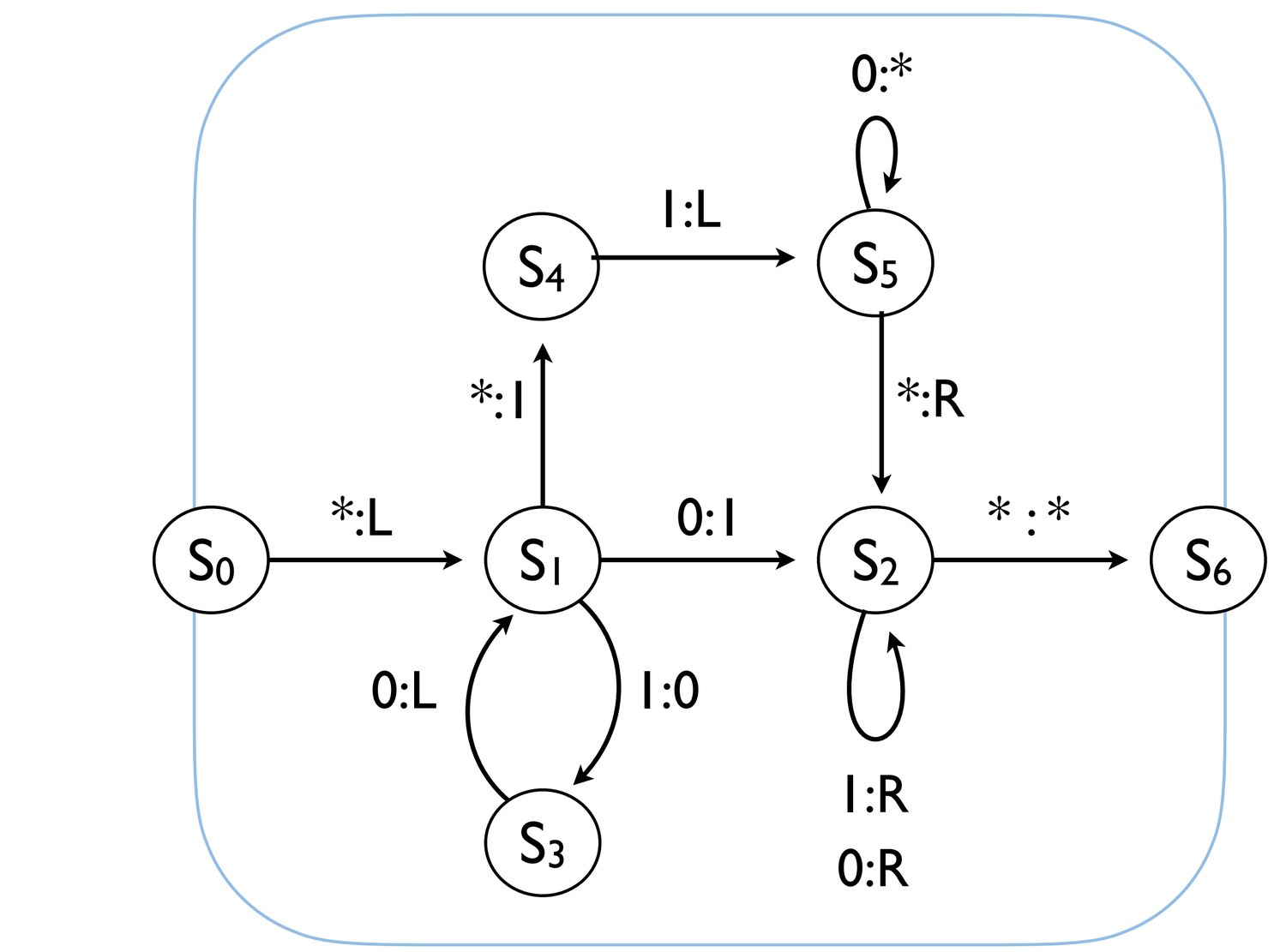
To facilitate boxing, let it return to standard position. Let's also add a terminal node.



This machine can be boxed:



And composed:



When dealing with unbounded binary problems, there is a wrinkle. Suppose the whole binary block is filled with 1's. To compute successor on this numeral, we must overwrite the left-most * with a 1, it then rewrites the * one cell to the left.

32. The Mathematical Tree

The Hyper-operation Sequence

Successor is a primitive function: it is defined in terms of no simpler function.

$$x + 1$$

Addition is defined recursively in terms of successor.

$$x + y = x + \underbrace{1 \ldots + 1}_{y \text{ times}}$$

y copies of 1 added to x

Multiplication is defined recursively in terms of addition.

$$x \times y = \underbrace{x + x \ldots + x}_{y \text{ times}}$$

y copies of x combined by addition

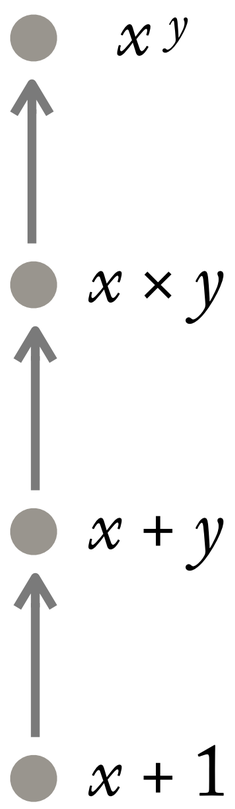
Exponentiation is defined recursively in terms of multiplication.

$$x^y = \underbrace{x \times x \ldots \times x}_{y \text{ times}}$$

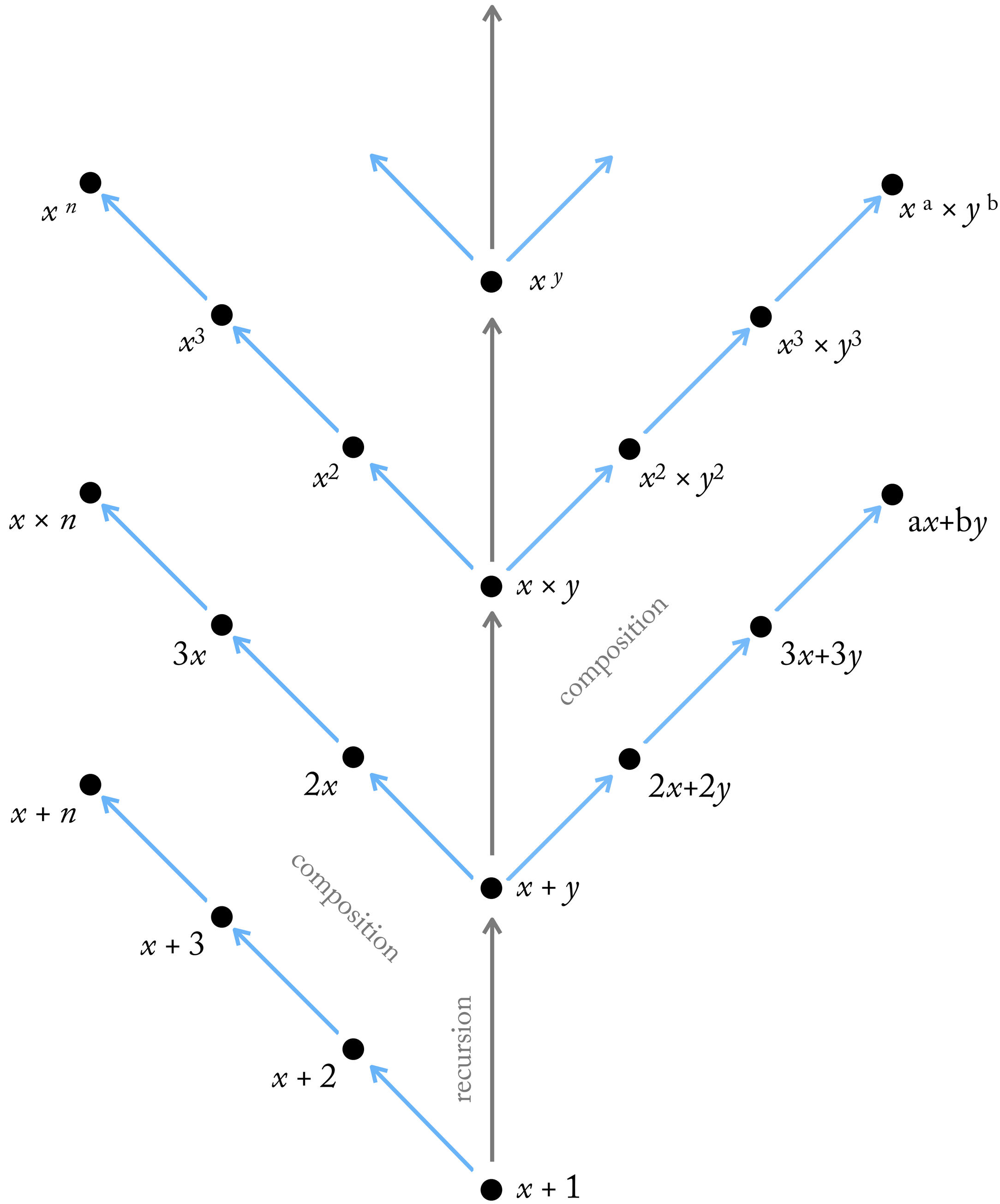
y copies of x combined by multiplication

Through recursion, we can build steadily up from successor to the further reaches of mathematical space.

Each function in the sequence can be computed by an algorithm that relies on a sub-routine for the previous function in the sequence.

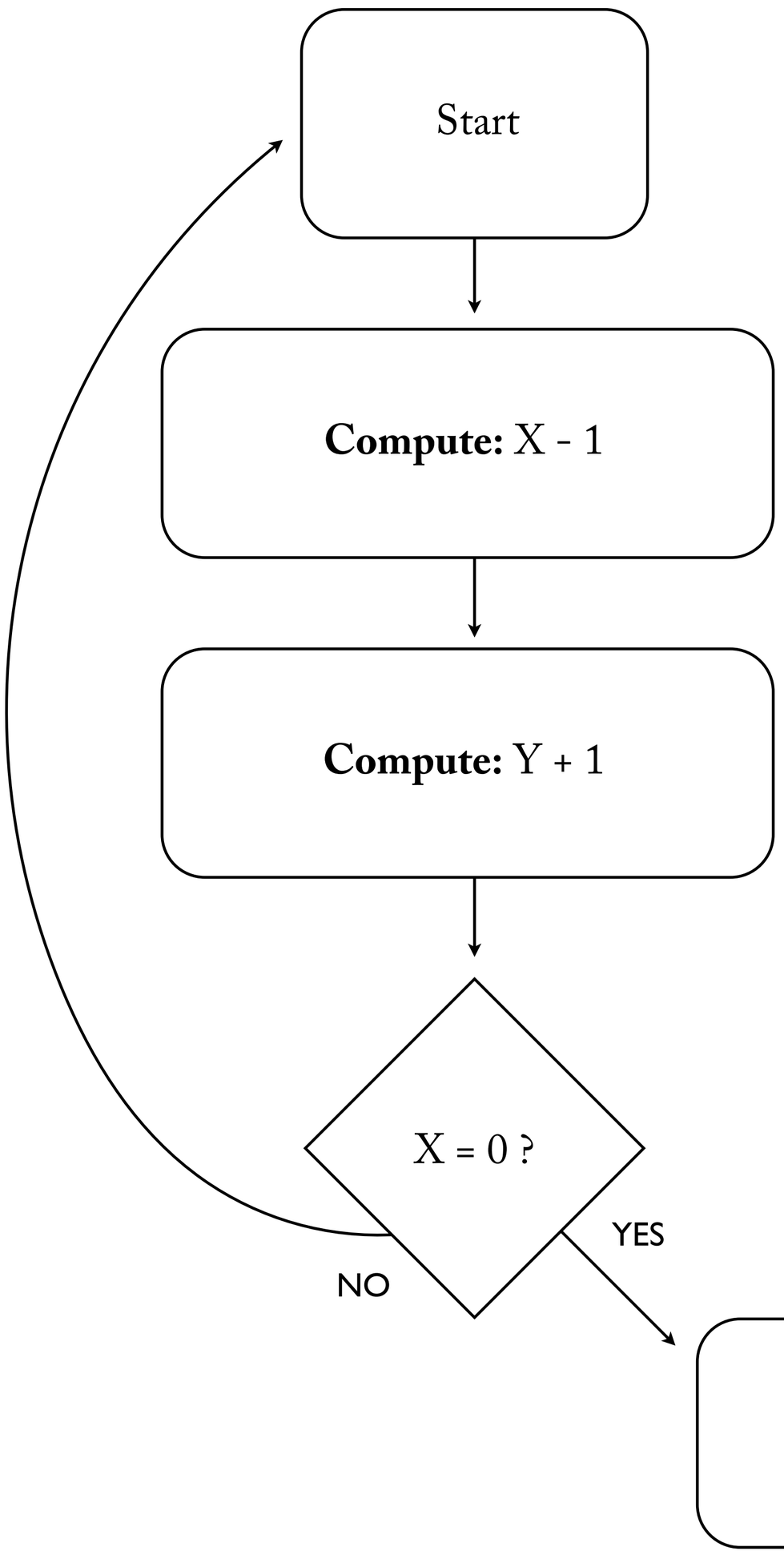


Recursion + Composition = Computable Mathematics



Recursive Addition

Compute $f(x,y) = x + y$



How can we break addition up into a simple, repeating operation?

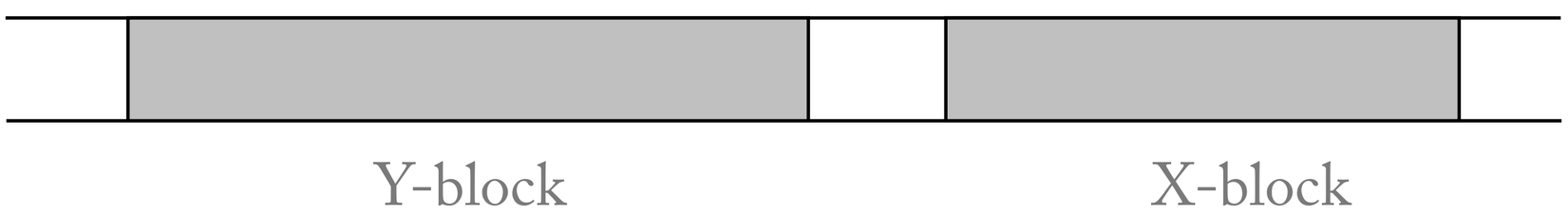
Adding x to y is equivalent to adding 1 to x, y times.

$$
\begin{aligned}
2 + 3 &= 2 + 1 \\
&= 3 + 1 \\
&= 4 + 1 \\
&= 5
\end{aligned}
$$

3 times

$$
\begin{aligned}
4 + 5 &= 4 + 1 \\
&= 5 + 1 \\
&= 6 + 1 \\
&= 7 + 1 \\
&= 8 + 1 \\
&= 9
\end{aligned}
$$

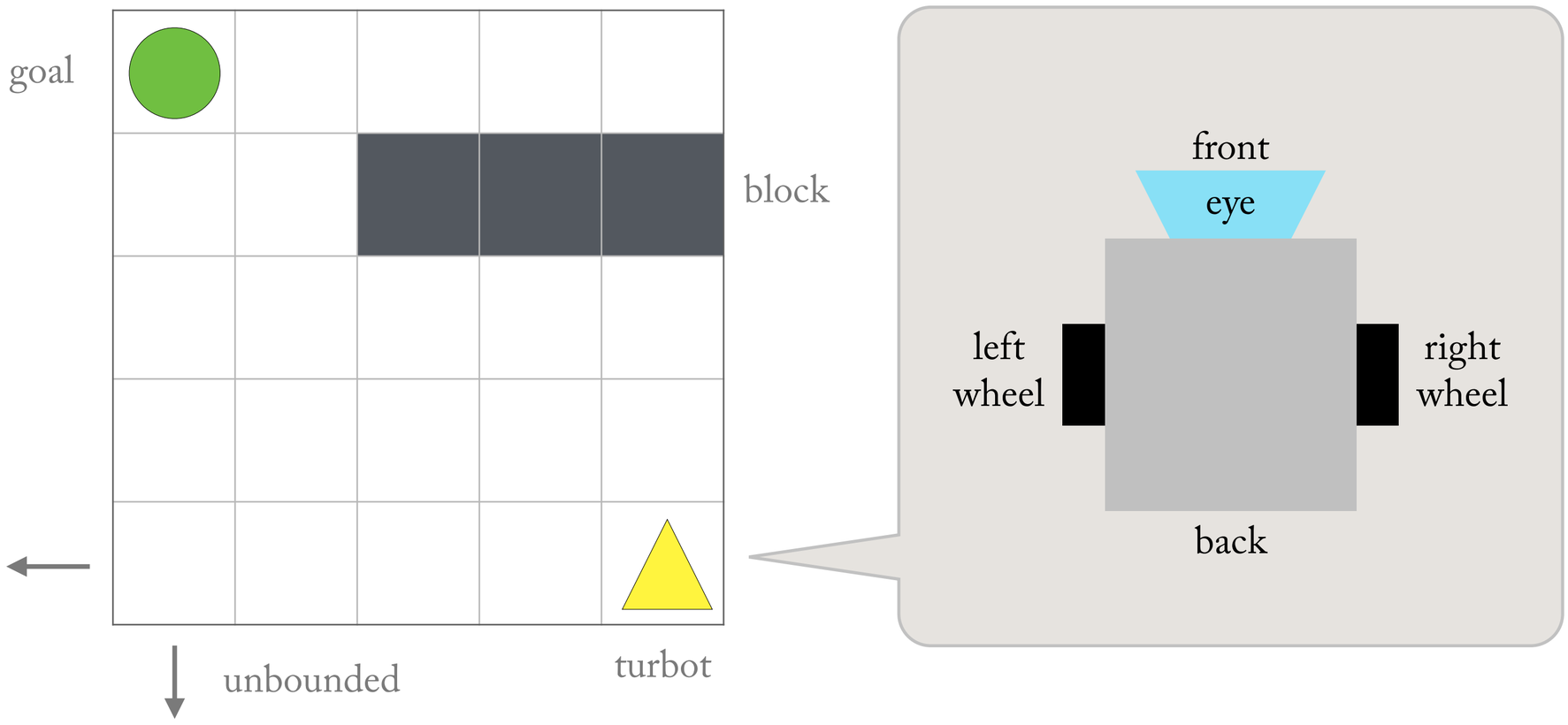
5 times



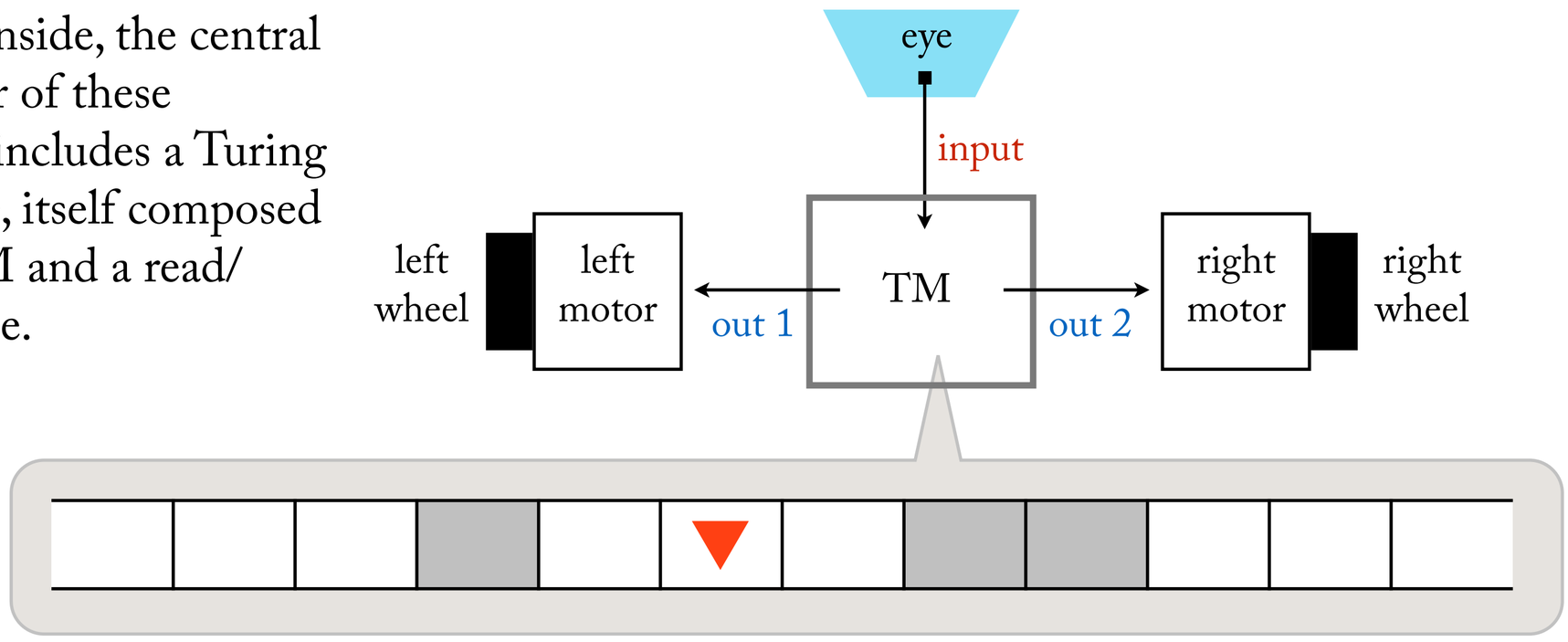
33. Computational Navigation

Turbots as Turing Machines

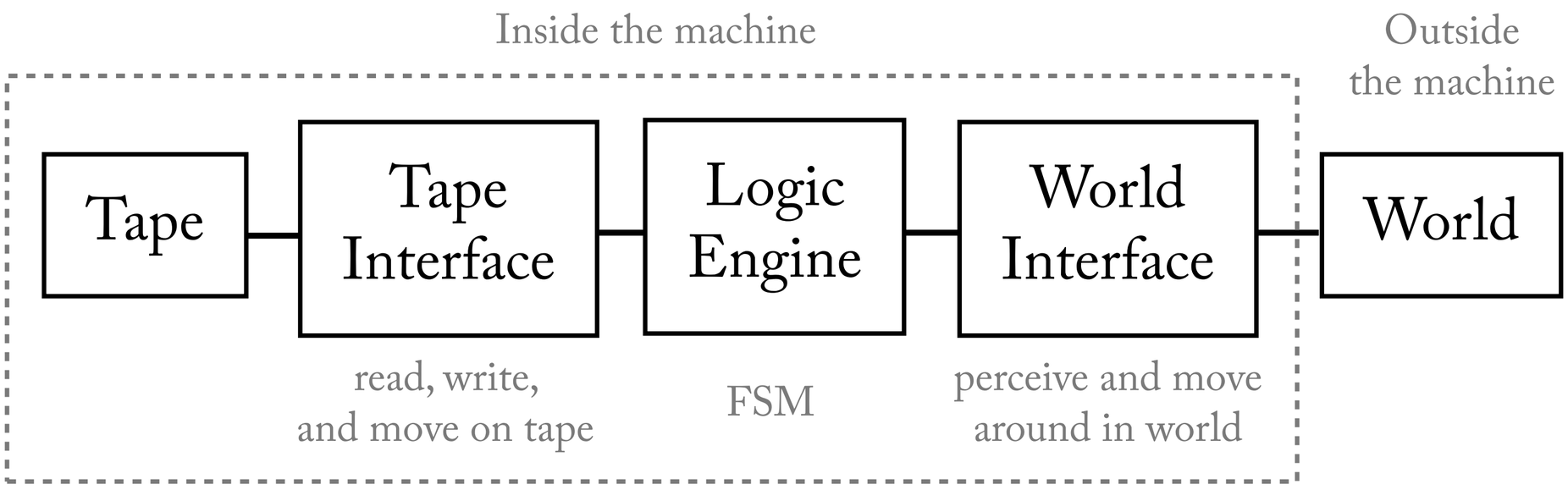
We next introduce Turbots based on Turing Machines. Viewed from the outside, they are just like the other navigational agents we've designed.



On the inside, the central processor of these Turbots includes a Turing Machine, itself composed of a FSM and a read/ write tape.



Turbots negotiate between two worlds: the *internal* world of the tape, and the *external* world of the environment. In this way they are just like you or me.



Turbots: Operation

- Turbots have an *internal* tape, but also act on an *external* world.

- At any moment, a Turbot has both an **internal position** (on the tape) and an **external position** (in the world).

- At each transition, a Turbot can perform either an internal operation, or an external operation, but not both.

- When a Turbot is performing an internal operation, its external position remains unchanged; when it is performing an external operation, its internal position remains unchanged.

- Turbots start with a blank tape (unless otherwise noted).

- Here are the operations available to Turbots:

Internal:



Input	**Output**
0 = read 0	0 = write 0
1 = read 1	1 = write 1
* = read *	* = write *
	R = move right
	L = move left

External:



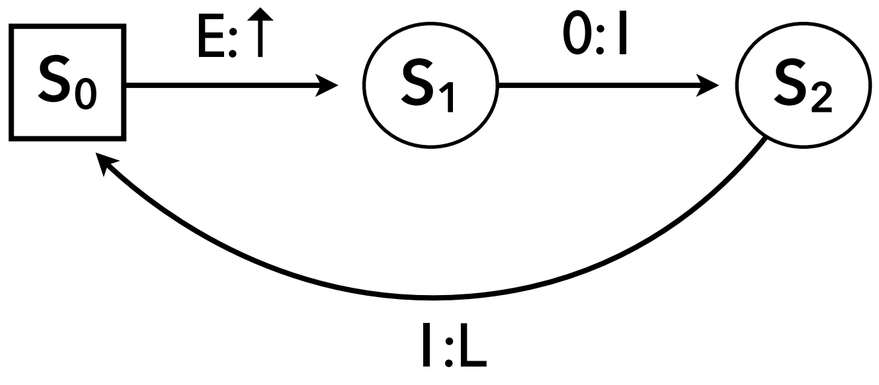
Input	**Output**
B = see block	↑ = move forward
E = see empty	↱ = right turn
F = see food	↰ = left turn

- By convention, states are either internal or external, but not both. External states are **square**, internal states are **circular**.

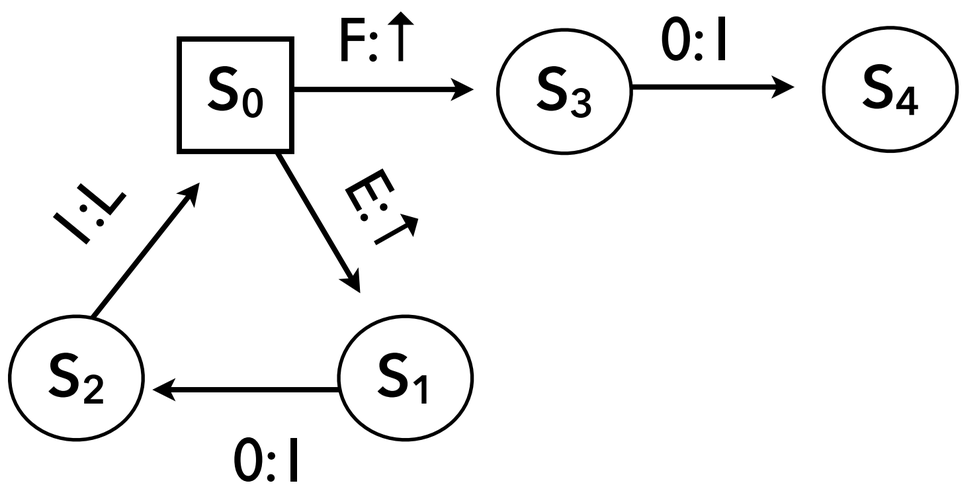
- The Turbot **cannot** pass through blocks. It **can** pass through food.

- Internally, a Turing Machine only perceives the cell it is **currently reading**.
- Externally, a Turbot only perceives the cell **in front of it**.

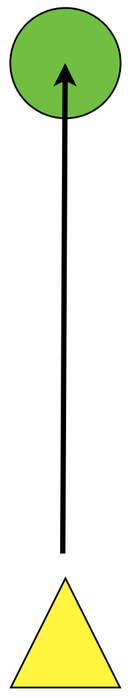
- This turbot moves forward, and records a "1" each time it does so.



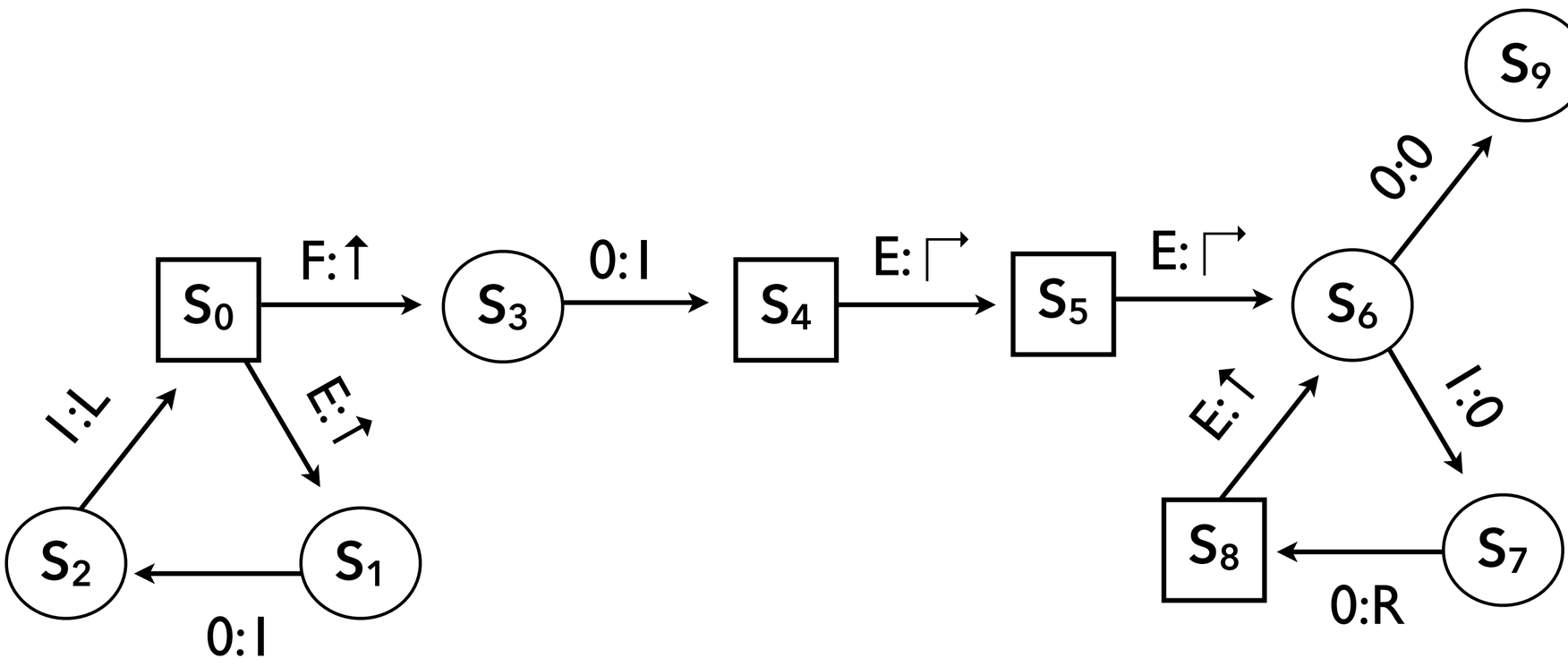
Half-Max Machine



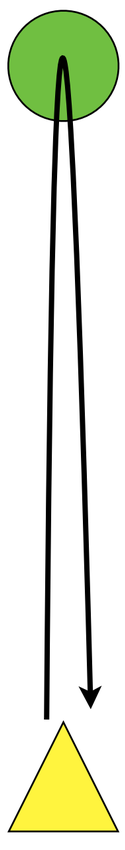
Move to food directly in front of you, recording a "1" each time you move forward, then halt.



Mad-Max Machine



Move to food directly in front of you, but an unknown distance away, then return to the starting point and halt.



34. The Desert Ant

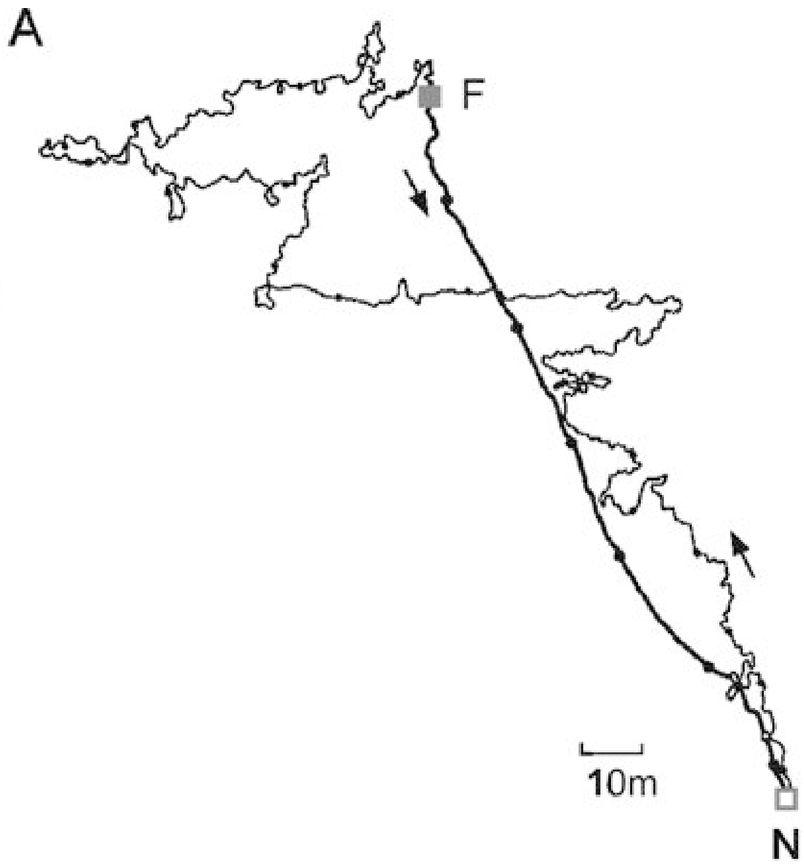
The Ant



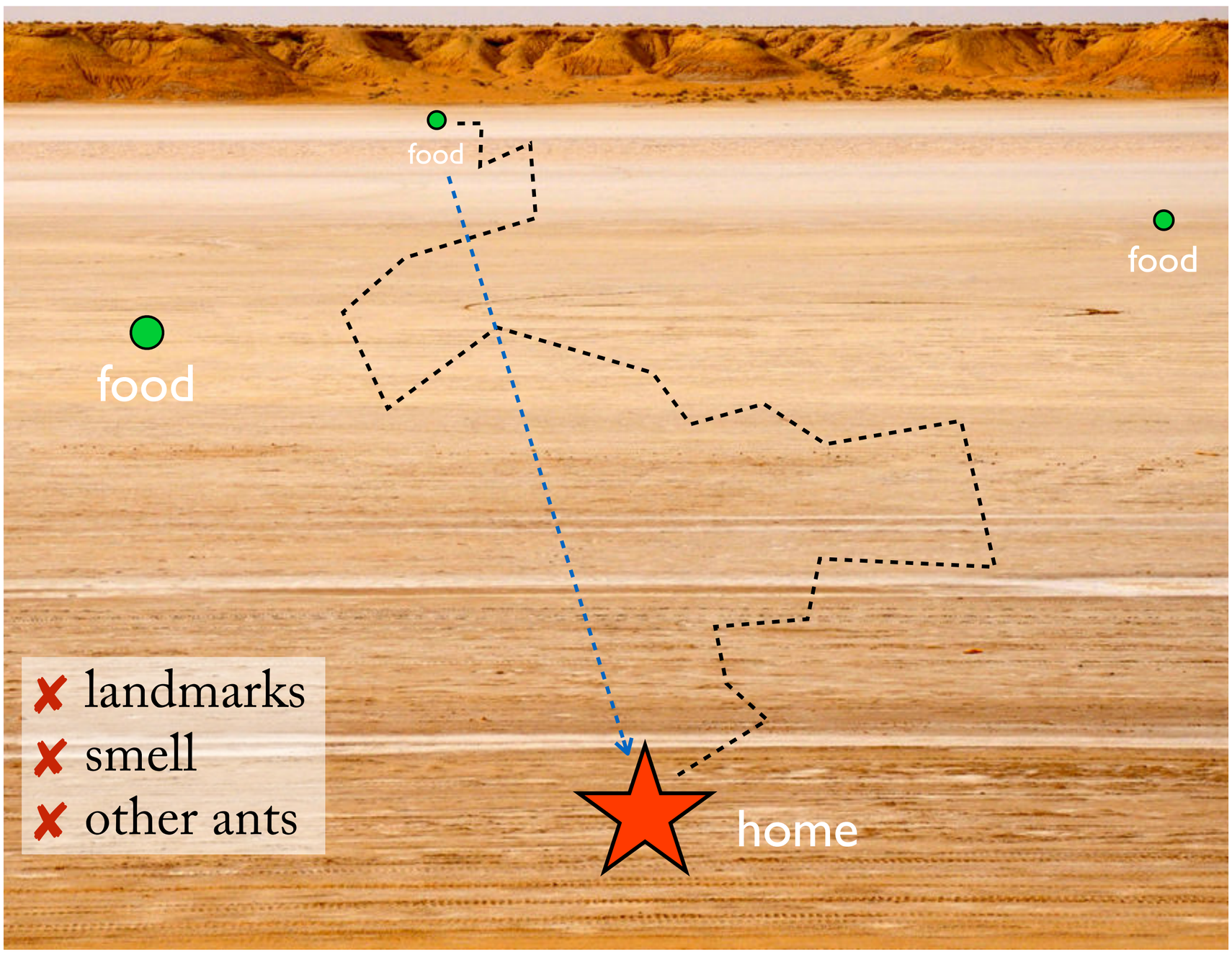


250,000 neurons

The Journey

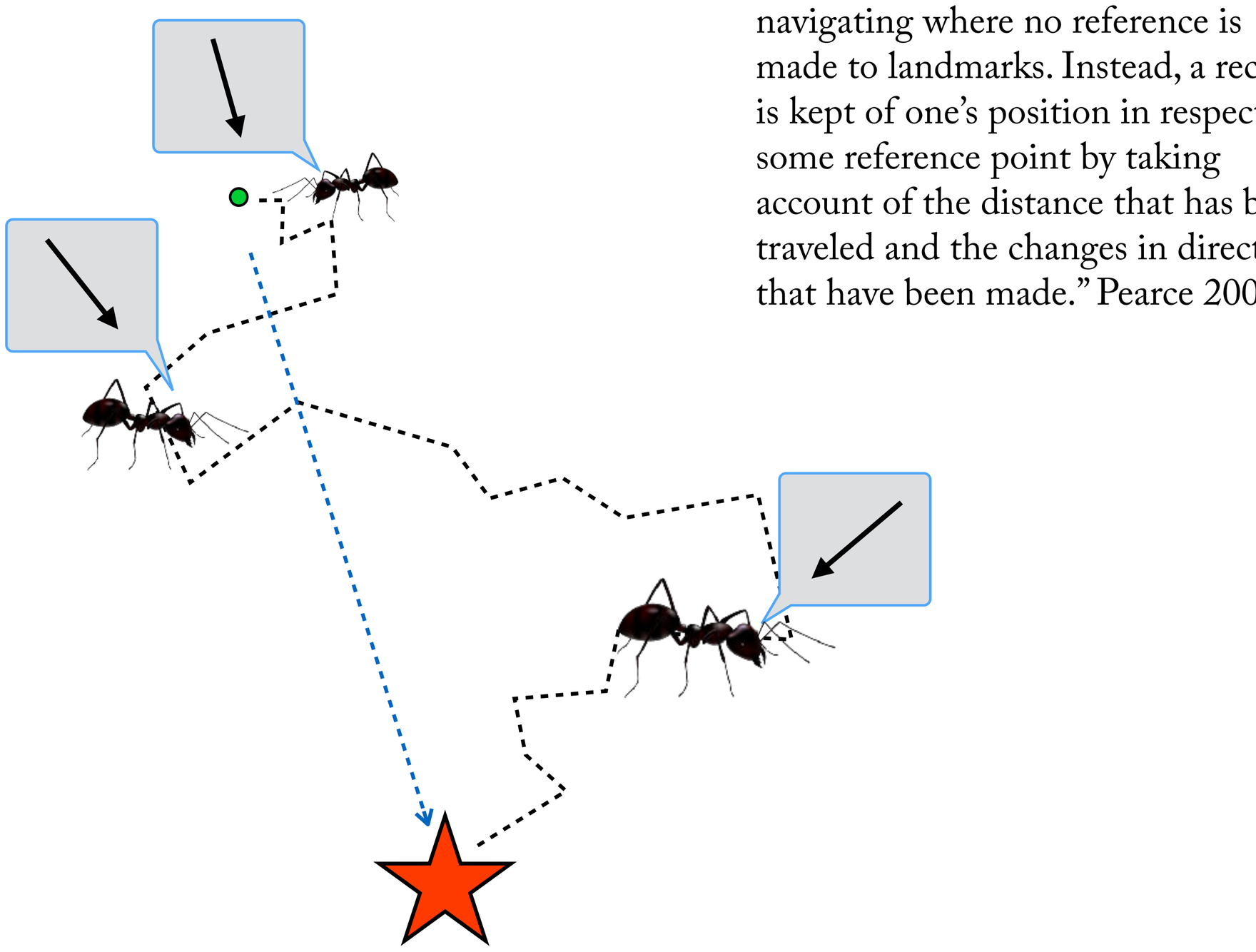


"Scientists observed that instead of retracing their winding route back home, they head straight back to the nest. How does the ant keep track of its current location relative to home? Path integration is the name given to the process thought to be used by animals (human and non-human alike) for what some might call dead reckoning." — SA



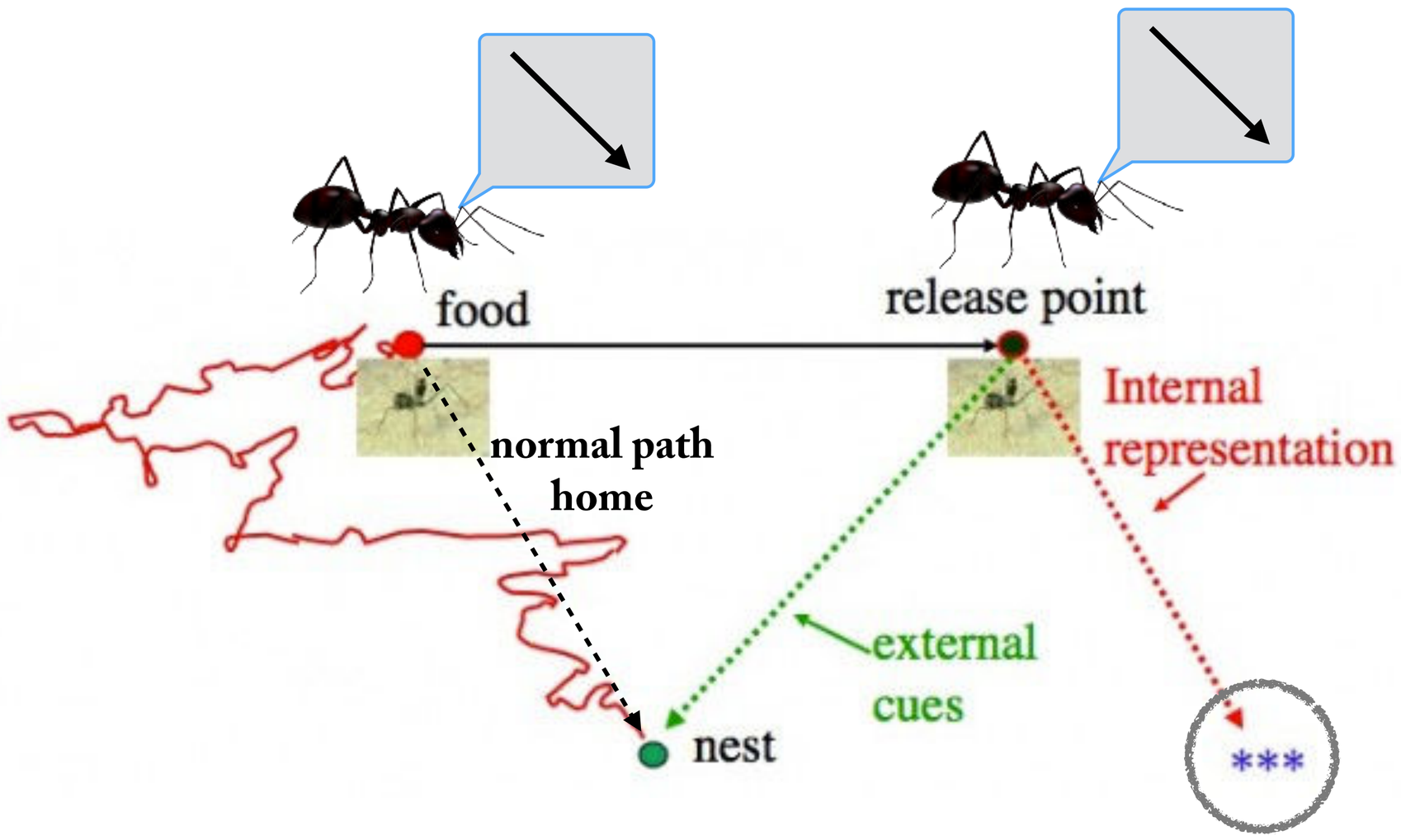
Ant = ~ 0.5 cm
Travel = >150 m
= over 20,000x
their length

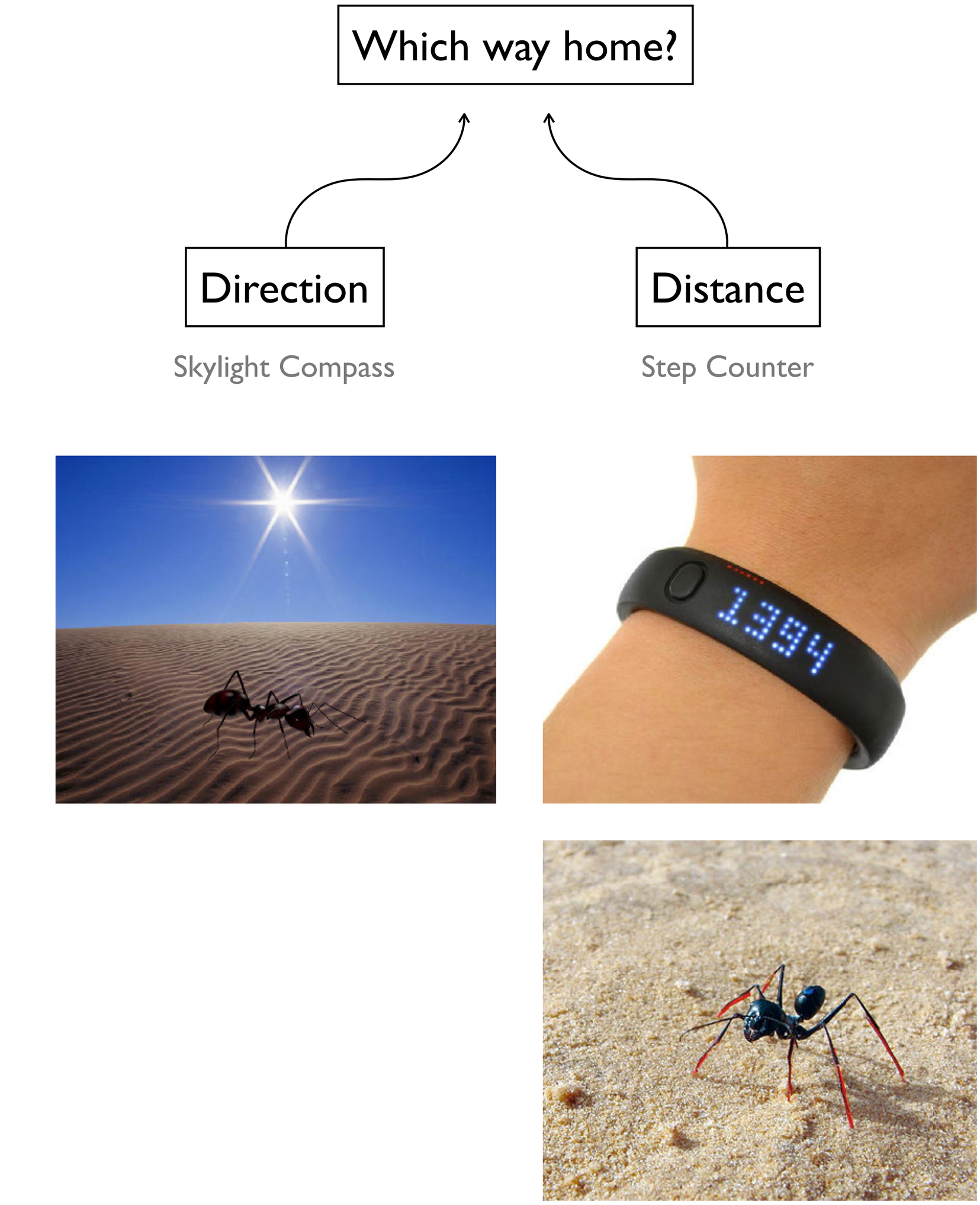
The Representational Hypothesis

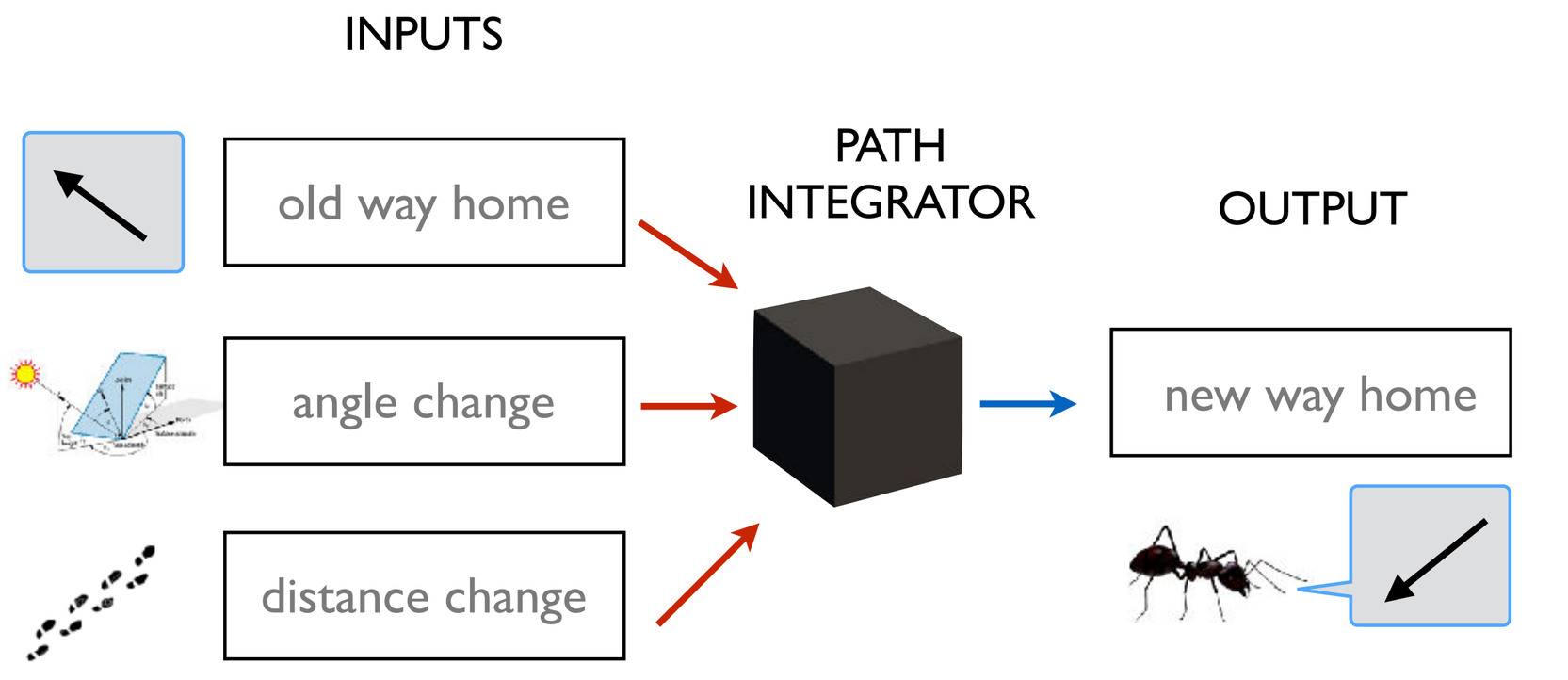


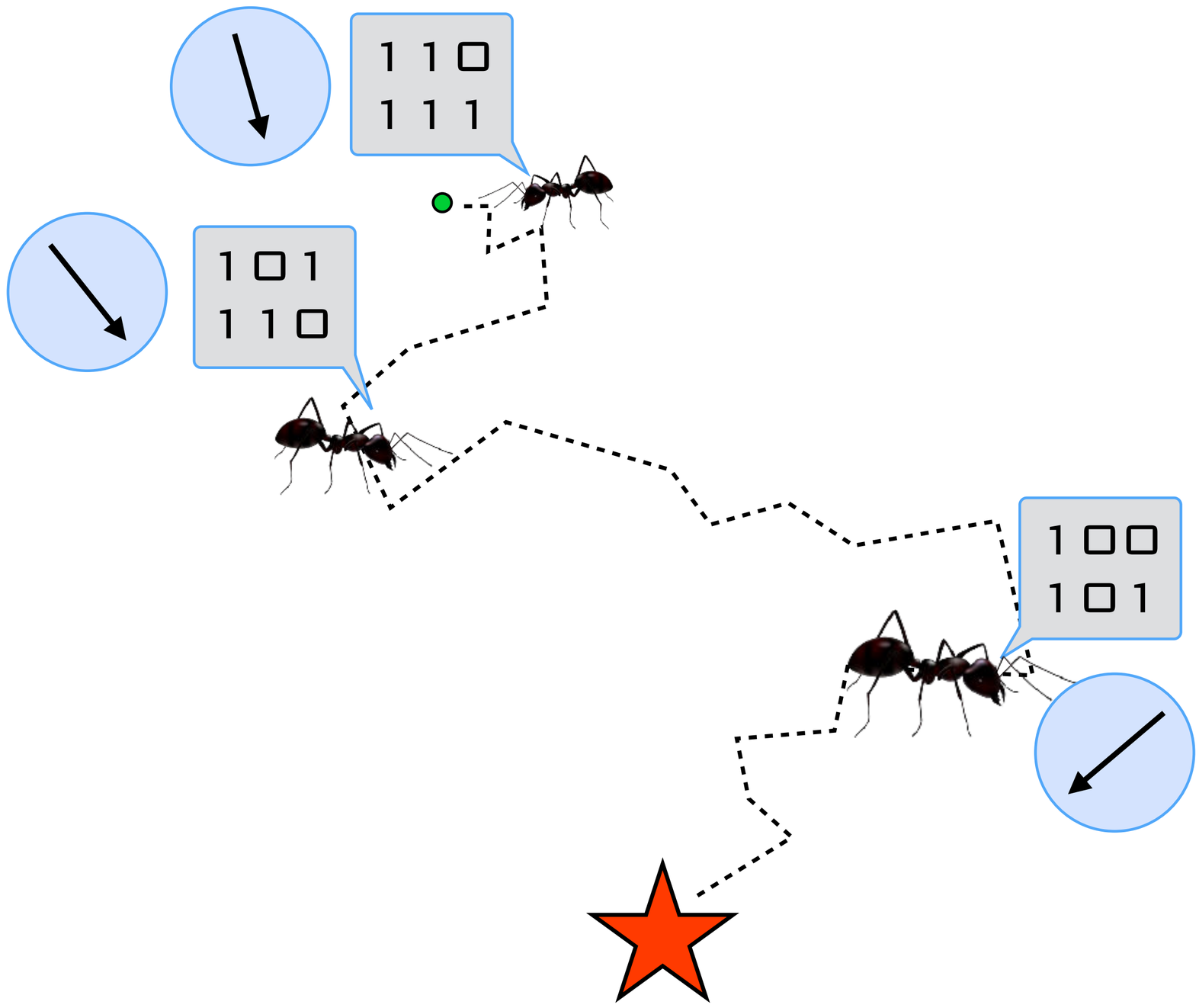
"**Dead reckoning**… is the method of navigating where no reference is made to landmarks. Instead, a record is kept of one's position in respect to some reference point by taking account of the distance that has been traveled and the changes in direction that have been made." Pearce 2008

Testing the Representational Hypothesis



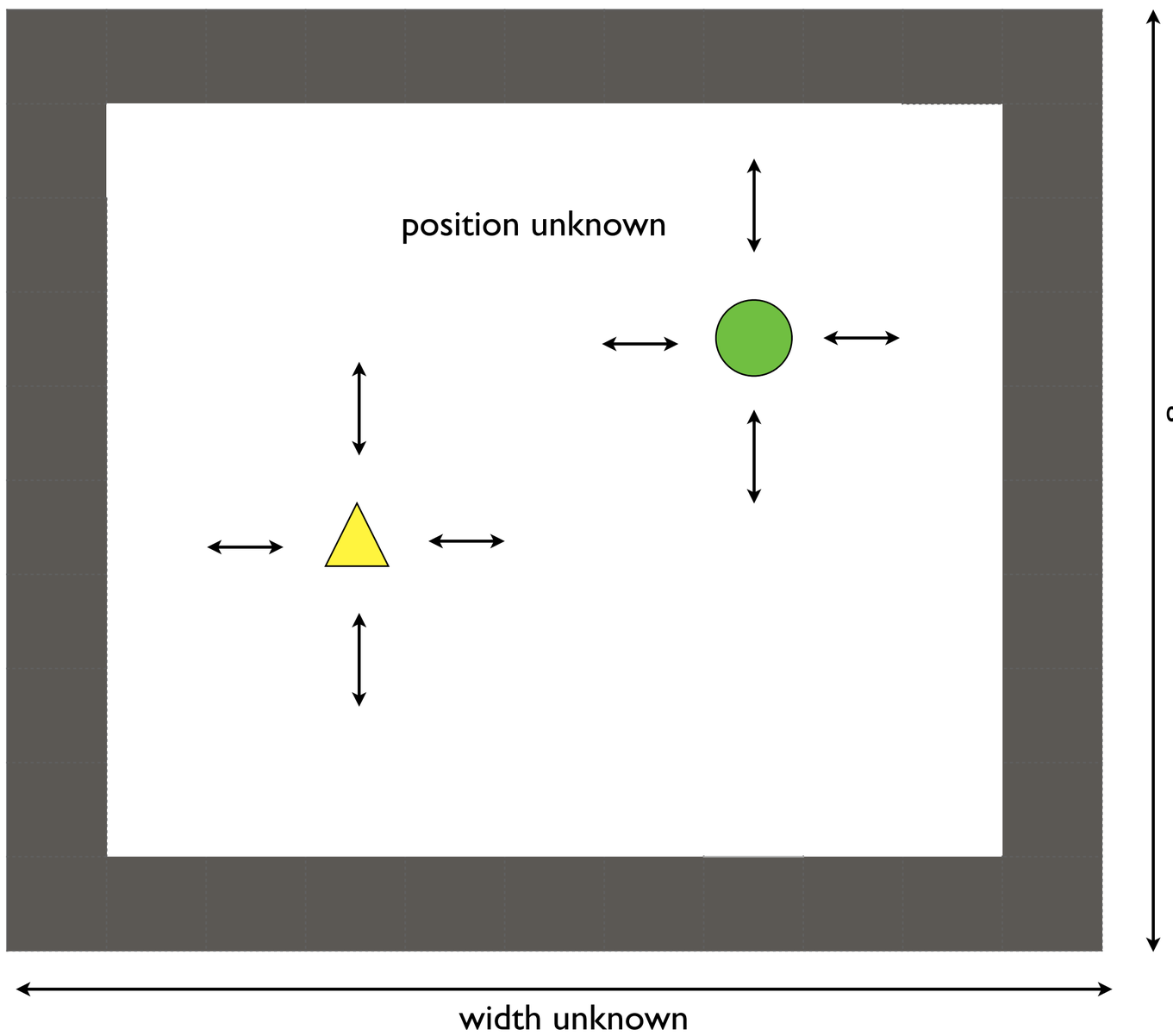

Which way home?
Direction
Distance
Skylight Compass
Step Counter
13:34


INPUTS
old way home
angle change
distance change
PATH
INTEGRATOR
OUTPUT
new way home



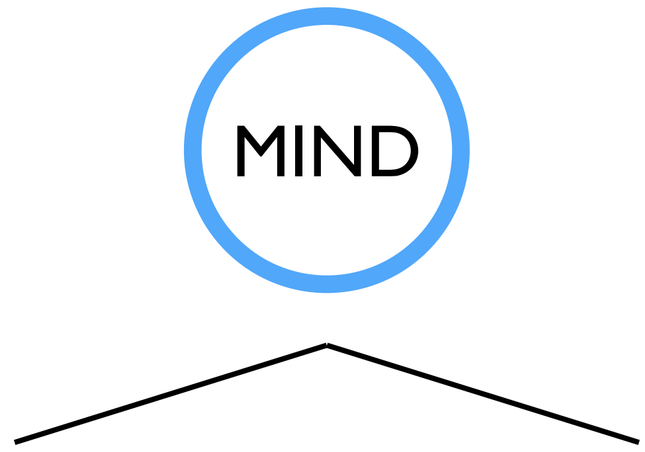
The Desert Ant

First, search until you find food (are placed over it). Second, return to where you started as efficiently as possible.

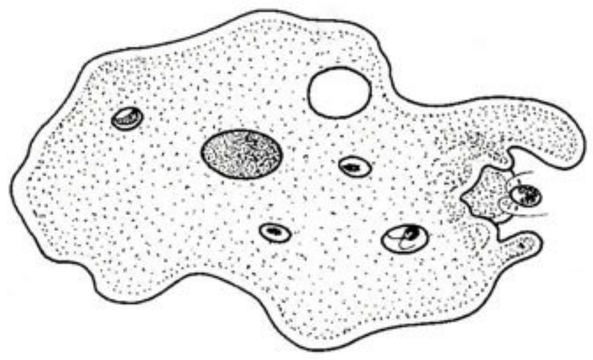


35. Physical Cognition

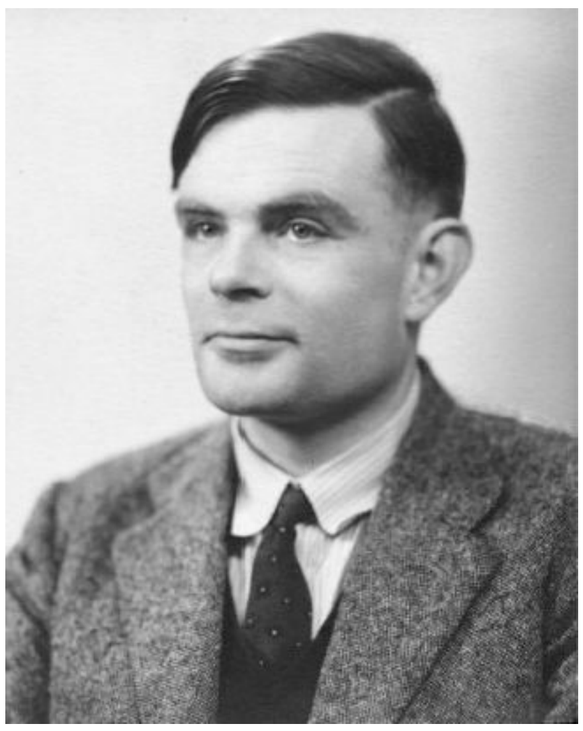
"A belief … is a map of neighbouring space by which we steer."
—Ramsey 1931









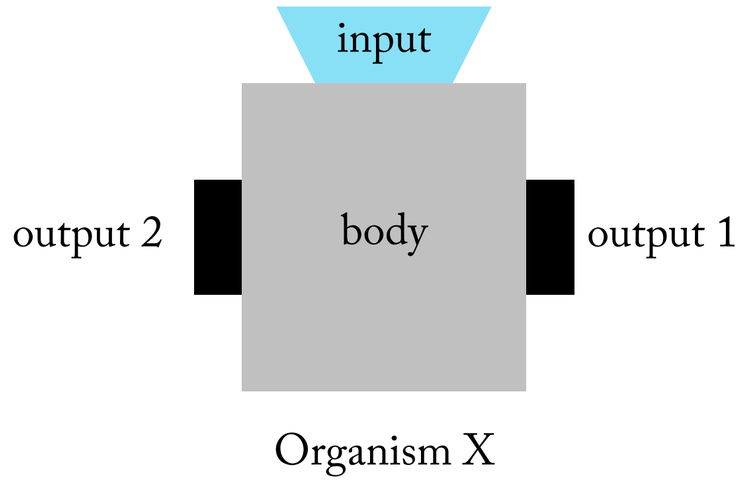


From Object to Animal

Suppose we found a strange object in the woods and attempted to understand its behavior.



After much experimentation, we determine that it is some sort of input-output system. Perhaps it is even an animal of some unknown kind. We'll call it Organism X.



Organism X

To study it's behavior, we schematize it's overall organization into four basic components: an input organ, two output organs, and a body connecting them.

Does Organism X have cognition? Does it think? Have beliefs? Consciousness? To put it to the test, we stimulate the input organ with a series of signals over time and record the output. Here are some snapshots of one of our experiments.

...	t13	t12	t11	t10	t9	t8	t7	t6	t5	t4	t3	t2	t1	
...	0	0	0	0	0	1	0	0	0	0	0	0	0	IN
...	11	11	11	01	11	01	11	11	11	11	11	11	10	OUT

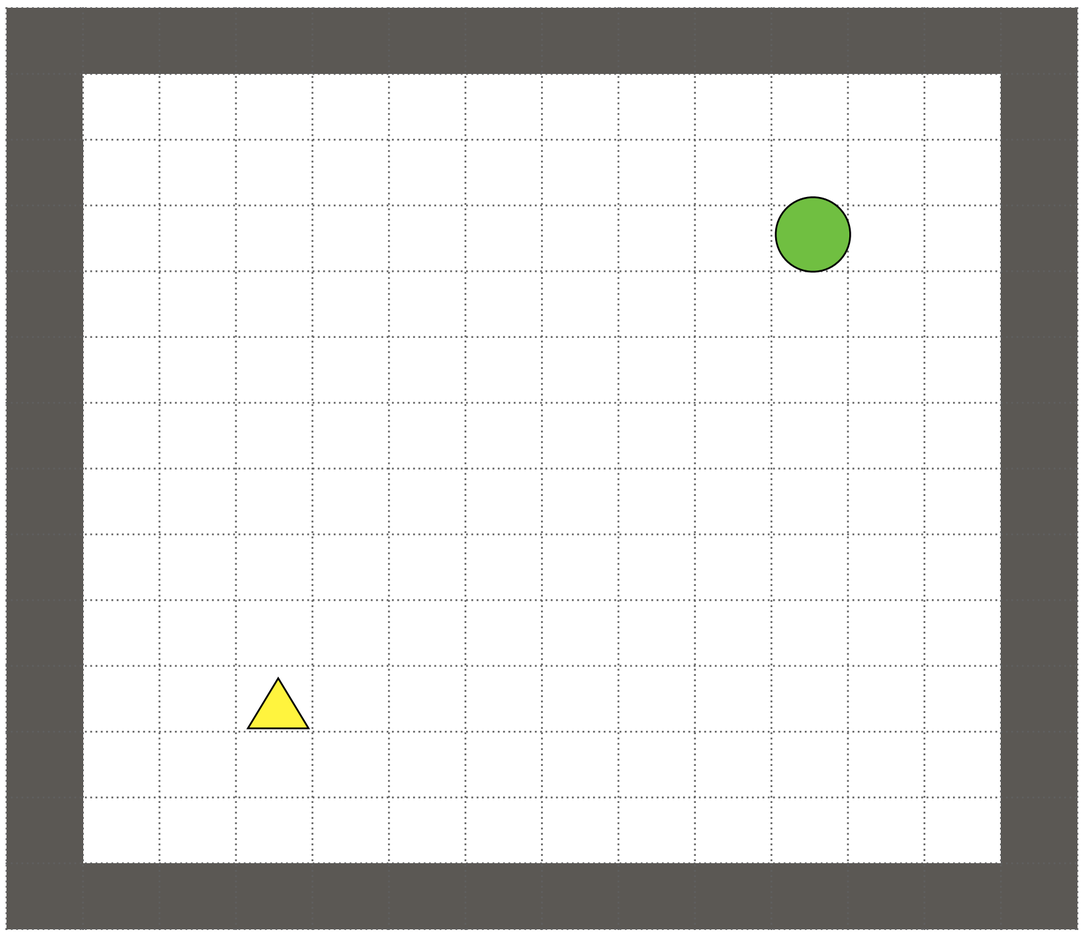
...	t111	t110	t109	t108	t107	t106	t105	t104	t103	t102	t101	t100	...	
...	0	0	0	0	0	0	*	0	0	1	0	0	...	IN
...	11	11	11	11	11	01	11	11	11	10	11	11	...	OUT

...	t198	t197	t196	t195	t194	t193	t192	t191	t190	t189	t188	t187	...	
...	0	0	0	0	0	0	0	0	0	0	0	0	...	IN
...	00	00	00	00	00	00	00	11	11	11	11	11	...	OUT

There is certainly something complex about Organism X's reactions. But are they random or rational? Is it trying to tell us something? Do the inputs *mean anything* to it?

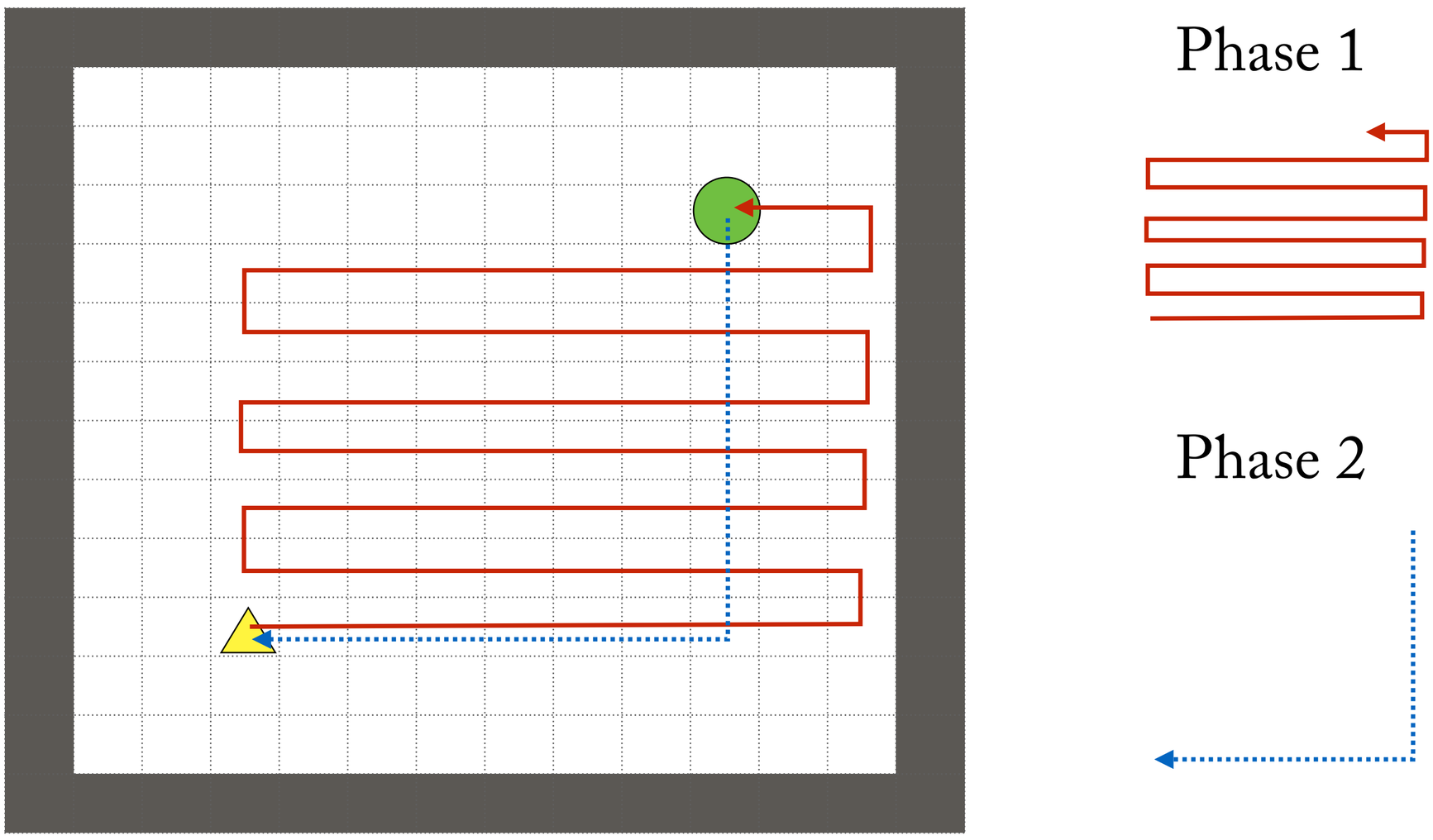
The Ecological View

We consult with a cognitive scientist, who gives us some wise advice: you won't find any answers to your questions unless you look at the **ecological environment** in which Organism X evolved. Only then can we attempt to decode its inputs and outputs.



Consulting with some naturalists, it turns out that X's tend to live in enclosed spaces, where they have single food source— a kind of plant that lives only a single day— and no predators. In other words, the X ecology looks a bit like this.

Observed Behavior



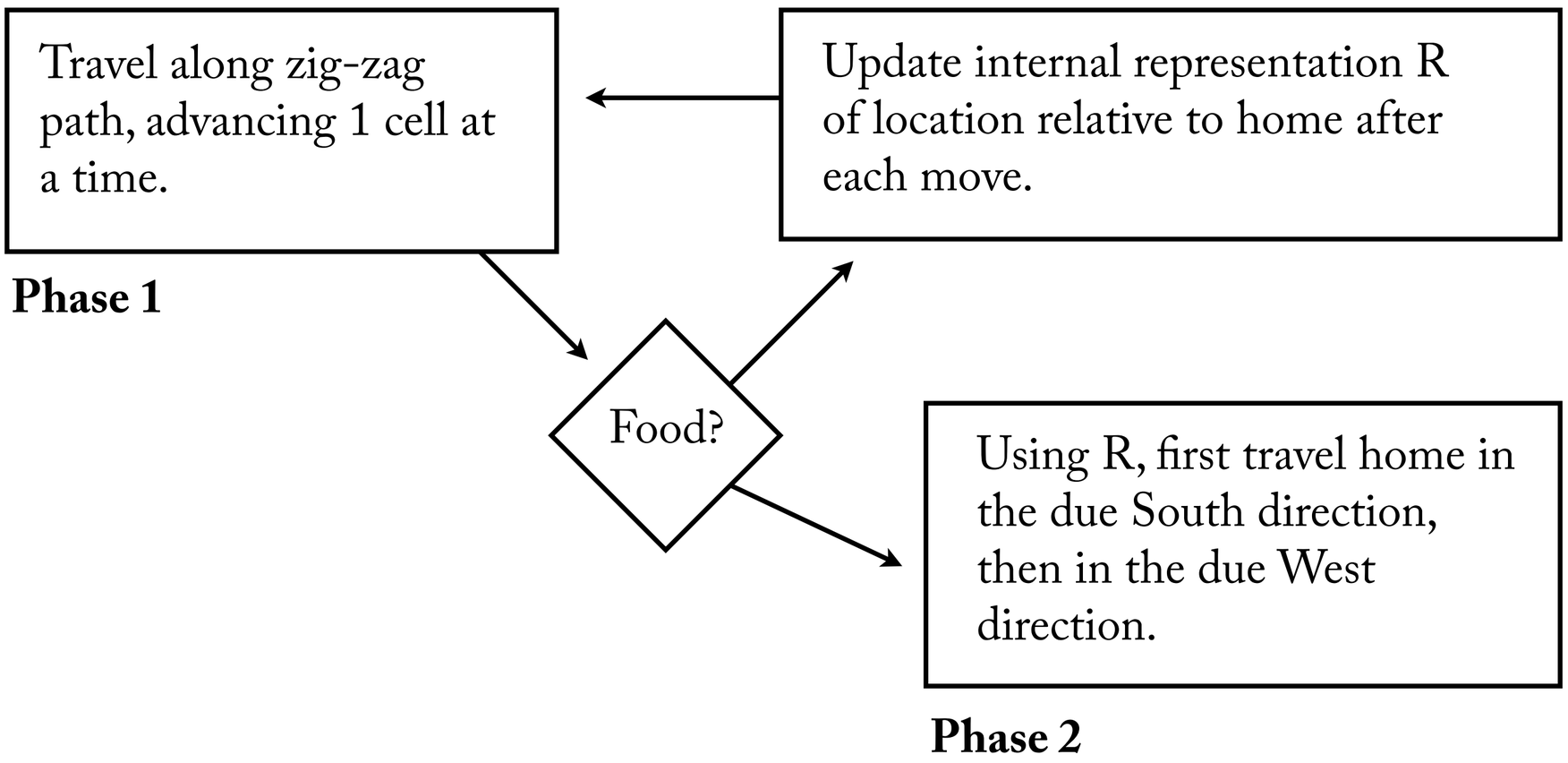
Evolutionary Analysis

Phase 1:
Exhaustive search for food through NE quadrant.

→

Phase 2:
Return home as efficiently as possible.

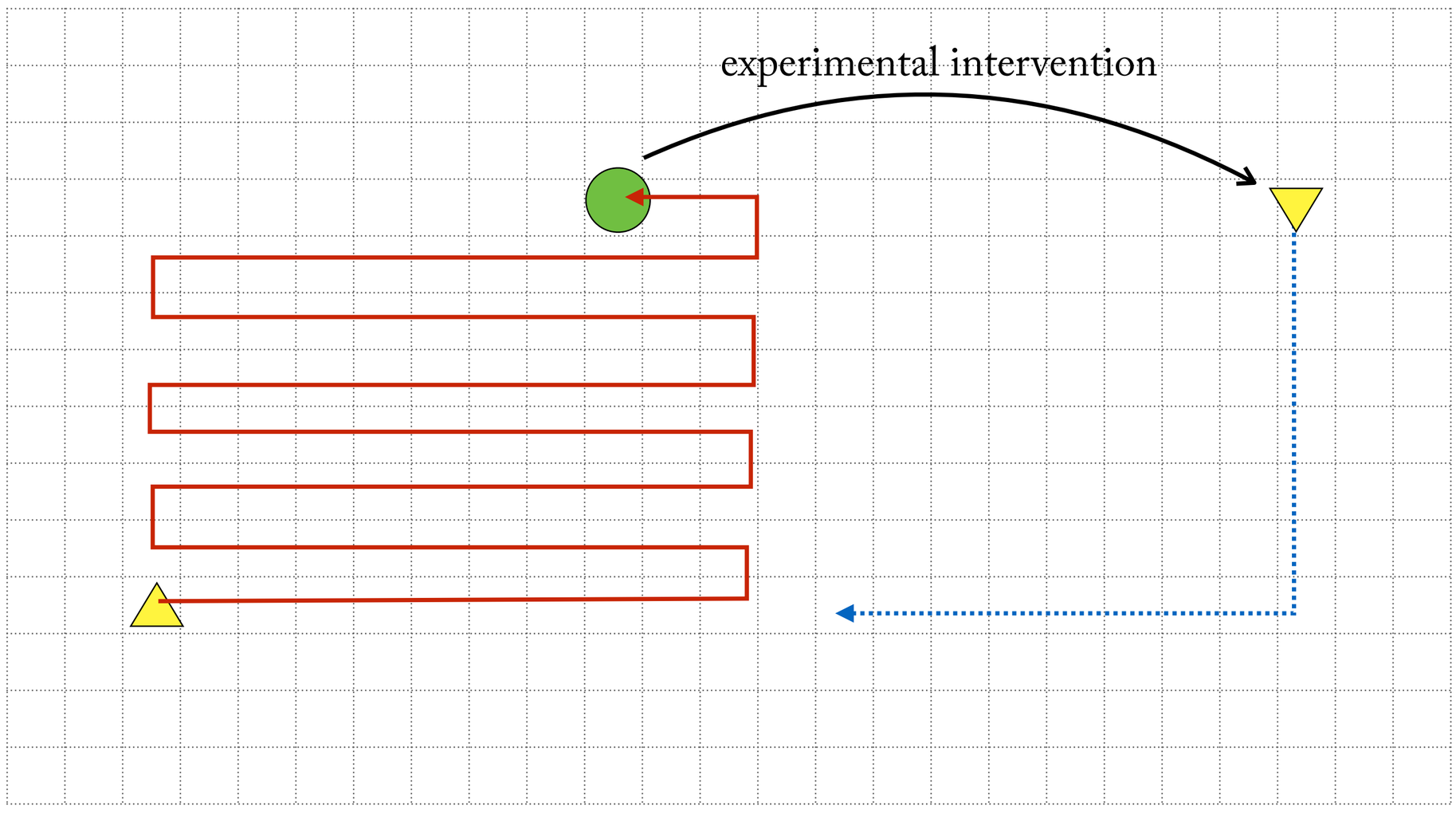
The Representational Hypothesis



"What makes this of interest to cognitive scientists is that it suggests that the brain of an insect is a symbol processing organ. It suggests that the brain uses computation to construct a cognitive map and to compute courses from the positional values stored in that map. The positional data stored on a map together with the computations that utilize those data to set courses constitute a **representation**, a model of the world used to determine courses of action in that world. Thus, the homing abilities of insect might be taken as strong evidence in favor of a computational-representational theory of mind."

Testing the Representational Hypothesis

An experiment with Turbots: evidence for mental representations.



Neuroscience

Next we capture a Turbot, bring it to the lab, and look inside. When we inspect the tape, this is what we find:

*	0	0	0	0	1	*	0	0	0	1	0	*

We capture other Turbots, find variations of our first result.

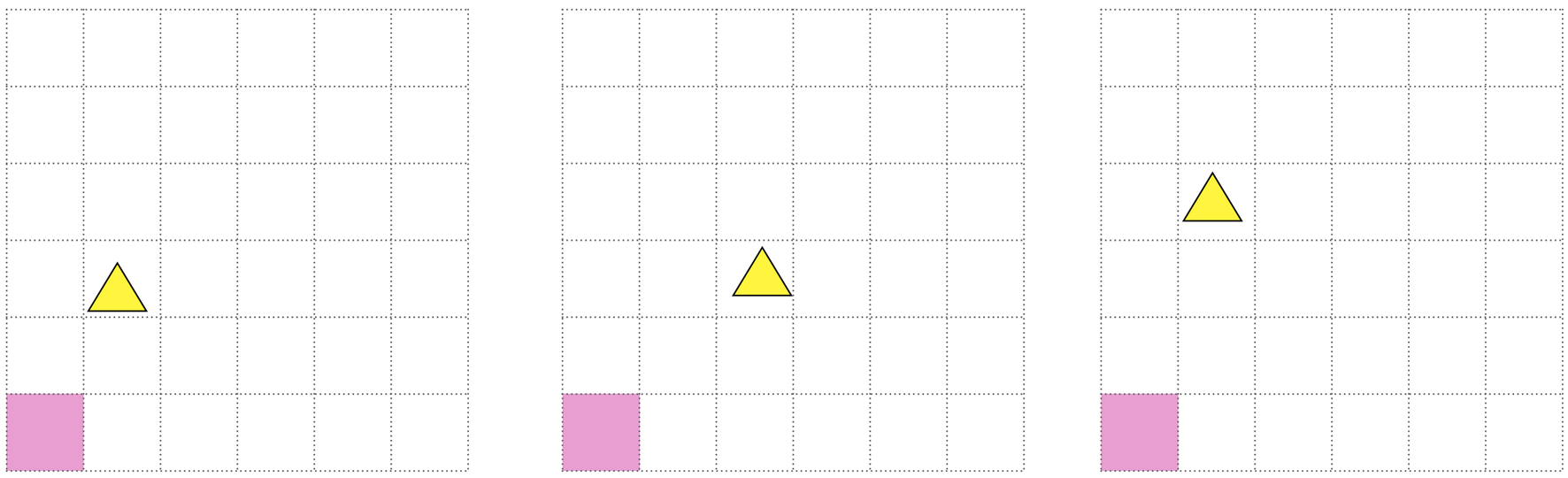
*	0	0	0	0	1	*	0	0	0	1	0	*

*	0	0	0	1	0	*	0	0	0	1	1	*

There are clearly some stable patterns here, but also variation. What does this all mean?

Cognitive Science

After testing several different hypotheses, someone in our lab comes up with the idea that the mark on the tape are correlated to the X- and Y- position of the Turbot. This makes sense from an evolutionary perspective. So we capture three different Turbots at different stages of their search for food.



00001 00010 00010 00010 00001 00011

We still don't really know what these marks *mean*, but we have a clue. When the Turbot moves in the X- direction, the left-hand block changes; when the Turbot moves in the Y-direction, the right-hand block changes. Otherwise they stay the same. This gives rise to the following hypothesis:

*	0	0	0	0	1	*	0	0	0	1	0	*

records X-position records Y-position

Externalism

According to **externalism,** for an internal symbol S to be a genuine mental representation of X, it must satisfy two criteria:

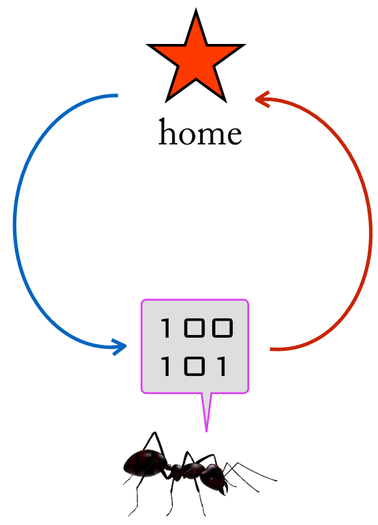
(1) it must function to mirror X in the world;

(2) it must be guided by perception (of X) and guide action (towards X).

When these criteria are met, mental representations are capable of **truth** and **falsity.**

Navigational symbols have meaning for the ant because:

- internal symbols c**arry information** about the direction home.
- internal symbols **guide the action** of the ant with respect to the direction home



Information Semantics

Mirroring

Further study reveals that the symbols in the Turbot memory systematically covary with the physical distance between the turbot and home. In fact, when the system is working properly, the following generalizations hold:

> binary(S_x) = the number of steps **east** between **here** and **home**.
> binary(S_y) = the number of steps **north** between **here** and **home**.

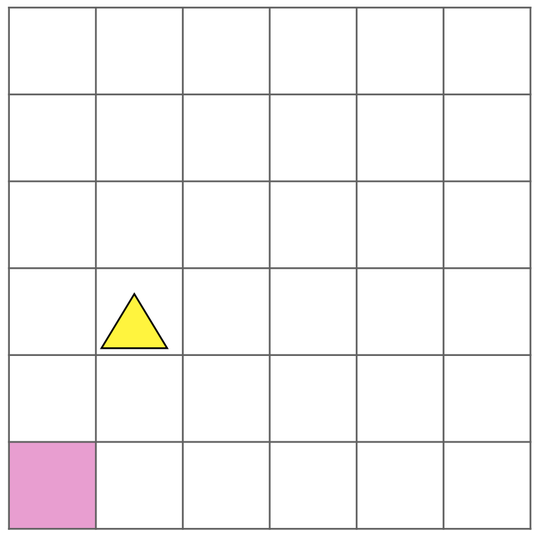
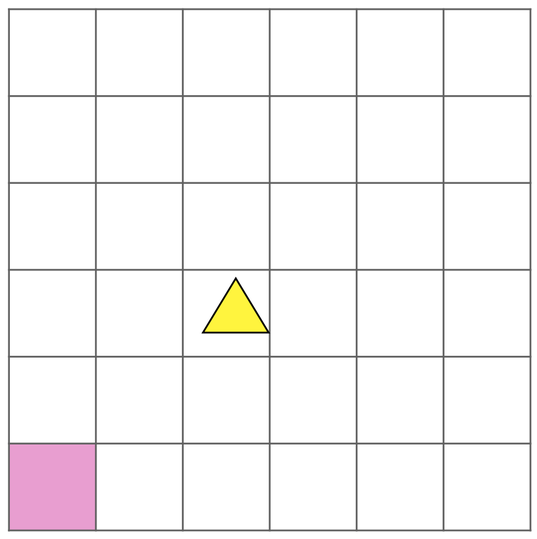
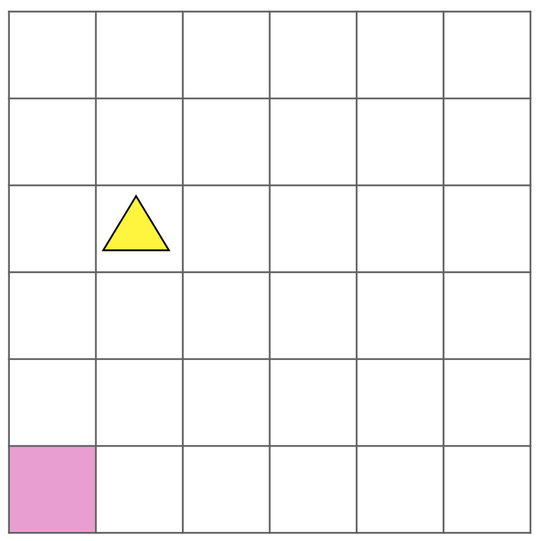
Mediating action and perception.

Truth and Falsity

This leads the idea that these symbols are **mental representations:** internal symbols which represent the "beliefs" or "cognitive commitments" of the Turbot. At any given time, the Turbot has the "beliefs":

> The number of steps **east** between **here** and **home** is binary(S_x).
> The number of steps **north** between **here** and **home** is binary(S_y).

These beliefs may be **true** or **false**. As we saw in our original displacement experiment, it is clear that the Turbots follow their internal symbols, *as if* the equations above were true, even when they are not.

Internal world	External world
00001 00010	
00010 00010	
00001 00011	

"A *mental representation* is a functioning isomorphism between a set of processes in the brain and a behaviorally important aspect of the world…"

Mental Representation

Further study reveals that the symbols in the Turbot memory systematically covary with the physical distance between the turbot and home. In fact, when the system is working properly, the following generalizations hold:

$\text{distance}_x(\text{here, home}) = \text{Bin}(S_x)$
$\text{distance}_y(\text{here, home}) = \text{Bin}(S_y)$

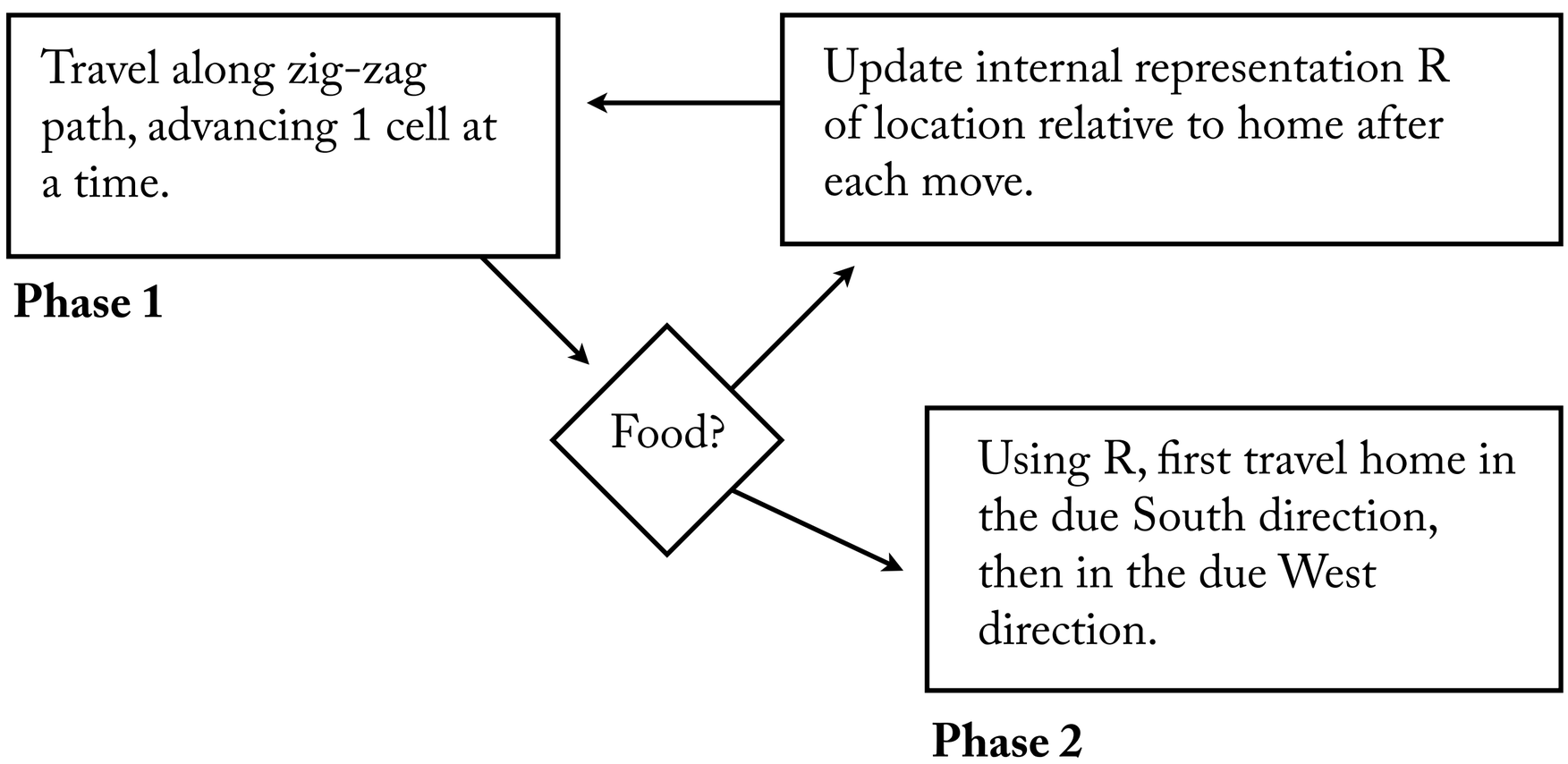
Yet, as we saw in our original displacement experiment, the system is not always working properly. Still, it is clear that the Turbots follow their internal symbols, *as if* the equations above were true, even when they are not.

This leads the idea that these symbols are **mental representations:** internal symbols which represent the "beliefs" or "cognitive commitments" of the Turbot.

Representational content = (at a time)

$\text{distance}_x(\text{here, home}) = a$ &
$\text{distance}_y(\text{here, home}) = b$

where $a = \text{Bin}(S_x)$ and $b = \text{Bin}(S_y)$



Mirroring the world. These representations can be thought of creating an internal way of "tracking" or "mirroring" an external phenomena.

Truth and falsity. The machine misrepresents the environment when the content of its symbols is false.

"A *mental representation* is a functioning isomorphism between a set of processes in the brain and a behaviorally important aspect of the world…

"Is the brain a symbol-processing system? The rise of cognitive psychology in the sixties reflected the emergence of the (then novel) view that the brain was a symbol-processing system analogous to a computer, and that it used its symbol-processing capacity to construct representations of the environment in which it functioned."

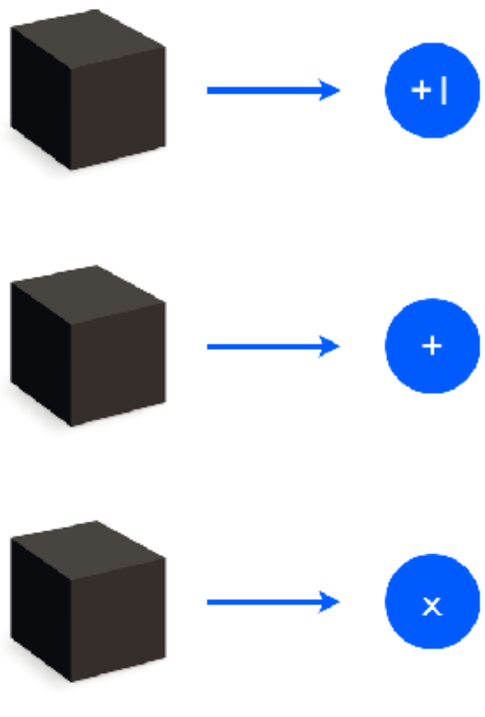
Suppose that we discover some Turbots in the wild, and we attempt to describe them as a cognitive scientist would.

Suppose further that they are the products of natural selection: over millions of cycles of reproduction, mutation, and selective pressure, our turbots have evolved. We assume that:

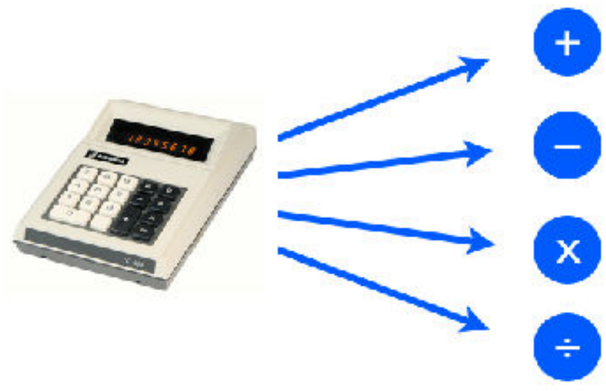
- **The environment** has stayed the same over the course of natural selection (it always involves the surrounding walls, and the placement of food).
- **Reproduction** involves the duplication of machines in similar environments.
- **Variation** between machines involves random permutations of logic gates or tape configurations.
- **Selective pressure**: if a turbot doesn't find food and return home within a certain time span, it dies without having a chance to reproduce.

36. Machines and Programs

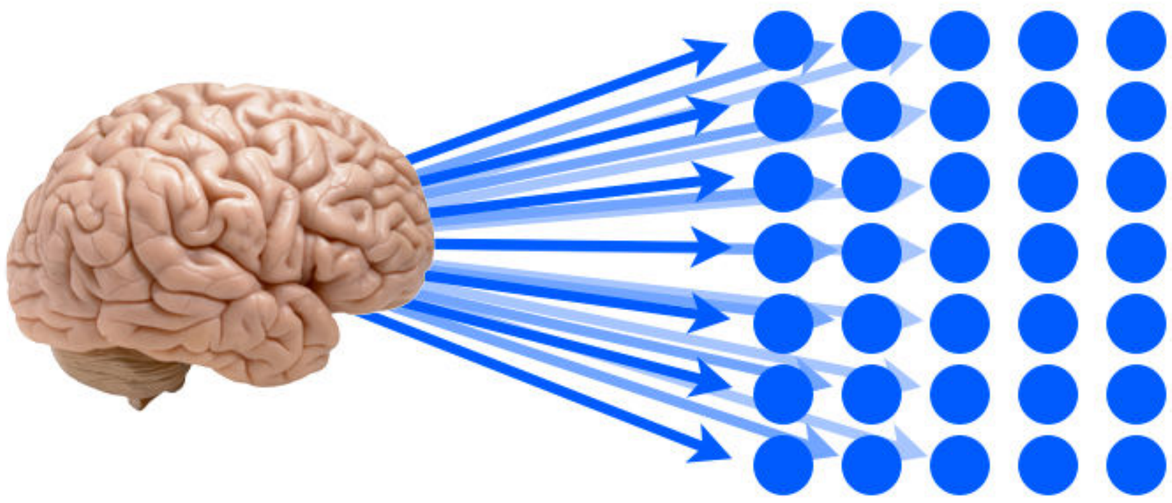
Multi-function Machines



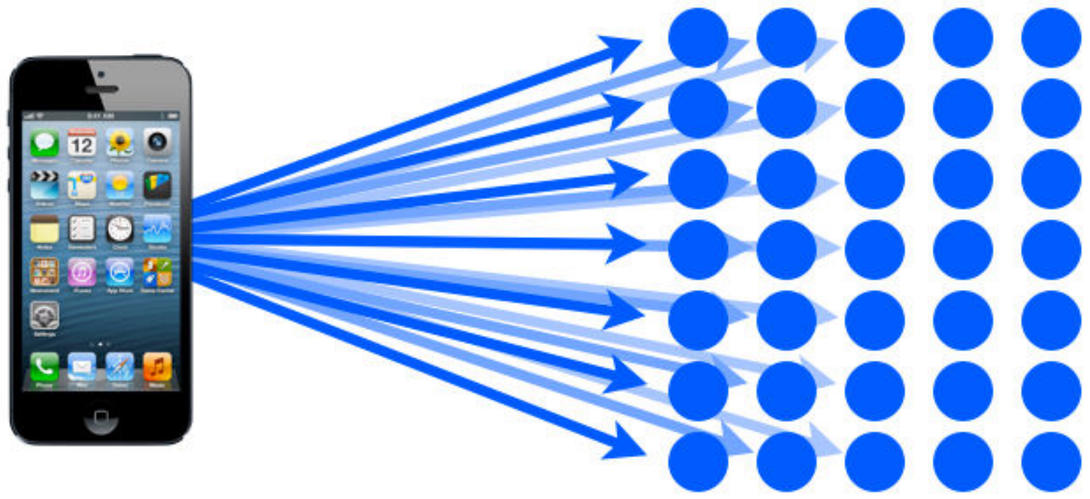
So far, all of the machines we have encountered compute one function each. Of course, it is possible to construct a machine which computes more than one function. A simple calculator computes four functions.



Our brains can compute a lot more than four functions. In fact, it seems like there is no practical limit on the number of functions a brain could compute. It is an *infinite* function machine.

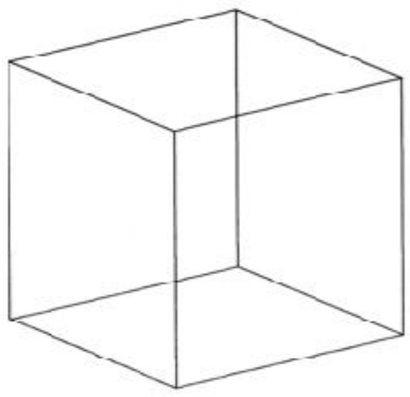


This isn't just true of our brains. It's true of any computer which can read *programs*. Such devices are known as **universal computers**. Your laptop and your phone are universal computers.



Universal Turing Machines: The Goal

The goal: Create a Turing Machine which can perform the task of any other Turing Machine. This is called a **Universal Turing Machine** (UTM).



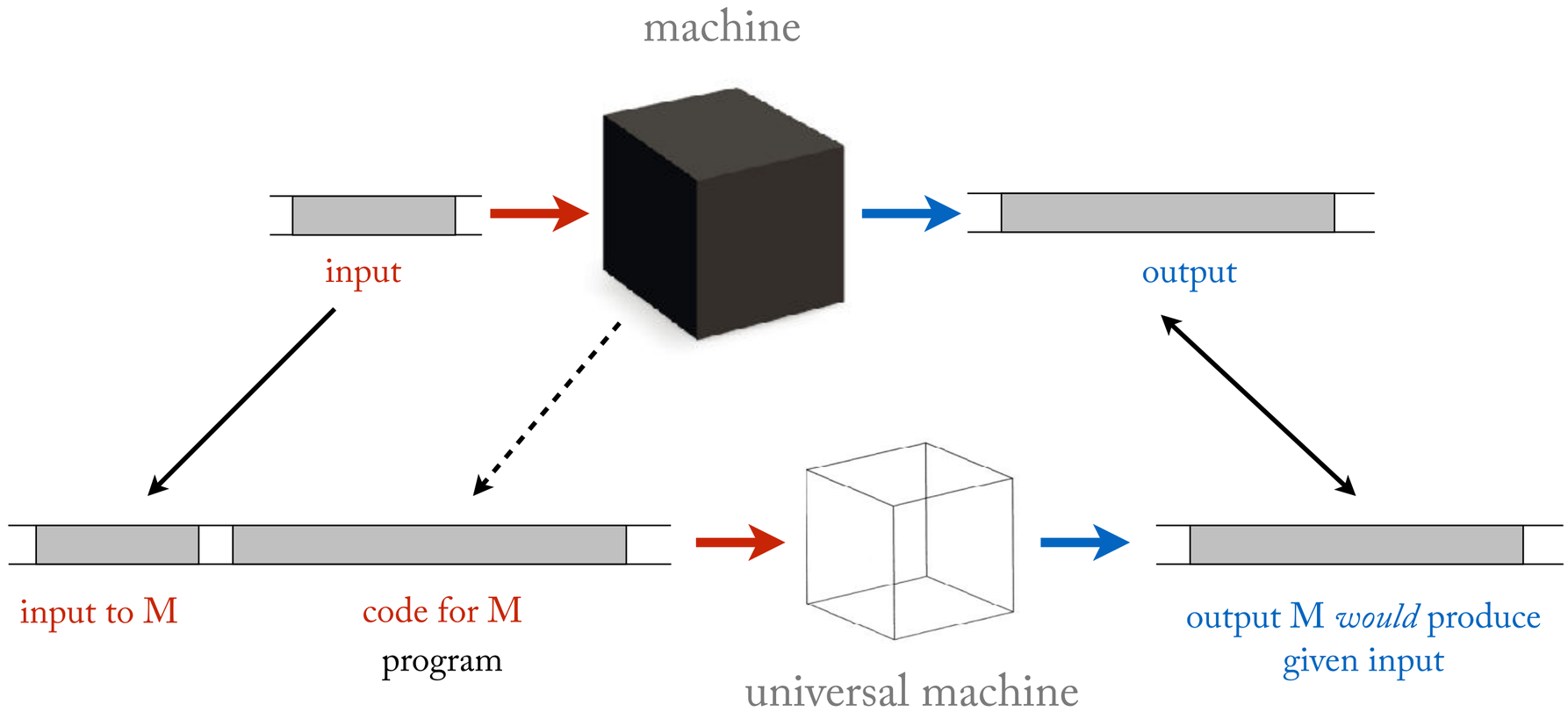
universal machine

Universal Turing Machines: The Idea

Strategy for designing a UTM:

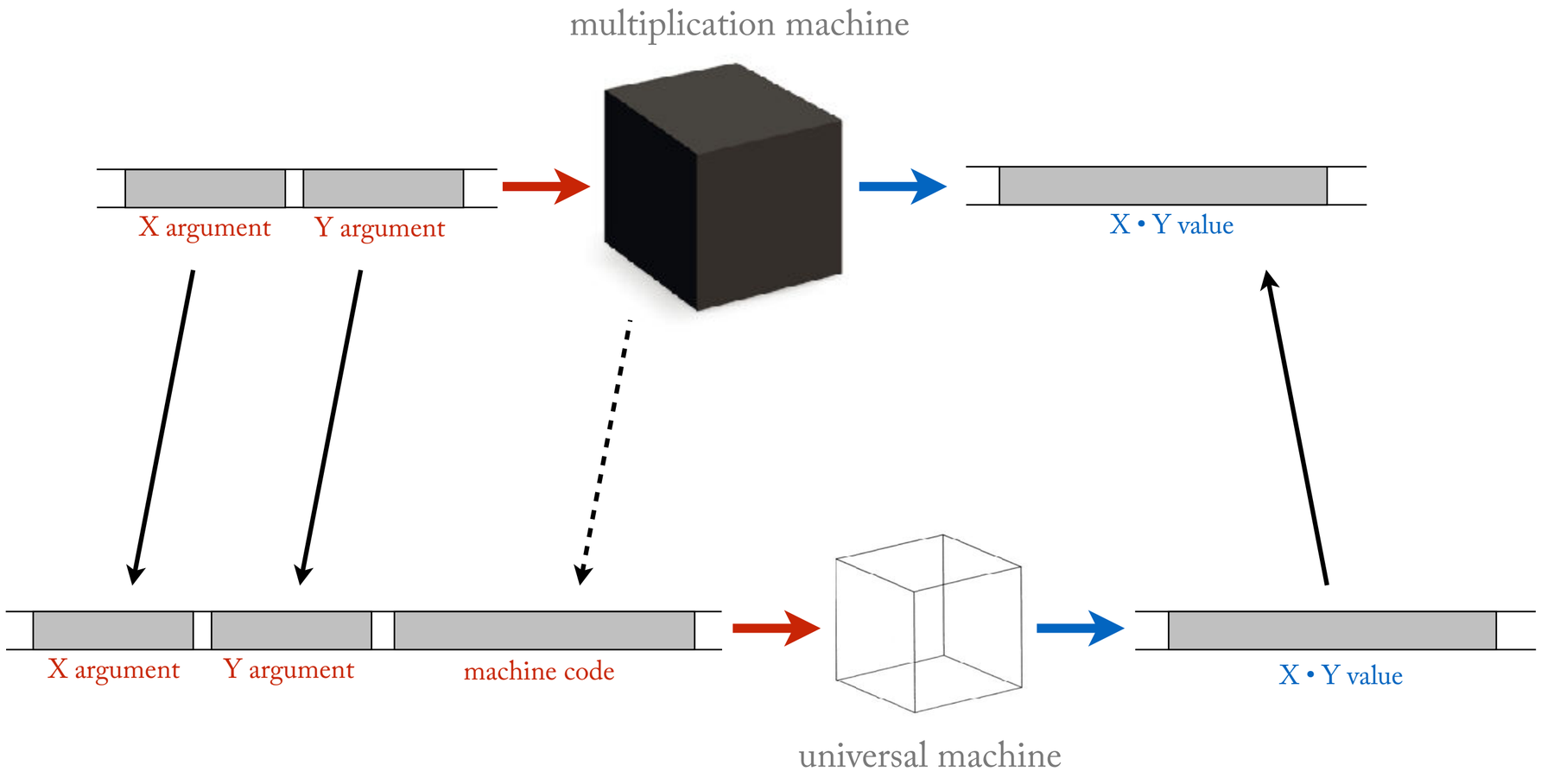
Stage 1: find a method for turning an arbitrary Turing Machine (the simulated machine) into something a Turing Machine can read— i.e. 1's and 0's on tape. This something is called the simulated machine's **code number**.

Stage 2: invent a UTM which takes as input (i) any simulated machine's code number, and (ii) the simulated machine's input; the UTM then outputs the simulated machine's output.



For example, let M be a multiplication machine. It takes two inputs (corresponding to the X and Y arguments) and returns one output (corresponding to the value X•Y). And let's say that M's code number is C.

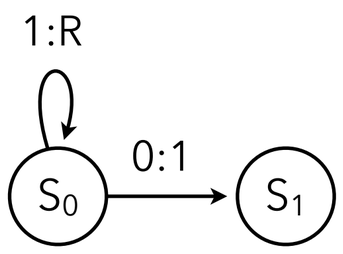
Then if we give a tape which contains, X, Y, *and* C to the UTM, it will return the value X•Y.



Codes Numbers (aka Programs)

Derive a machine's code number in five easy steps.

Step 1. Choose a Turing Machine, and draw up it's state table.



STATE	IN	OUT	NEXT STATE
S_0	1	R	S_0
	0	1	S_1
S_1	1	H	
	0	H	

Step 2. Divide each state cell into rows according to input. Eliminate rows which do not lead to another state.

STATE	IN	OUT	NEXT STATE
S_0	1	R	S_0
S_0	0	1	S_1
S_1	1	H	
S_1	0	H	

Step 3. Replace all symbols with numbers, according to a translation system.

State S_n	⟿	*n*
No state	⟿	0
Read a "0"	⟿	0
Read a "1"	⟿	1
Write a "0"	⟿	0
Write a "1"	⟿	1
Move left (L)	⟿	2
Move right (R)	⟿	3
Halt (H)	⟿	4

STATE	IN	OUT	NEXT STATE
0	1	3	0
0	0	1	1
1	1	4	0
1	0	4	0

Step 4. Represent the table by a sequence of quadruples:

state, in, out, next

state, in, out, next

etc...

0, 1, 3, 0
0, 0, 1, 1
1, 1, 4, 0
1, 0, 4, 0

Step 5. Represent each number in tally. Represent commas with 0's and line breaks with *s. The result is a string 0's and 1's.

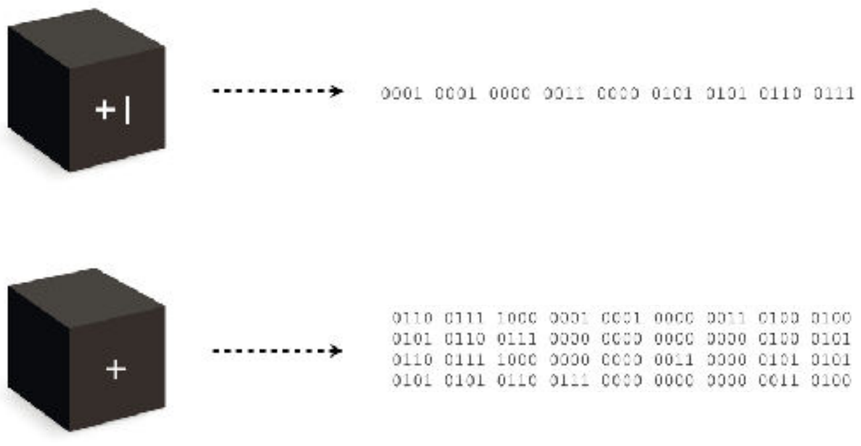
001011100*000010

1*10101111100*100

0111100

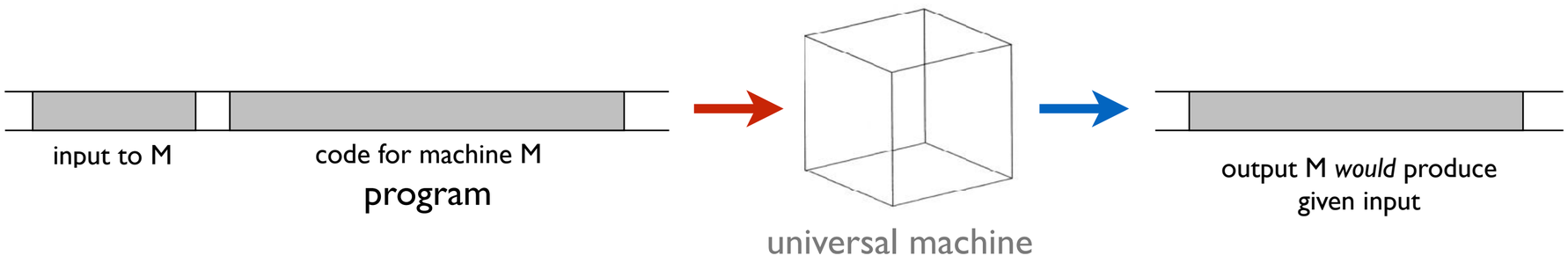
Now every single Turing Machine can be associated with a code number. And every single (well-formed) code number corresponds to a Turing Machine.

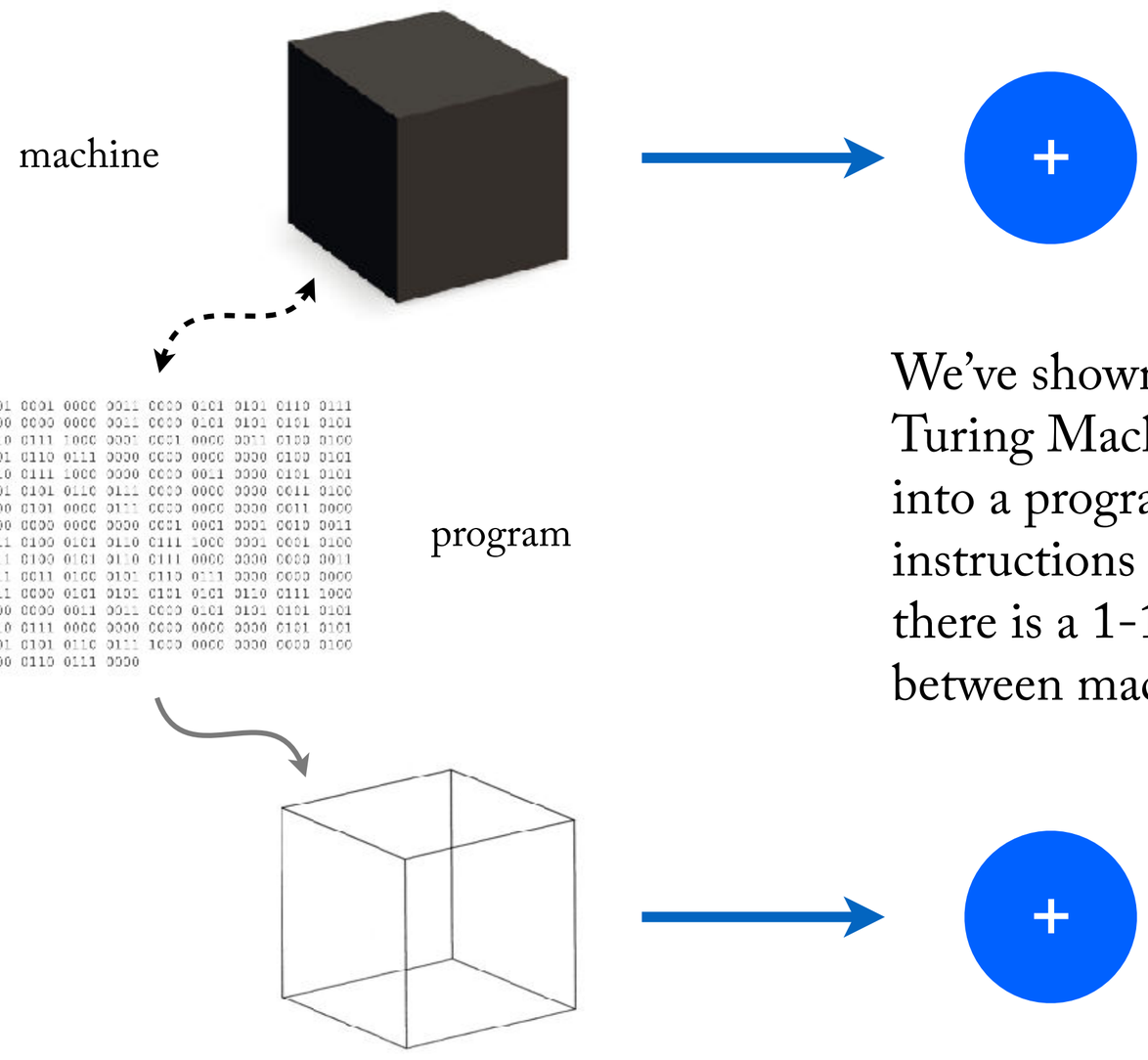
These code numbers will now become the programs which are given to the Universal Machine as input.



Machines as Programs

The code for a machine is also known as its **program**. A program is simply an algorithm written down in a language that a universal machine can interpret.





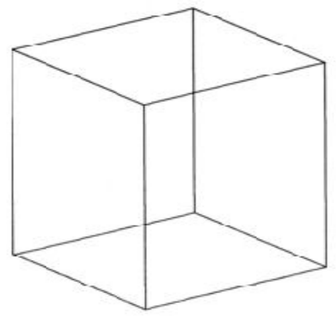
We've shown that any given Turing Machine can be turned into a program which serves as instructions for the UTM. So there is a 1-1 correspondence between machines and programs.

Up until now algorithms have been used to describe machines at a higher level of abstraction. Now they are used as something which is inputted *into* the machine. The universal machine, in effect, *reads* higher levels of abstraction.

37. The Universal Machine

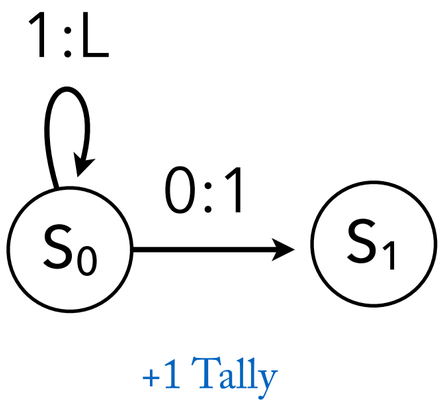
The Idea

The goal: Create a Turing Machine which can perform the task of any other Turing Machine. This is called a **Universal Turing Machine** (UTM).



universal machine

Step 1: Identify a method for translating a machine into a **code number**, which will function as the **program** for that machine.



+1 Tally

STATE	IN	OUT	NEXT
S_0	1	L	S_0
S_0	0	1	S_1
S_1	1	H	
S_1	0	H	

0, 1, 2, 0
0, 0, 1, 1
1, 1, 4, 0
1, 1, 4, 0

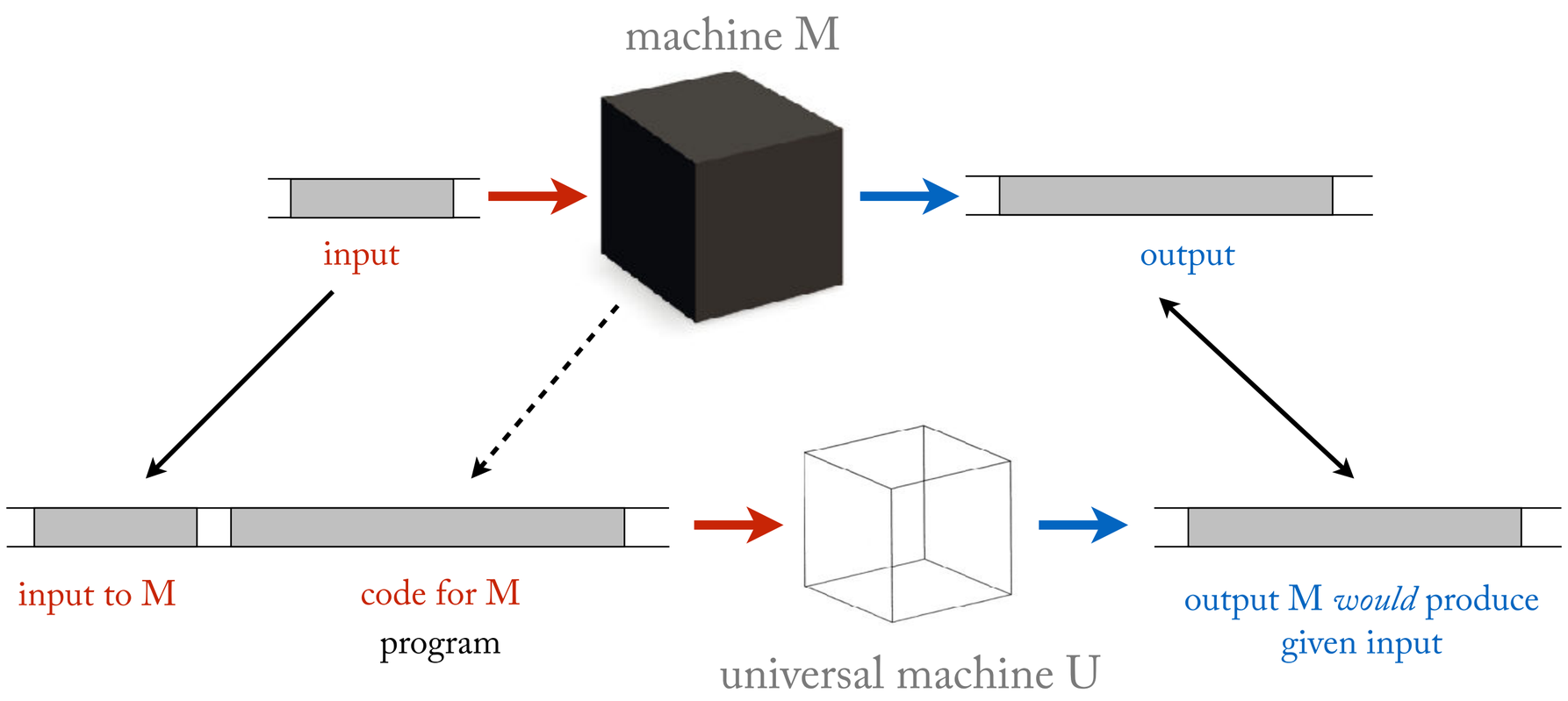
Turing machine → state table → numeral array

* 001011000 * 00001010 * 10101111000 * 10101111000 *

row 1 row 2 row 3 row 4

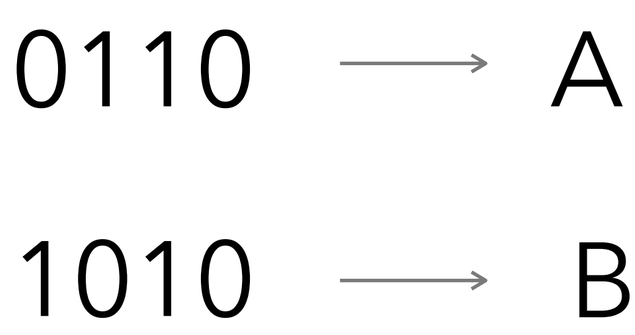
→ code number

Step 2: design a machine U which takes as input (i) any simulated machine M's code number, and (ii) M's input on a given occasion; U then produces the output which M would have produced in response to that input.



Markers

It's often useful to design an algorithm which makes use of more than two symbols. We can do this with 0's and 1's, by making *sequences* of 0's and 1's stand for additional symbols. This is more or less how your computer stores the 26+ letters of the English alphabet.

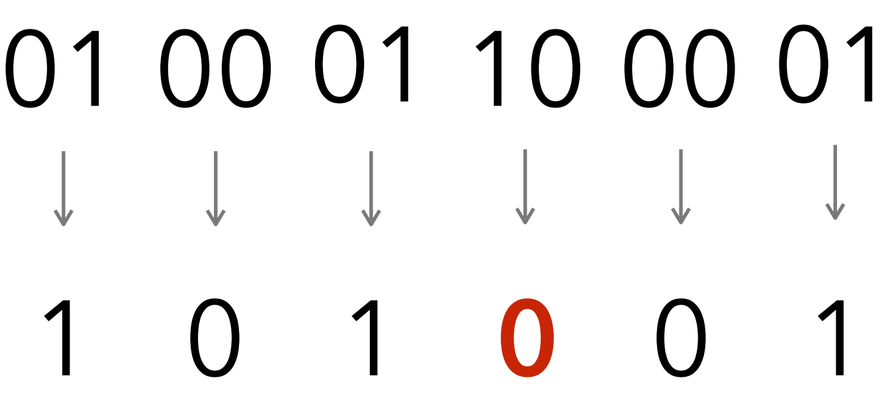


In particular, it is valuable to introduce a symbol which serves as a **marker** to indicate that a given symbol is in someway different from other symbols. Markers can be used to hold your place while computing, or two mark the end or beginning of a block.

A simple and useful scheme is to allow each *two* bits to represent a symbol. Then the the right hand bit will represent the symbol itself, and the left hand bit will indicate whether it is "marked".

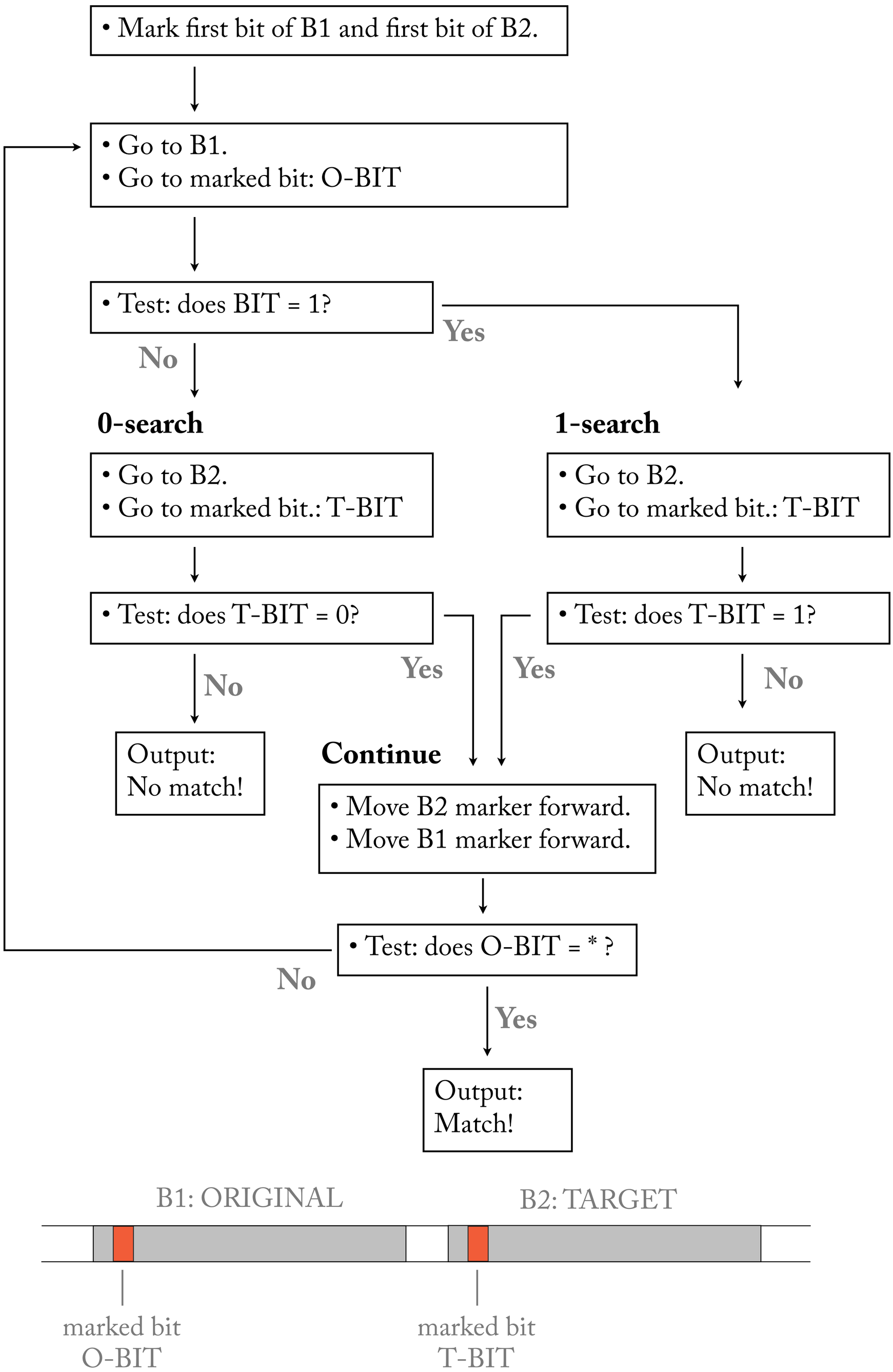


Now we can pick a marked symbol out of a crowd:



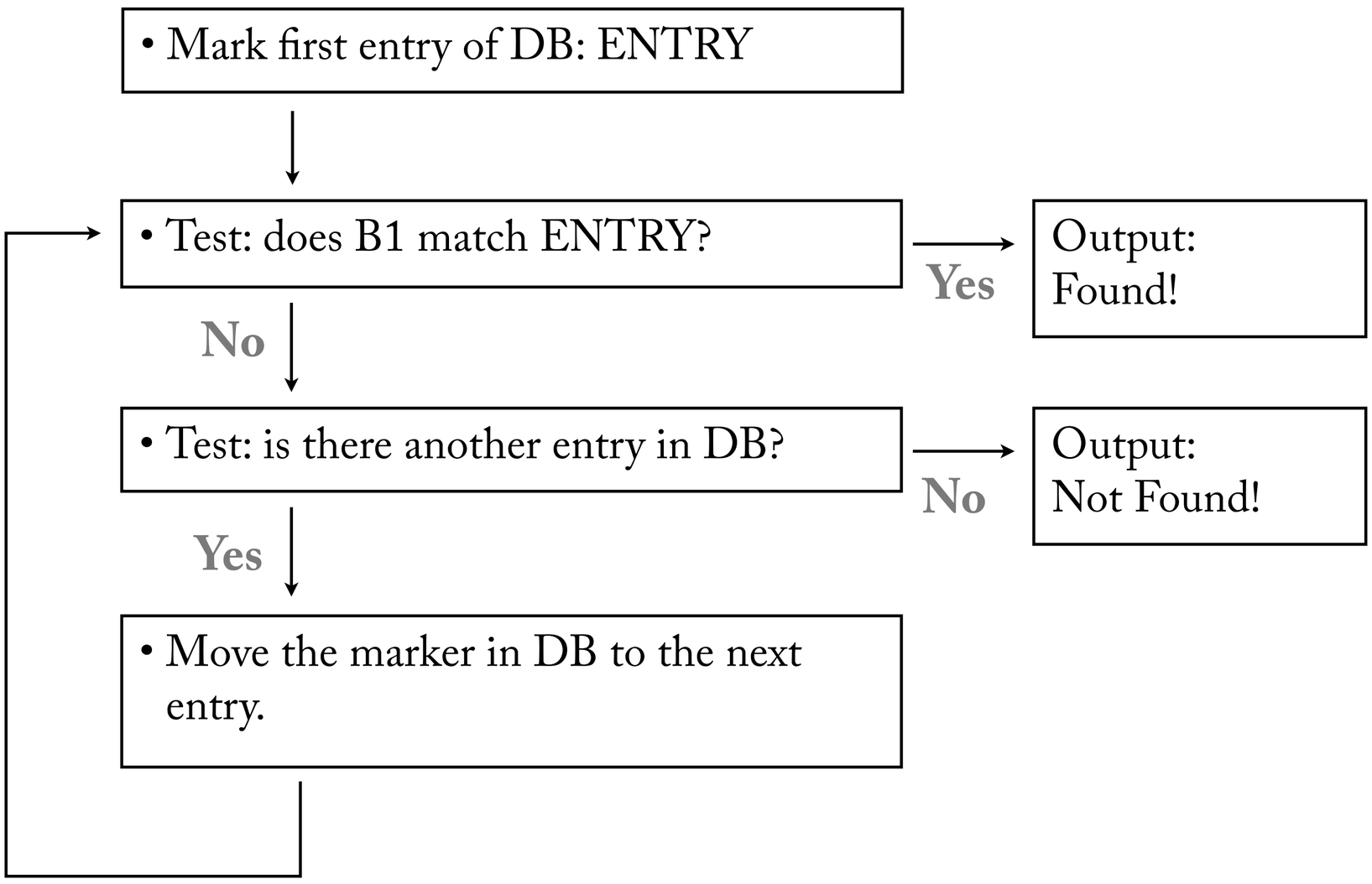
Matching

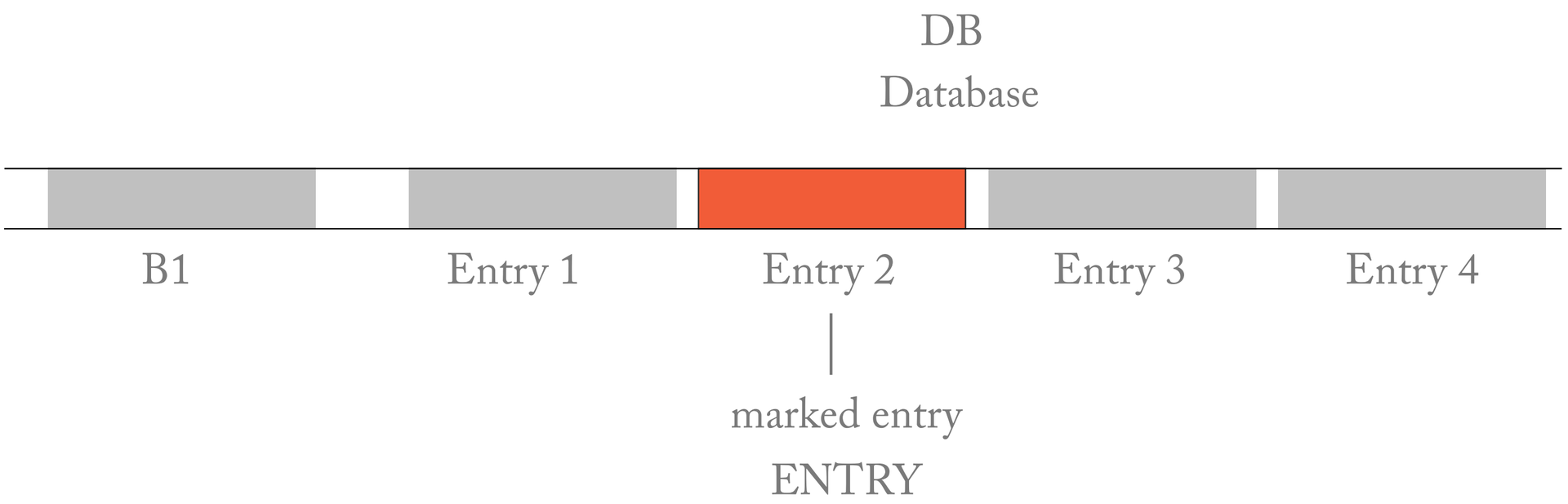
This algorithm compares an original string to a target string, and tests whether the two match or not.



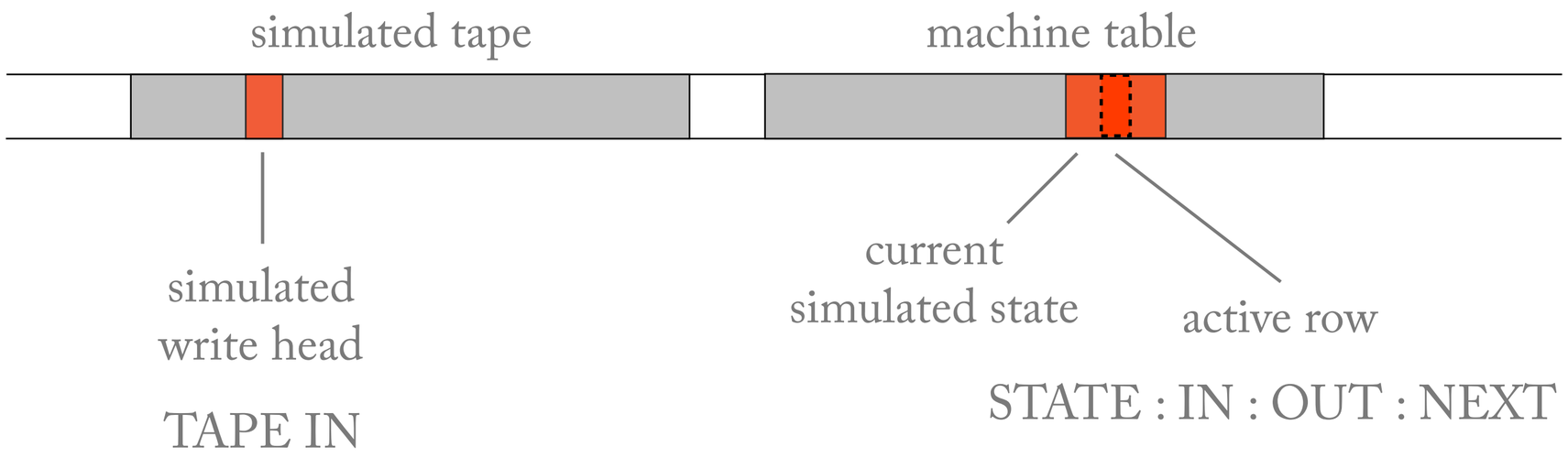
Search

This algorithm searches through a database to find an entry that matches some original sample. It outputs “Found!” or “Not Found!” depending on the result of the search.





Design for a Universal Turing Machine



Initialize + select first state
- Move the simulated write head to the first symbol of the simulated tape.
- Makes the first state of the machine table the current simulated state.

Initialize the simulated tape set the current state to S0.

Select current instruction
- Search the input column of the current simulated state for TAPE IN.
- Make that row the active row: STATE : IN : OUT : NEXT

Search for the current row based on the input from the simulated tape…

Check for final instruction
- Test: does OUT = Halt?

Yes → **Clean up**
- Clean up and halt.

No

Perform current instruction
- Perform OUT on the simulated tape, at the position of the simulated write head.

…make the appropriate simulated output: writing or moving on the simulated tape.

Select next state
- Search the state column of the machine table for NEXT.
- Make that state the current simulated state.

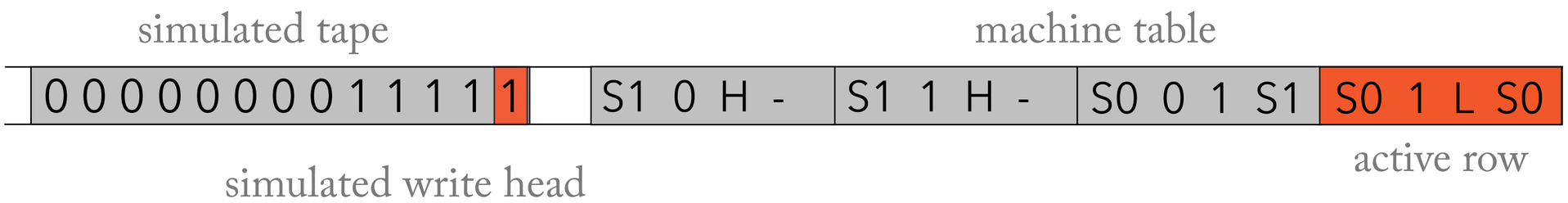
Find the next state named by the current row of the machine table, and make that the new current state.

Operation of a Universal Turing Machine

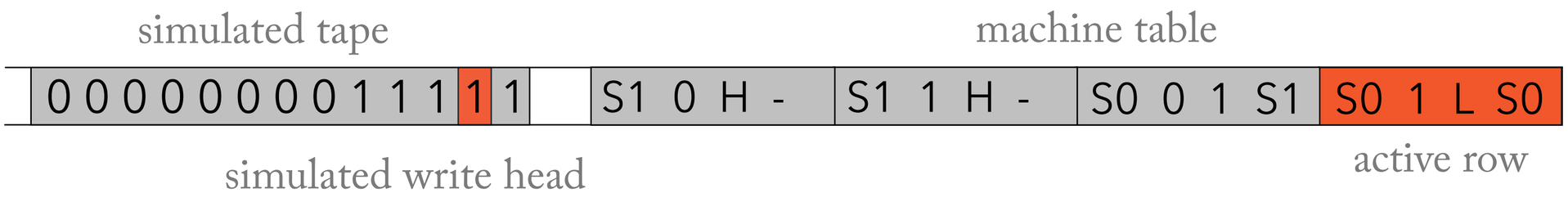
The following sequence illustrates the progress of the UTM, given the following machine table as its program.

STATE	IN	OUT	NEXT
S_0	1	L	S_0
S_0	0	1	S_1
S_1	1	H	
S_1	0	H	

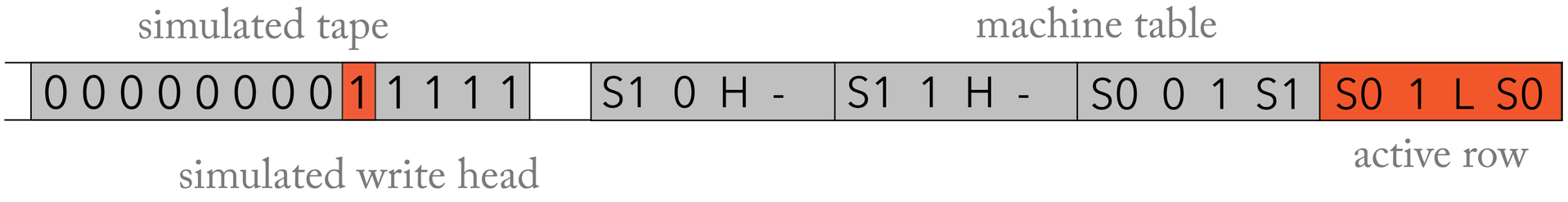
Initial state of the tape:



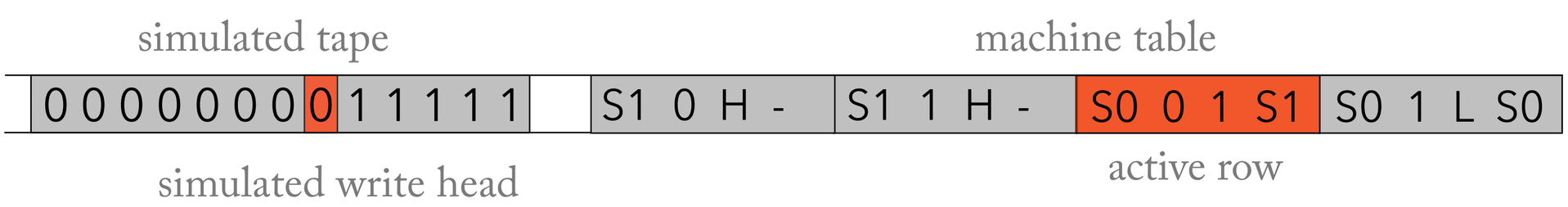
After 1 input/output cycle:



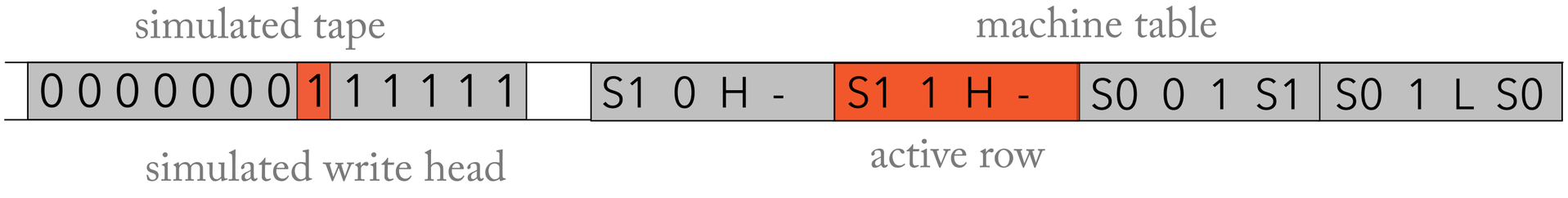
After 4 cycles:



After 5 cycles:



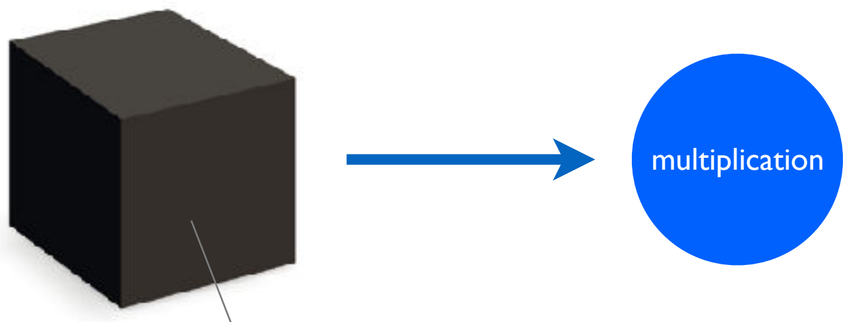
After 6 cycles:



38. Cognitive Architecture

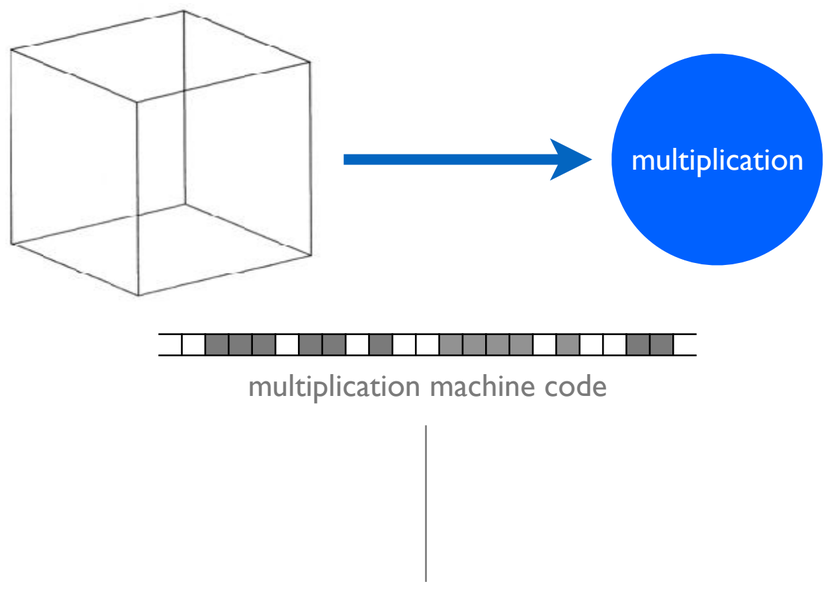
Implicit and Explicit Knowledge

Computes multiplication **without** any representation of a multiplication algorithm.

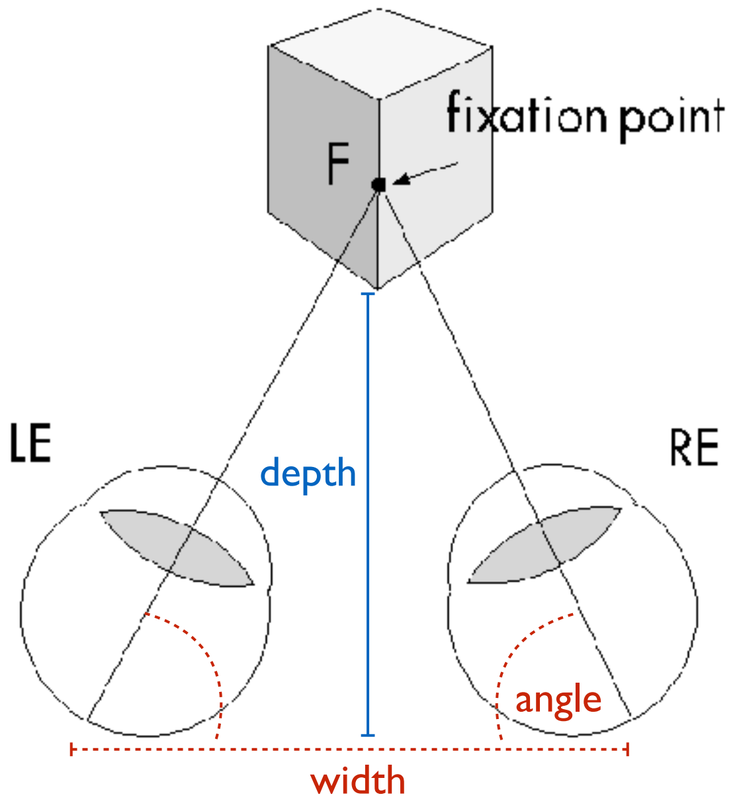


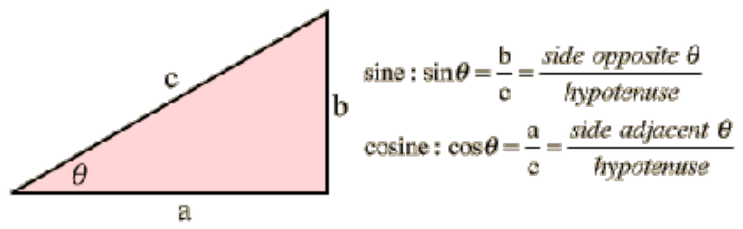
IMPLICIT KNOWLEDGE

Computes multiplication **with** a representation of a multiplication algorithm.



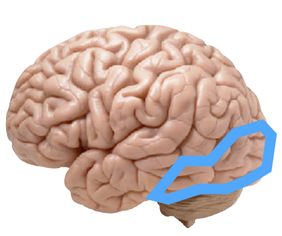
EXPLICIT KNOWLEDGE







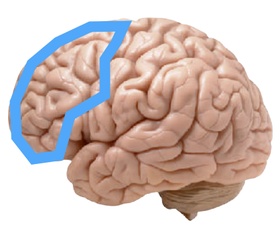
IMPLICIT KNOWLEDGE



Computes *tangent* **without** an explicit representation of a tangent algorithm.

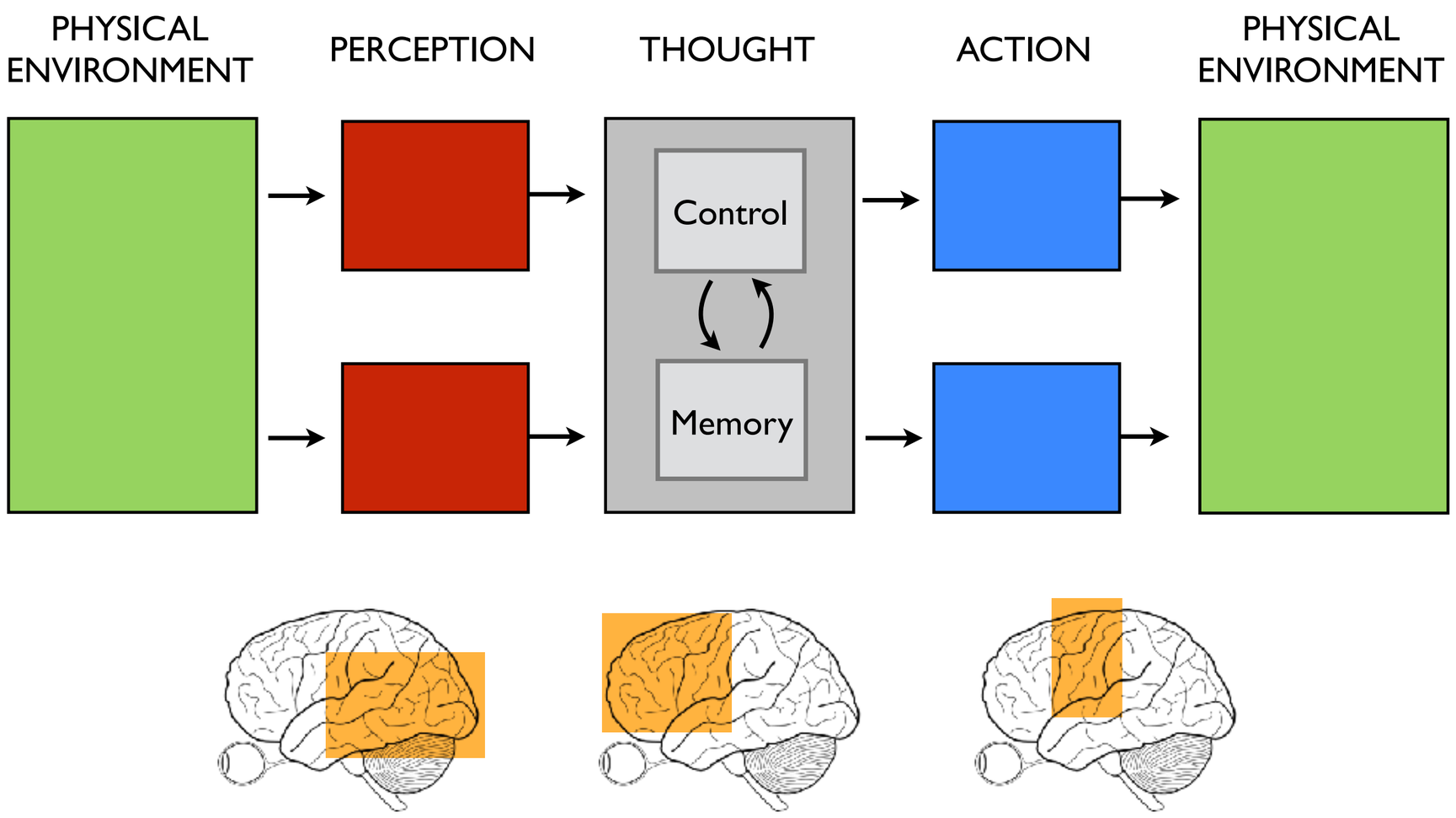


EXPLICIT KNOWLEDGE



Computes *tangent* **with** an explicit representation of a tangent algorithm.

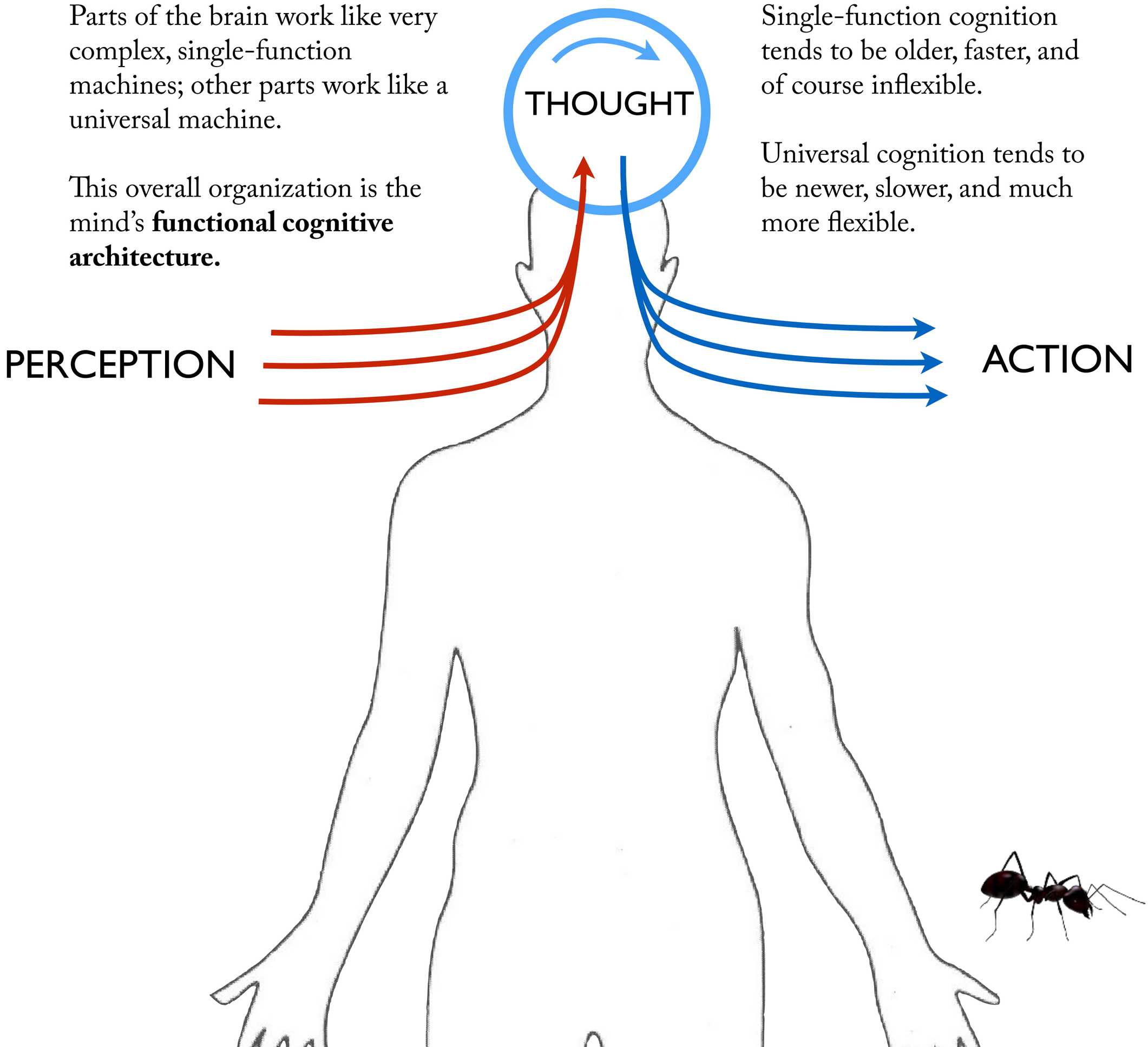
Cognitive Architecture



LOW-LEVEL COGNITION | **HIGH-LEVEL** COGNITION | LOW-LEVEL COGNITION

Parts of the brain work like very complex, single-function machines; other parts work like a universal machine.

This overall organization is the mind's **functional cognitive architecture.**

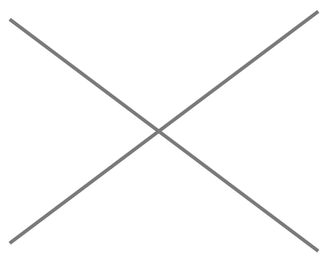


Single-function cognition tends to be older, faster, and of course inflexible.

Universal cognition tends to be newer, slower, and much more flexible.

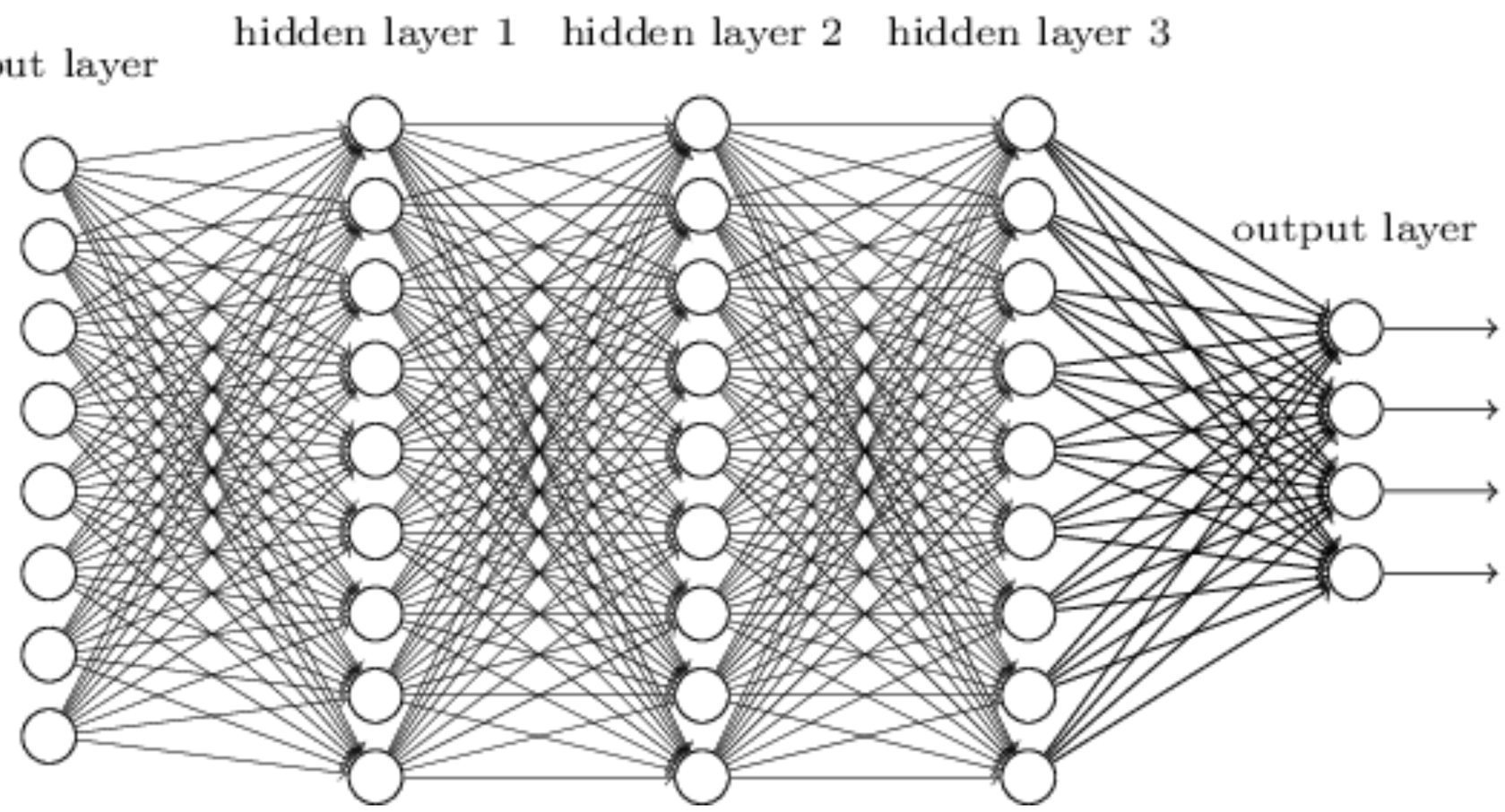
Implicit and Explicit Knowledge

There are two ways to draw the distinction between "implicit" and "explicit" knowledge.

	Implicit Knowledge Encoded in Architecture (not represented)	**Explicit Knowledge** Encoded in Memory (represented)
Implicit Knowledge Unconscious Knowledge	perceptual rules, motor control rules, knowledge how Turbot: Binary +1	implicit bias, blindsight, ? language rules Turbot: x-y coordinates UTM: Binary +1
Explicit Knowledge Consciously Accessible Knowledge		knowledge that, memories, beliefs, intentions, attitudes, reasoning, mental imagery

---

Some computer architectures, such as "neural nets" (not necessarily what's in our brains), don't draw a sharp distinction between logic engine and memory. Hence they don't draw a sharp distinction between knowledge encoded in *architecture* and knowledge encoded in *representations*.



39. Computation as Consciousness

The Architecture of Human Cognition

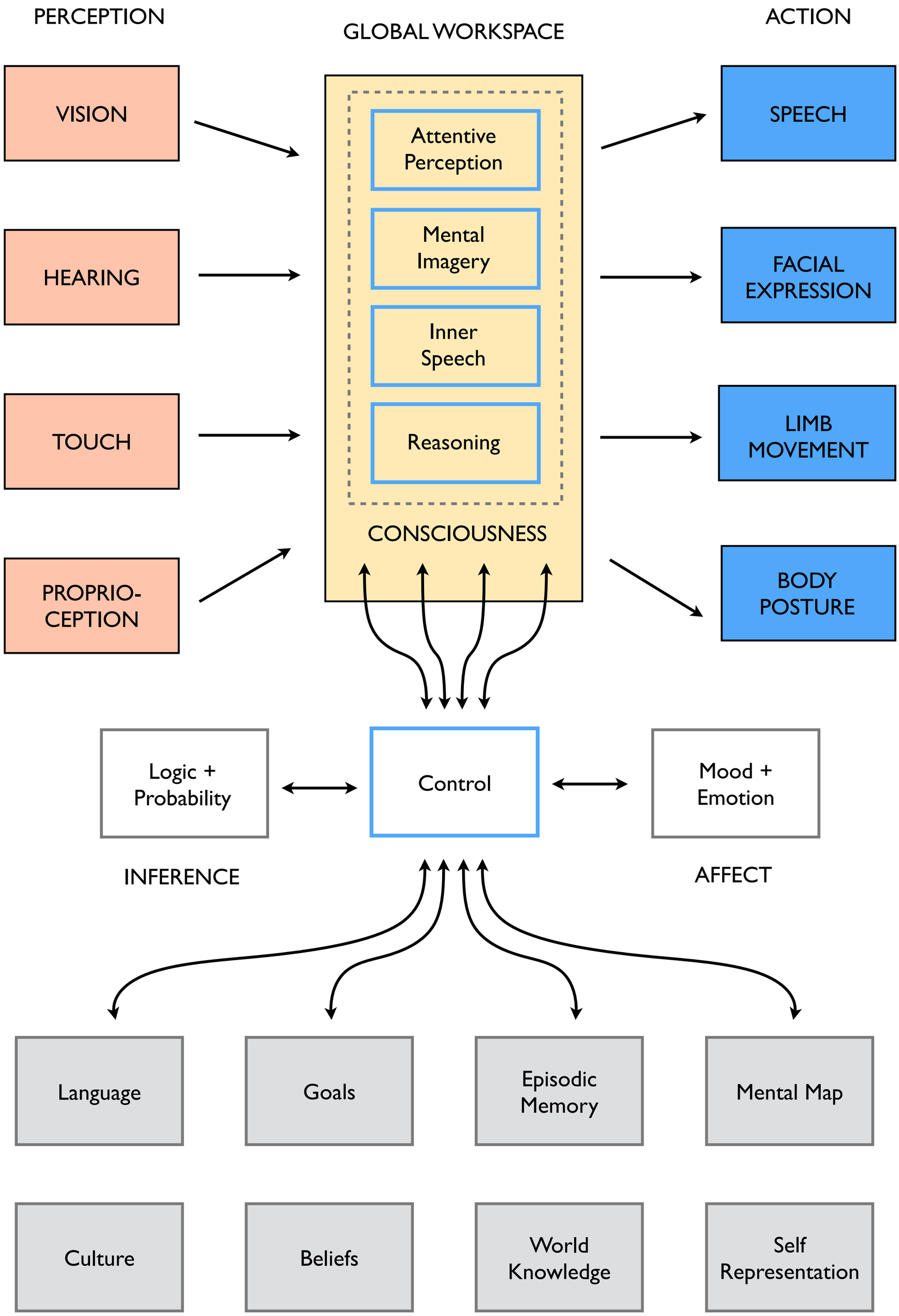
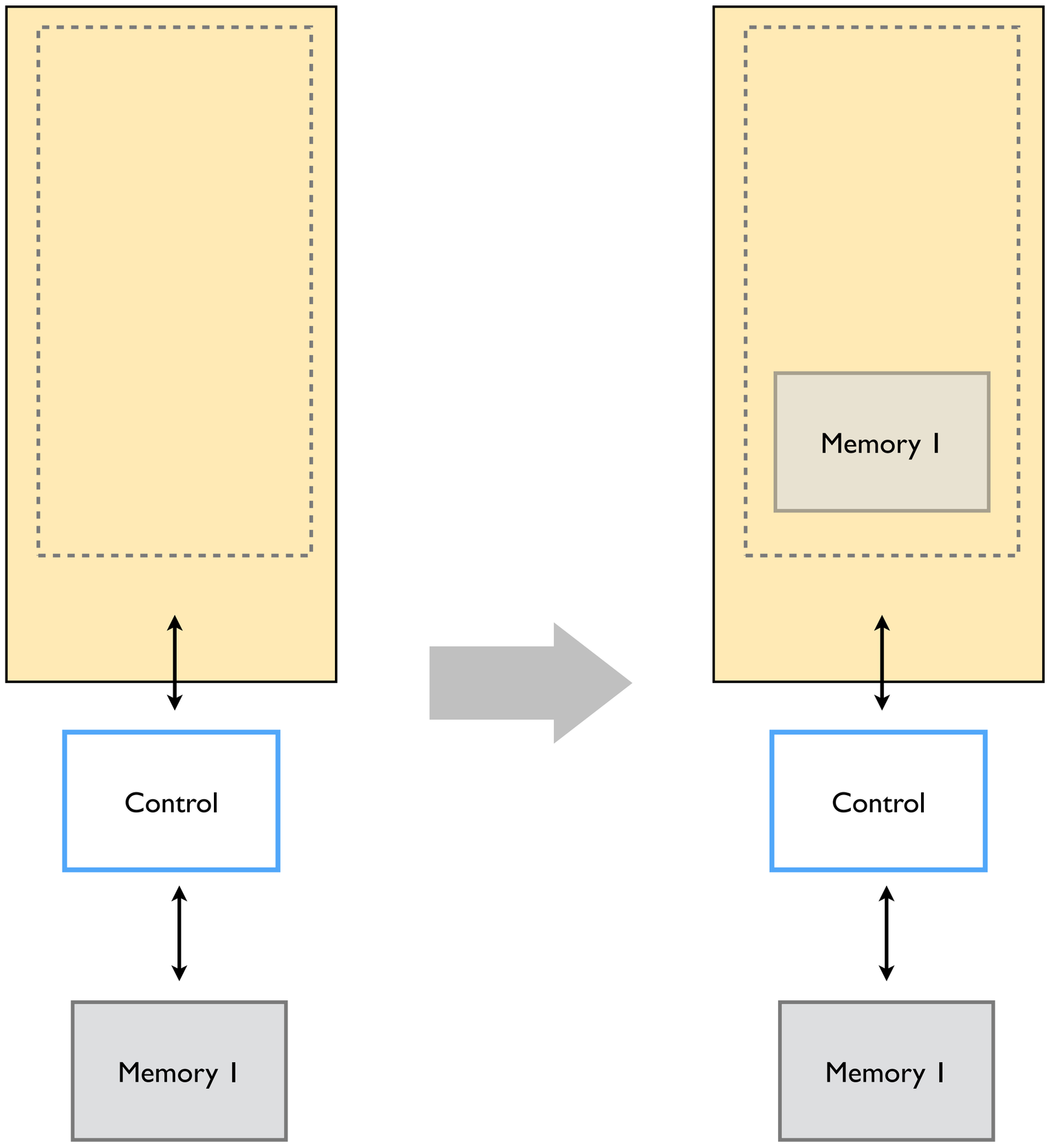


Illustration: recalling a memory.



Consciousness as center of information integration.

> "The ancient theater metaphor of consciousness gives us one plausible way to think about this evidence. A theater combines limited events taking place on stage with a vast audience, just as consciousness involves very limited information that with access to a vast number of unconscious sources of knowledge." (Baars)

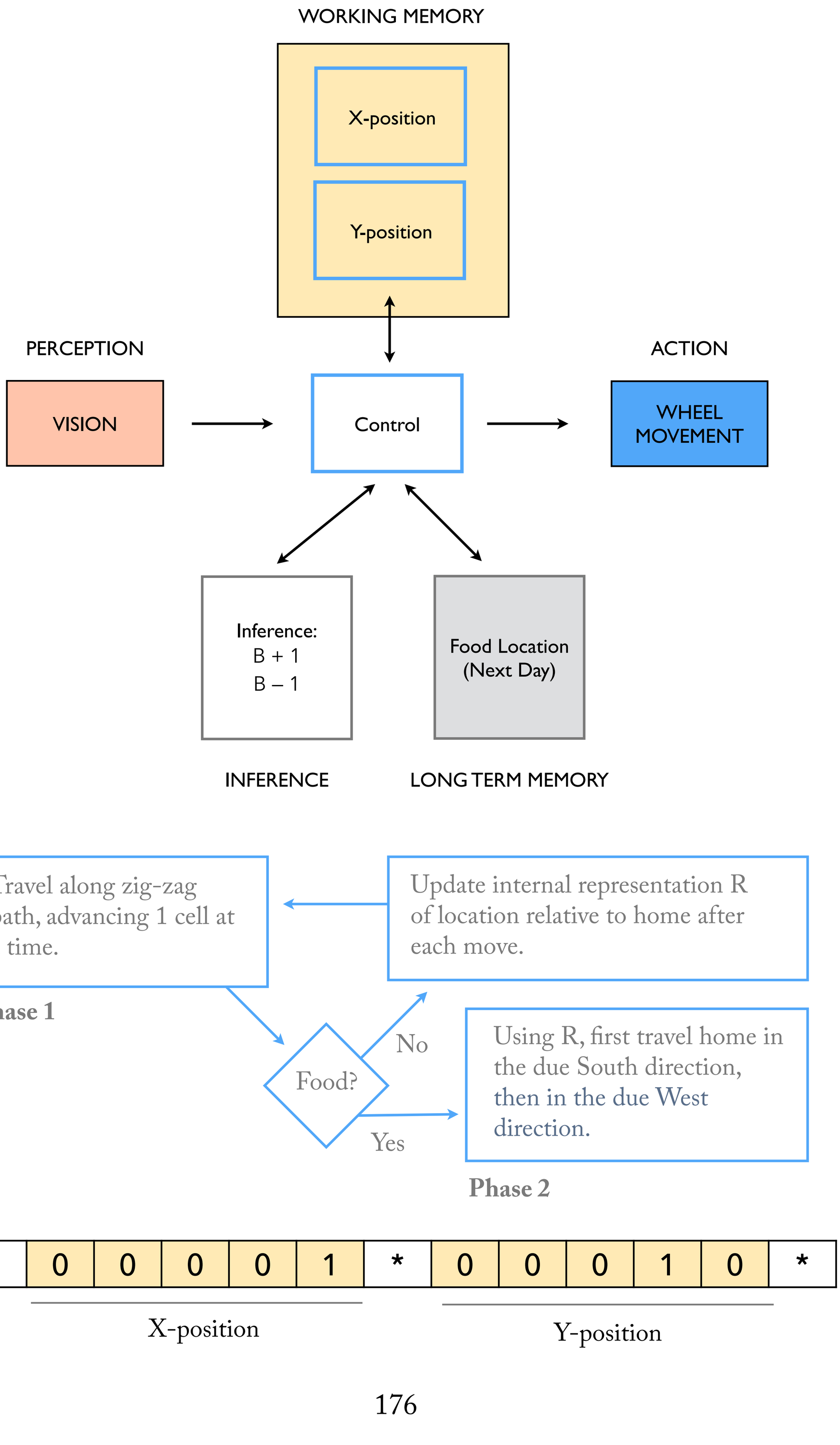
Conscious Cognition

- Limited capacity
- Domain general
- Voluntary
- Informationally open
- Slow
- Adaptable

Unconscious Cognition

- Large capacity
- Domain specific
- Automatic
- Informally encapsulated
- Fast
- Fixed function

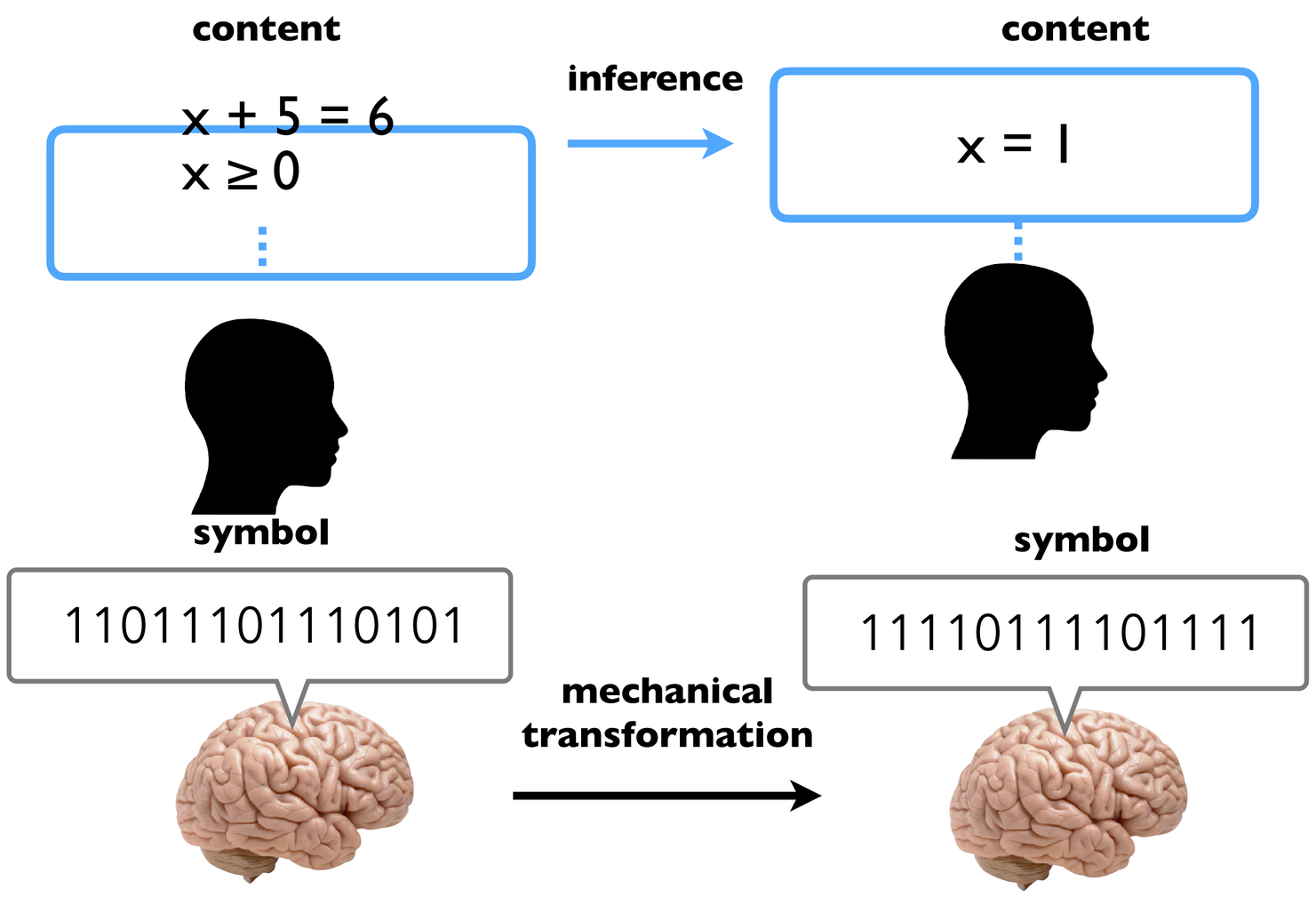
The Architecture of Turbot Cognition

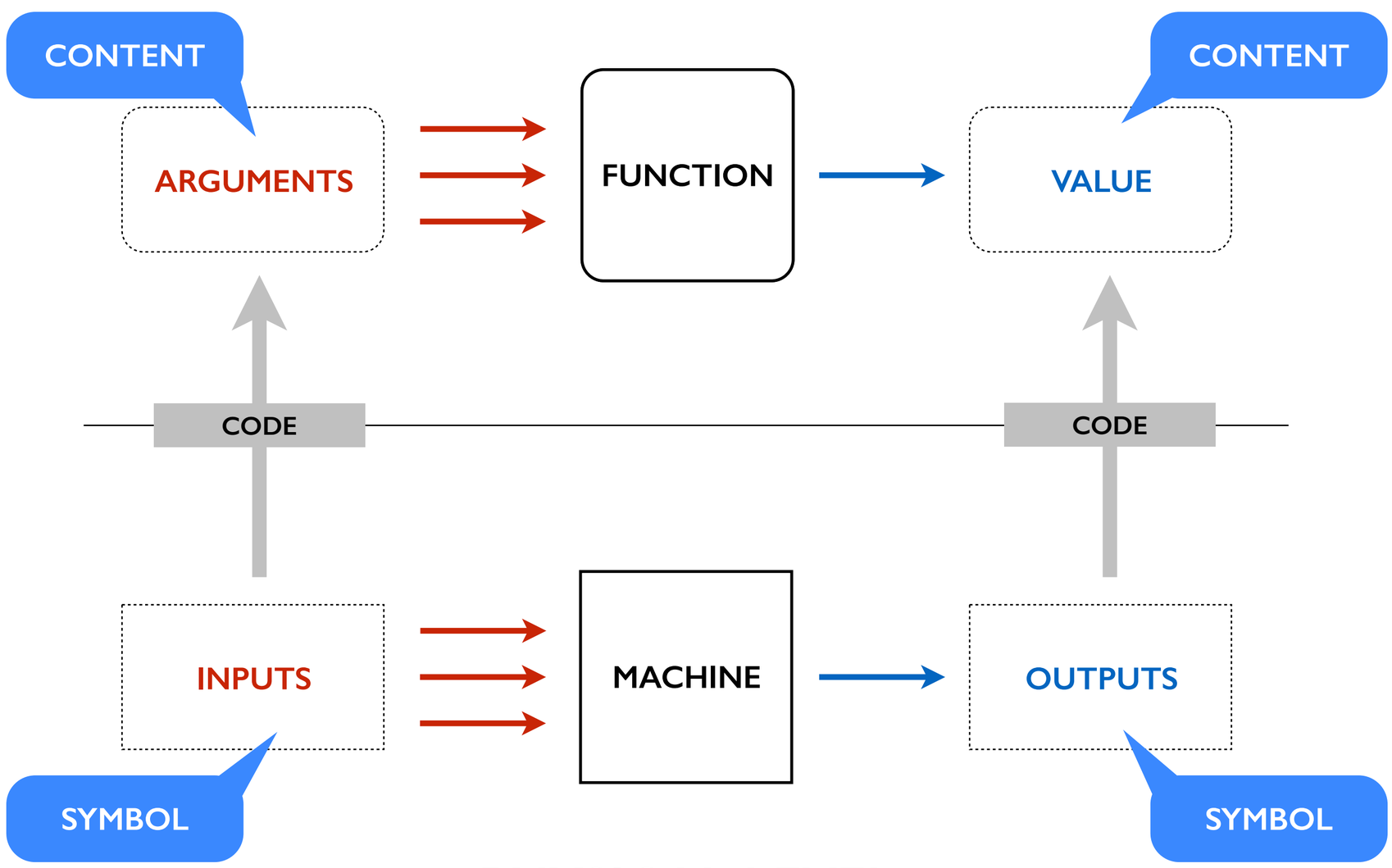


40. Computational Theory of Mind: The Long View

Cognition as Computation

> My hypothesis then is that thought models, or parallels, reality—that its essential feature is not 'the mind', 'the self', 'sense-data', nor propositions but symbolism, and that this symbolism is largely of the same kind as that which is familiar to us in mechanical devices which aid thought and calculation.

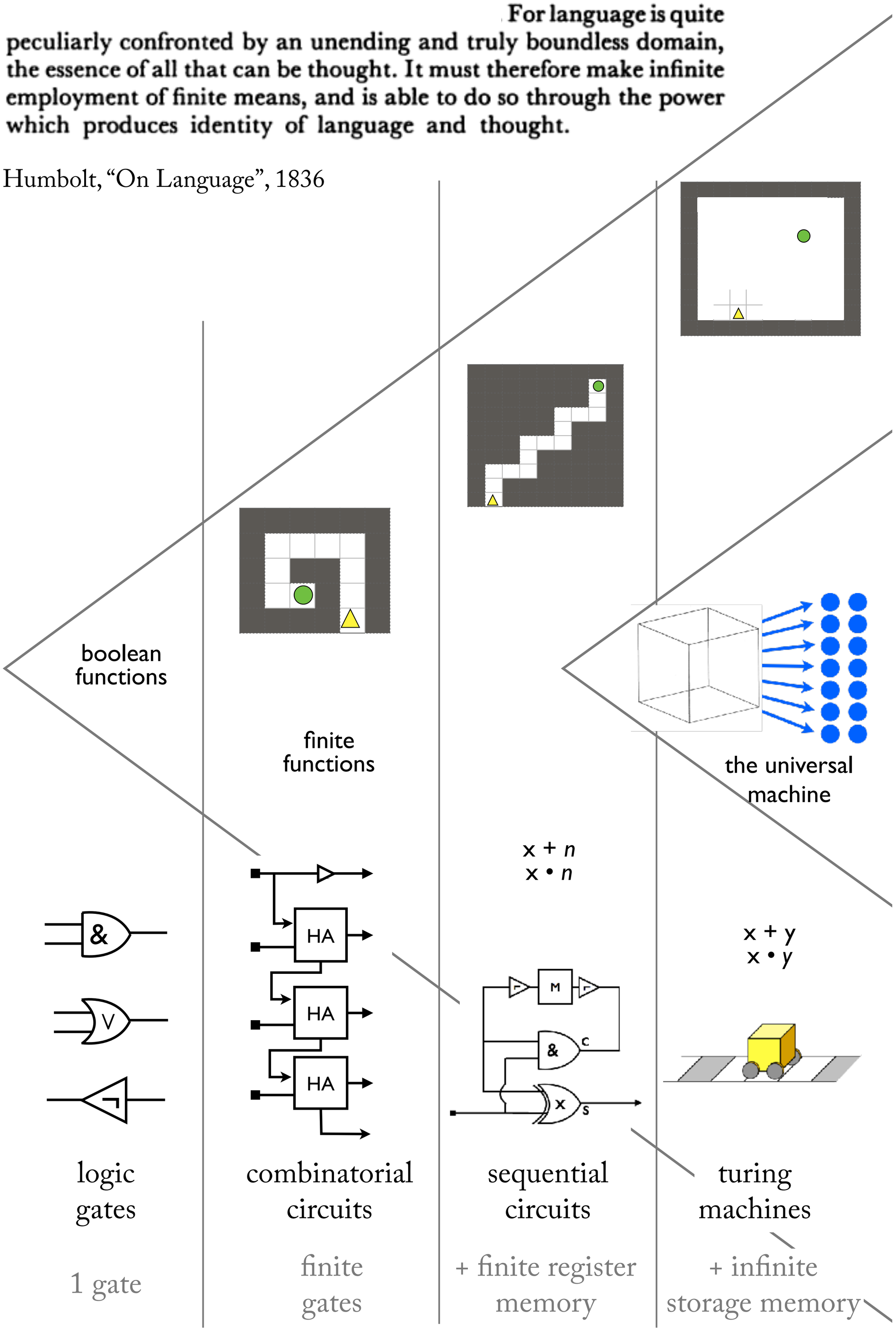




Orders of Computational Power

> For language is quite peculiarly confronted by an unending and truly boundless domain, the essence of all that can be thought. It must therefore make infinite employment of finite means, and is able to do so through the power which produces identity of language and thought.

Humboldt, "On Language", 1836



Levels of Computational Abstraction

"If one hopes to achieve a full understanding of a system as complicated as a nervous system… then one must be prepared to contemplate different kinds of explanation at different levels of description that are linked, at least in principle, into a cohesive whole, even if linking the levels in complete detail is impractical."

Marr, *Vision*, 1982

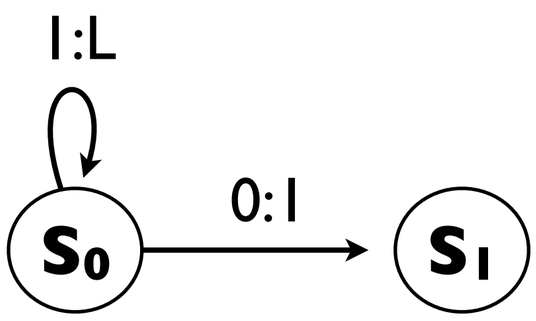
Ecological-Cognitive Level

+1

Functions

Semantic Abstraction

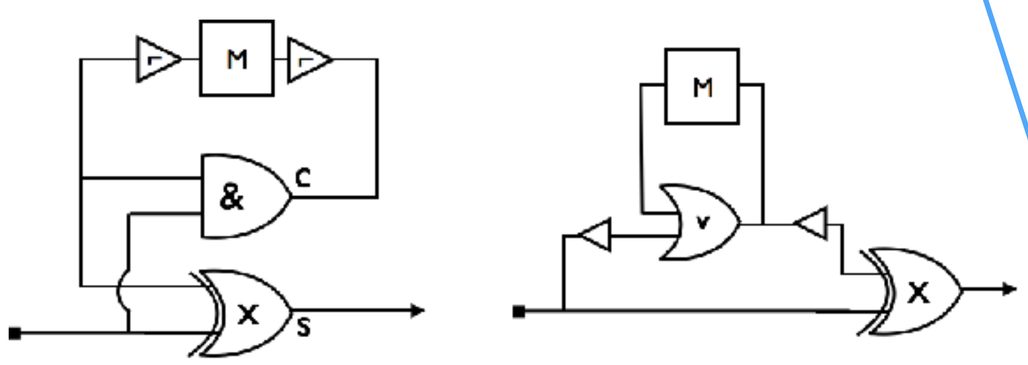
Algorithmic Level



State Machines

State Abstraction

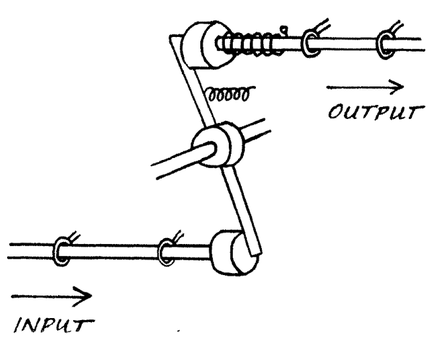
Mechanical Level

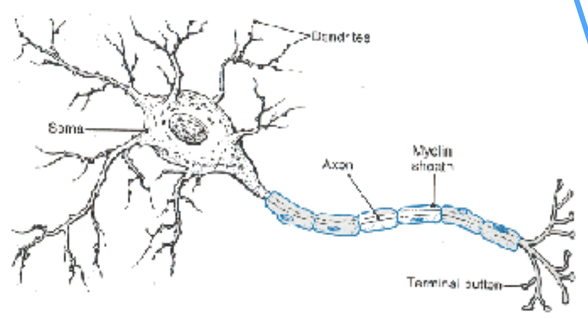


Logic Circuits

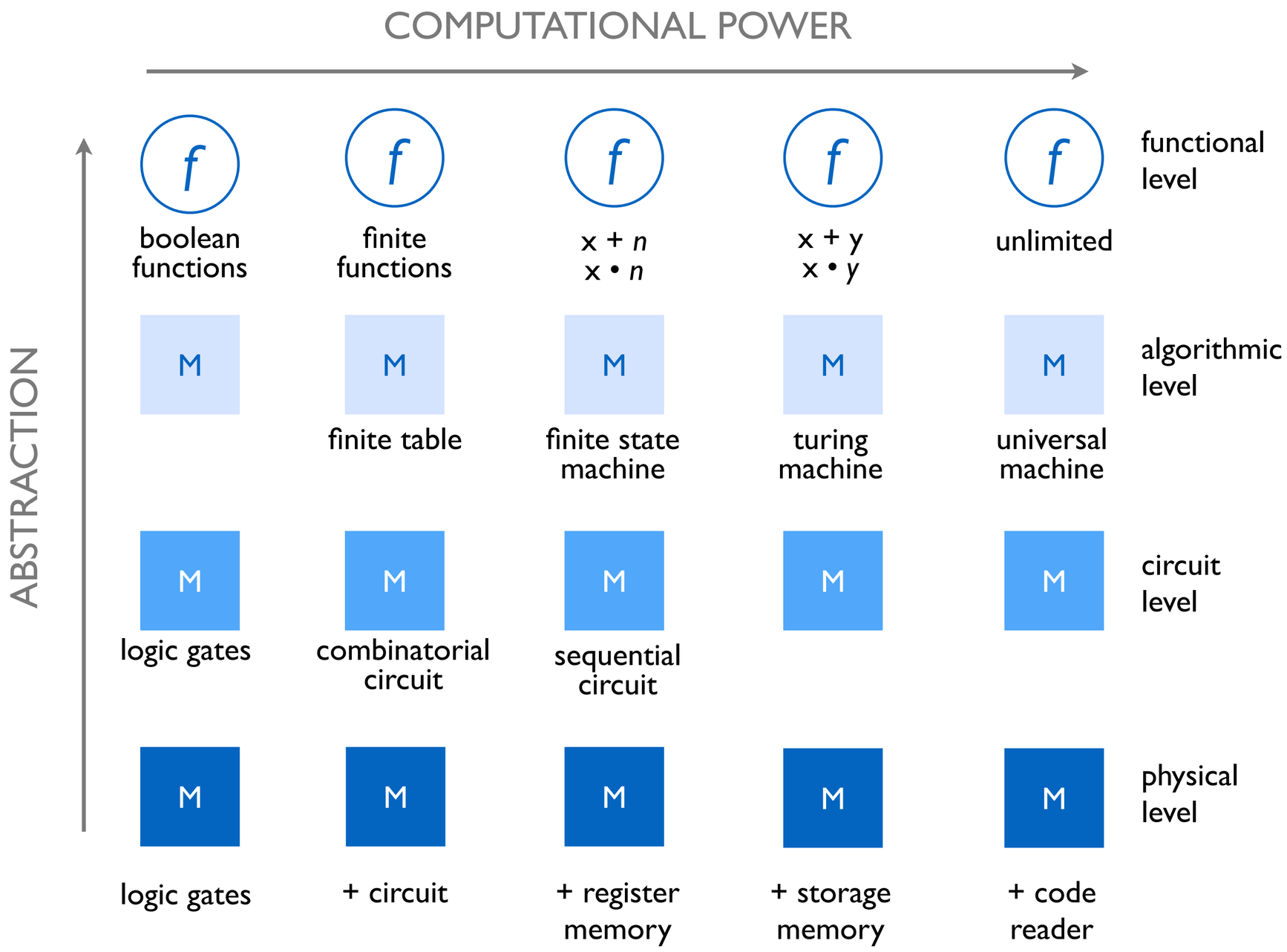
Functional Abstraction

Physical Level

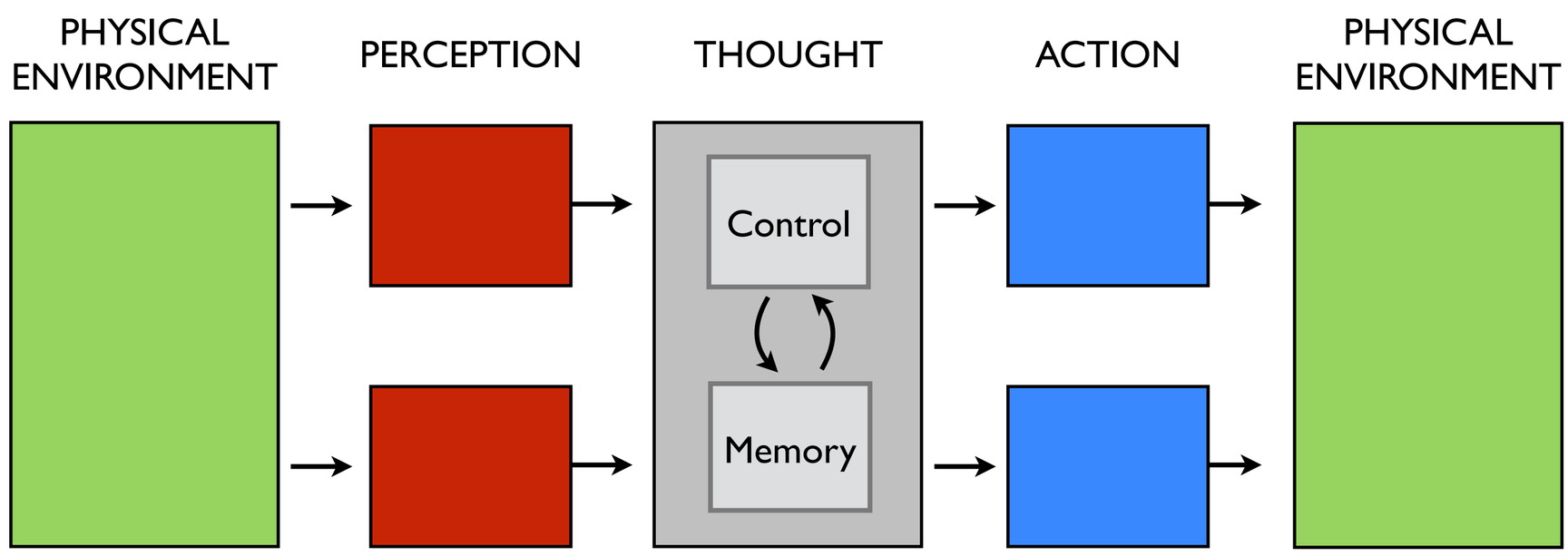


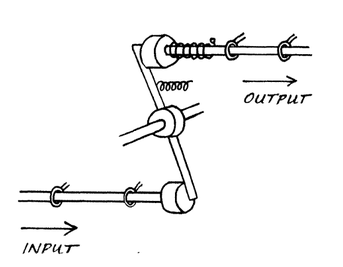


Physical Machines



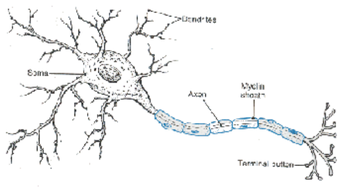
Cognition as Natural Computation







Home is 389 steps away.





I think therefore I am.

Editorial Assistance: Bill Kowalsky, Jon Ma, Christian DeLeon

Problem Sets

HW1. Basics: Circuits, Functions, and Representations

Note: I have put key words in **bold** in the text below. You may look these up (e.g. in the reading, or on Wikipedia) for help or context on a given problem.

I. Combinatorial Circuits

Design **combinatorial circuits** with the following input-output profiles:

1. **NAND**

IN 1	IN 2	OUT
0	0	1
0	1	1
1	0	1
1	1	0

2. Reconstructing OR

IN 1	IN 2	OUT
0	0	0
0	1	1
1	0	1
1	1	1

<< For this problem, do not use OR-gates.

DeMorgan's Law is useful here!

3. Splitting outputs

IN 1	IN 2	OUT1	OUT2
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

4. **XOR** (**exclusive OR**)

IN 1	IN 2	OUT
0	0	0
0	1	1
1	0	1
1	1	0

5. Combining circuits

IN 1	IN 2	OUT1	OUT2
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

<< You can box your XOR circuit from Problem 4 and reuse it here.

Hint: you might think about the circuits here as systematically analogous to equations in propositional logic, and the input-output tables as analogous to truth-tables. In many cases, you can use your knowledge of logic to work backwards to find the appropriate circuit.

II. Functions

6. Consider the function $m(\cdot)$ defined by the following **equation**:
   for any integers x, y, and z: $m(x, y, z) = 3 + (x \bullet y \bullet z)$
   a. Define the function in English, with no mathematical symbolism, using the terms "function", "argument", "value", "apply", and "return" correctly.
   b. Describe the function with another English sentence using the phrase "maps ... to".
   c. How many **places** does this function have?
7. Consider the function $k(\cdot)$ that takes two integers x and y as arguments, and returns the **successor** of their product as its value. Define this function with an equation.
8. This is a problem about **function composition**. Let $s(\cdot)$ be the 1-place **successor function**. Let $a(\cdot)$ be the 2-place **addition function**. Suppose that $p(\cdot)$ is a 2-place function composed of successor functions and addition functions. Given that $p(1, 2) = 5$, provide two possible definitions of $p(\cdot)$. Your two definitions should define different functions: functions that agree on the arguments (1, 2) but disagree on some other arguments.
9. Consider the function $j(\cdot)$ that maps two integers x and y to their product, so long as both x and y are between 0 and 2, inclusive. For any other arguments, it is undefined.
   a. Define $j(\cdot)$ with an equation.
   b. Define $j(\cdot)$ with a table.

   This is a problem about **partial functions**.
10. The **Boolean numbers** are 0 and 1. A **Boolean function** is a function which has only Boolean numbers as arguments and values.
    a. Using an equation, define a 1-place Boolean function which maps 1's to 0's, and 0's to 1's.
    b. Using an equation, define a 1-place Boolean function which maps any Boolean number to itself. This is called the **identity function**.
    c. Using an equation, define a 1-place Boolean function which maps any Boolean number to the number one. This is called a **constant function**.

III. Representations

11. Represent the numbers zero through ten in **binary**.

12. Invent a new **base-6 counting system**, with all new symbols for the 6 digits (including zero!). List your basic symbols and their meanings. Then represent the number thirty-two in this system.

13. a. Add the binary number 1111 to the binary number 1.
    b. Add the binary number 1011 to the binary number 11001.
    Use one of the methods of addition from the Petzold reading.

14. Define a function *f* which maps the symbol "@" to the number zero and the symbol "#" to the number one. Use a table or an equation.

15. Suppose we use the symbols in Problem 14 to form a base-2 counting system. Let's call these symbols "marks". A mark is either @ or #. Using an equation, define a 3-place function *g* which takes any three marks in a row and maps them onto the numerical value of that string of marks in that base-2 system. You will have to use the function in Problem 14 as part of the definition.

    So, for example, your function should predict that
    $g(@, @, \#)$ = one $g(@, \#, @)$ = two and so on…
    Your answer should take the form:
    "for any series of three marks x, y, z: $g(x, y, z)$ = …."

Hint: there are two things going on in Problem 15. It asks you to use your knowledge of functions to define the numerical values of symbols in a base-2 counting system. The method of computing binary values described by Petzold will be of help here. But it is also asking you to keep track of the difference between a *symbol* and its *content*, in this case the difference between "#" and *the number one*. Remember that only numbers, but not symbols, can be added and multiplied! At a deeper level, this problem asks you to give a partial **semantics** for a 3-place representational system. The science of formal semantics extends this kind of analysis to the domain of language.

Challenge problem (optional): define a function which maps strings of 1's and 0's of *arbitrary length* to their numerical values. This is the general problem of semantics for binary.

IV. Synthesis

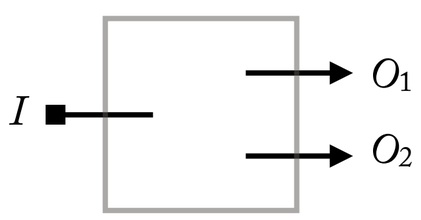
16. Let $tal(\cdot)$ be a function which maps strings of tally 1's and 0's to their numerical values. For example, $tal(0)$=zero, $tal(1)$=one, $tal(00)$=zero, $tal(01)$=one, $tal(11)$=two, and so on.

    Consider a machine *M* which takes as input a single atomic symbol *I*, and produces two-place symbol O_1O_2 as output (in the form of two atomic symbols O_1 and O_2), such that, for any input, $successor(tal(I)) = tal(O_1O_2)$. For example, 1 → *M* → 11. Design a combinatorial circuit for machine *M*, with the schematic form below.

17. Let $bin(\cdot)$ be a function which maps strings of binary 1's and 0's to their numerical values. For example, $bin(0)$=zero, $bin(1)$=one, $bin(00)$=zero, $bin(01)$=one, $bin(10)$=two, $bin(11)$=three, and so on.

    Consider a machine *N* which takes as input a single atomic symbol *I*, and produces two-place symbol O_1O_2 as output (in the form of two atomic symbols O_1 and O_2), such that, for any input, $successor(bin(I)) = bin(O_1O_2)$. For example, 1 → *N* → 10. Design a combinatorial circuit for machine *N*, with the schematic form below.

Hint: these two problems look complex, but they are just asking you to apply the intuitive concept of **computation** we have been discussing in the course so far. First figure out what table of input and output symbols would satisfy the equations given. Only then solve for the underlying circuit.



Schematic for *M* and *N*

HW2. Computing with CCs

10.10.25

I. CC Arithmetic

Starting with **the three basic logic gates only**, design CCs that compute the following functions. At any stage you may define a new boxed circuit from a previously established circuit or logic gate. Useful circuits to define in this way include **All-1**, **XOR**, **half-adder**, and (if you wish) **full-adder**. You will also want to introduce boxed circuits for specific mathematical functions as the need arises.

1. Half adder
2. +1 [0-15] B
3. +2 [0-15] B
4. +3 [0-15] B
5. $2x$ [0-15] B
6. $2x$+1 [0-15] B
7. 2(x+1) [0-15] B (only use *one* sub-part which computes +1)
8. x+y [0-15] B (i.e. addition, bounded to 0-15 for each argument)
   Reference**:** Petzold (2000) "A binary adding machine."
9. For any integer n, in principle, could you design a CC that computes +1 [0- n] B? Why or why not? Short paragraph.
10. Could you design a CC that computes +1 B (i.e. unbounded)? Why or why not? Short paragraph.

Challenge problem: prove the following: for any numerical function f and any integer n, there is a CC which computes f[0 - n] B.

On binary addition

Hint 1: Use the grade school algorithm for long addition.

Start by thinking of the problem in terms of long addition, e.g.:

	1	1	Input 1
+	1	0	Input 2
1	0	1	Output

Then set up your machine in the same format:

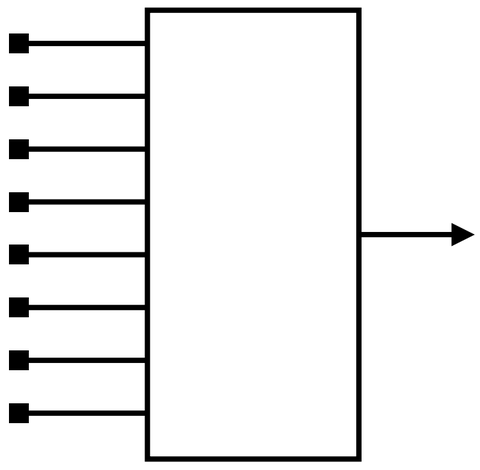
	Digit 4	Digit 3	Digit 2	Digit 1	
	☐	☐	☐	☐	Input 1
	Digit 4	Digit 3	Digit 2	Digit 1	
	☐	☐	☐	☐	Input 2
Digit 5	Digit 4	Digit 3	Digit 2	Digit 1	
☐	☐	☐	☐	☐	Output

When filling in the circuit, think of your machine first in terms of columns (e.g. how does the Output Digit 1 relate to the Input Digit 1's?). Then think about how the columns relate to each other. A key idea here is the "carry digit"– remember how this works in standard long addition, and see if you can apply the same idea to the construction of your machine.

Hint 2: Add twice. When you are doing long addition, note that you really have to do basic addition *twice* for each column. First you have to add the digit from the top row to the digit from the bottom row, and then you have to add the carry to the result. (You can also do these in the opposite order.) The same thing is true in binary, a fact you should convince yourself of before trying to solve the problem. You'll find this idea has a direct correlate in the circuit which computes binary addition.

II. CC Perception

For each of the following problems assume a CC that has 8 inputs and 1 output, like the schematic circuit below. Think of these as computing **perception**: they take as their input an array of stimulations (like hitting the retina) and they output a symbol which represents a specific shape or category.



10. **Edge detector.** Design a machine that obeys the following rule: output 1 iff there is a string of *at least* three consecutive 1's anywhere in the input.

11. **Object detector.** Design a machine that obeys the following rule: output 1 iff there is a string of *exactly* three consecutive 1's anywhere in the input.

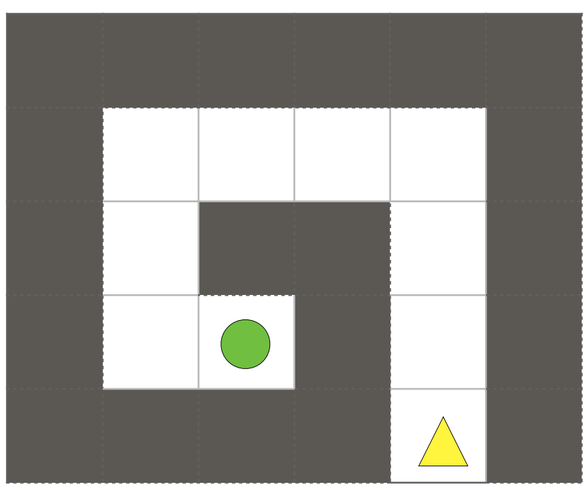
12. **Landmark recognition.** Design a machine that obeys the following rule: output 1 iff the input = 11010111

III. CC Navigation

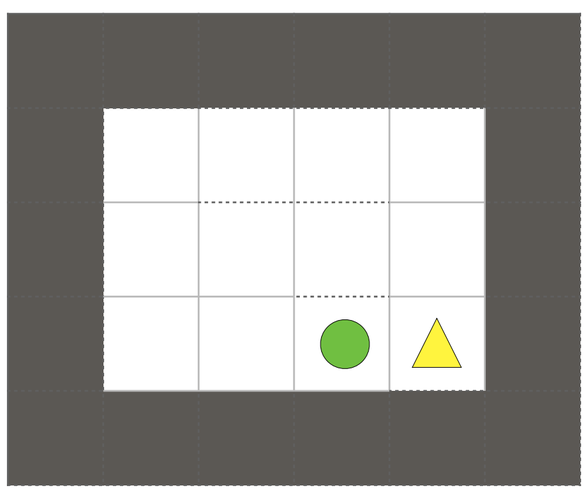
In these navigation tasks, the yellow triangle indicates the turbot's starting position and orientation. Navigation is "completed," and the problem solved, when the turbot *crosses over* the goal. It does not need to stop on the goal. (Think of Pac-Man.)

For each of the following problems, answer the question: can it be completed by a combinatorial circuit embedded in a turbot? If so, give the circuit. If not, explain why in a short paragraph.

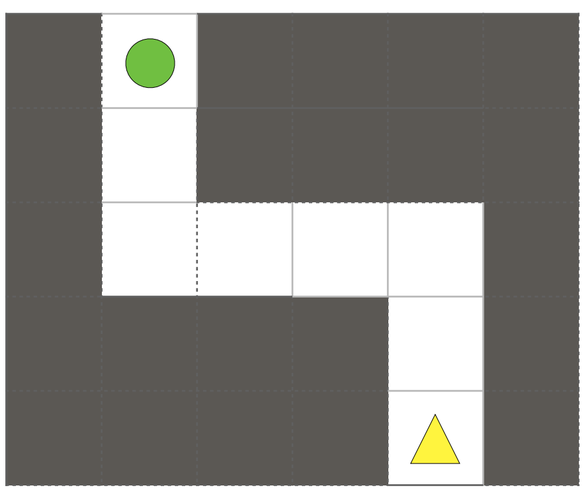
13. **Spiral**



14. **Full circle**



15. **Zig-zag**



IV. Reflection

16. **Representational systems.** Suppose you were designing a calculator for solving a high-stakes problem. (Perhaps "you" are Nature, the "calculator" is the Brain, and the "problem" is Survival.) Would you design it to calculate in binary or in tally? Why? Use specific examples to justify your answer. ~1 paragraph

17. **Embodied computation.** Consider a Turbot like the one in problem 13. What function, if any, does it compute? If it does compute a function, what are the arguments and what is the value of this function? How does this compare to the function computed by the "directions" mode in Google Maps? ~1 paragraph

HW3. Computing with SCs

10.27.25

I. SC Arithmetic

At any stage you may define a new boxed circuit from a previously established circuit. You may assume boxed circuits for **XOR** and **half-adder.** You will also want to introduce boxed circuits for specific mathematical functions as the need arises.

Design SCs that compute the following functions.

1. +1 B
2. +2 B
3. $2x$ B
4. $2x$+1 B [Use one $2x$ circuit and one +1 circuit]
5. $2(x+1)$ B [Use one $2x$ circuit and one +1 circuit]
6. $x+y$ B

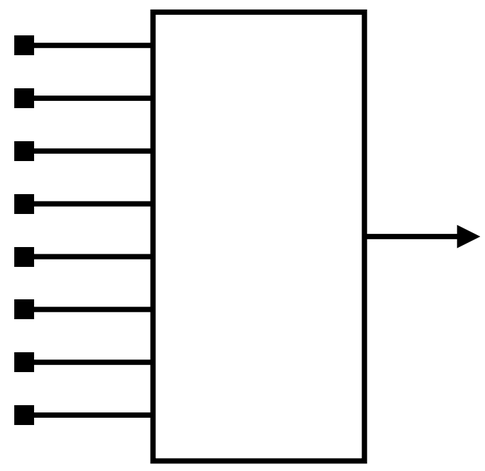
For each of the following functions, is it possible to compute this function with an SC? If so, construct the circuit. If not, explain why in a short paragraph: be as clear and precise as you can.

7. +3 T
8. $2x$ T
9. $x + y$ T
10. $x \bullet y$ B (multiplication)

II. SC Perception

For each of the following problems assume a SC that has 8 inputs and 1 output, like the schematic circuit below. Think of these as computing **perception**: they take as their input an array of stimulations (like hitting the retina) and they output a symbol which represents a specific shape or category.

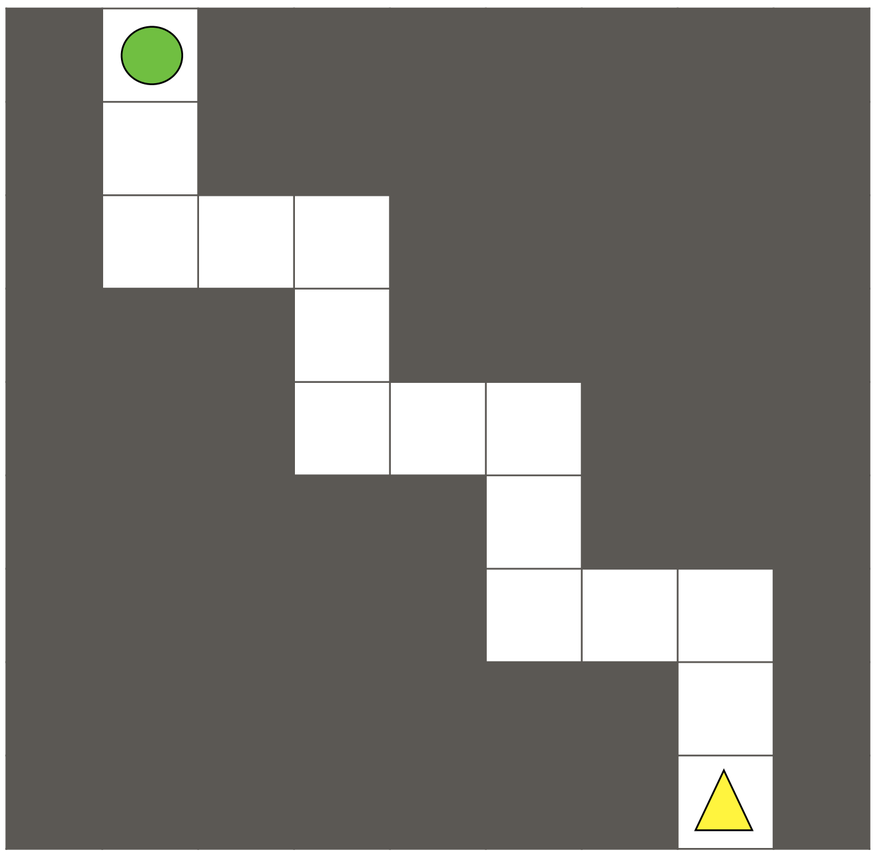
11. **Change detector.** Design a machine that obeys the following rule: output 1 iff the current input differs in any way from the previous input.
12. **Motion detector.** Let an *object image* be a string of exactly three consecutive 1's. Design a machine that obeys the following rule: output 1 iff there is an object image anywhere in the input moving upwards 1 unit per unit of time.



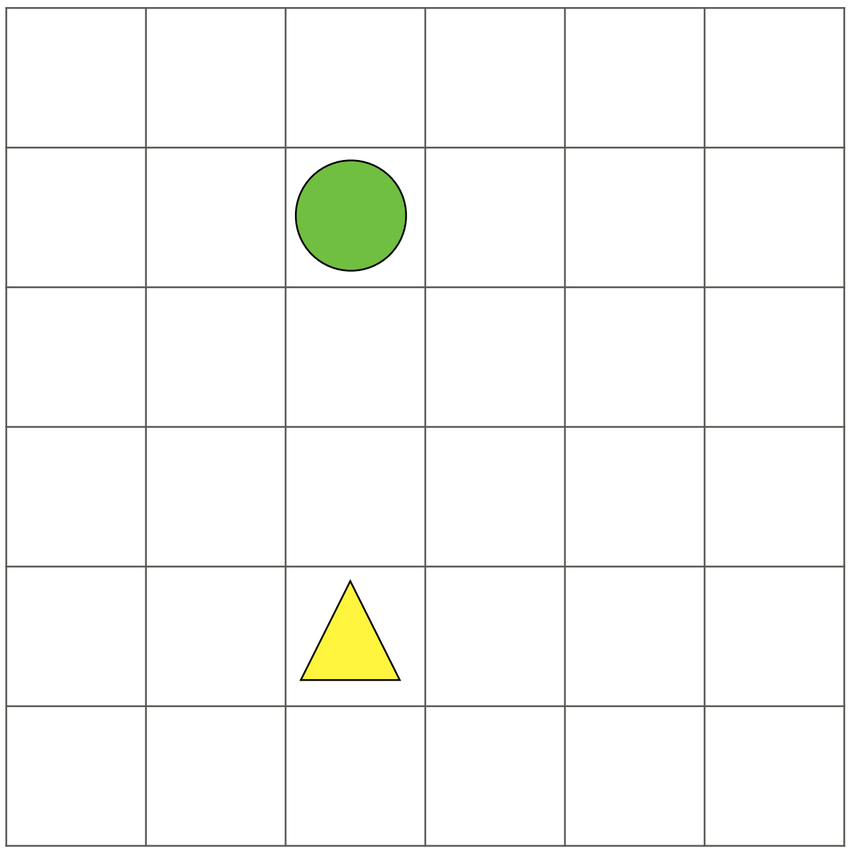
II. SC Navigation

For each of the following navigation tasks: can it be completed by a combinatorial circuit embedded in a turbot? If so, give the circuit. If not, explain why in a short paragraph.

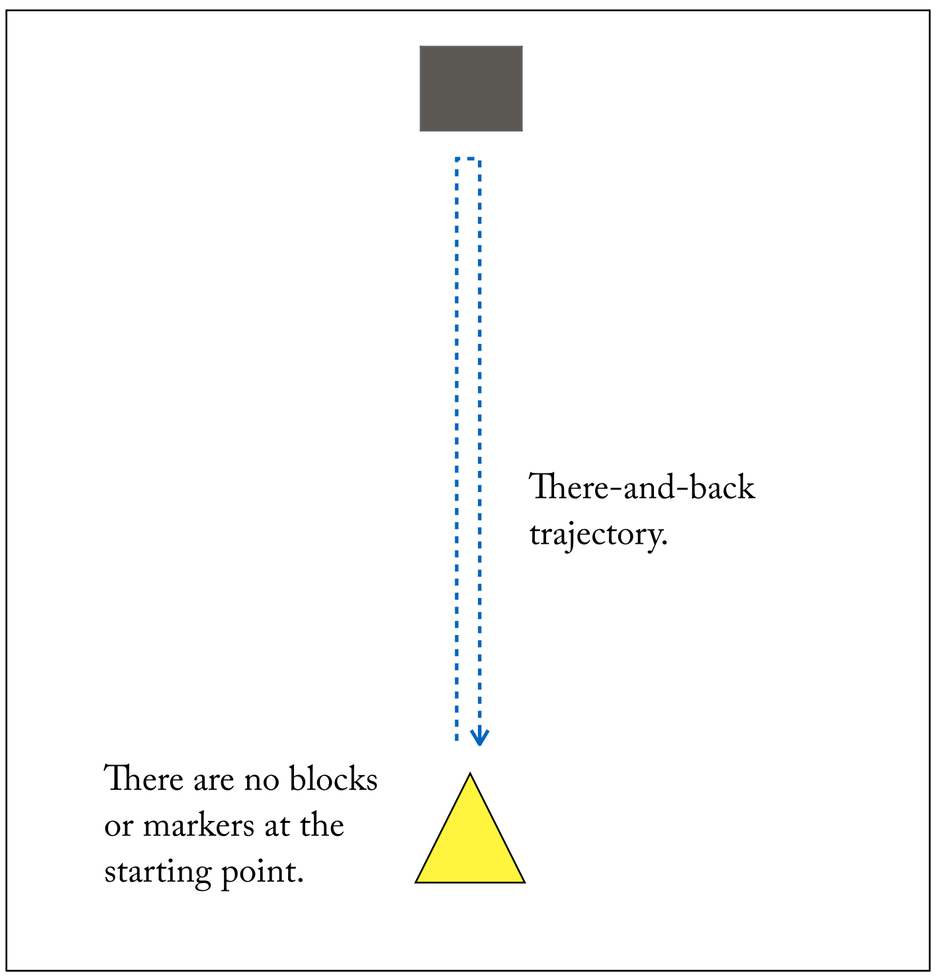
13. **Zig-Zag:** reach the goal and **keep going**.



14. **Three ahead:** reach the goal and **stop**.



15. **Mad Max:** go to a block directly ahead, but an unknown distance away, then return to the starting point and **stop**.



Note 1: SC Turbots have two outputs, one for each wheel.

Note 2: The goal (the green circle) is invisible to the turbot; it reads as an empty block.

III. Reflection

Short paragraph answers.

16. SCs are more computational powerful than CCs: SCs can solve problems that CCs cannot. What physical abilities would you add to an SC if you wanted to make it even more powerful?

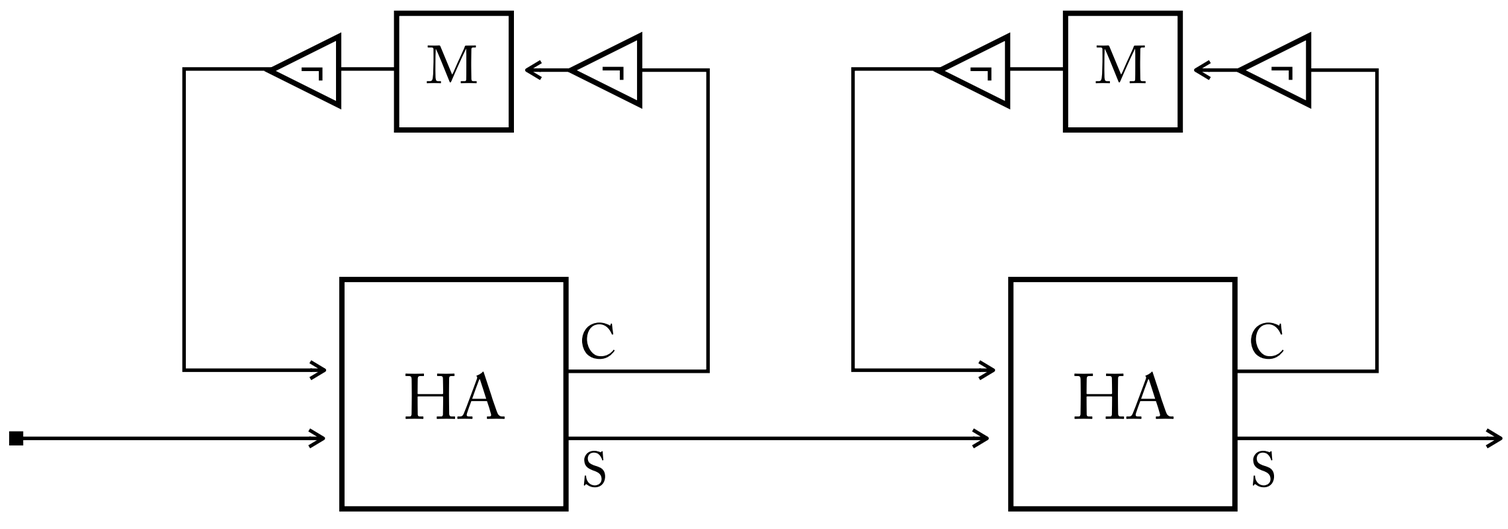
17. Could there be an intelligent being with no memory?

HW4. Computing with FSMs

11.11.25

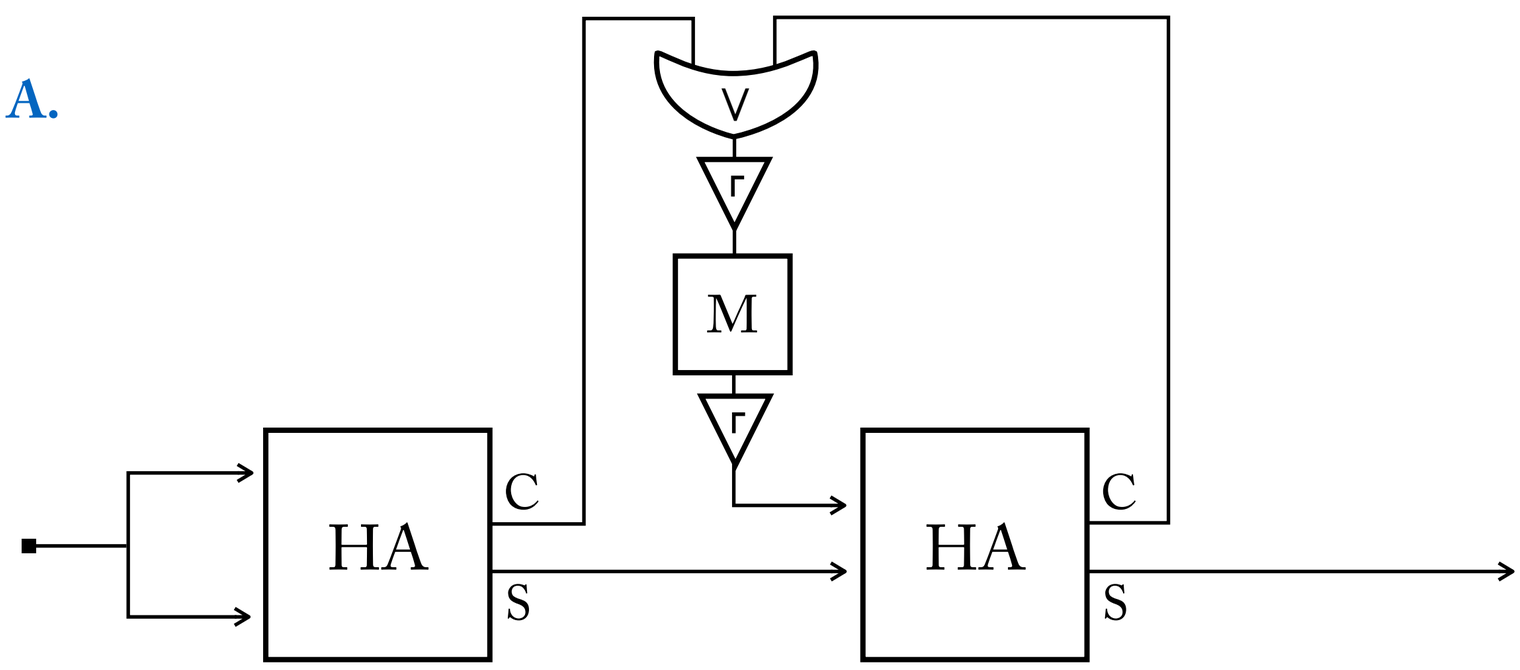
I. State Abstraction

1. **State abstraction.** Construct a state table for the following machine, then convert it to a FSM diagram. What function does it compute in binary?

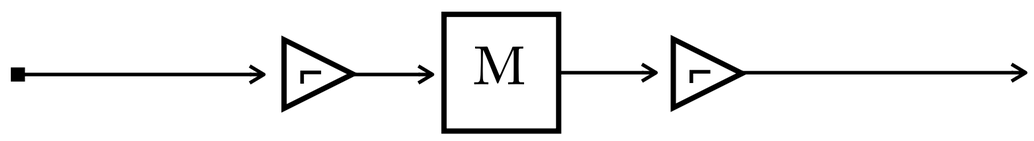


2. **Multiple realizability.** Consider the following two SCs. What function does each compute in binary? Do they implement the same FSM? Why or why not? Show your work.

A.



B.



II. FSM Arithmetic and Beyond

Design FSMs that compute the following functions. (Don't design an SC first to get to your solution.)

3. +1 T

4. +2 T

5. +1 B

6. +2 B

7. +3 B

8. $2x$ B

9. $2x$+1 B

10. $x + y$ B

> **Challenge problem:** prove the following: for any numerical function f and any integer n, there is an FSM which computes f[0 - n] B.

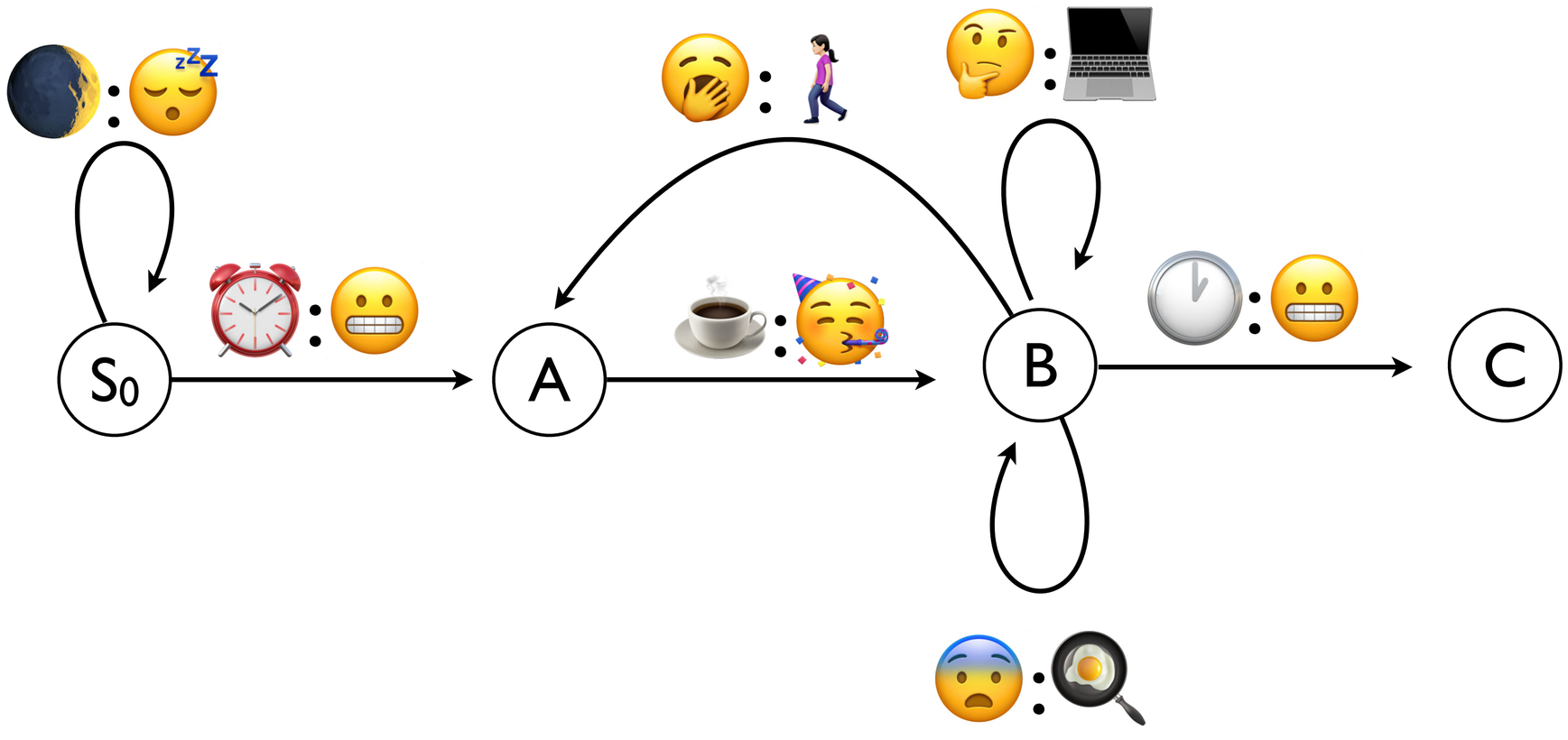
11. **ON/OFF Switch**

    This circuit toggles between outputting 1's or 0's; it switches each time it receives a 1, but keeps the same output if it receives a 0. Start in the off state, where output = 0.

12. **The Daily Grind**

    Design an FSM which captures the cycles and possible paths of your typical morning routine, using only emojis as inputs and outputs.

For example, here is a fragment of my day:



III. FSM Navigation

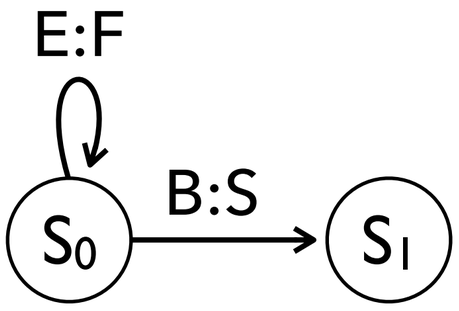
For each of the following navigations tasks, design an FSM which solves it.

Use the following abbreviations:

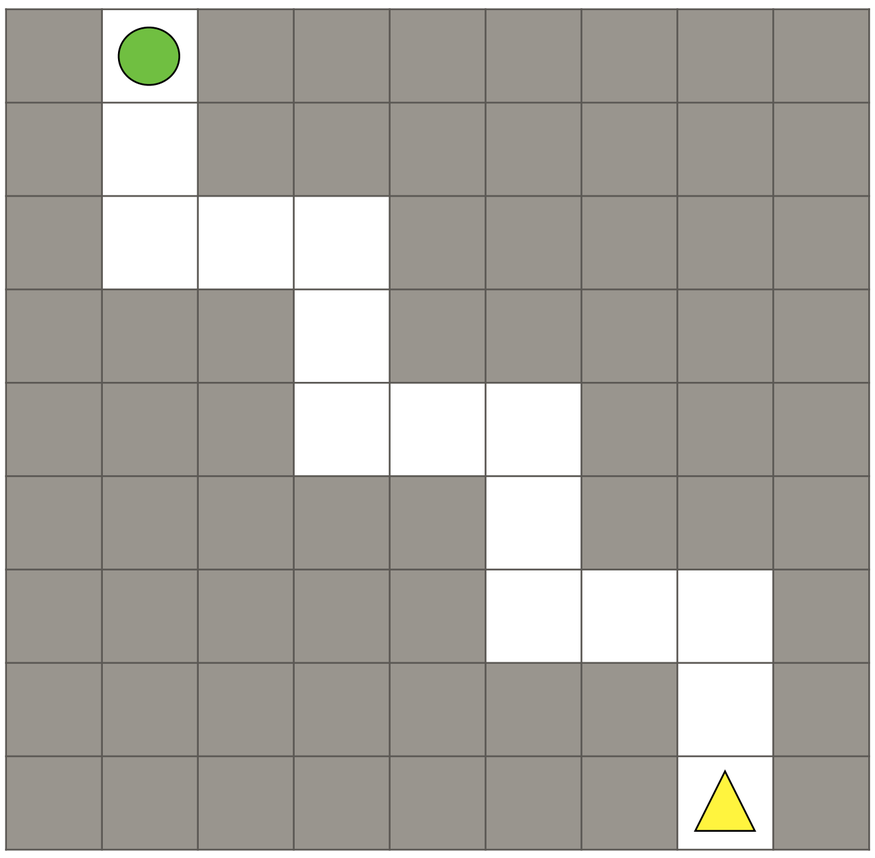
Inputs:
0 = **E** (Empty)
1 = **B** (Block)

Outputs:
11 = **F** (Forward)
00 = **S** (Stop)
10 = **R** (Right)
01 = **L** (Left)

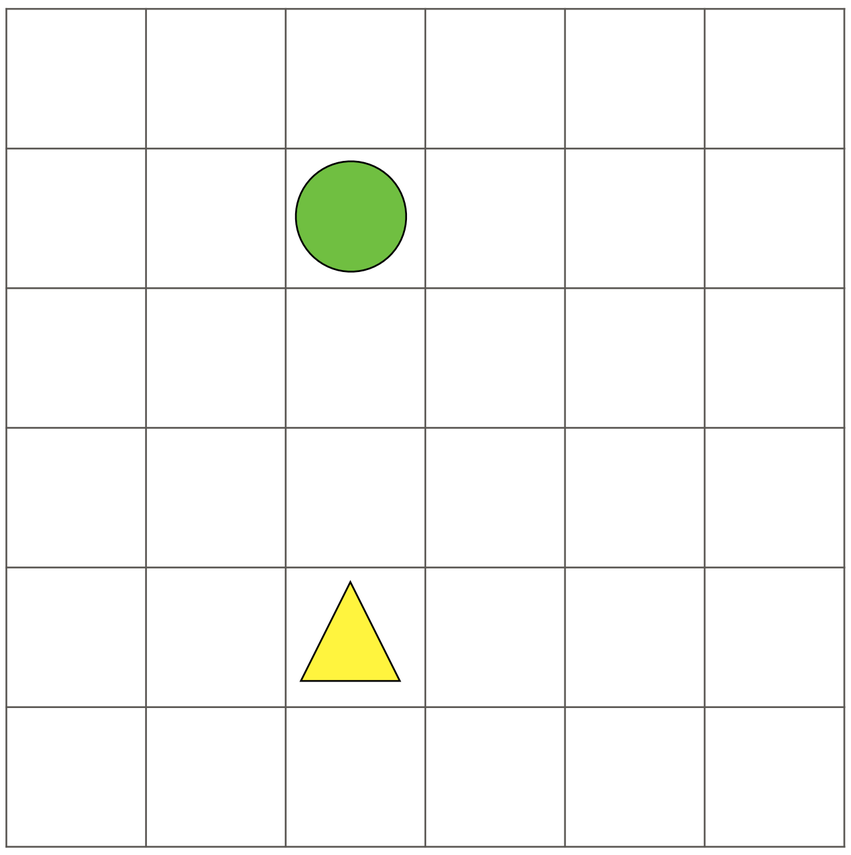
For example, this machine goes forward until it hits a block, then stops:



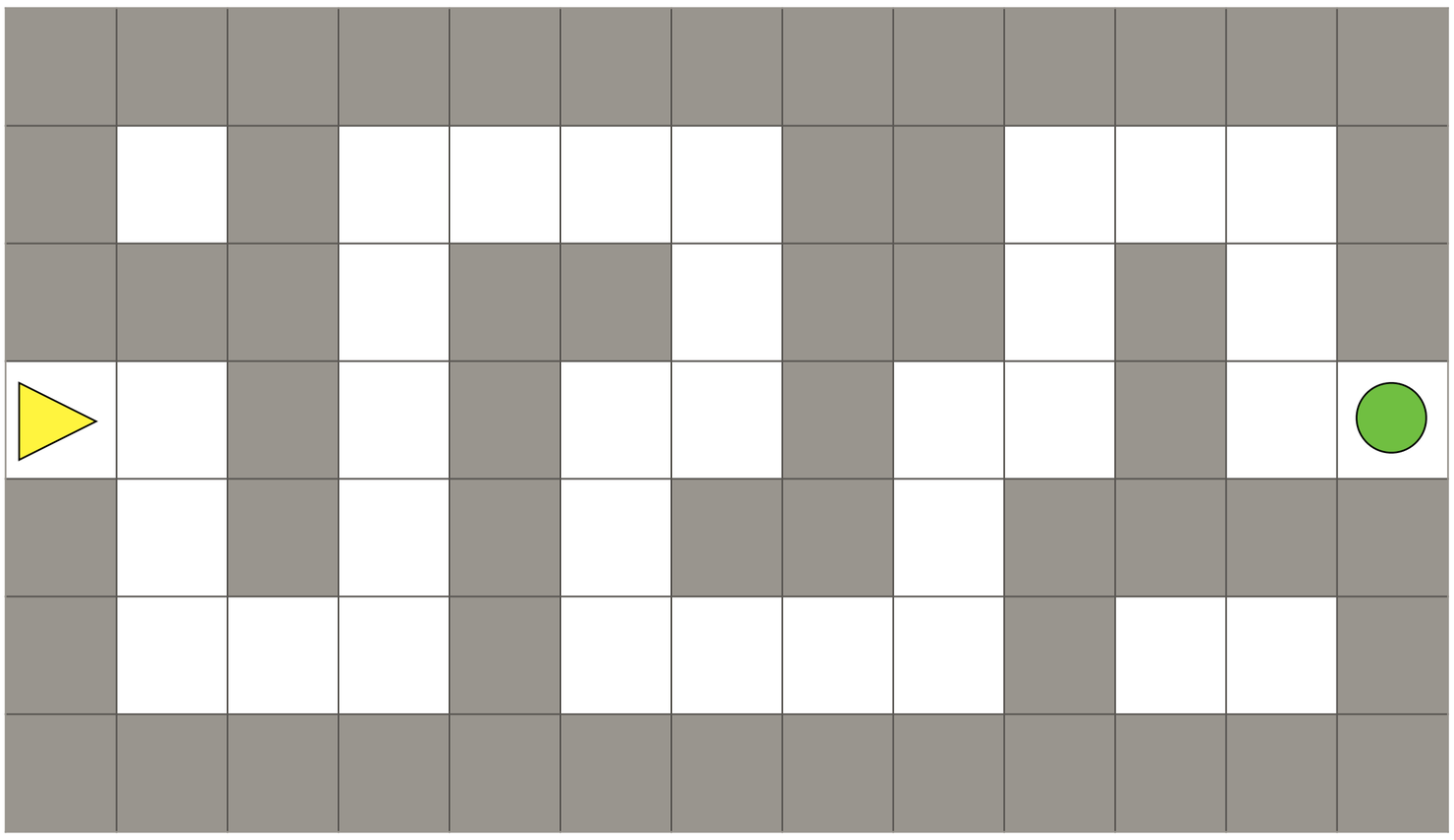
12. **Zig-Zag.** Reach the goal and **keep going**.



13. **Three Ahead.** Reach the goal and **stop**.



14. **Way Finder.** Given *any* non-branching maze, make your way all the way through, and keep going.
A **non-branching maze** is a path that never branches and is never wider than 1-cell across, except on corners where two 1-cell width paths may intersect. Here is an example.*



* This problem was designed by Isaijah Johnson and Jack Schaeffer, Phil 133 W19.

II. FSM Arithmetic and Beyond

Design FSMs that compute the following functions. **Don't** design an SC first to get your solution; **do** think through the FSM directly.

4. +1 T
5. +2 T
6. +1 B
7. +2 B
8. +4 B
9. 2*x* B

Take away problem 9! (Or: replace it with the problem: given the circuit, was is the FSM?)

Caution!

Function composition **does not work** for FSMs. You can't combine two +1 machines to get a +2 machine.

This has to do with the fact that FSM diagrams show states of the *overall* machine, not the flow of information *through* it.

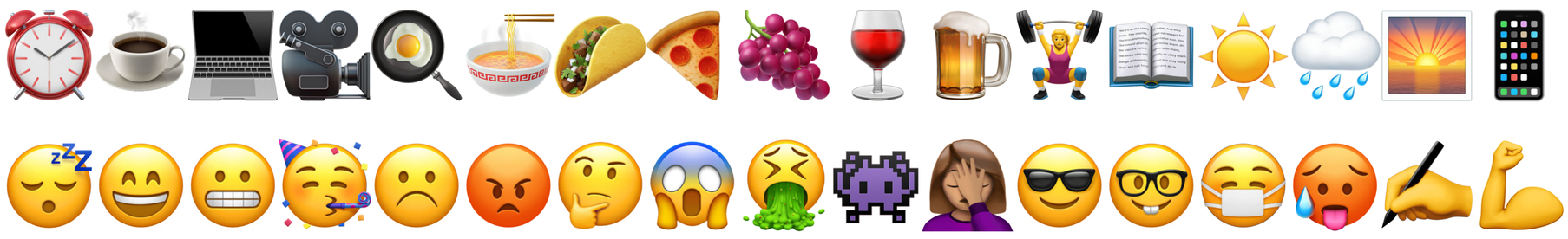
10. **ON/OFF Switch**

This circuit toggles between outputting 1's or 0's; it switches each time it receives a 1, but keeps the same output if it receives a 0. Start in the off state, where output = 0.

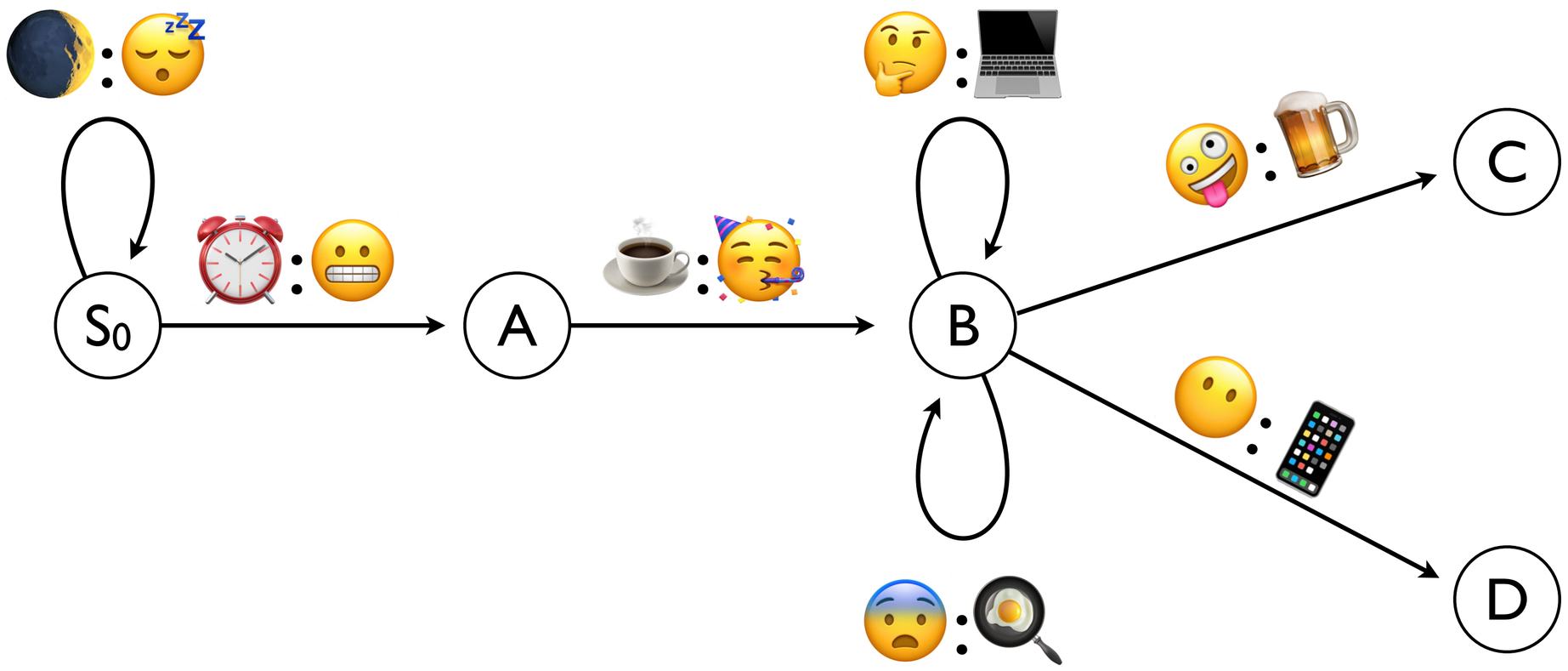
11. **The Daily Grind**

Design an FSM which captures the cycles and possible paths of your typical day during quarantine. Make it one big loop to take you from morning to morning. Feel free to add alternative inputs and outputs as appropriate.

Possible inputs and outputs:



For example, here is a fragment of my day:



HW5. Computing with TMs

2.18.25

Rules

- **Annotation:** all machines must be annotated.
- **Arguments:** Tally problems are defined on arguments $x \geq 1$; for binary problems, $x \geq 0$.
- **Blocks:** blocks are define differently for tally and binary.
  - Tally block = a single block of 1's.
  - Binary block = a block of 0's or 1's, flanked by *s.
- **X- and Y-Blocks:**
  - For 1-place functions, the tape starts with a single block.
    For 2-place functions, the tape starts with two blocks.
  - When a problem involves two arguments x and y:
    - The x-block is on the **right**, and the y-block is on the **left.**
- **Standard Position.**
  - All problems begin in standard position, and must return to standard position.
    - Tally SP = the right most cell of the right most block.
    - Binary SP = the right most * of the right most block.
- **Final configuration:**
  - All problems must end in standard position.
  - All problems must end with a well-formed output: a single block.
- **Boxes**
  - You can use a boxed version of any problem you have solved in an earlier problem, provided it satisfies the criteria for boxing.

I. TM Arithmetic: Tally

Assume $x,y \geq 1$.

1. $x+1$ T

2. $x+3$ T.
   Use your solution from (1).

3. $3x$ T.
   Use your solution from (2).

4. $x+y$ T.
   Use your solution from (1).
   For this problem, do not assume that the input blocks will be only separated by 1 cell. They may be separated by any number of cells.

5. $3(x + y)$ T.
   Use your solutions from (3) and (4).

6. $x + 3y$ T.
   Use your solutions from (3) and (4).

II. TM Arithmetic: Binary

Assume $x,y \geq 0$. The following problems have some arbitrary constraints built-in to make them simpler.

7. $x+1$ B
   • Assume the input block will always have an extra 0 in the left-most cell.

8. $x - 1$ B
   $f(x) = x - 1$ if $x \geq 1$
   $f(x) = 0$ if $x = 0$

9. $x + y$ B
   Use your solutions from (7) and (8).
   • Assume that y-block contain enough 0's padding the left side initially so that the entire block may contain the binary representation of $x+y$.
   • Don't forget to "clean up" your tape at the end of computation.

III. Algorithmic Design

10. Multiplication: $x \bullet y$

    Using a combination of flow charts, prose, and "snapshots" of the tape, describe a step-by-step *strategy* for computing multiplication. This should not be a complete Turing Machine. (That is a difficult and laborious problem.) But it should be sufficiently detailed that you think someone *could* build a Turing Machine from your instructions, if they knew what they were doing. Your choice of Tally or Binary.

11. Multi-function machine

    Design a machine which can perform different functions depending on the user input (at least successor, addition, multiplication), similar to a calculator. There are many way to do this, so be creative. Use a combination of flow charts, prose, and "snapshots" of the tape; don't design a completed Turing Machine.

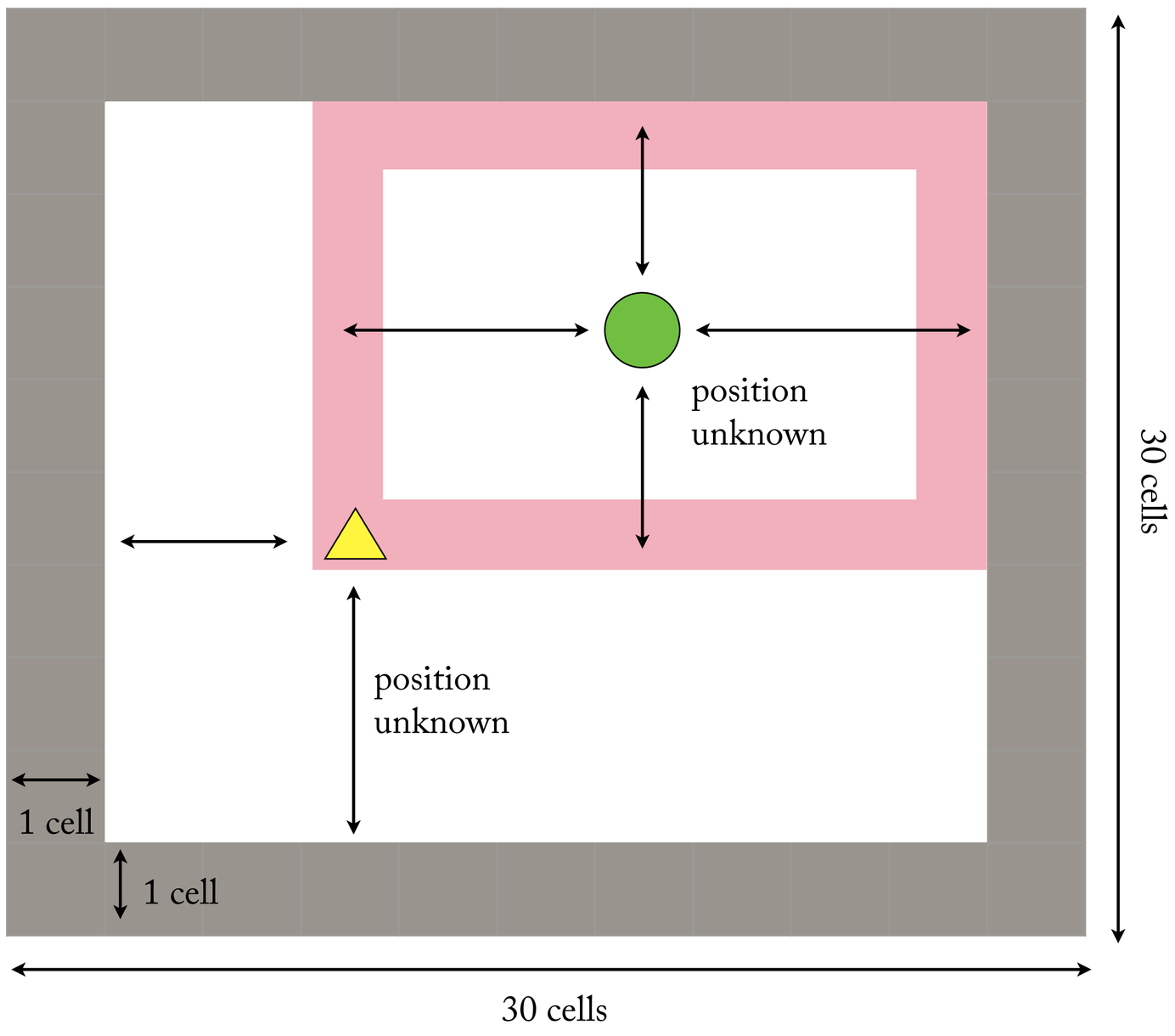
HW6. Navigation with Turbots

2.18.25

The Desert Ant

Goal: Search until you find food (and pass over it), then return to where you started as quickly as possible. You may choose any initial configuration for the tape.

- **Assumption 1:** the food will always appear in the NE quadrant relative to the Turbot's starting position (and not at the same position as the Turbot itself.)
- **Assumption 2:** the world is 30 X 30 cells, surrounded by a wall.
- **Constraint:** use a maximum of 20 cells on the Turing Machine tape.
- **Unknowns:** the starting position of the Turbot in the world is unknown, the position of the food within the NE quadrant of the Turbot is also unknown.



1. Flow-chart.

Design a high-level flow-chart that describes your solution to the Desert Ant Problem. Be sure to describe both the design of the machine and the configuration of the tape.

2. Turbot

Construct a TM that follows flow-chart in in (1) and solves the Desert Ant Problem. Use modular boxed functions when you can. Label all the major parts of your design according to their function.

For very similar functions, it is sufficient to provide one solution, and indicate the obvious change to the other. (For example, if you have one sub-part that handles left turns, and a symmetrical one that handles right turns, you can just design one of these, and describe the other one in words.)

3. The Life Cycle

Suppose that, after solving the Desert Ant Problem, your Turbot must stay at home for a fixed amount of time (say: 20 clock cycles), then return to the food, and the whole cycle must repeat again.

Assuming the food never moves, how would you change your initial design to accommodate this challenge? How would you change the tape? Describe your approach in prose, or with the help of a flow chart, but don't design the actual Turbot.